栈与队列

栈与队列

栈的定义与实现

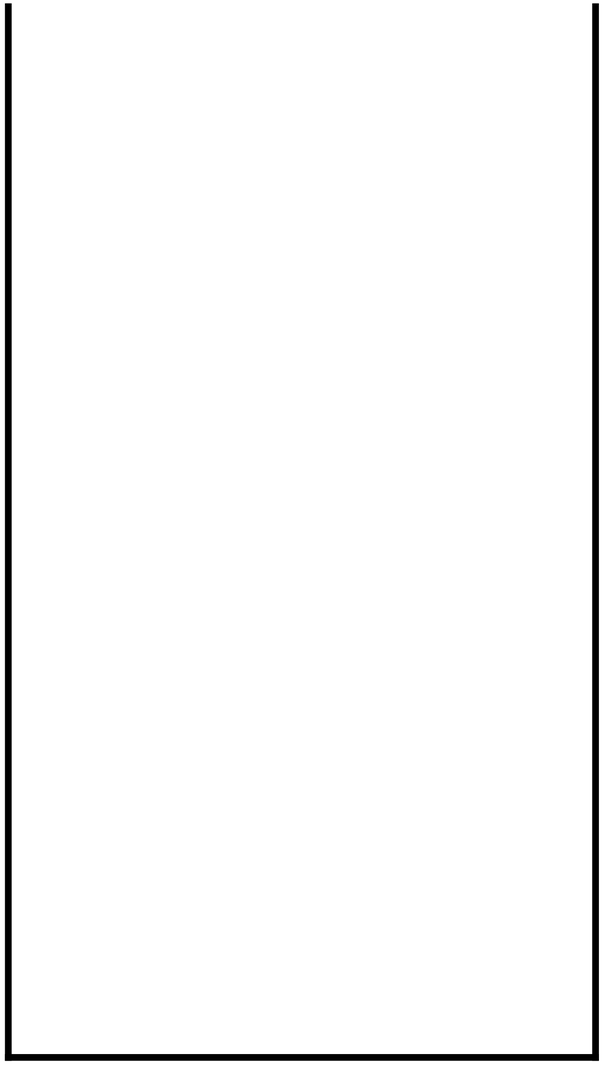
队列的定义与实现

栈与队列的应用

栈

栈（stack）是限定仅在表尾进行插入或删除操作的线性表。因此对栈来说，表尾端有其特殊含义，称为**栈顶**（top），相应地，表头端称为**栈底**（bottom）。不含元素的空表称为**空栈**。

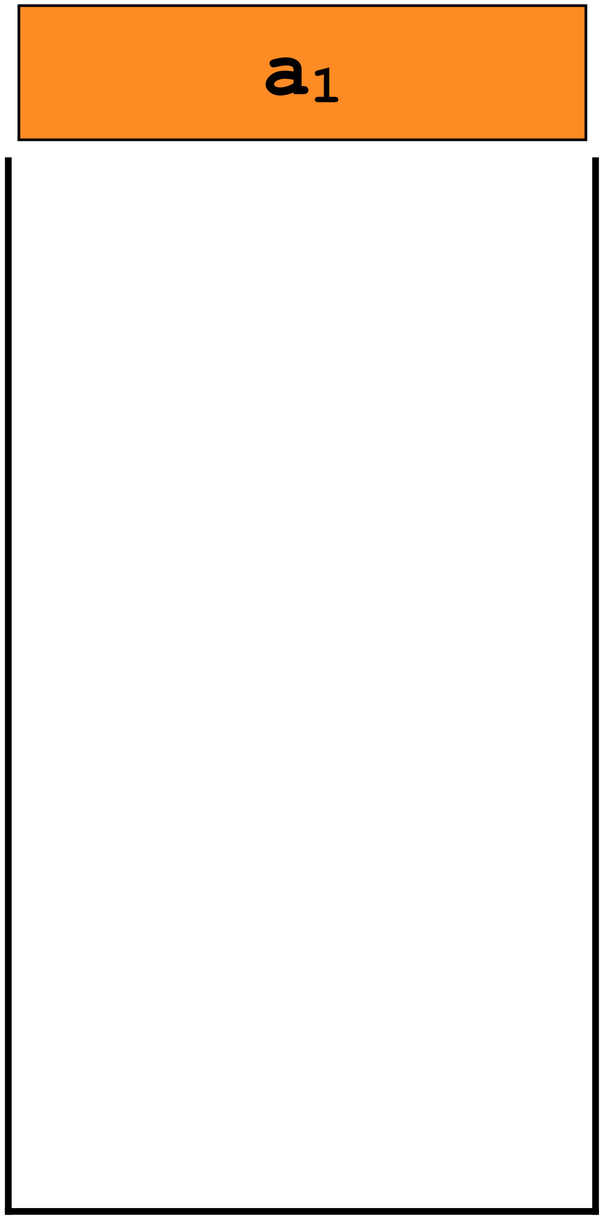
假设 $S=(a_1, a_2, \cdots, a_n)$，则称 a_1 为栈底元素，a_n 为栈顶元素。栈中元素按 $a_1, a_2, \cdots, a_n$ 的次序进栈，退栈的第一个元素应为栈顶元素。换句话说，栈的修改是按照后进先出的原则进行的。因此，栈又称为后进先出（Last In First Out, LIFO）的线性表。



栈

栈（stack）是限定仅在表尾进行插入或删除操作的线性表。因此对栈来说，表尾端有其特殊含义，称为**栈顶**（top），相应地，表头端称为**栈底**（bottom）。不含元素的空表称为**空栈**。

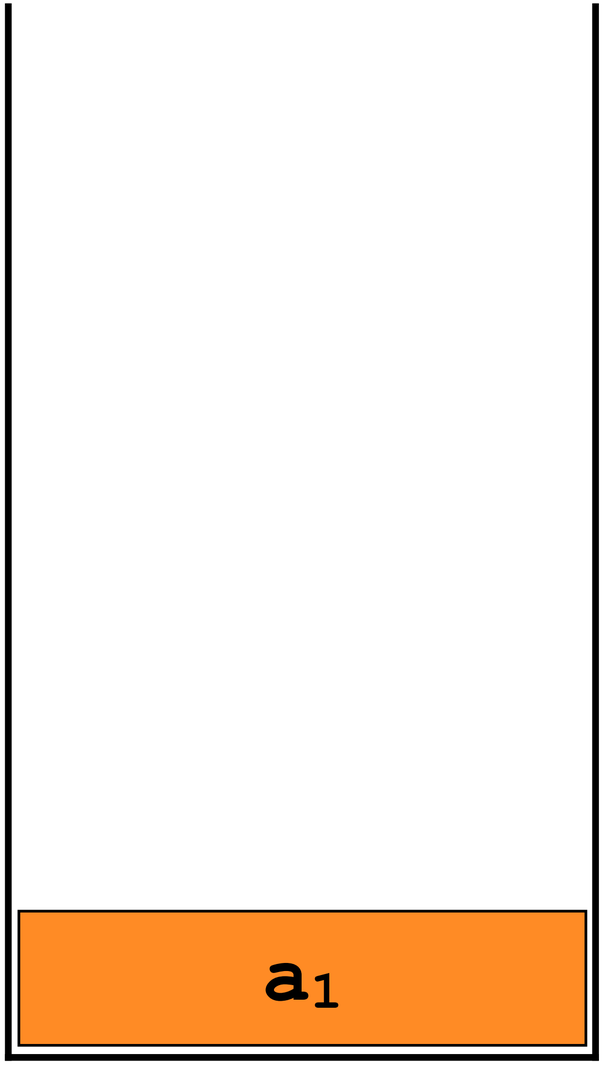
假设 $S=(a_1, a_2, \cdots, a_n)$，则称 a_1 为栈底元素，a_n 为栈顶元素。栈中元素按 $a_1, a_2, \cdots, a_n$ 的次序进栈，退栈的第一个元素应为栈顶元素。换句话说，栈的修改是按照后进先出的原则进行的。因此，栈又称为后进先出（Last In First Out, LIFO）的线性表。



栈

栈（stack）是限定仅在表尾进行插入或删除操作的线性表。因此对栈来说，表尾端有其特殊含义，称为**栈顶**（top），相应地，表头端称为**栈底**（bottom）。不含元素的空表称为**空栈**。

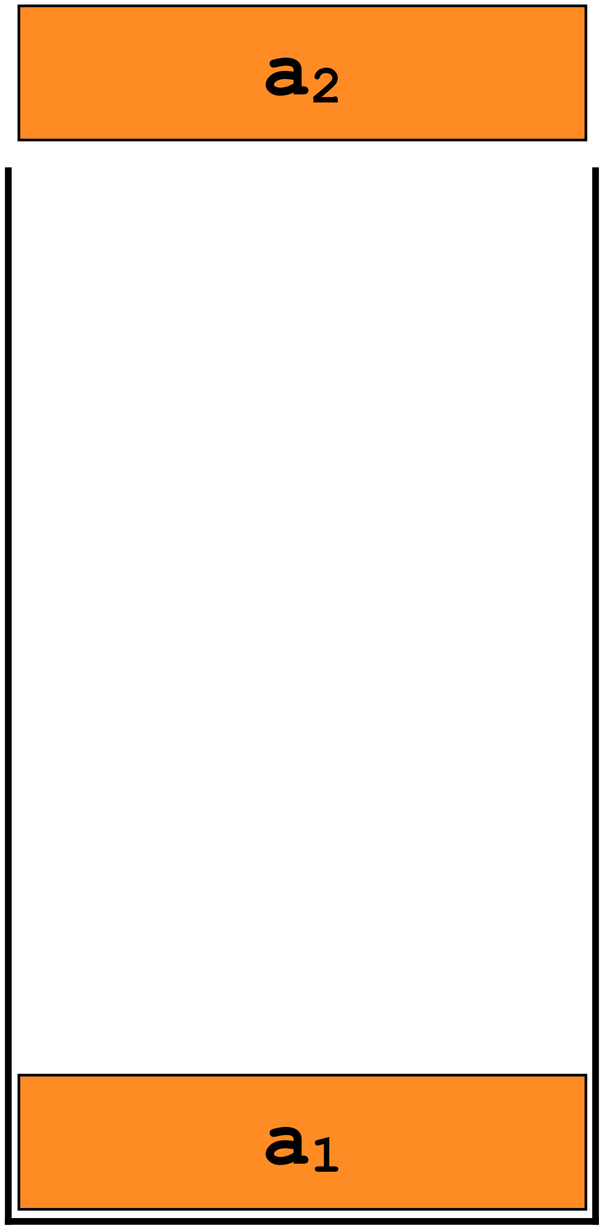
假设 $S=(a_1, a_2, \cdots, a_n)$，则称 a_1 为栈底元素，a_n 为栈顶元素。栈中元素按 $a_1, a_2, \cdots, a_n$ 的次序进栈，退栈的第一个元素应为栈顶元素。换句话说，栈的修改是按照后进先出的原则进行的。因此，栈又称为后进先出（Last In First Out, LIFO）的线性表。



栈

栈（stack）是限定仅在表尾进行插入或删除操作的线性表。因此对栈来说，表尾端有其特殊含义，称为**栈顶**（top），相应地，表头端称为**栈底**（bottom）。不含元素的空表称为**空栈**。

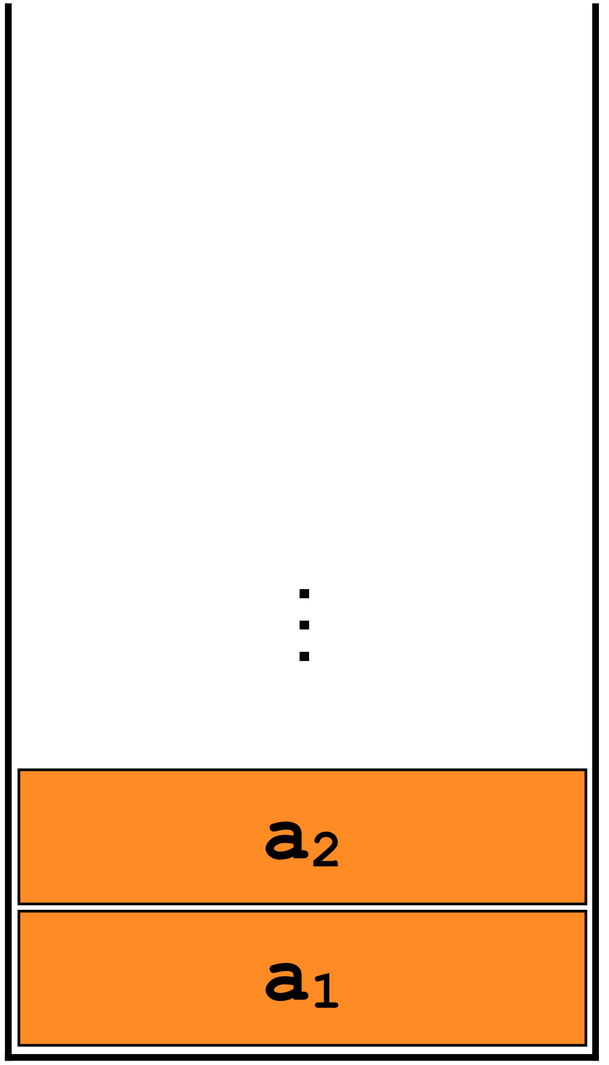
假设 $S=(a_1, a_2, \cdots, a_n)$，则称 a_1 为栈底元素，a_n 为栈顶元素。栈中元素按 $a_1, a_2, \cdots, a_n$ 的次序进栈，退栈的第一个元素应为栈顶元素。换句话说，栈的修改是按照后进先出的原则进行的。因此，栈又称为后进先出（Last In First Out, LIFO）的线性表。



栈

栈（stack）是限定仅在表尾进行插入或删除操作的线性表。因此对栈来说，表尾端有其特殊含义，称为**栈顶**（top），相应地，表头端称为**栈底**（bottom）。不含元素的空表称为**空栈**。

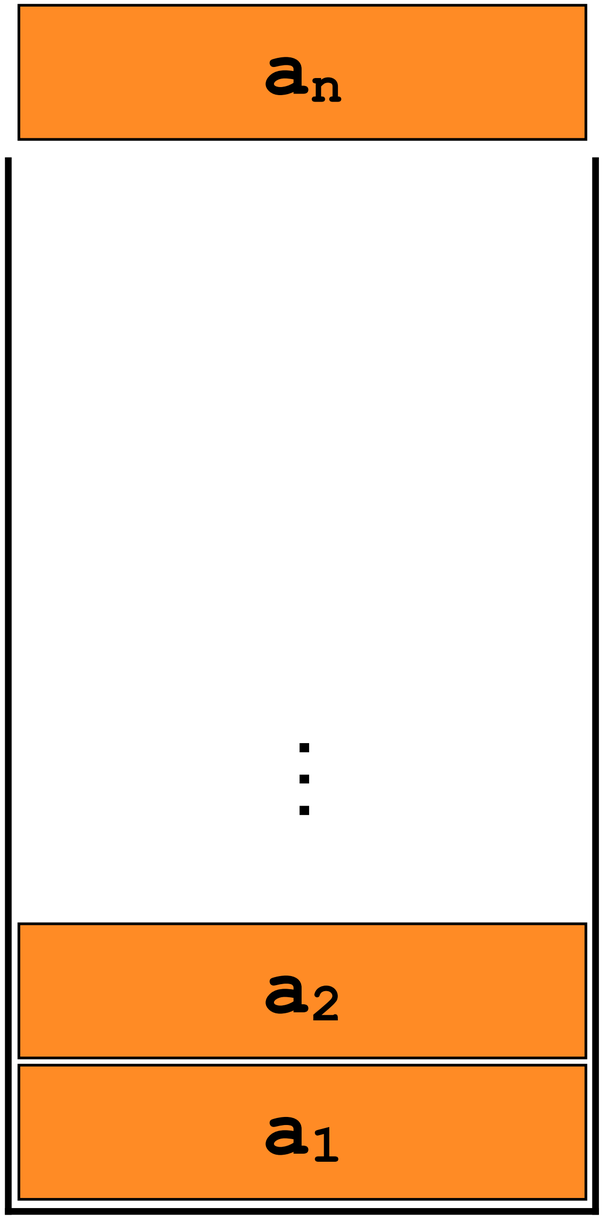
假设 $S=(a_1, a_2, \cdots, a_n)$，则称 a_1 为栈底元素，a_n 为栈顶元素。栈中元素按 $a_1, a_2, \cdots, a_n$ 的次序进栈，退栈的第一个元素应为栈顶元素。换句话说，栈的修改是按照后进先出的原则进行的。因此，栈又称为后进先出（Last In First Out, LIFO）的线性表。



栈

栈（stack）是限定仅在表尾进行插入或删除操作的线性表。因此对栈来说，表尾端有其特殊含义，称为**栈顶**（top），相应地，表头端称为**栈底**（bottom）。不含元素的空表称为**空栈**。

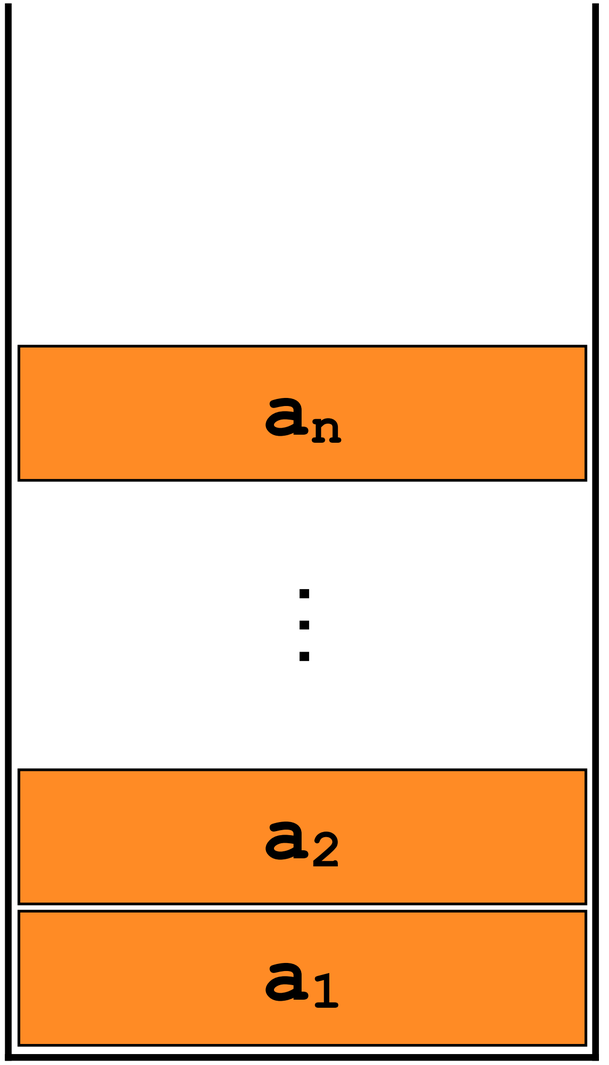
假设 $S=(a_1, a_2, \cdots, a_n)$，则称 a_1 为栈底元素，a_n 为栈顶元素。栈中元素按 $a_1, a_2, \cdots, a_n$ 的次序进栈，退栈的第一个元素应为栈顶元素。换句话说，栈的修改是按照后进先出的原则进行的。因此，栈又称为后进先出（Last In First Out, LIFO）的线性表。



栈

栈（stack）是限定仅在表尾进行插入或删除操作的线性表。因此对栈来说，表尾端有其特殊含义，称为**栈顶**（top），相应地，表头端称为**栈底**（bottom）。不含元素的空表称为**空栈**。

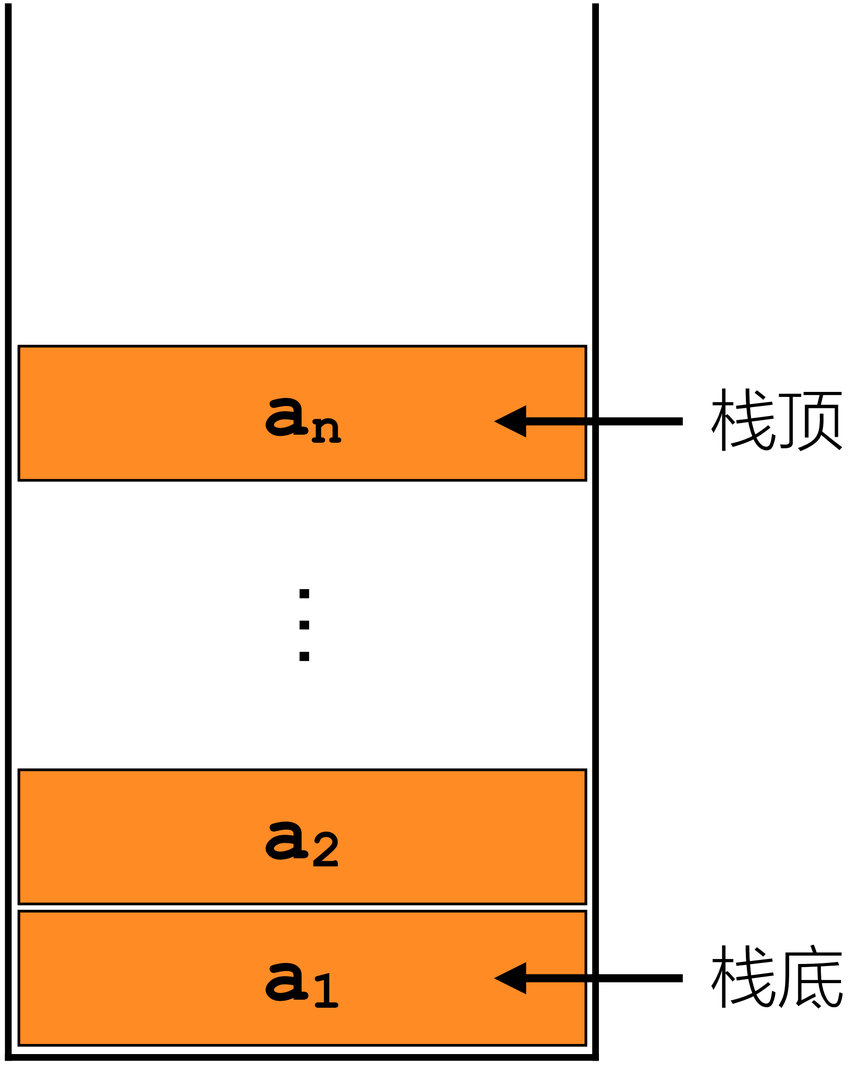
假设 $S=(a_1, a_2, \cdots, a_n)$，则称 a_1 为栈底元素，a_n 为栈顶元素。栈中元素按 $a_1, a_2, \cdots, a_n$ 的次序进栈，退栈的第一个元素应为栈顶元素。换句话说，栈的修改是按照后进先出的原则进行的。因此，栈又称为后进先出（Last In First Out, LIFO）的线性表。

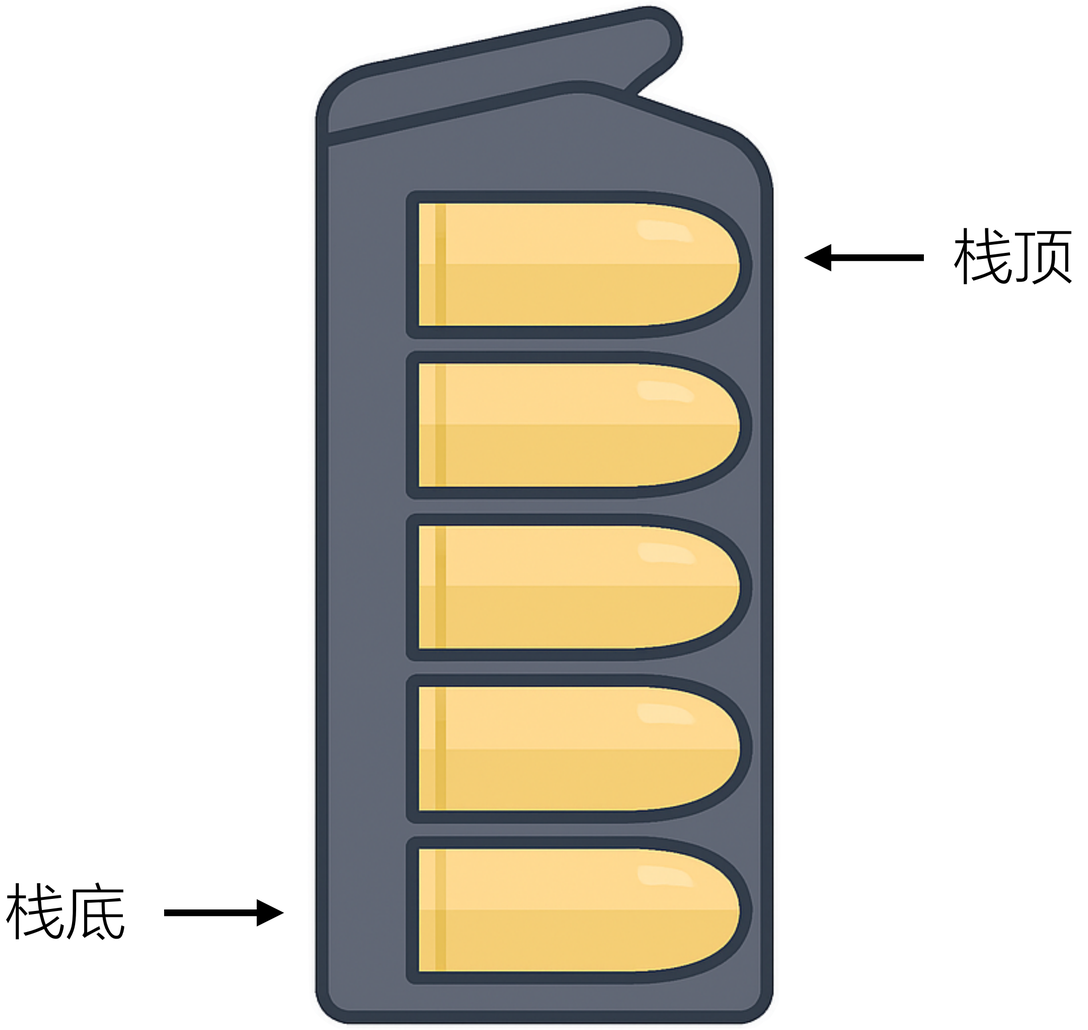


栈

栈（stack）是限定仅在表尾进行插入或删除操作的线性表。因此对栈来说，表尾端有其特殊含义，称为**栈顶**（top），相应地，表头端称为**栈底**（bottom）。不含元素的空表称为**空栈**。

假设 $S=(a_1, a_2, \cdots, a_n)$，则称 a_1 为栈底元素，a_n 为栈顶元素。栈中元素按 $a_1, a_2, \cdots, a_n$ 的次序进栈，退栈的第一个元素应为栈顶元素。换句话说，栈的修改是按照后进先出的原则进行的。因此，栈又称为后进先出（Last In First Out, LIFO）的线性表。




栈顶
栈底

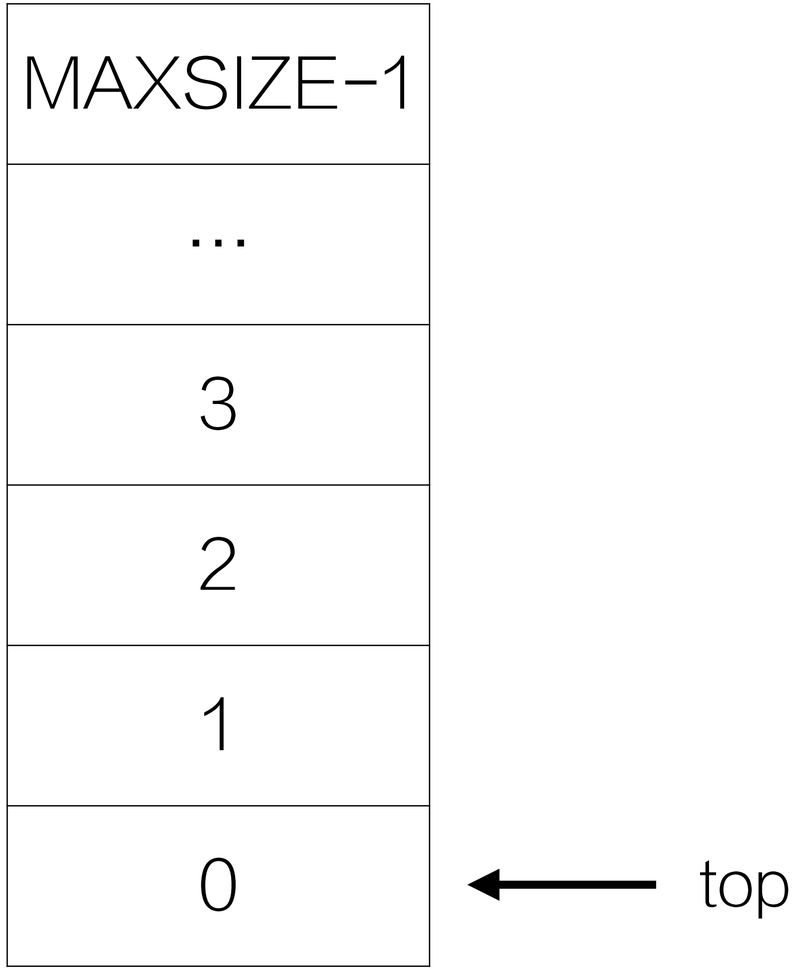
栈是限制插入和删除操作只能在一个位置进行的表，该位置是表的末端，叫作栈顶（top）。对栈的基本操作有进栈（push）和出栈（Pop），前者相当于插入，后者则是删除最后插入的元素。

栈的顺序结构实现

```
#define MAXSIZE = 100
typedef int ElemType;


typedef struct{
    ElemType data[MAXSIZE];
    int top;
}Stack;
```

案例：栈与队列 / 栈 / test1.c



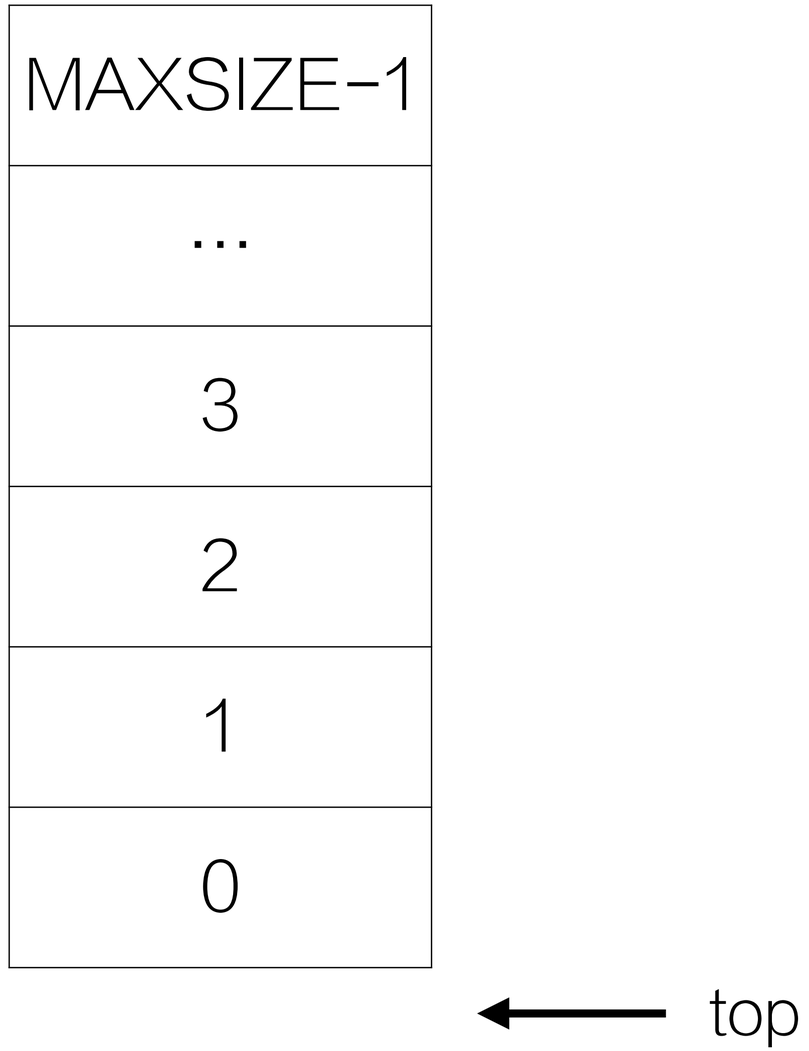
栈的顺序结构-初始化

案例：栈与队列 / 栈 / test1.c

```
#define MAXSIZE = 100
typedef int ElemType;

typedef struct{
    ElemType data[MAXSIZE];
    int top;
}Stack;

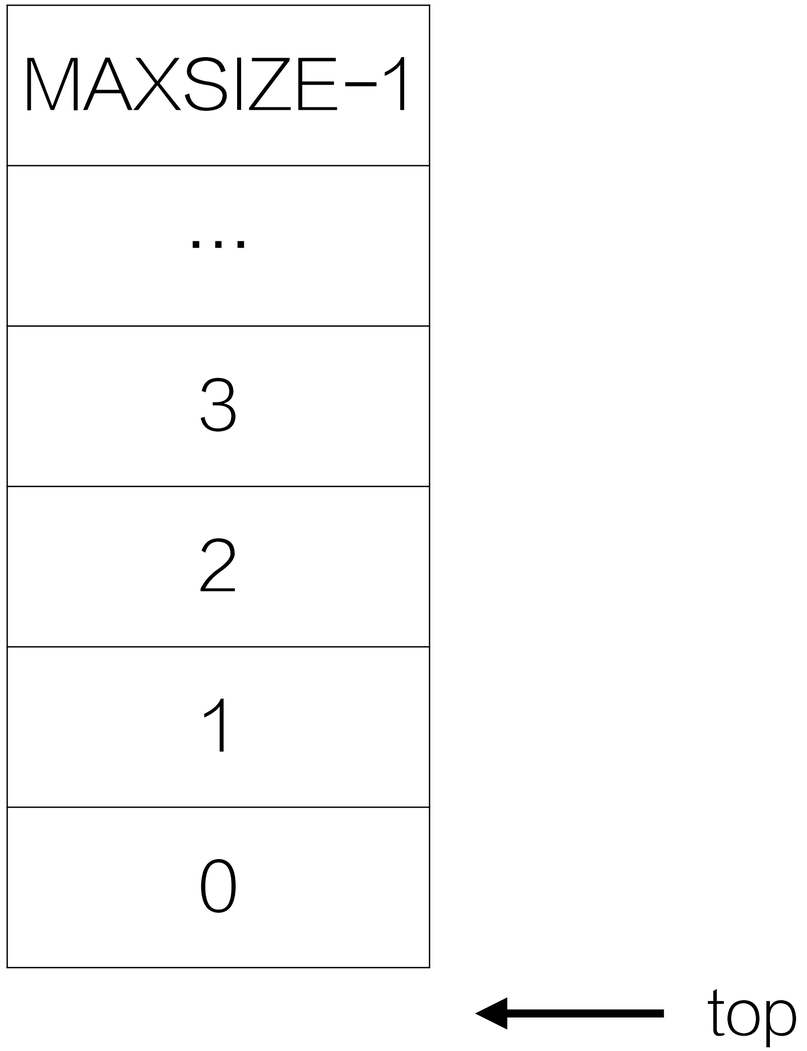
void initStack(Stack *s)
{
    s->top = -1;
}
```



栈的顺序结构-判断栈是否为空

案例：栈与队列 / 栈 / test1.c

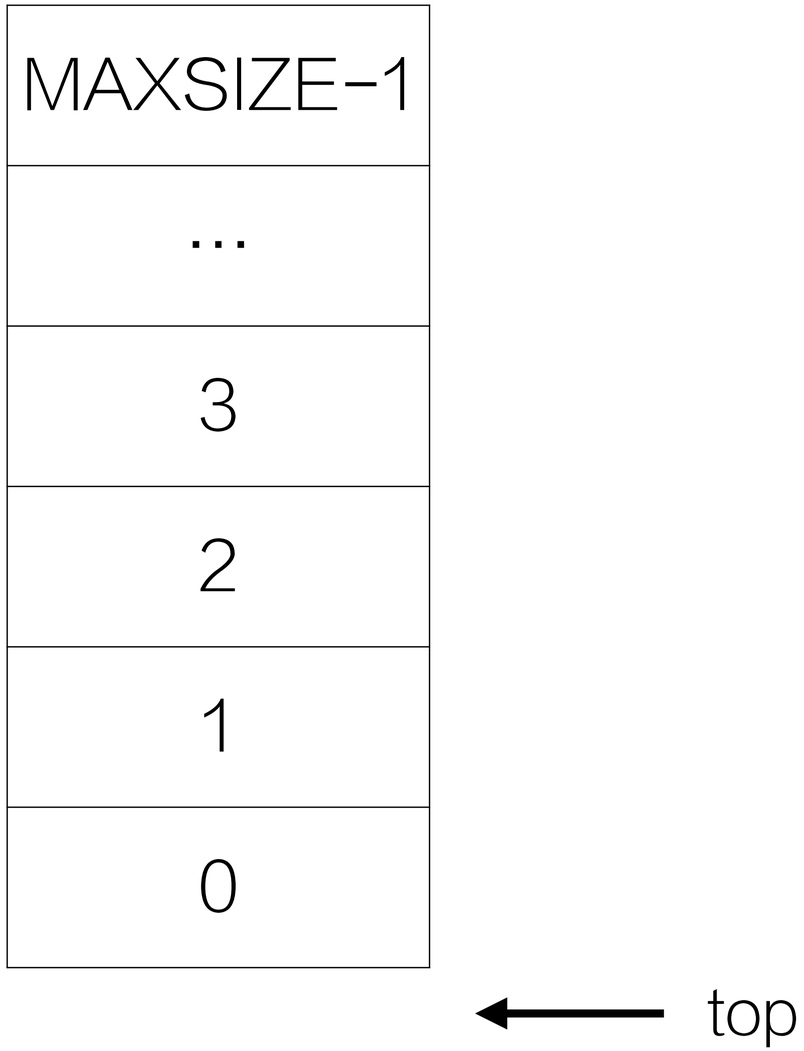
```
int isEmpty(Stack *s)
{
    if (s->top == -1)
    {
        printf("空的\n");
        return 1;
    }
    else
    {
        return 0;
    }
}
```



栈的顺序结构-进栈 / 压栈

案例：栈与队列 / 栈 / test1.c

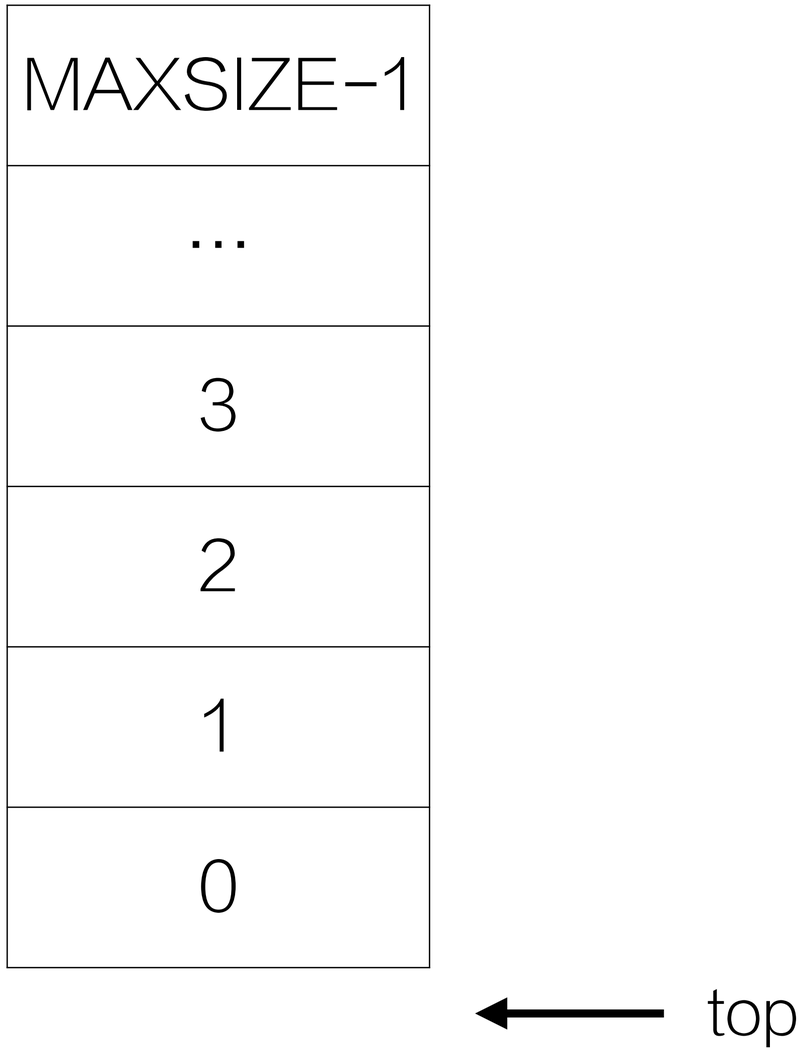
```
int push(Stack *s, ElemType e)
{
    if (s->top >= MAXSIZE - 1)
    {
        printf("满了\n");
        return 0;
    }
    s->top++;
    s->data[s->top] = e;
    return 1;
}
```



栈的顺序结构-进栈 / 压栈

案例：栈与队列 / 栈 / test1.c

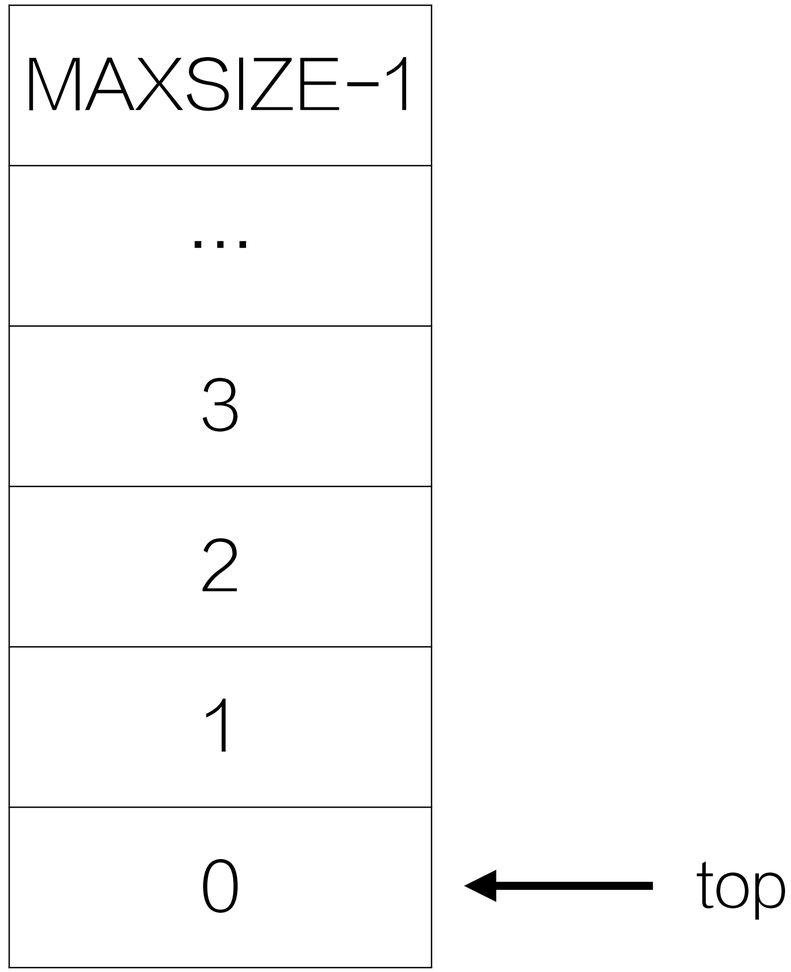
```
int push(Stack *s, ElemType e)
{
    if (s->top >= MAXSIZE - 1)
    {
        printf("满了\n");
        return 0;
    }
    s->top++;
    s->data[s->top] = e;
    return 1;
}
```



栈的顺序结构-进栈 / 压栈

案例：栈与队列 / 栈 / test1.c

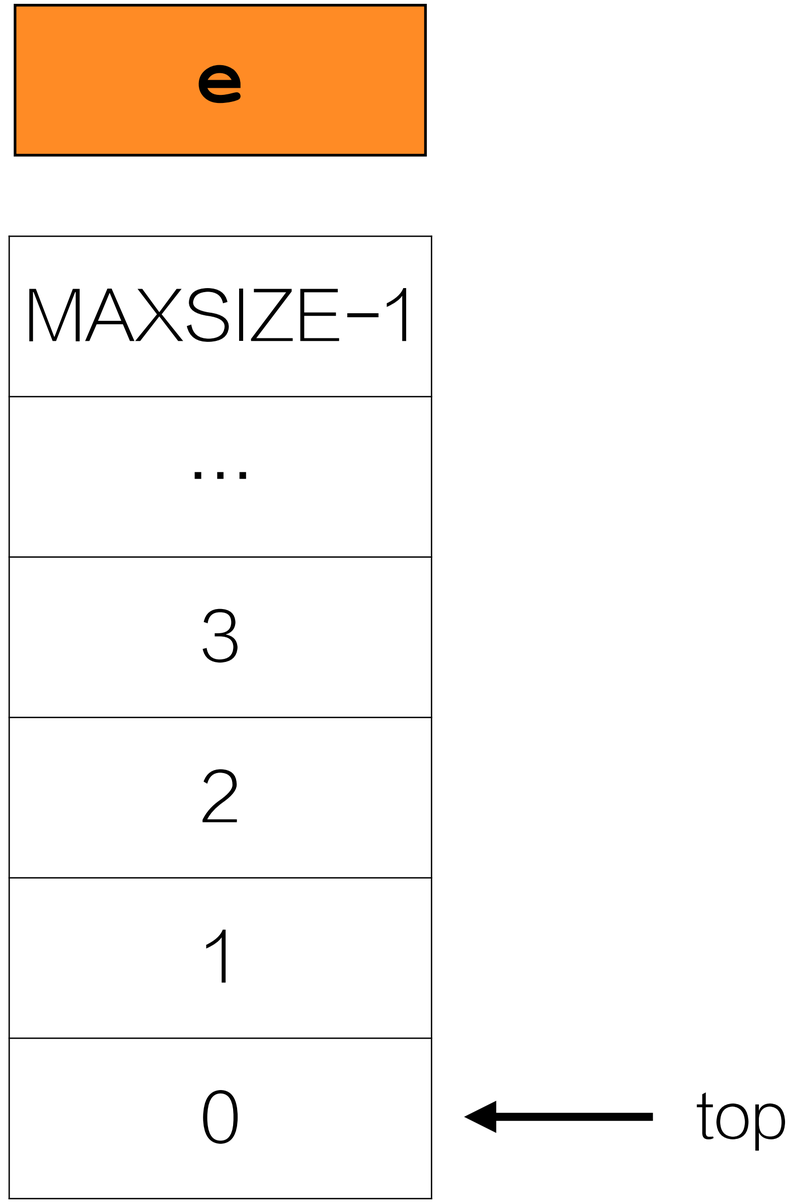
```
int push(Stack *s, ElemType e)
{
    if (s->top >= MAXSIZE - 1)
    {
        printf("满了\n");
        return 0;
    }
    s->top++;
    s->data[s->top] = e;
    return 1;
}
```



栈的顺序结构-进栈 / 压栈

```
int push(Stack *s, ElemType e)
{
    if (s->top >= MAXSIZE - 1)
    {
        printf("满了\n");
        return 0;
    }
    s->top++;
    s->data[s->top] = e;
    return 1;
}
```

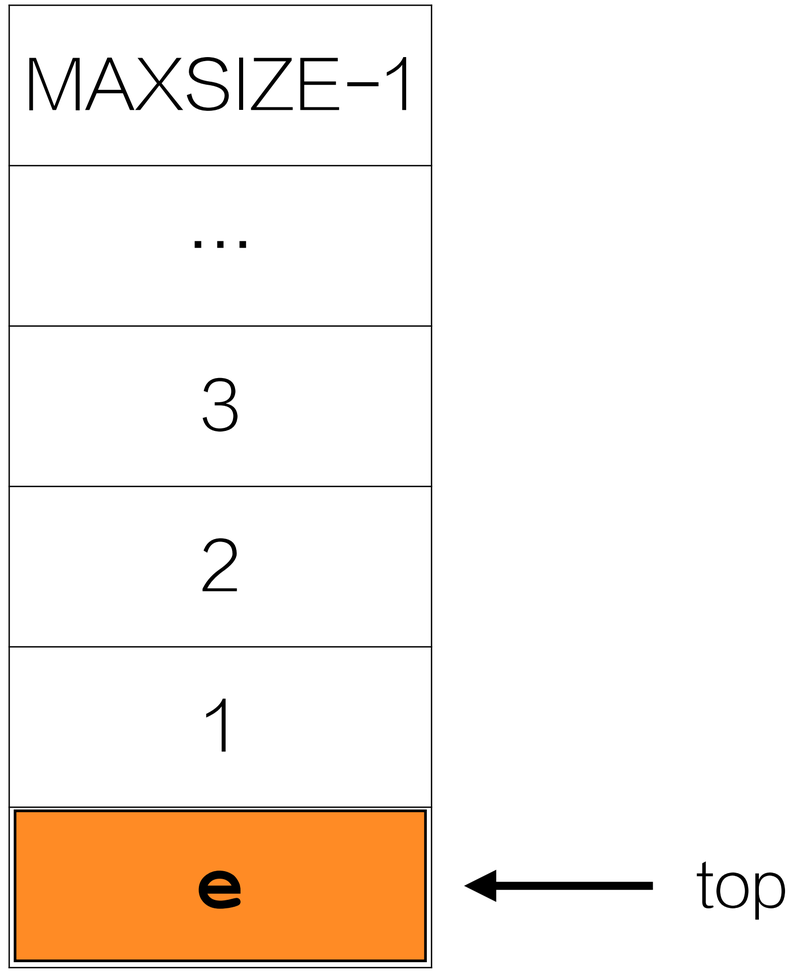
案例：栈与队列 / 栈 / test1.c



栈的顺序结构-进栈 / 压栈

```
int push(Stack *s, ElemType e)
{
    if (s->top >= MAXSIZE - 1)
    {
        printf("满了\n");
        return 0;
    }
    s->top++;
    s->data[s->top] = e;
    return 1;
}
```

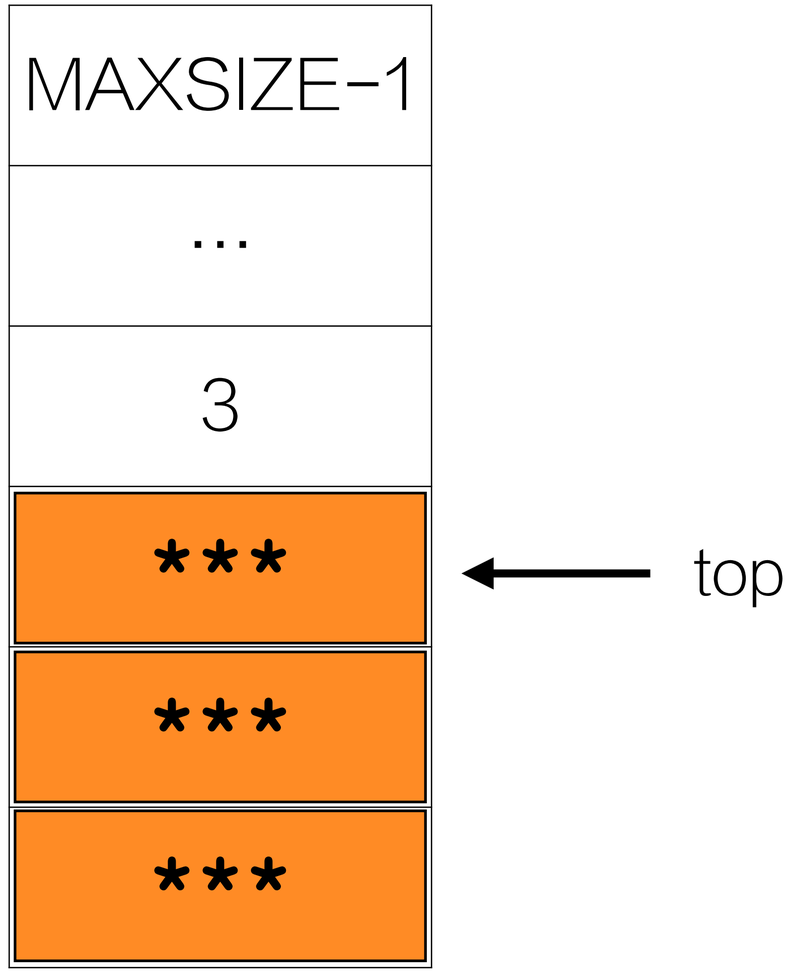
案例：栈与队列 / 栈 / test1.c



栈的顺序结构-出栈

```
ElemType pop(Stack *s, ElemType *e)
{
    if (s->top == -1)
    {
        printf("空的\n");
        return 0;
    }
    *e = s->data[s->top];
    s->top--;
    return 1;
}
```

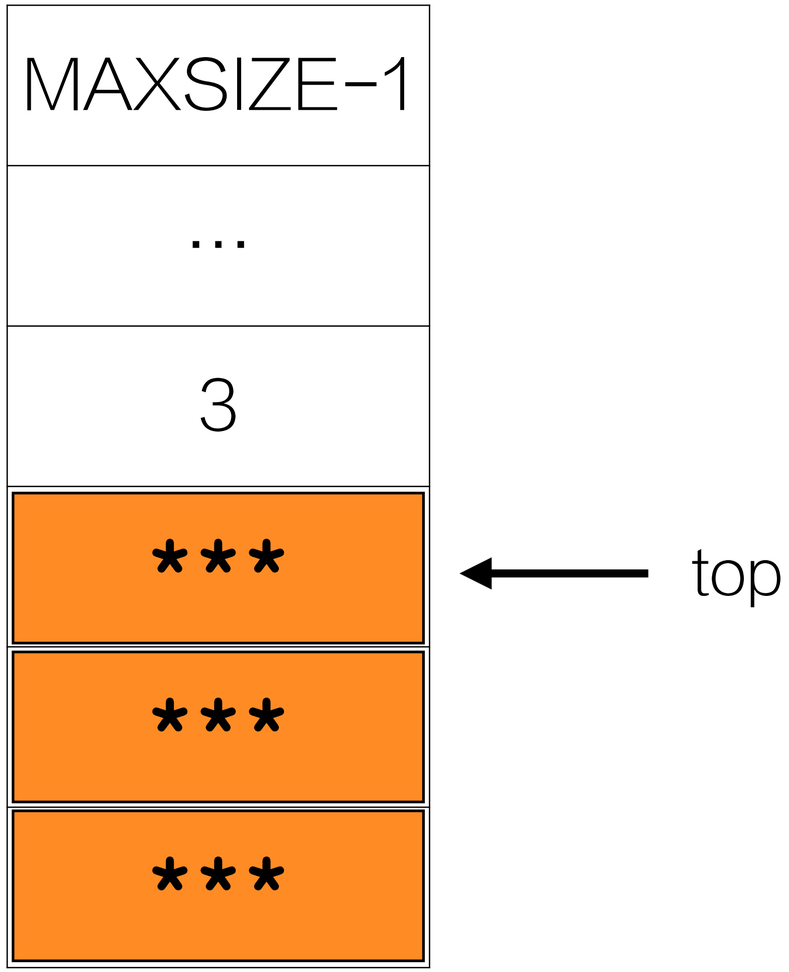
案例：栈与队列 / 栈 / test1.c



栈的顺序结构-出栈

```
ElemType pop(Stack *s, ElemType *e)
{
    if (s->top == -1)
    {
        printf("空的\n");
        return 0;
    }
    *e = s->data[s->top];
    s->top--;
    return 1;
}
```

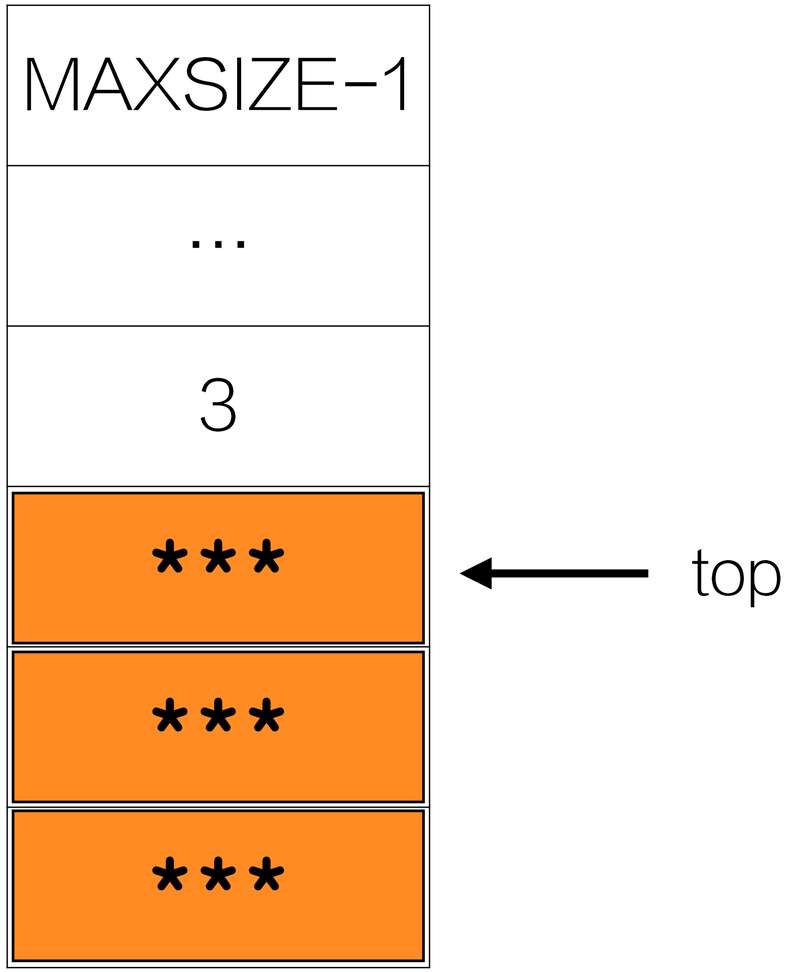
案例：栈与队列 / 栈 / test1.c



栈的顺序结构-出栈

```
ElemType pop(Stack *s, ElemType *e)
{
    if (s->top == -1)
    {
        printf("空的\n");
        return 0;
    }
    *e = s->data[s->top];
    s->top--;
    return 1;
}
```

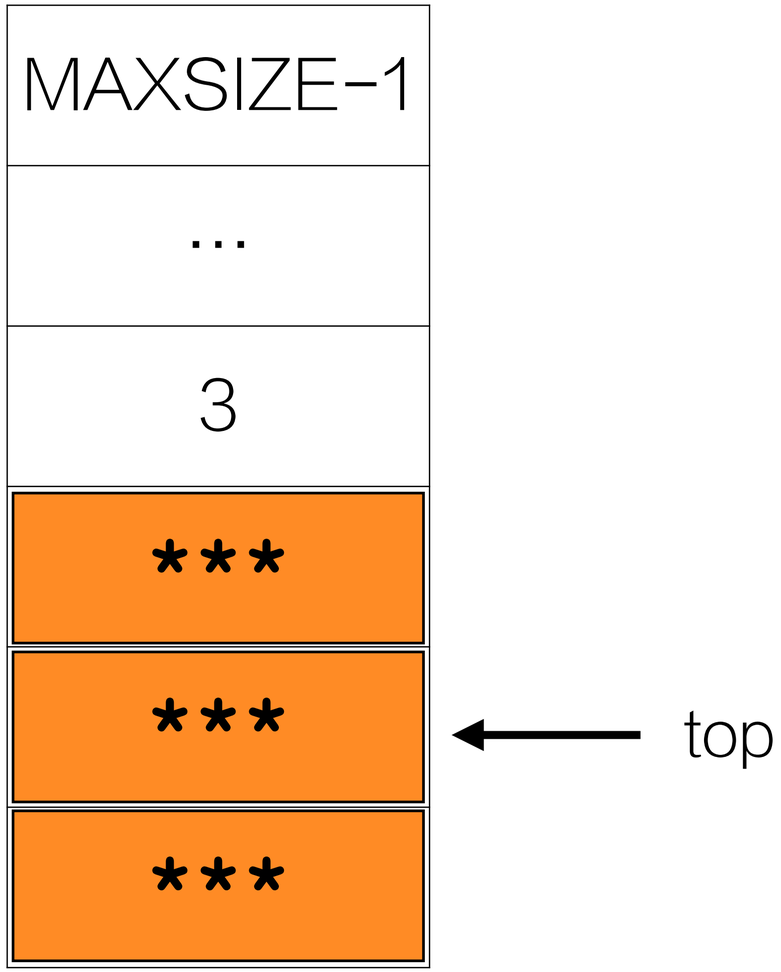
案例：栈与队列 / 栈 / test1.c



栈的顺序结构-出栈

```
ElemType pop(Stack *s, ElemType *e)
{
    if (s->top == -1)
    {
        printf("空的\n");
        return 0;
    }
    *e = s->data[s->top];
    s->top--;
    return 1;
}
```

案例：栈与队列 / 栈 / test1.c



栈的顺序结构-获取栈顶元素

```
int getTop(Stack *s, ElemType *e)
{
    if (s->top == -1)
    {
        printf("空的\n");
        return 0;
    }
    *e = s->data[s->top];
    return 1;
}
```

案例：栈与队列 / 栈 / Test1.c

案例：栈与队列 / 栈 / test1.c

栈的顺序结构初始化 – 动态内存分配

案例：栈与队列 / 栈 / test2.c

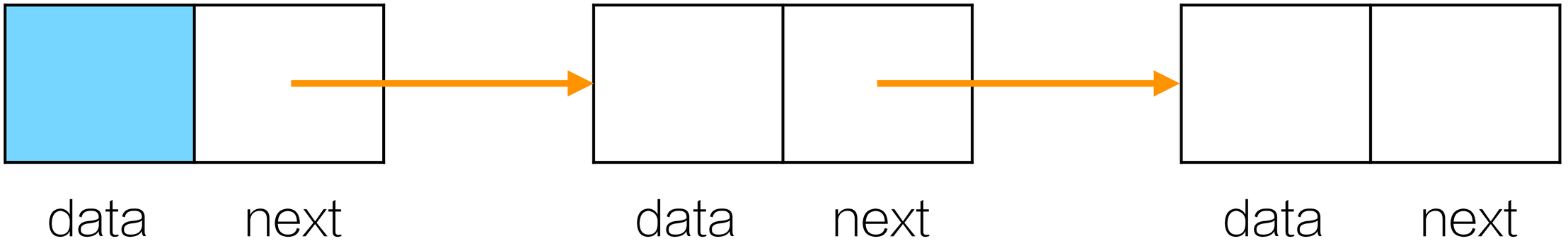
```
#define MAXSIZE 100
typedef int ElemType;
typedef struct
{
    ElemType *data;
    int top;
}Stack;
Stack* initStack()
{
    Stack *s = (Stack*)malloc(sizeof(Stack));
    s->data = (ElemType*)malloc(sizeof(ElemType) * MAXSIZE);
    s->top = -1;
    return s;
}
```

栈的链式结构实现

```
typedef int ElemType;


typedef struct stack
{
    ElemType data;
    struct stack *next;
}Stack;
```

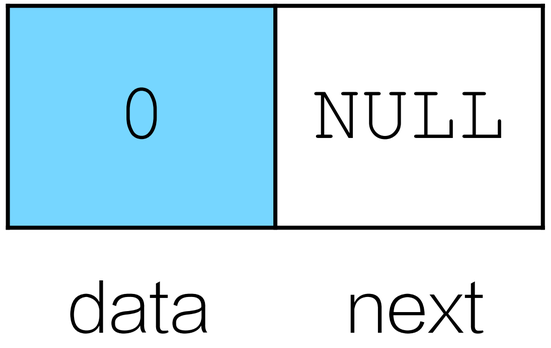
案例：栈与队列 / 栈 / test3.c



栈的链式结构实现 – 初始化

案例：栈与队列 / 栈 / test3.c

```
typedef int ElemType;
typedef struct stack
{
    ElemType data;
    struct stack *next;
}Stack;
Stack* initStack()
{
    Stack *s = (Stack*)malloc(sizeof(Stack));
    s->data = 0;
    s->next = NULL;
    return s;
}
```



栈的链式结构实现 – 判断栈是否为空

案例：栈与队列 / 栈 / test3.c

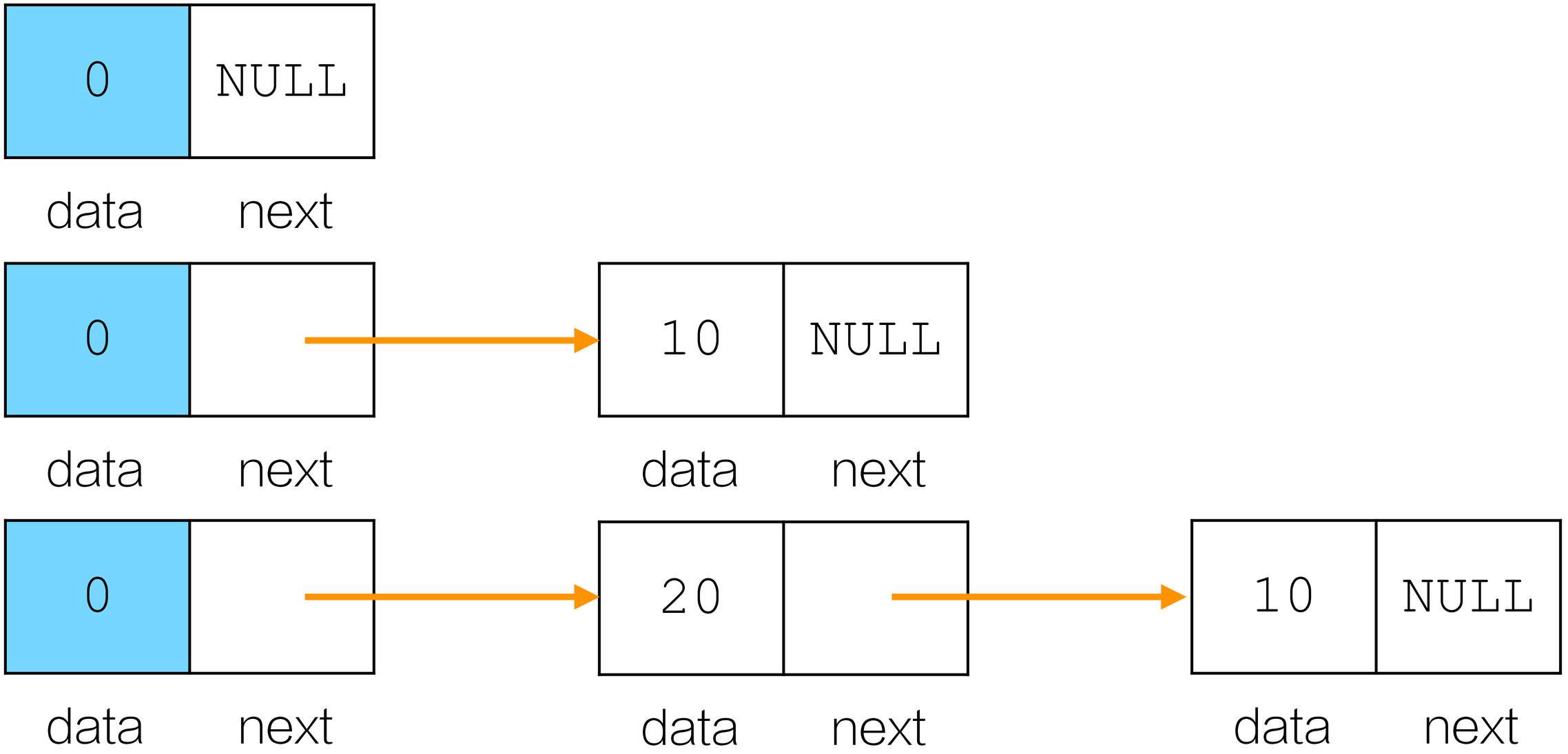
```
int isEmpty(Stack *s)
{
    if (s->next == NULL)
    {
        printf("空的\n");
        return 1;
    }
    else
    {
        return 0;
    }
}
```

栈的链式结构实现 – 进栈 / 压栈

```
int push(Stack *s, ElemType e)
{
    Stack *p = (Stack*)malloc(sizeof(Stack));
    p->data = e;
    p->next = s->next;
    s->next = p;
    return 1;
}
```

案例：栈与队列 / 栈 / test3.c

栈的链式结构实现 – 进栈 / 压栈

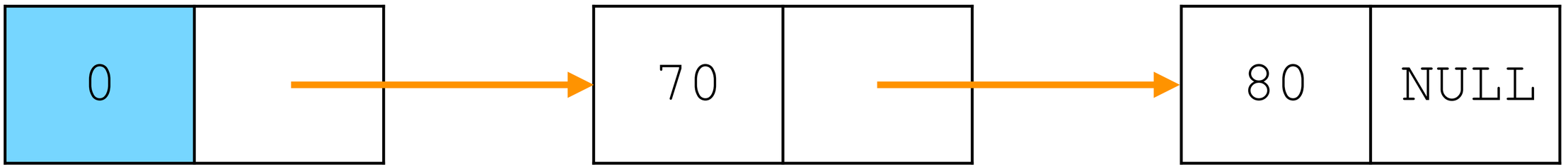


栈的链式结构实现 – 出栈

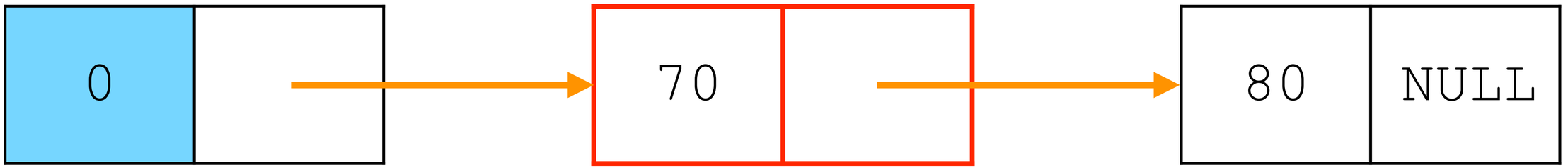
```
int pop(Stack *s, ElemType *e)
{
    if(s->next == NULL)
    {
        printf("空的\n");
        return 0;
    }
    *e = s->next->data;
    Stack *q = s->next;
    s->next = q->next;
    free(q);
    return 1;
}
```

案例：栈与队列 / 栈 / test3.c

栈的链式结构实现 – 出栈

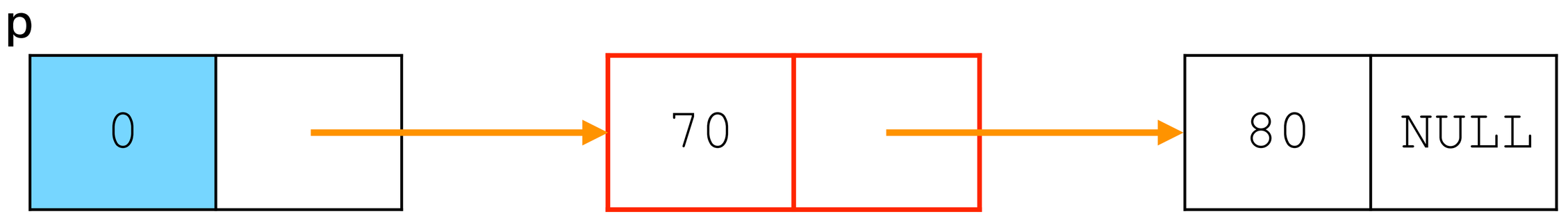


栈的链式结构实现 – 出栈



栈的链式结构实现 – 出栈

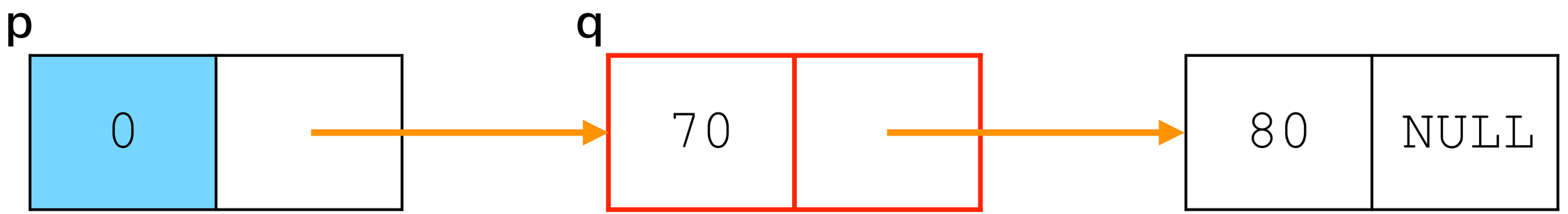
找到头节点 p



栈的链式结构实现 – 出栈

找到头节点 p

用指针 q 记录要出栈的节点

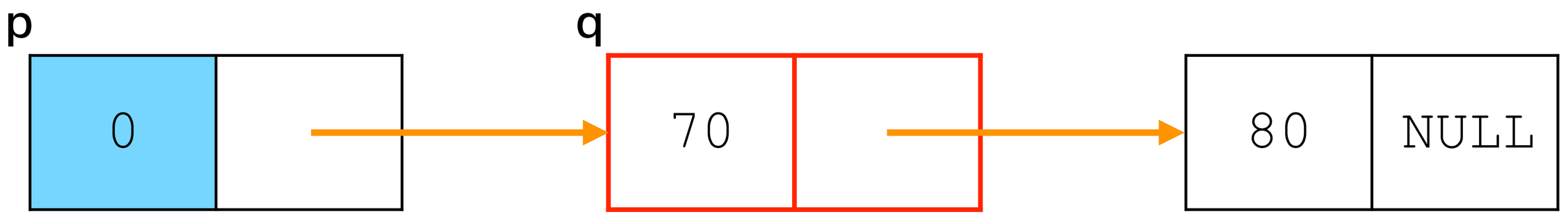


栈的链式结构实现 – 出栈

找到头节点 p

用指针 q 记录要出栈的节点

通过改变 p 的后继节点实现出栈

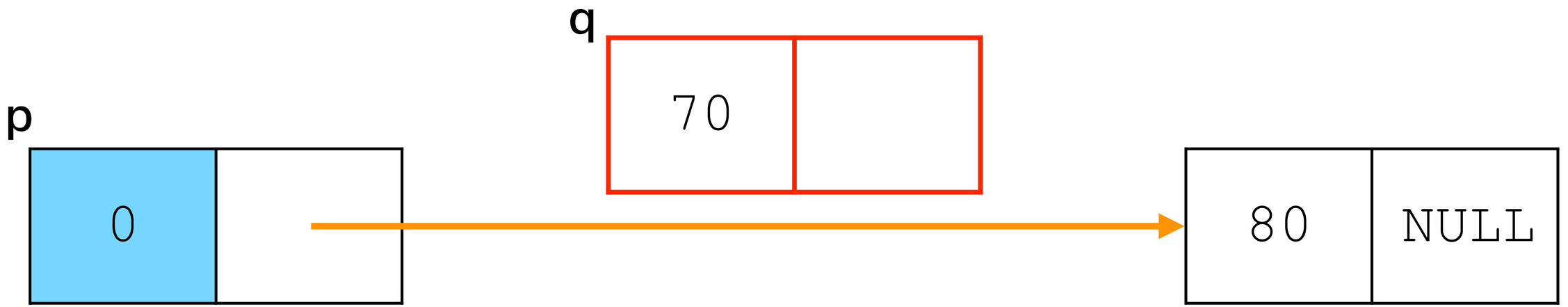


栈的链式结构实现 – 出栈

找到头节点 p

用指针 q 记录要出栈的节点

通过改变 p 的后继节点实现出栈



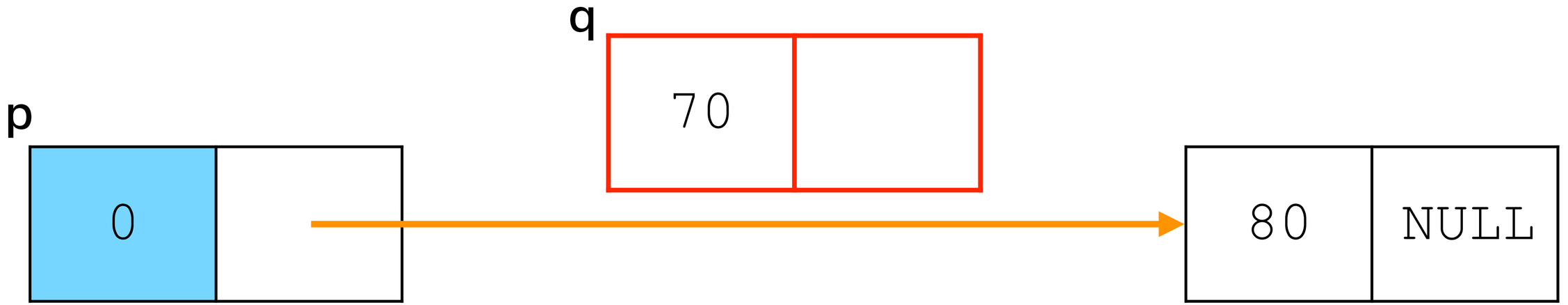
栈的链式结构实现 – 出栈

找到头节点 p

用指针 q 记录要出栈的节点

通过改变 p 的后继节点实现出栈

释放出栈节点的空间



栈的链式结构实现 – 获取栈顶元素

```
int getTop(Stack *s, ElemType *e)
{
    if (s->next == NULL)
    {
        printf("空的\n");
        return 0;
    }
    *e = s->next->data;
    return 1;
}
```

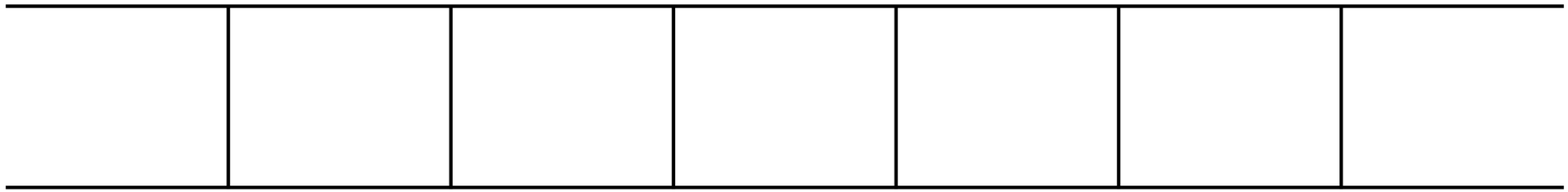
案例：栈与队列 / 栈 / test3.c

队列

队列（queue）是一种先进先出（First In First Out, FIFO）的线性表。它只允许在表的一端进行插入，而在另一端删除元素。在队列中，允许插入的一端称为队尾（rear）,允许删除的一端则称为队头（front）。假设队列为 $q = (a_1, a_2, \cdots, a_n)$，那么，a1 就是队头元素，$a_n$ 就是队尾元素。队列中的元素是按照 $a_1, a_2, \cdots, a_n$ 的顺序进入的，退出队列也只能按照这个次序依次退出，也就是说，只有在 $a_1, a_2, \cdots, a_{n-1}$ 都离开队列之后，a_n 才能退出队列。

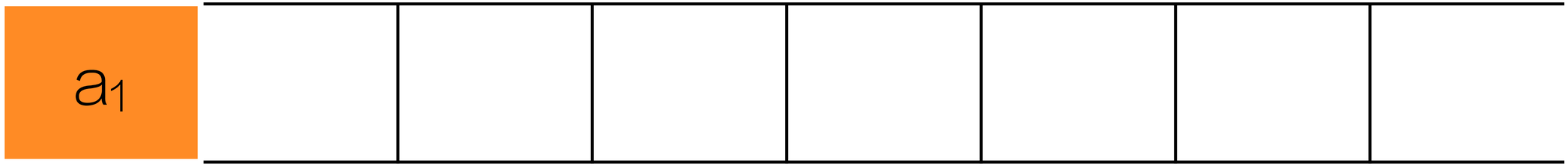
队列

队列（queue）是一种先进先出（First In First Out, FIFO）的线性表。它只允许在表的一端进行插入，而在另一端删除元素。在队列中，允许插入的一端称为队尾（rear）,允许删除的一端则称为队头（front）。假设队列为 $q = (a_1, a_2, \cdots, a_n)$，那么，a1 就是队头元素，$a_n$ 就是队尾元素。队列中的元素是按照 $a_1, a_2, \cdots, a_n$ 的顺序进入的，退出队列也只能按照这个次序依次退出，也就是说，只有在 $a_1, a_2, \cdots, a_{n-1}$ 都离开队列之后，a_n 才能退出队列。



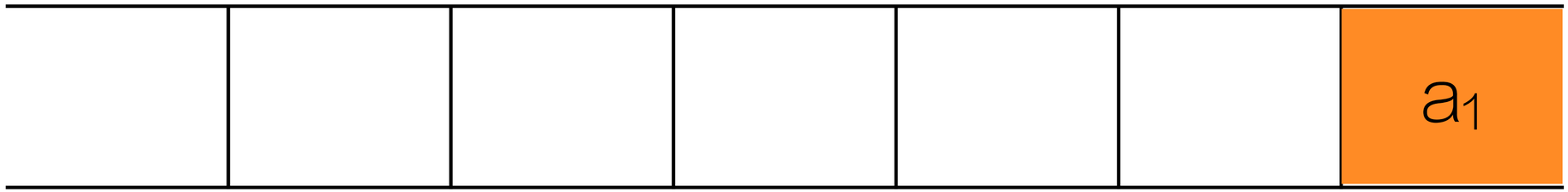
队列

队列（queue）是一种先进先出（First In First Out, FIFO）的线性表。它只允许在表的一端进行插入，而在另一端删除元素。在队列中，允许插入的一端称为队尾（rear）,允许删除的一端则称为队头（front）。假设队列为 $q = (a_1, a_2, \cdots, a_n)$，那么，a1 就是队头元素，$a_n$ 就是队尾元素。队列中的元素是按照 $a_1, a_2, \cdots, a_n$ 的顺序进入的，退出队列也只能按照这个次序依次退出，也就是说，只有在 $a_1, a_2, \cdots, a_{n-1}$ 都离开队列之后，a_n 才能退出队列。



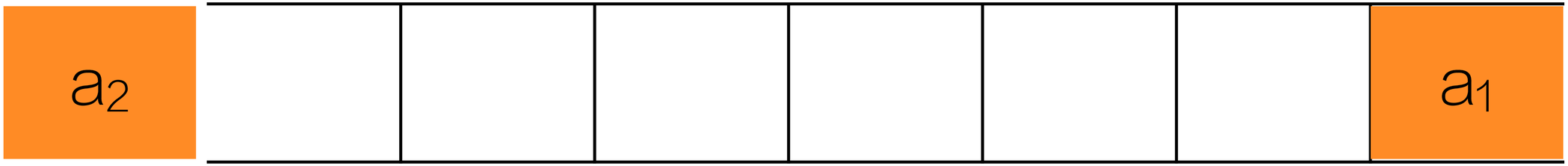
队列

队列（queue）是一种先进先出（First In First Out, FIFO）的线性表。它只允许在表的一端进行插入，而在另一端删除元素。在队列中，允许插入的一端称为队尾（rear）,允许删除的一端则称为队头（front）。假设队列为 q = (a_1, a_2, …, a_n)，那么，a1 就是队头元素，a_n 就是队尾元素。队列中的元素是按照 a_1, a_2, …, a_n 的顺序进入的，退出队列也只能按照这个次序依次退出，也就是说，只有在 a_1, a_2, …, a_{n-1} 都离开队列之后，a_n 才能退出队列。



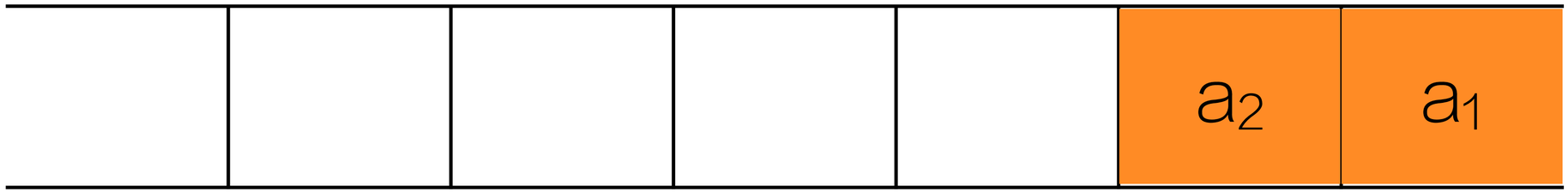
队列

队列（queue）是一种先进先出（First In First Out, FIFO）的线性表。它只允许在表的一端进行插入，而在另一端删除元素。在队列中，允许插入的一端称为队尾（rear）,允许删除的一端则称为队头（front）。假设队列为 $q = (a_1, a_2, \cdots, a_n)$，那么，a1 就是队头元素，$a_n$ 就是队尾元素。队列中的元素是按照 $a_1, a_2, \cdots, a_n$ 的顺序进入的，退出队列也只能按照这个次序依次退出，也就是说，只有在 $a_1, a_2, \cdots, a_{n-1}$ 都离开队列之后，a_n 才能退出队列。



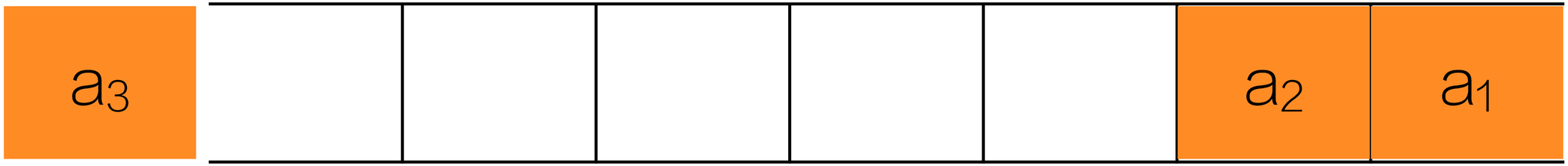
队列

队列（queue）是一种先进先出（First In First Out, FIFO）的线性表。它只允许在表的一端进行插入，而在另一端删除元素。在队列中，允许插入的一端称为队尾（rear）,允许删除的一端则称为队头（front）。假设队列为 $q = (a_1, a_2, \cdots, a_n)$，那么，a1 就是队头元素，$a_n$ 就是队尾元素。队列中的元素是按照 $a_1, a_2, \cdots, a_n$ 的顺序进入的，退出队列也只能按照这个次序依次退出，也就是说，只有在 $a_1, a_2, \cdots, a_{n-1}$ 都离开队列之后，a_n 才能退出队列。



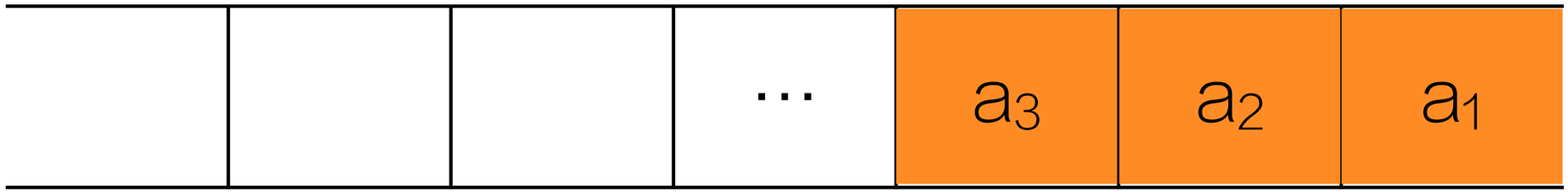
队列

队列（queue）是一种先进先出（First In First Out, FIFO）的线性表。它只允许在表的一端进行插入，而在另一端删除元素。在队列中，允许插入的一端称为队尾（rear）,允许删除的一端则称为队头（front）。假设队列为 q = (a_1, a_2, ⋯, a_n)，那么，a1 就是队头元素，a_n 就是队尾元素。队列中的元素是按照 a_1, a_2, ⋯, a_n 的顺序进入的，退出队列也只能按照这个次序依次退出，也就是说，只有在 a_1, a_2, ⋯, a_{n-1} 都离开队列之后，a_n 才能退出队列。



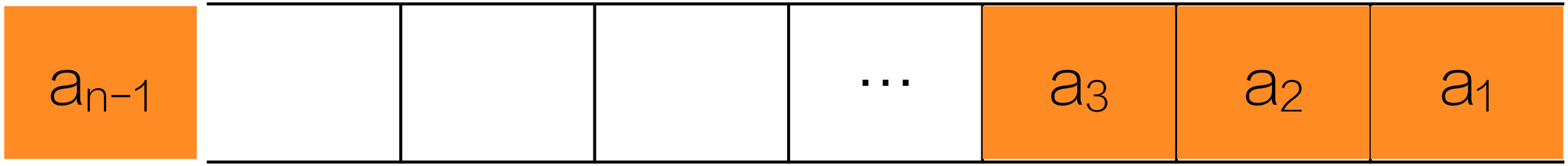
队列

队列（queue）是一种先进先出（First In First Out, FIFO）的线性表。它只允许在表的一端进行插入，而在另一端删除元素。在队列中，允许插入的一端称为队尾（rear）,允许删除的一端则称为队头（front）。假设队列为 $q = (a_1, a_2, \cdots, a_n)$，那么，a1 就是队头元素，$a_n$ 就是队尾元素。队列中的元素是按照 $a_1, a_2, \cdots, a_n$ 的顺序进入的，退出队列也只能按照这个次序依次退出，也就是说，只有在 $a_1, a_2, \cdots, a_{n-1}$ 都离开队列之后，a_n 才能退出队列。



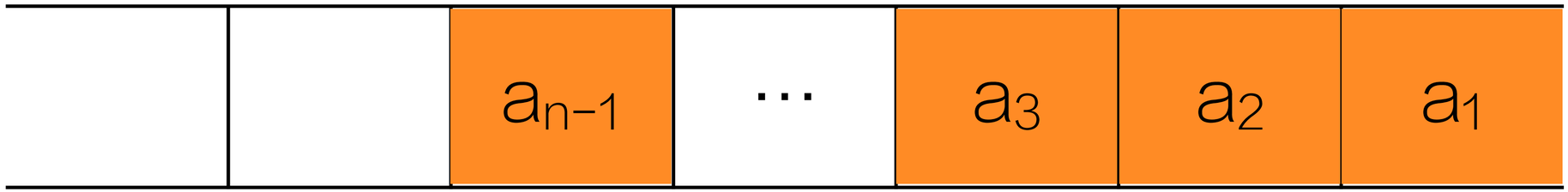
队列

队列（queue）是一种先进先出（First In First Out, FIFO）的线性表。它只允许在表的一端进行插入，而在另一端删除元素。在队列中，允许插入的一端称为队尾（rear）,允许删除的一端则称为队头（front）。假设队列为 $q = (a_1, a_2, \cdots, a_n)$，那么，a1 就是队头元素，$a_n$ 就是队尾元素。队列中的元素是按照 $a_1, a_2, \cdots, a_n$ 的顺序进入的，退出队列也只能按照这个次序依次退出，也就是说，只有在 $a_1, a_2, \cdots, a_{n-1}$ 都离开队列之后，a_n 才能退出队列。



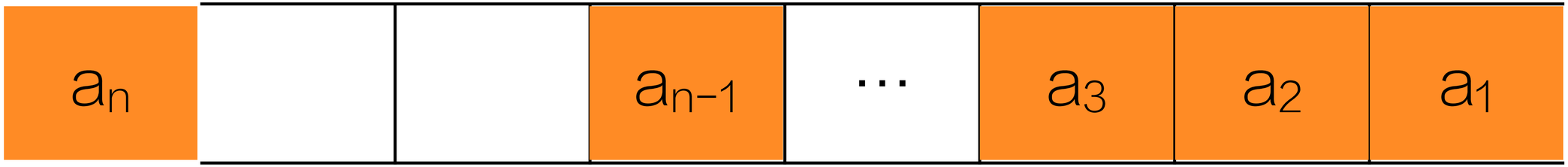
队列

队列（queue）是一种先进先出（First In First Out, FIFO）的线性表。它只允许在表的一端进行插入，而在另一端删除元素。在队列中，允许插入的一端称为队尾（rear）,允许删除的一端则称为队头（front）。假设队列为 $q = (a_1, a_2, \cdots, a_n)$，那么，a1 就是队头元素，$a_n$ 就是队尾元素。队列中的元素是按照 $a_1, a_2, \cdots, a_n$ 的顺序进入的，退出队列也只能按照这个次序依次退出，也就是说，只有在 $a_1, a_2, \cdots, a_{n-1}$ 都离开队列之后，a_n 才能退出队列。



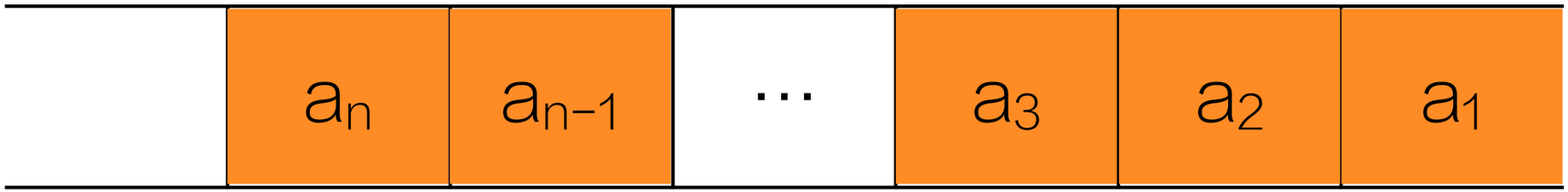
队列

队列（queue）是一种先进先出（First In First Out, FIFO）的线性表。它只允许在表的一端进行插入，而在另一端删除元素。在队列中，允许插入的一端称为队尾（rear）,允许删除的一端则称为队头（front）。假设队列为 $q = (a_1, a_2, \cdots, a_n)$，那么，a1 就是队头元素，$a_n$ 就是队尾元素。队列中的元素是按照 $a_1, a_2, \cdots, a_n$ 的顺序进入的，退出队列也只能按照这个次序依次退出，也就是说，只有在 $a_1, a_2, \cdots, a_{n-1}$ 都离开队列之后，a_n 才能退出队列。



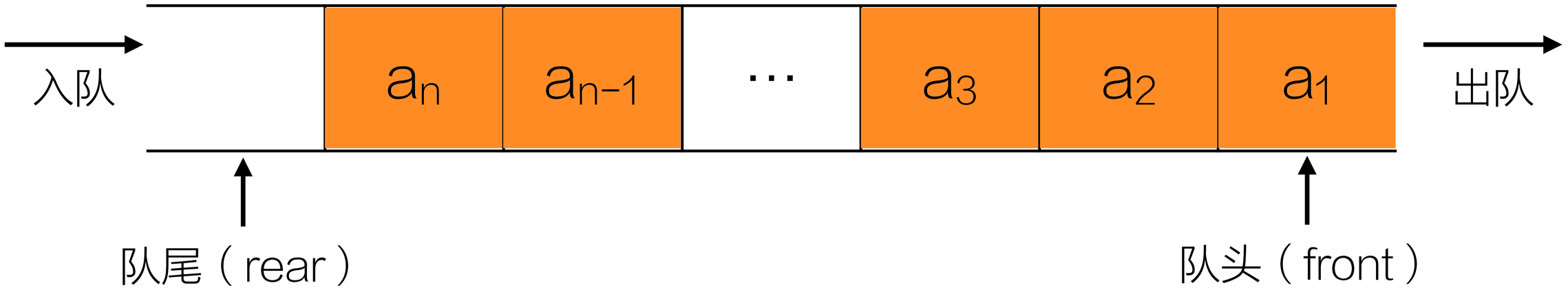
队列

队列（queue）是一种先进先出（First In First Out, FIFO）的线性表。它只允许在表的一端进行插入，而在另一端删除元素。在队列中，允许插入的一端称为队尾（rear）,允许删除的一端则称为队头（front）。假设队列为 $q = (a_1, a_2, \cdots, a_n)$，那么，a1 就是队头元素，$a_n$ 就是队尾元素。队列中的元素是按照 $a_1, a_2, \cdots, a_n$ 的顺序进入的，退出队列也只能按照这个次序依次退出，也就是说，只有在 $a_1, a_2, \cdots, a_{n-1}$ 都离开队列之后，a_n 才能退出队列。



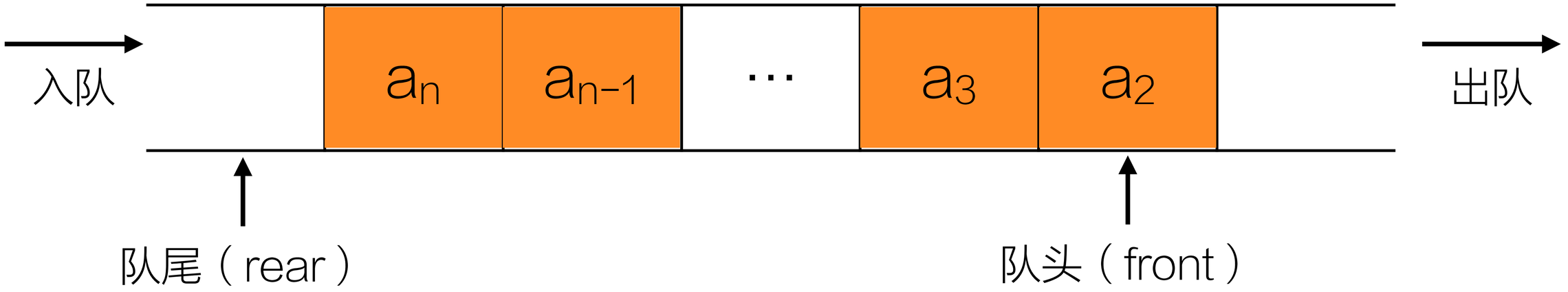
队列

队列（queue）是一种先进先出（First In First Out, FIFO）的线性表。它只允许在表的一端进行插入，而在另一端删除元素。在队列中，允许插入的一端称为队尾（rear）,允许删除的一端则称为队头（front）。假设队列为 $q = (a_1, a_2, \cdots, a_n)$，那么，a1 就是队头元素，$a_n$ 就是队尾元素。队列中的元素是按照 $a_1, a_2, \cdots, a_n$ 的顺序进入的，退出队列也只能按照这个次序依次退出，也就是说，只有在 $a_1, a_2, \cdots, a_{n-1}$ 都离开队列之后，a_n 才能退出队列。



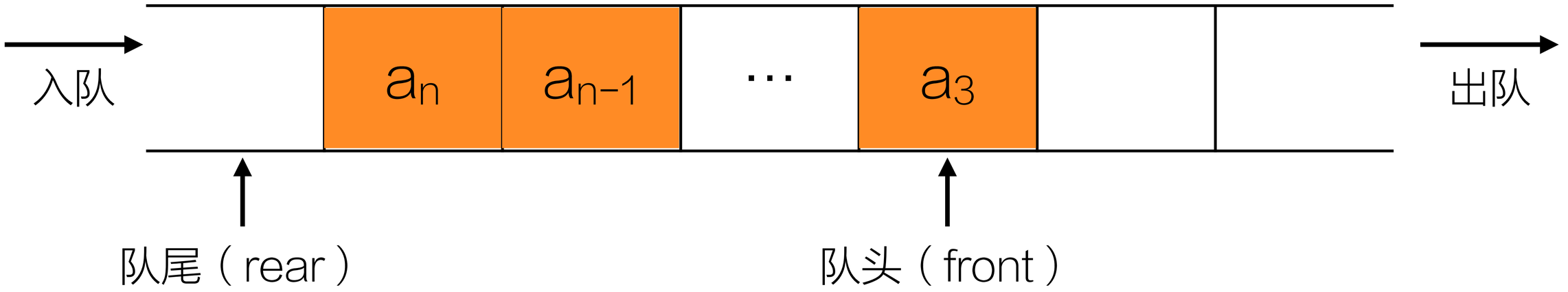
队列

队列（queue）是一种先进先出（First In First Out, FIFO）的线性表。它只允许在表的一端进行插入，而在另一端删除元素。在队列中，允许插入的一端称为队尾（rear）,允许删除的一端则称为队头（front）。假设队列为 $q = (a_1, a_2, \cdots, a_n)$，那么，a1 就是队头元素，$a_n$ 就是队尾元素。队列中的元素是按照 $a_1, a_2, \cdots, a_n$ 的顺序进入的，退出队列也只能按照这个次序依次退出，也就是说，只有在 $a_1, a_2, \cdots, a_{n-1}$ 都离开队列之后，a_n 才能退出队列。



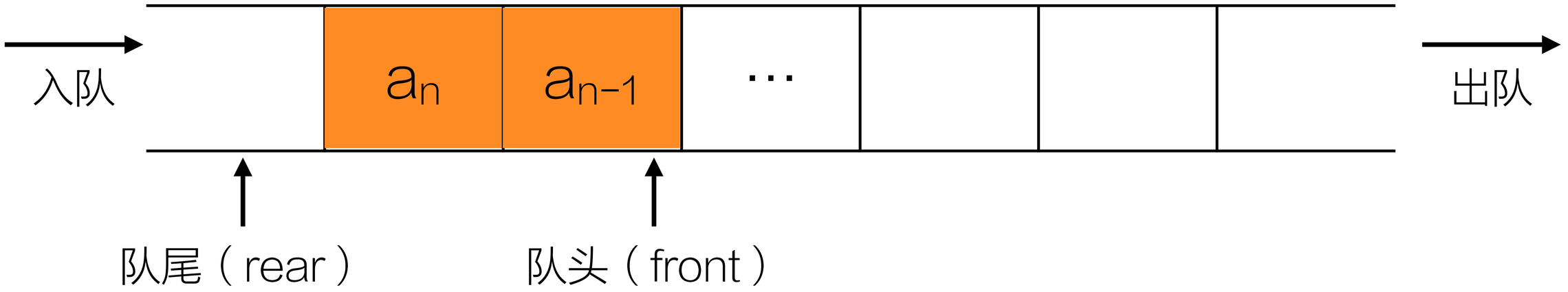
队列

队列（queue）是一种先进先出（First In First Out, FIFO）的线性表。它只允许在表的一端进行插入，而在另一端删除元素。在队列中，允许插入的一端称为队尾（rear）,允许删除的一端则称为队头（front）。假设队列为 $q = (a_1, a_2, \cdots, a_n)$，那么，a1 就是队头元素，$a_n$ 就是队尾元素。队列中的元素是按照 $a_1, a_2, \cdots, a_n$ 的顺序进入的，退出队列也只能按照这个次序依次退出，也就是说，只有在 $a_1, a_2, \cdots, a_{n-1}$ 都离开队列之后，a_n 才能退出队列。



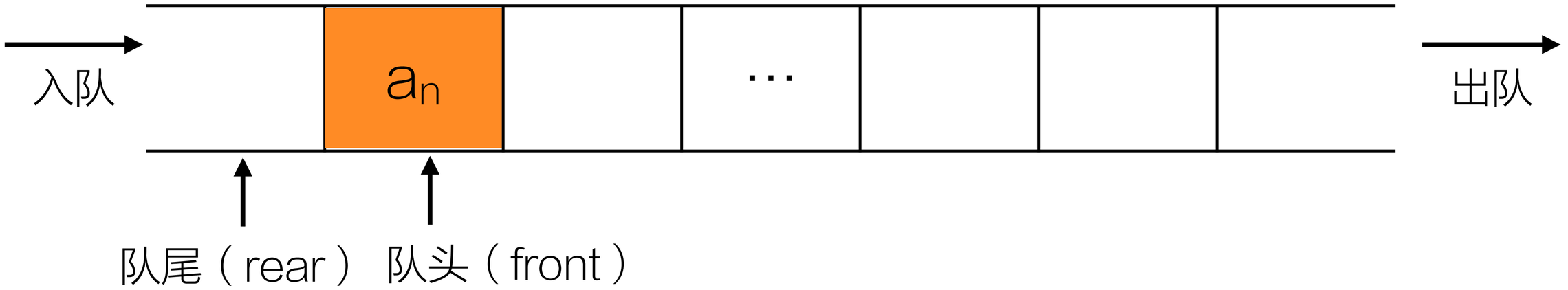
队列

队列（queue）是一种先进先出（First In First Out, FIFO）的线性表。它只允许在表的一端进行插入，而在另一端删除元素。在队列中，允许插入的一端称为队尾（rear）,允许删除的一端则称为队头（front）。假设队列为 $q = (a_1, a_2, \cdots, a_n)$，那么，a1 就是队头元素，$a_n$ 就是队尾元素。队列中的元素是按照 $a_1, a_2, \cdots, a_n$ 的顺序进入的，退出队列也只能按照这个次序依次退出，也就是说，只有在 $a_1, a_2, \cdots, a_{n-1}$ 都离开队列之后，a_n 才能退出队列。



队列

队列（queue）是一种先进先出（First In First Out, FIFO）的线性表。它只允许在表的一端进行插入，而在另一端删除元素。在队列中，允许插入的一端称为队尾（rear）,允许删除的一端则称为队头（front）。假设队列为 $q = (a_1, a_2, \cdots, a_n)$，那么，a1 就是队头元素，$a_n$ 就是队尾元素。队列中的元素是按照 $a_1, a_2, \cdots, a_n$ 的顺序进入的，退出队列也只能按照这个次序依次退出，也就是说，只有在 $a_1, a_2, \cdots, a_{n-1}$ 都离开队列之后，a_n 才能退出队列。



队列的顺序结构实现

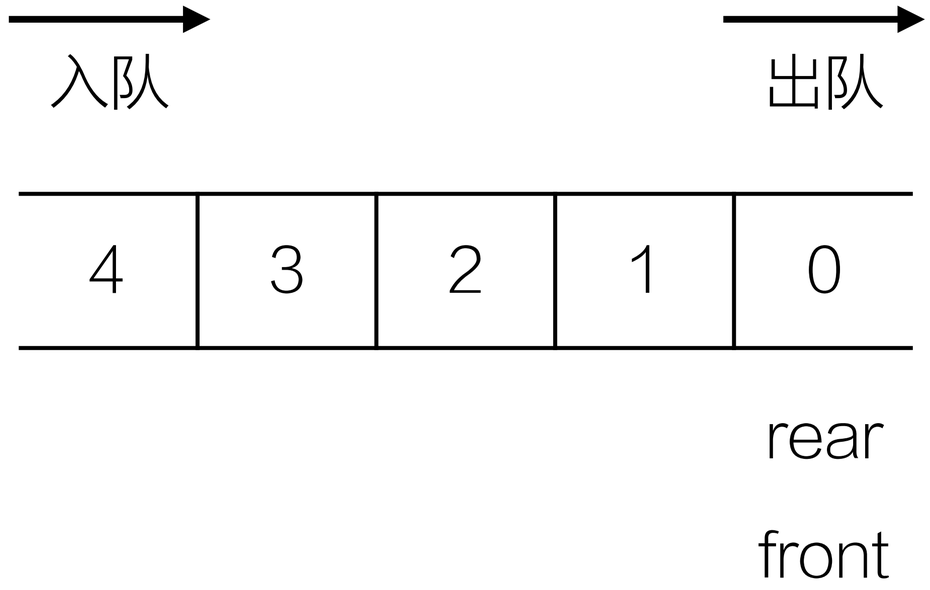
```
#define MAXSIZE 100
typedef int ElemType;
typedef struct
{
    ElemType data[MAXSIZE];
    int front;
    int rear;
}Queue;
```

案例：栈与队列 / 队列 / test1.c

队列的顺序结构 – 初始化

```
#define MAXSIZE 100
typedef int ElemType;
typedef struct
{
    ElemType data[MAXSIZE];
    int front;
    int rear;
}Queue;
void initQueue(Queue *Q)
{
        Q->front = 0;
        Q->rear = 0;
}
```

案例：栈与队列 / 队列 / test1.c

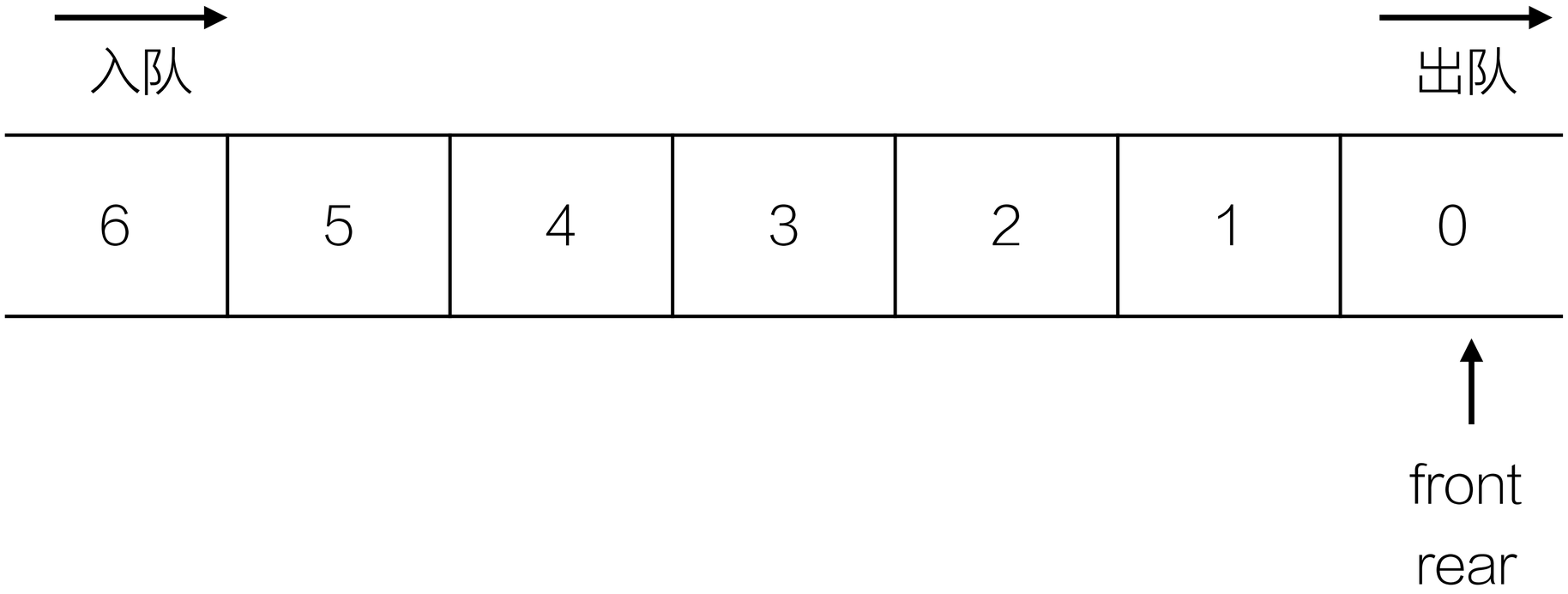


队列的顺序结构 – 判断队列是否为空

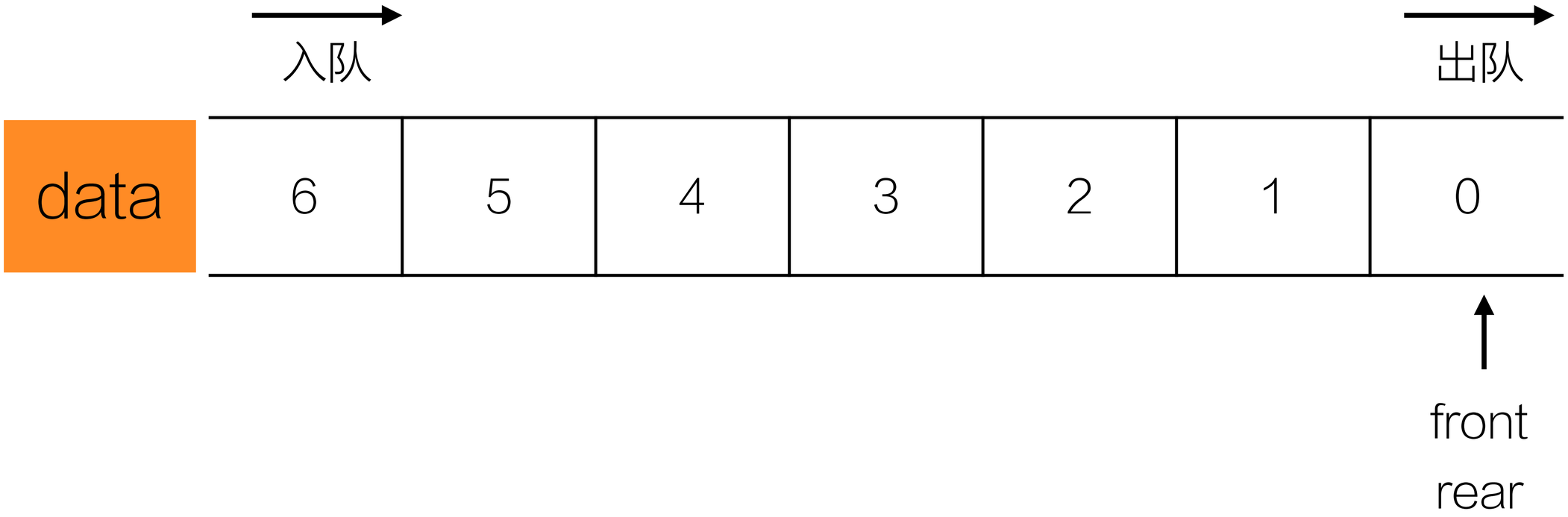
```
int isEmpty(Queue *Q)
{
    if (Q->front == Q->rear)
    {
        printf("空的\n");
        return 1;
    }
    else
    {
        return 0;
    }
}
```

案例：栈与队列 / 队列 / test1.c

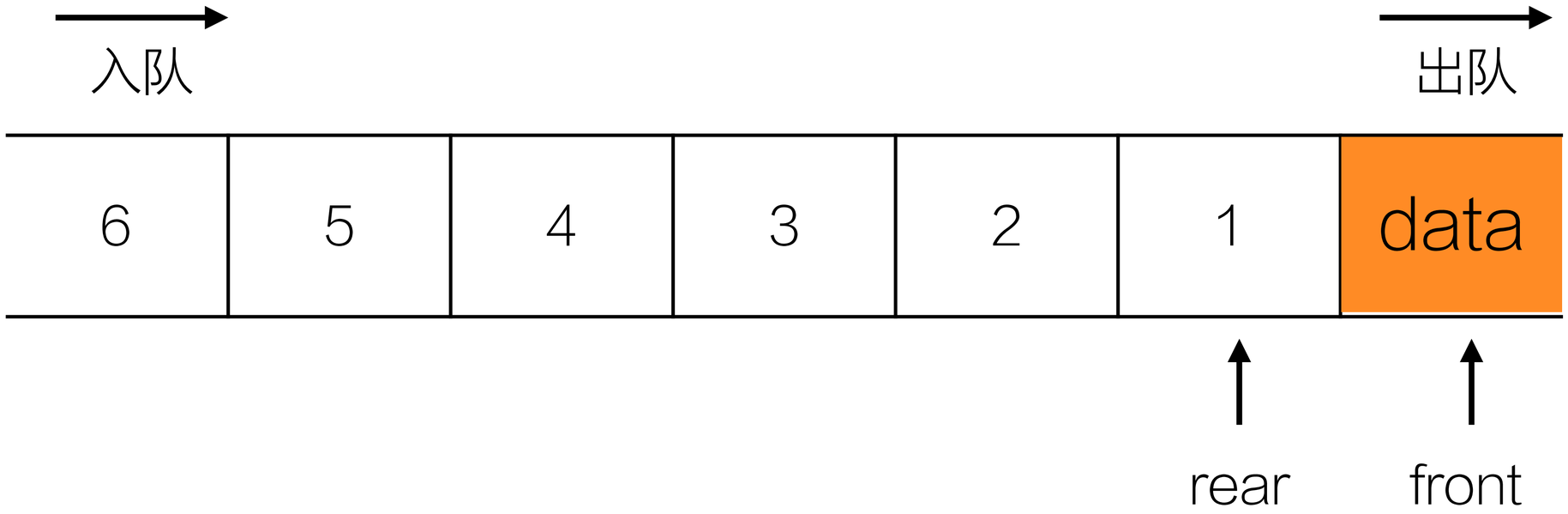
队列的顺序结构 – 判断队列是否为空



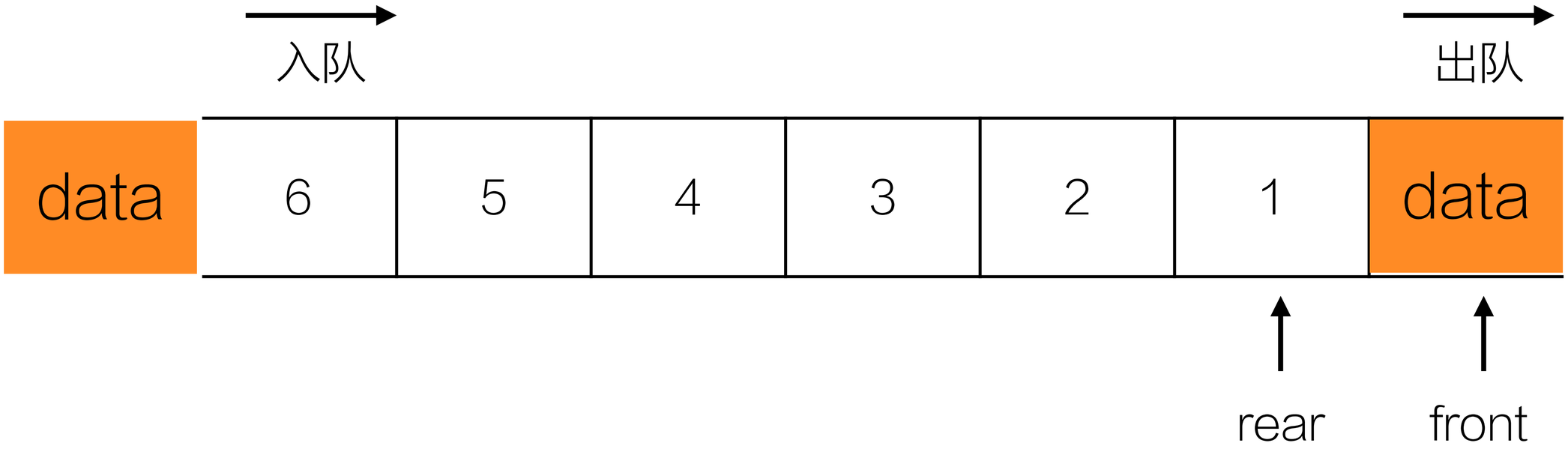
队列的顺序结构 – 判断队列是否为空



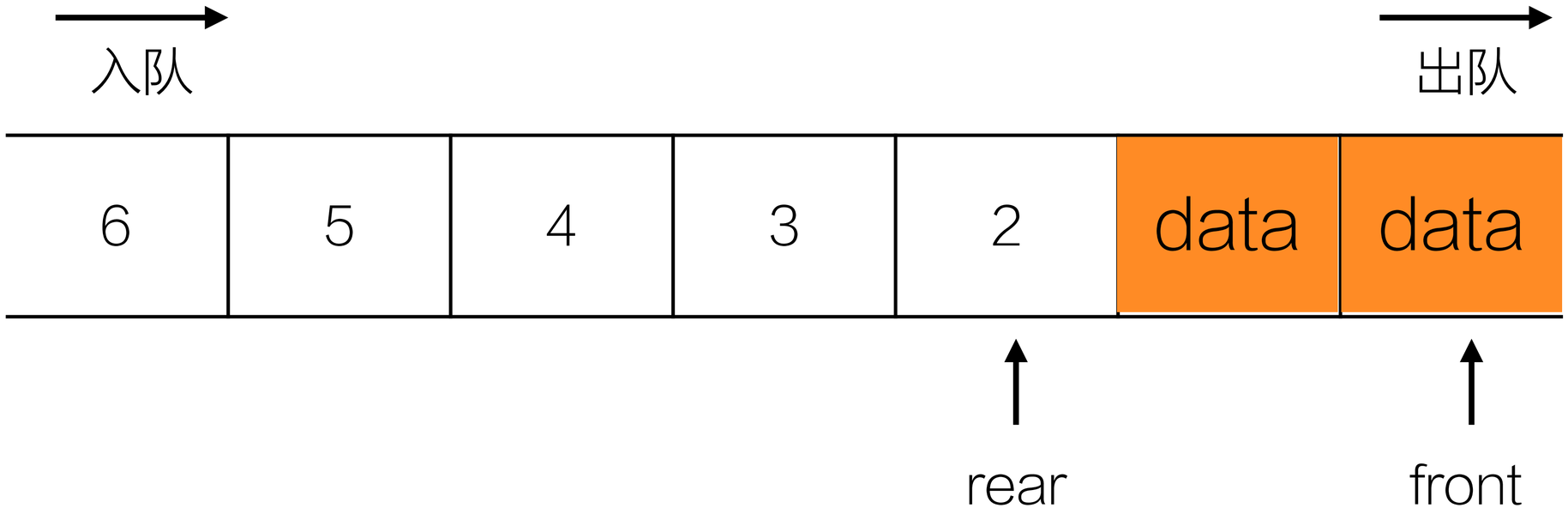
队列的顺序结构 – 判断队列是否为空



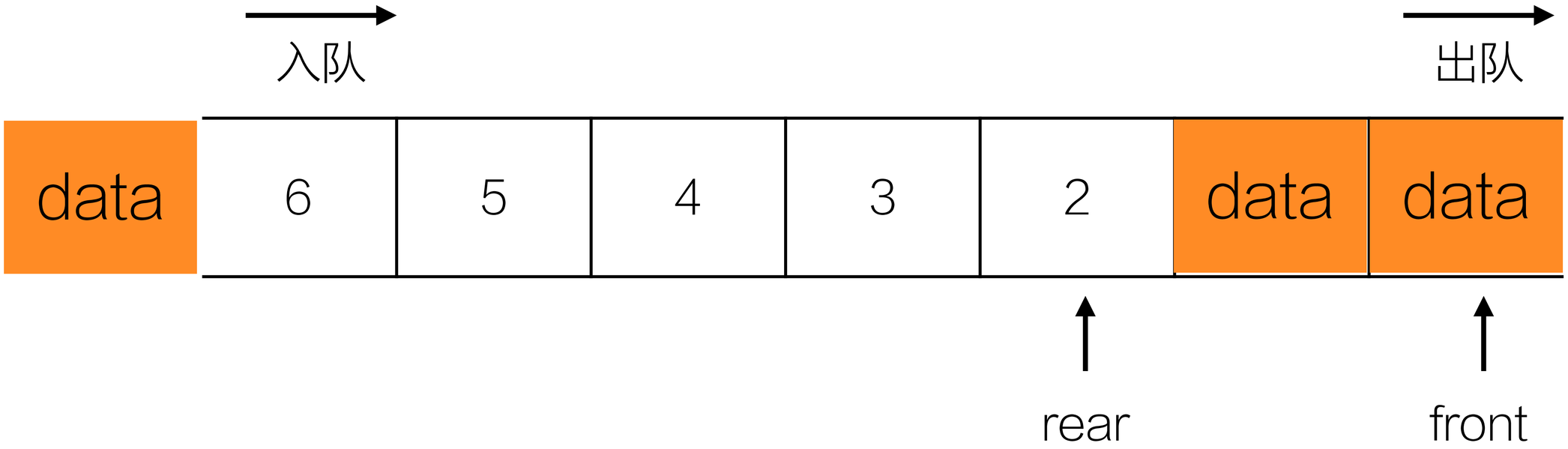
队列的顺序结构 – 判断队列是否为空



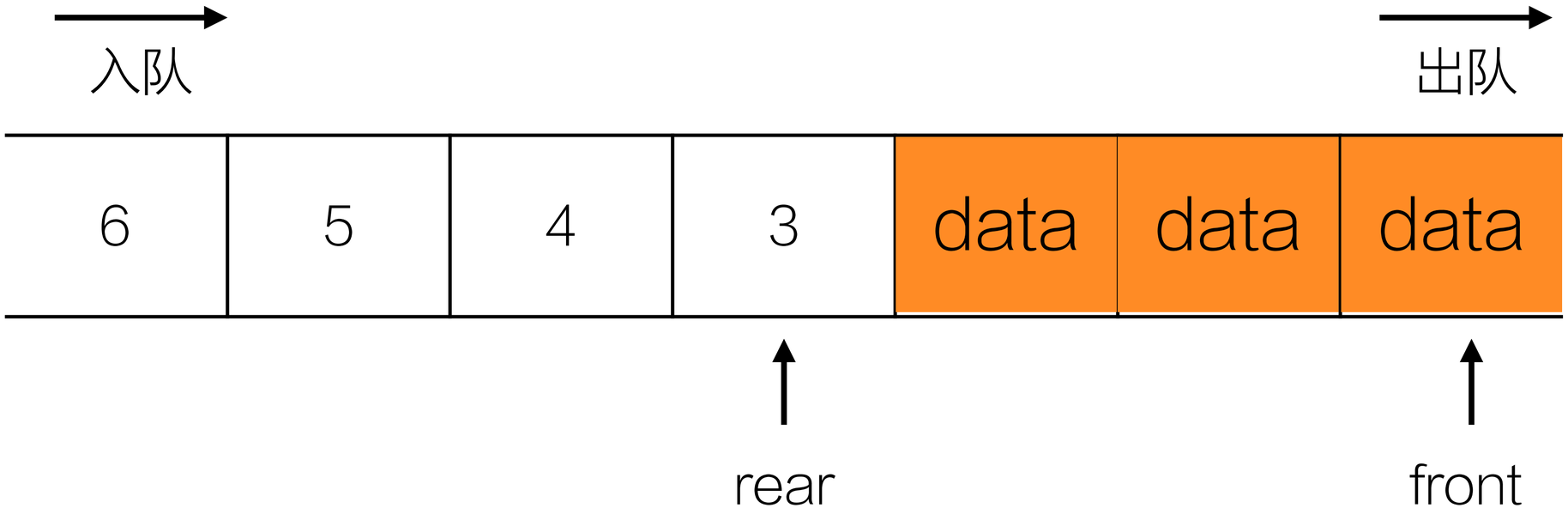
队列的顺序结构 – 判断队列是否为空



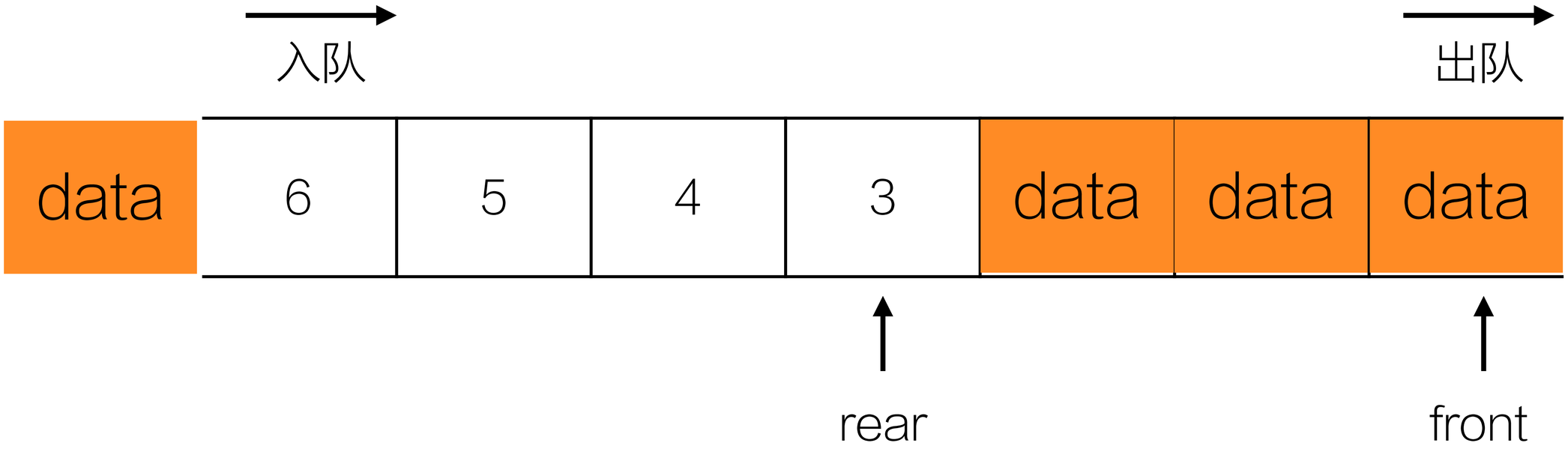
队列的顺序结构 – 判断队列是否为空



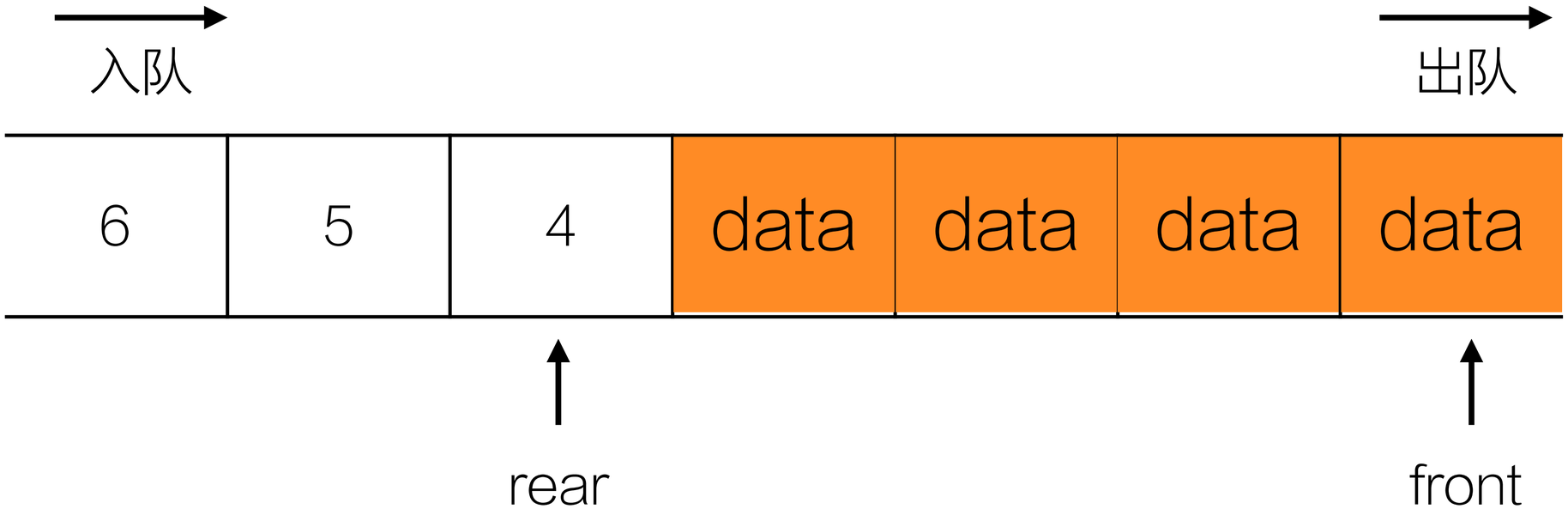
队列的顺序结构 – 判断队列是否为空



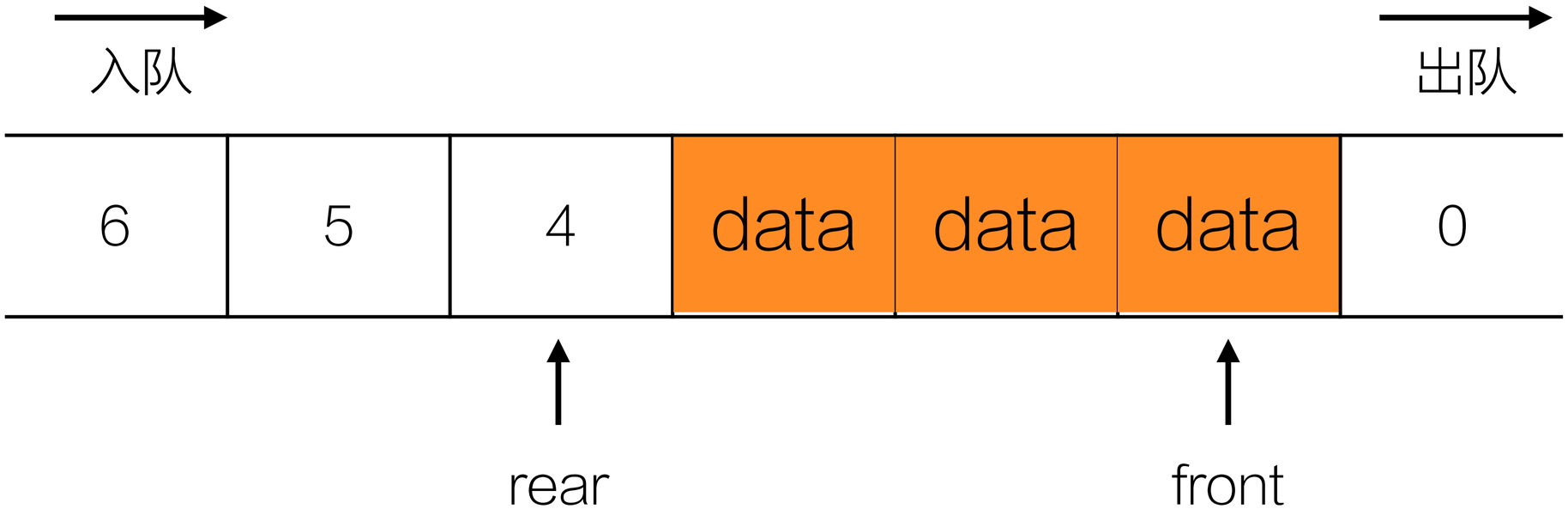
队列的顺序结构 – 判断队列是否为空



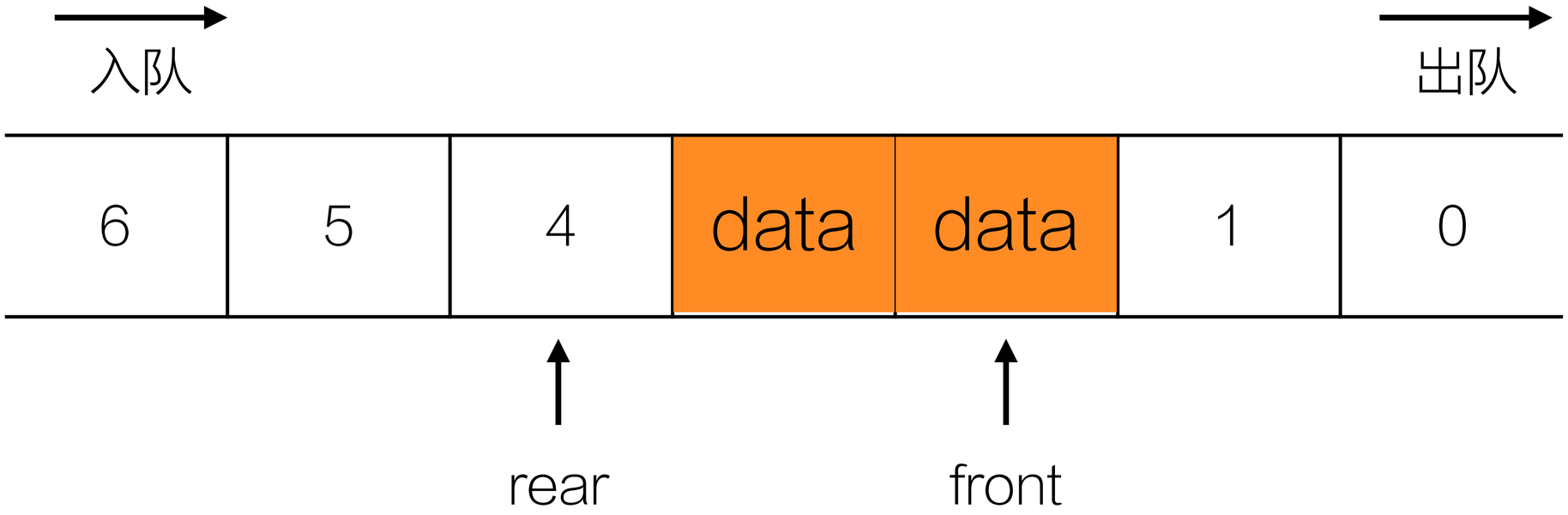
队列的顺序结构 – 判断队列是否为空



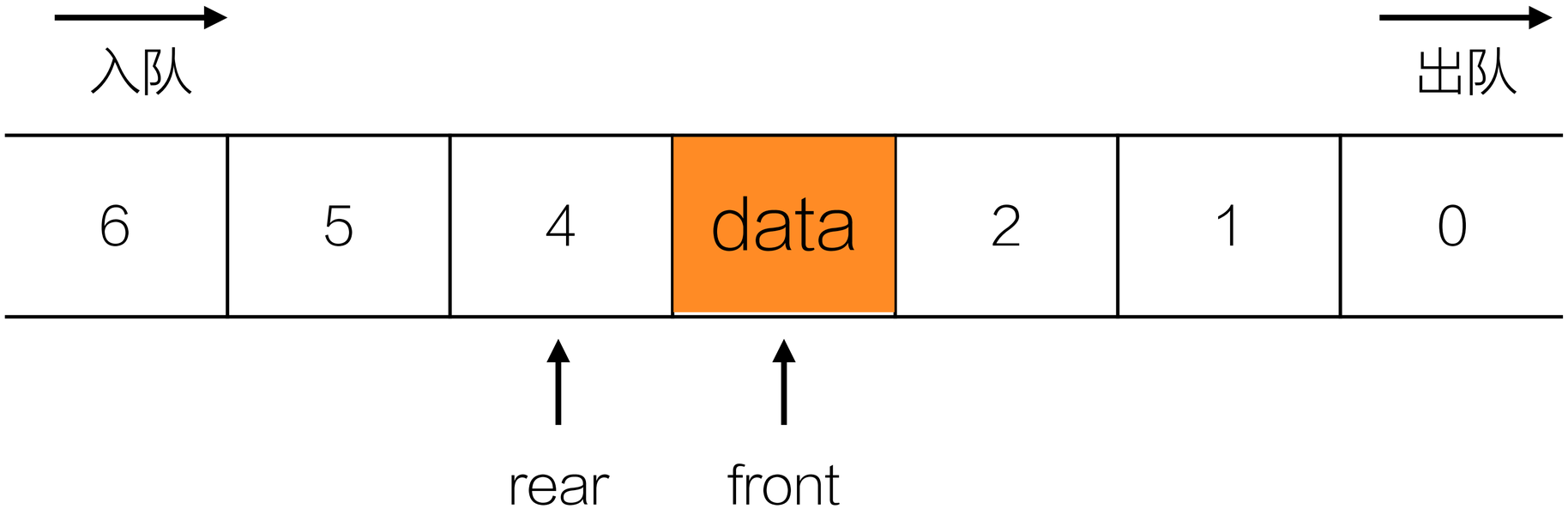
队列的顺序结构 – 判断队列是否为空



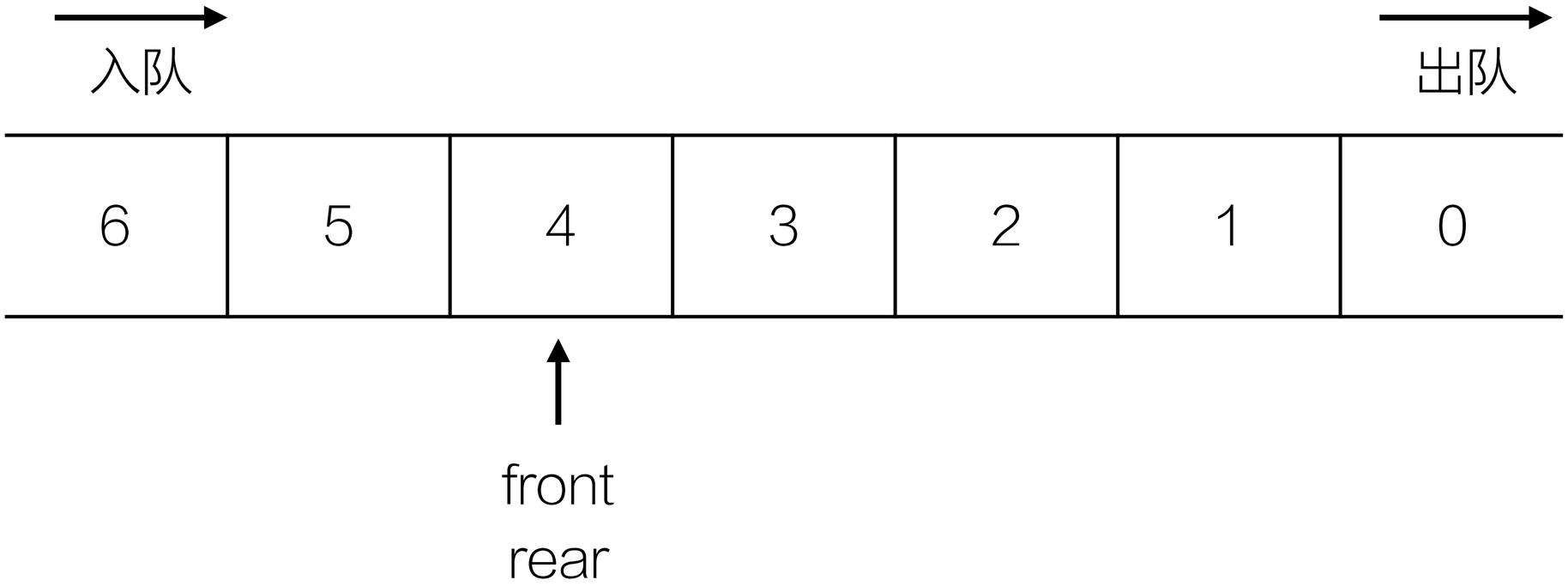
队列的顺序结构 – 判断队列是否为空



队列的顺序结构 – 判断队列是否为空



队列的顺序结构 – 判断队列是否为空

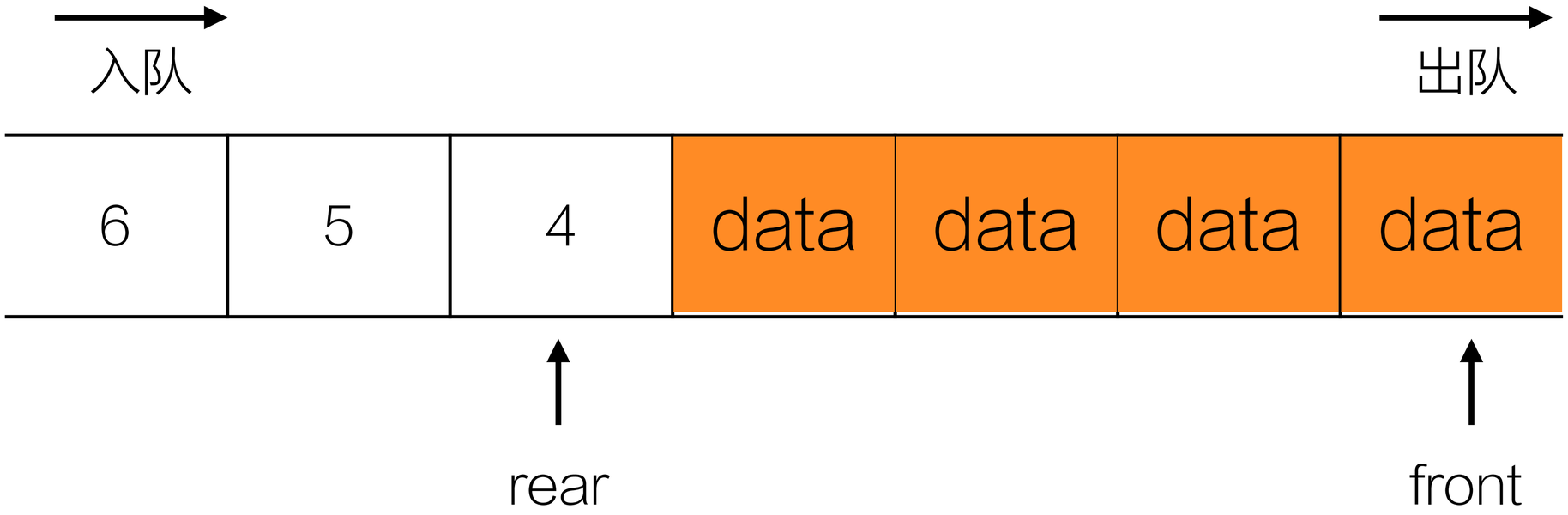


队列的顺序结构 – 出队

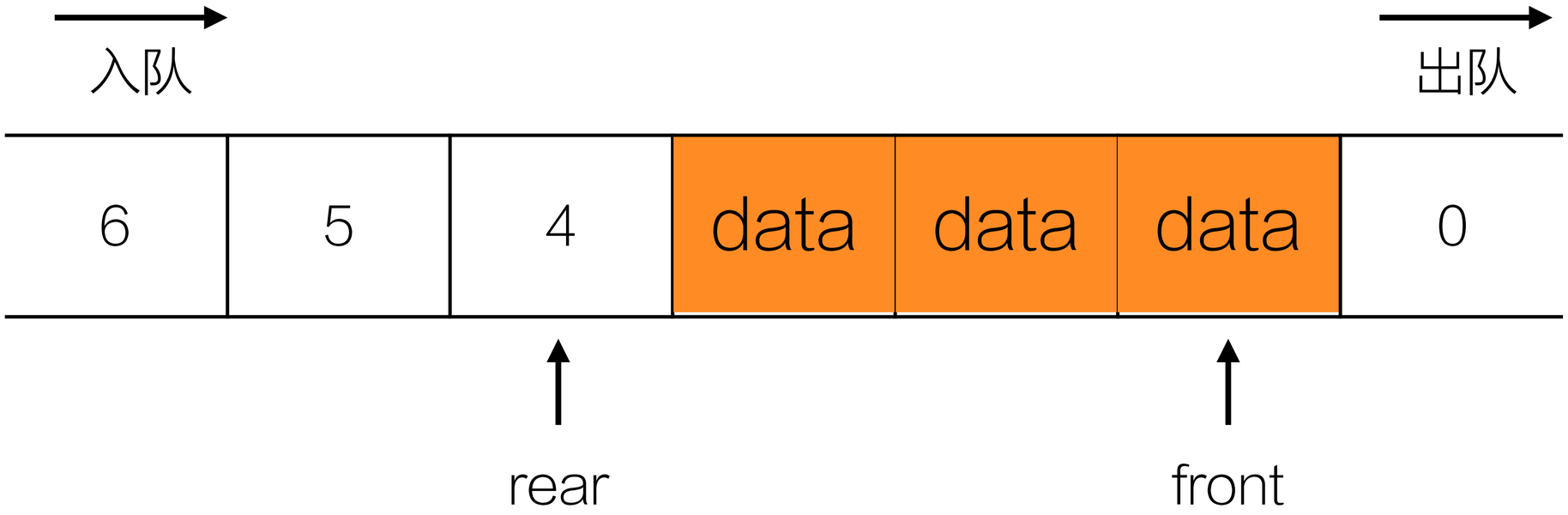
```
ElemType dequeue(Queue *Q)
{
    if (Q->front == Q->rear)
    {
        printf("空的\n");
        return 0;
    }
    ElemType e = Q->data[Q->front];
    Q->front++;
    return e;
}
```

案例：栈与队列 / 队列 / test1.c

队列的顺序结构 – 出队



队列的顺序结构 – 出队

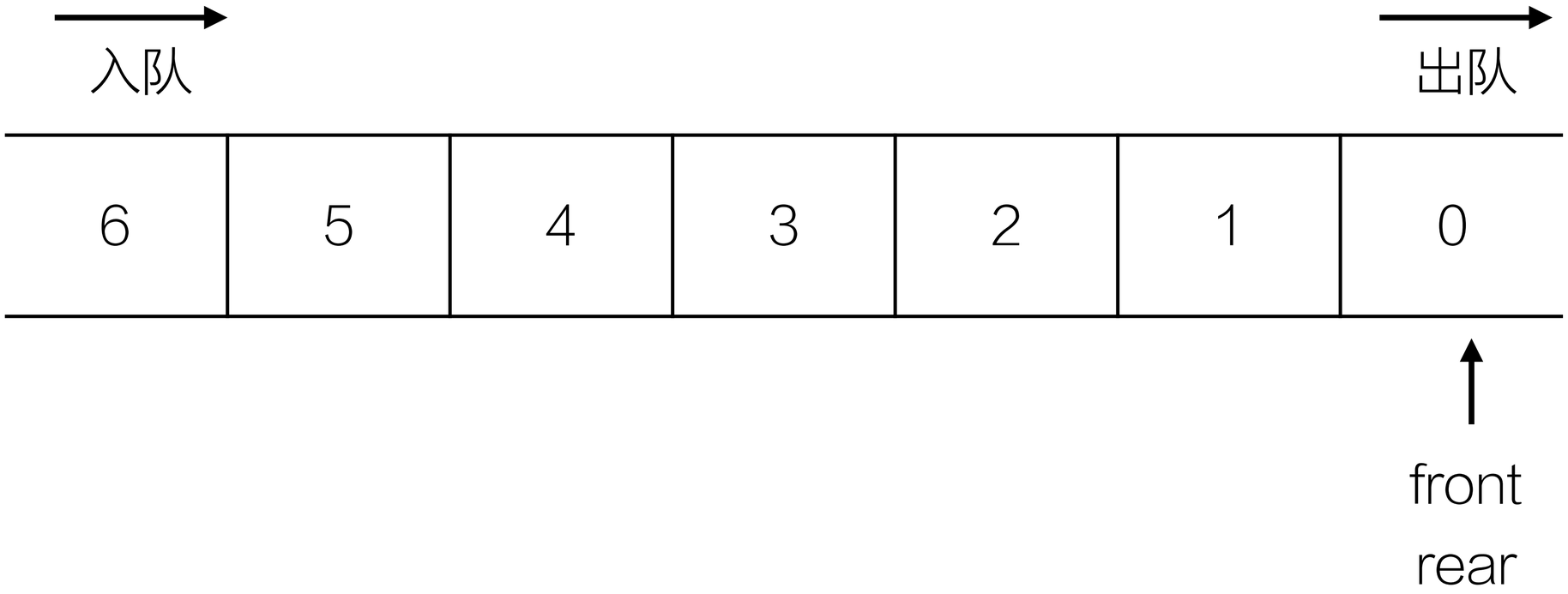


队列的顺序结构 - 入队

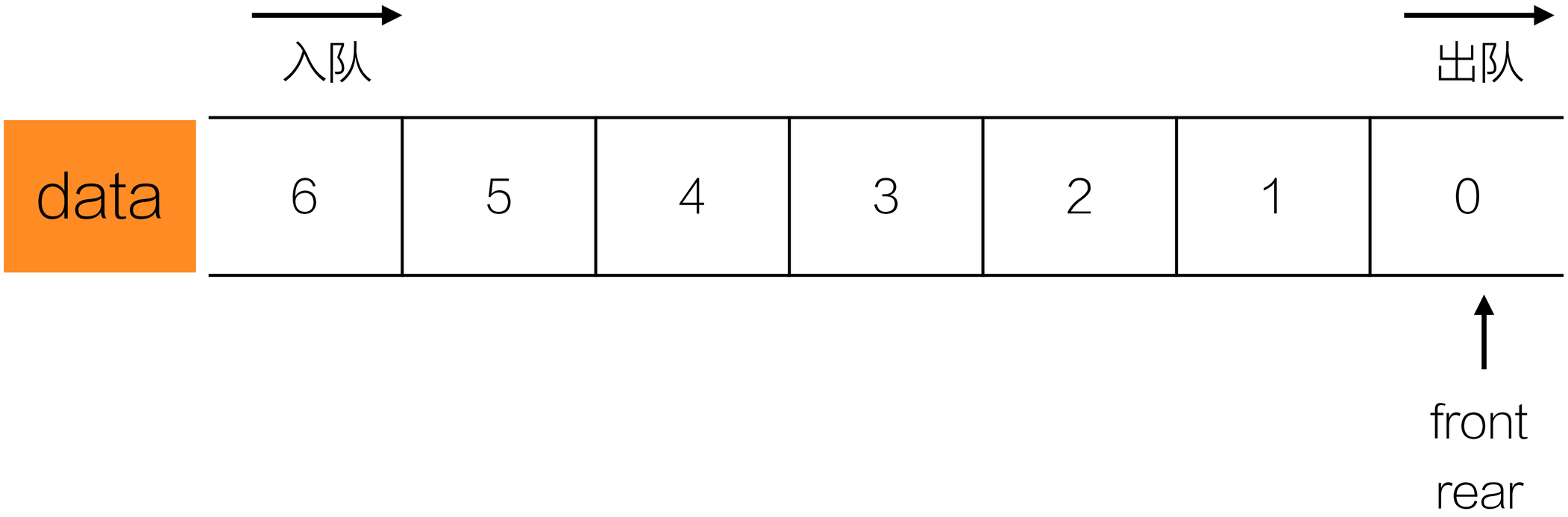
```
int equeue(Queue *Q, ElemType e)
{
    if (Q->rear >= MAXSIZE)
    {
        if(!queueFull(Q))
        {
            return 0;
        }
    }
    Q->data[Q->rear] = e;
    Q->rear++;
    return 1;
}
```

案例：栈与队列 / 队列 / test1.c

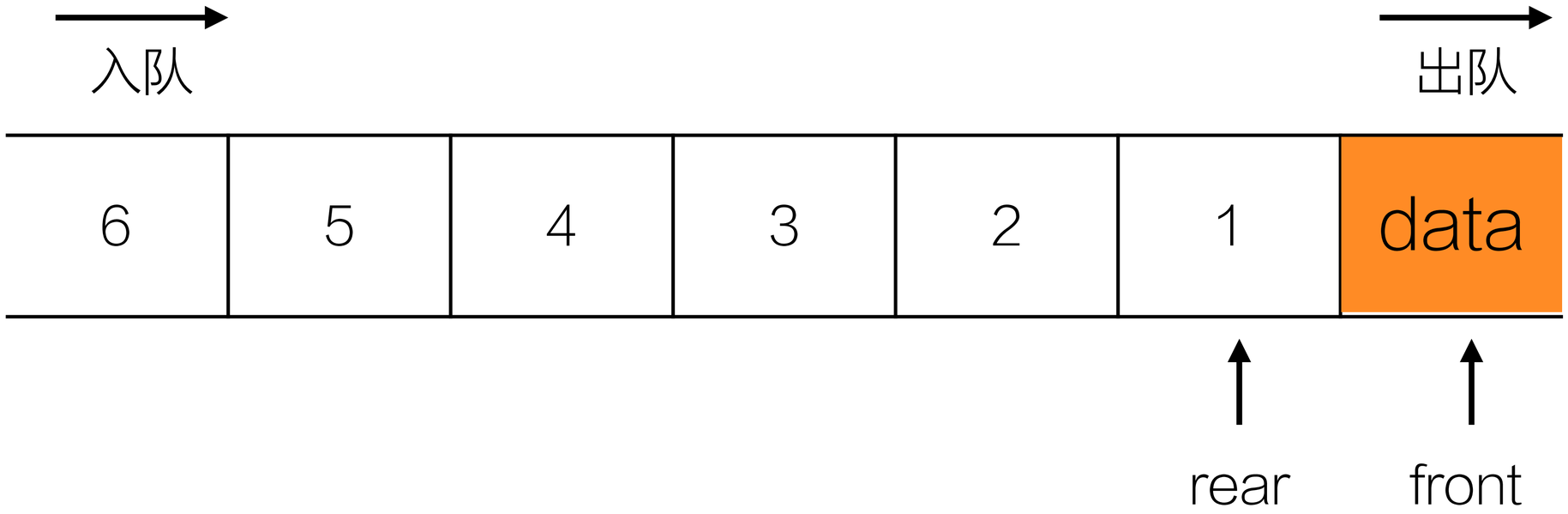
队列的顺序结构 – 入队



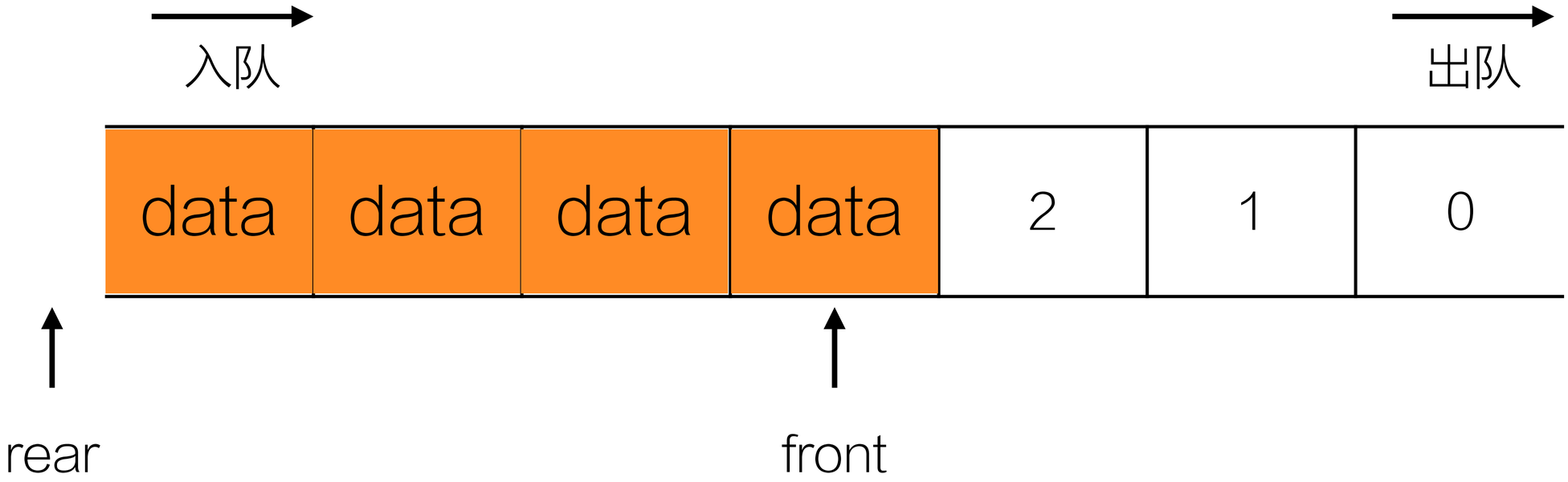
队列的顺序结构 – 入队



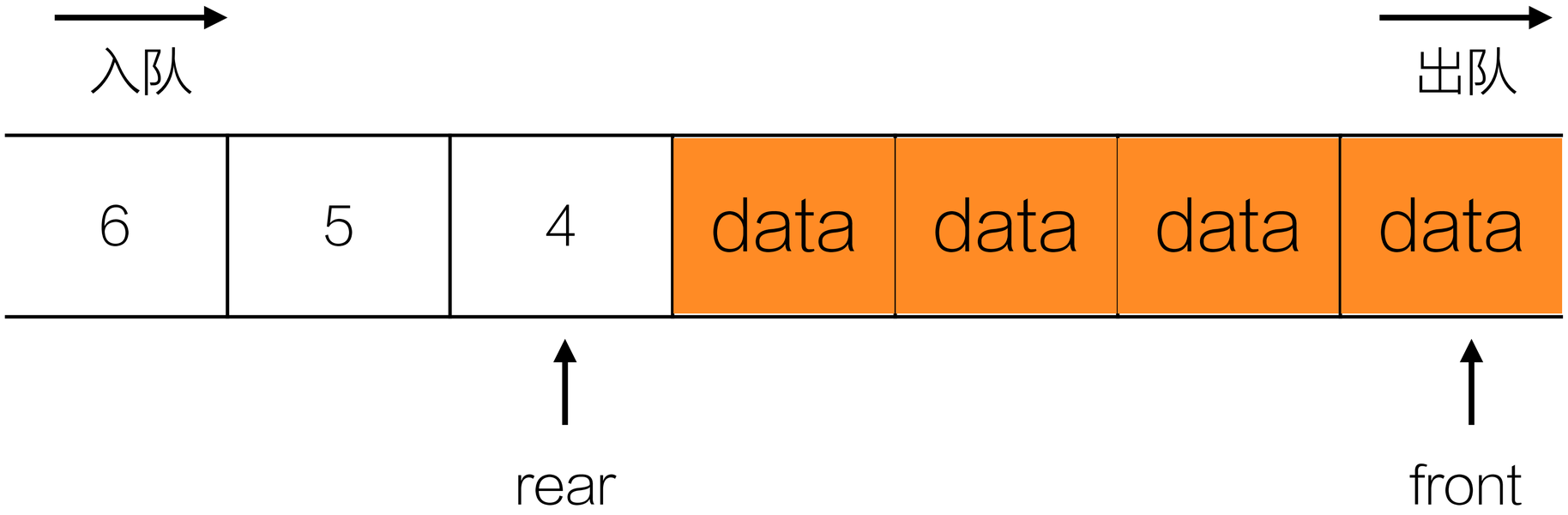
队列的顺序结构 – 入队



队列的顺序结构 – 调整队列



队列的顺序结构 – 调整队列



队列的顺序结构 - 调整队列

```
int queueFull(Queue *Q)
{
    if (Q->front > 0)
    {
        int step = Q->front;
        for (int i = Q->front; i <= Q->rear; ++i)
        {
            Q->data[i - step] = Q->data[i];
        }
        Q->front = 0;
        Q->rear = Q->rear - step;
        return 1;
    }
    else
    {
        printf("真的满了\n");
        return 0;
    }
}
```

队列的顺序结构 – 获取队头数据

```
int getHead(Queue *Q, ElemType *e)
{
    if (Q->front == Q->rear)
    {
        printf("空的\n");
        return 0;
    }
    *e = Q->data[Q->front];
    return 1;
}
```

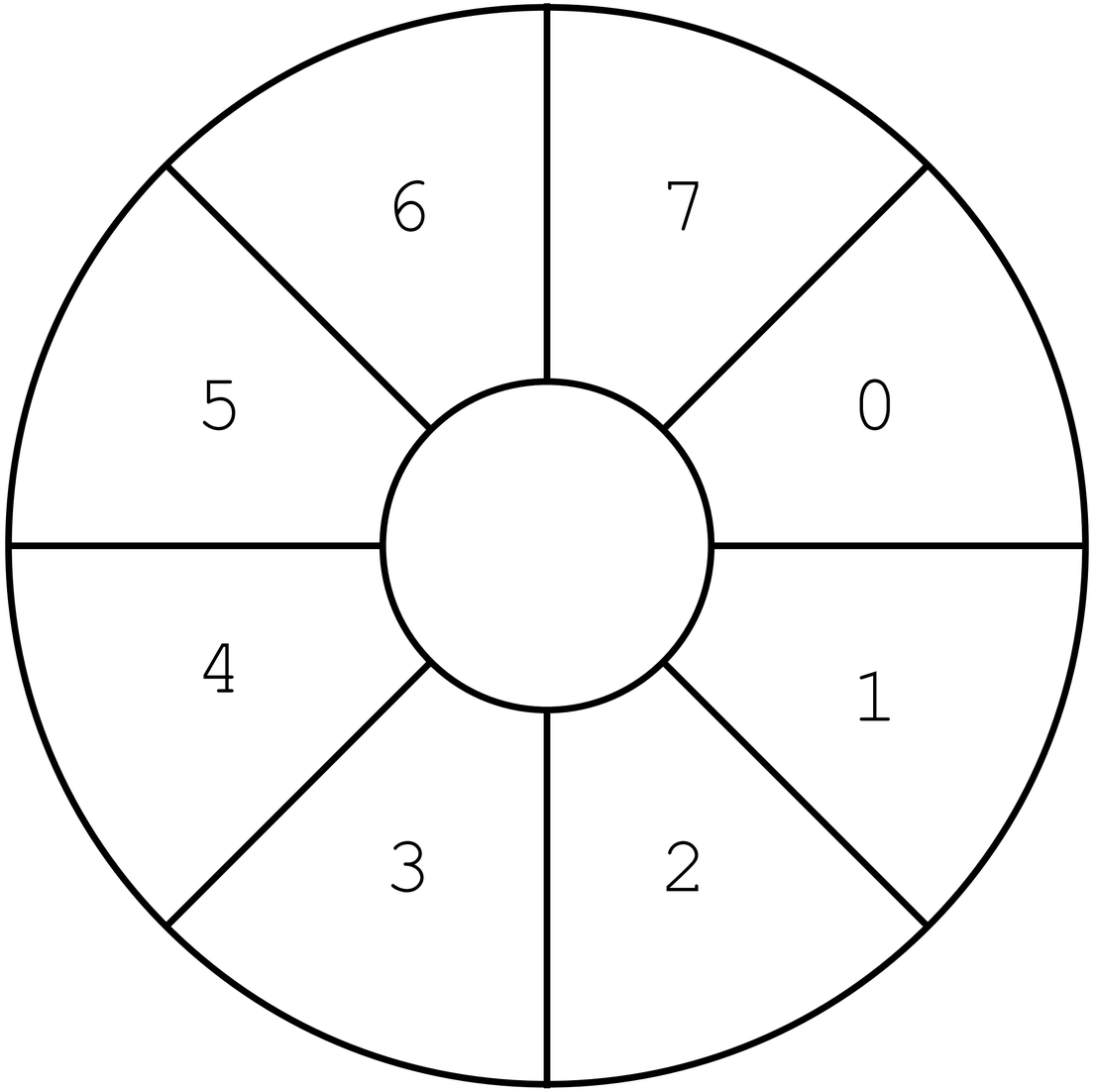
案例：栈与队列 / 队列 / test1.c

队列的顺序结构 – 动态内存分配

案例：栈与队列 / 队列 / test2.c

```
typedef struct
{
    ElemType *data;
    int front;
    int rear;
}Queue;
Queue* initQueue()
{
    Queue *q = (Queue*)malloc(sizeof(Queue));
    q->data = (ElemType*)malloc(sizeof(ElemType) * MAXSIZE);
    q->front = 0;
    q->rear = 0;
    return q;
}
```

队列的顺序结构 – 循环队列



队列的顺序结构 – 循环队列 – 入队

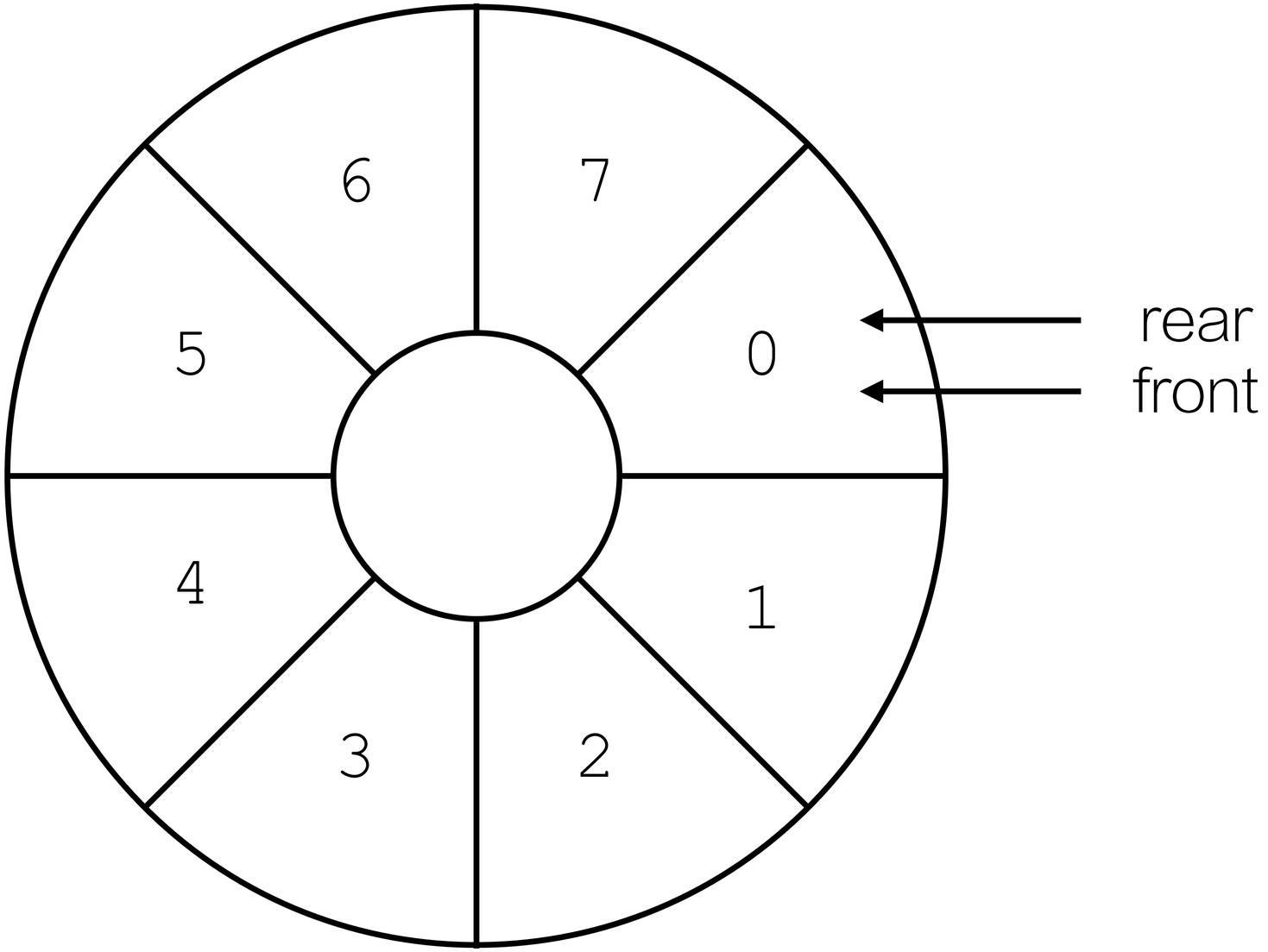
案例：栈与队列 / 队列 / test3.c

```
int equeue(Queue *Q, ElemType e)
{
    if ((Q->rear + 1) % MAXSIZE == Q->front)
    {
        printf("满了\n");
        return 0;
    }
    Q->data[Q->rear] = e;
    Q->rear = (Q->rear + 1) % MAXSIZE;
    return 1;
}
```

队列的顺序结构 – 循环队列

满队列：(rear + 1) % MAXSIZE == front

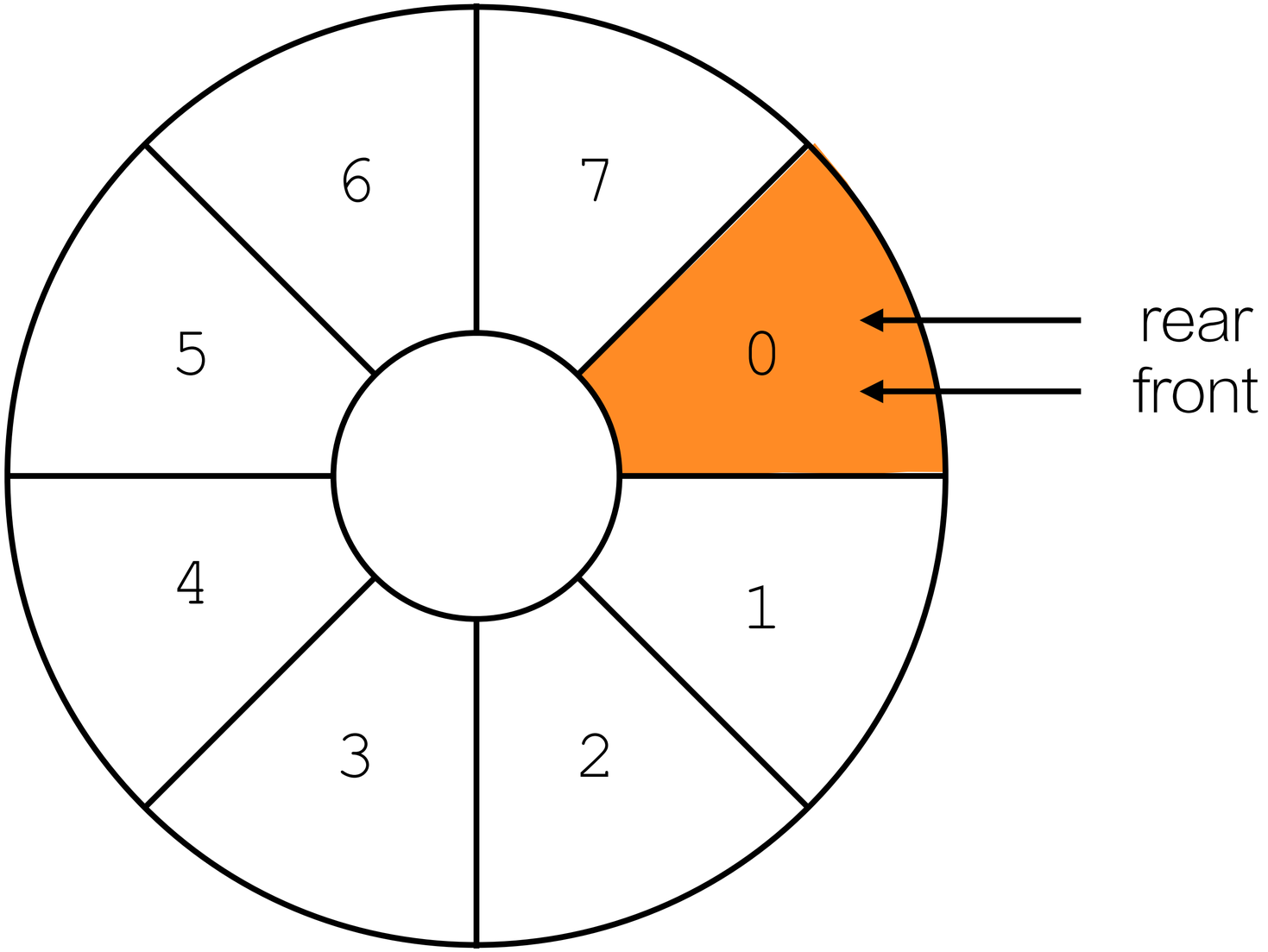
队尾：(rear + 1) % MAXSIZE



队列的顺序结构 – 循环队列

满队列：(rear + 1) % MAXSIZE == front

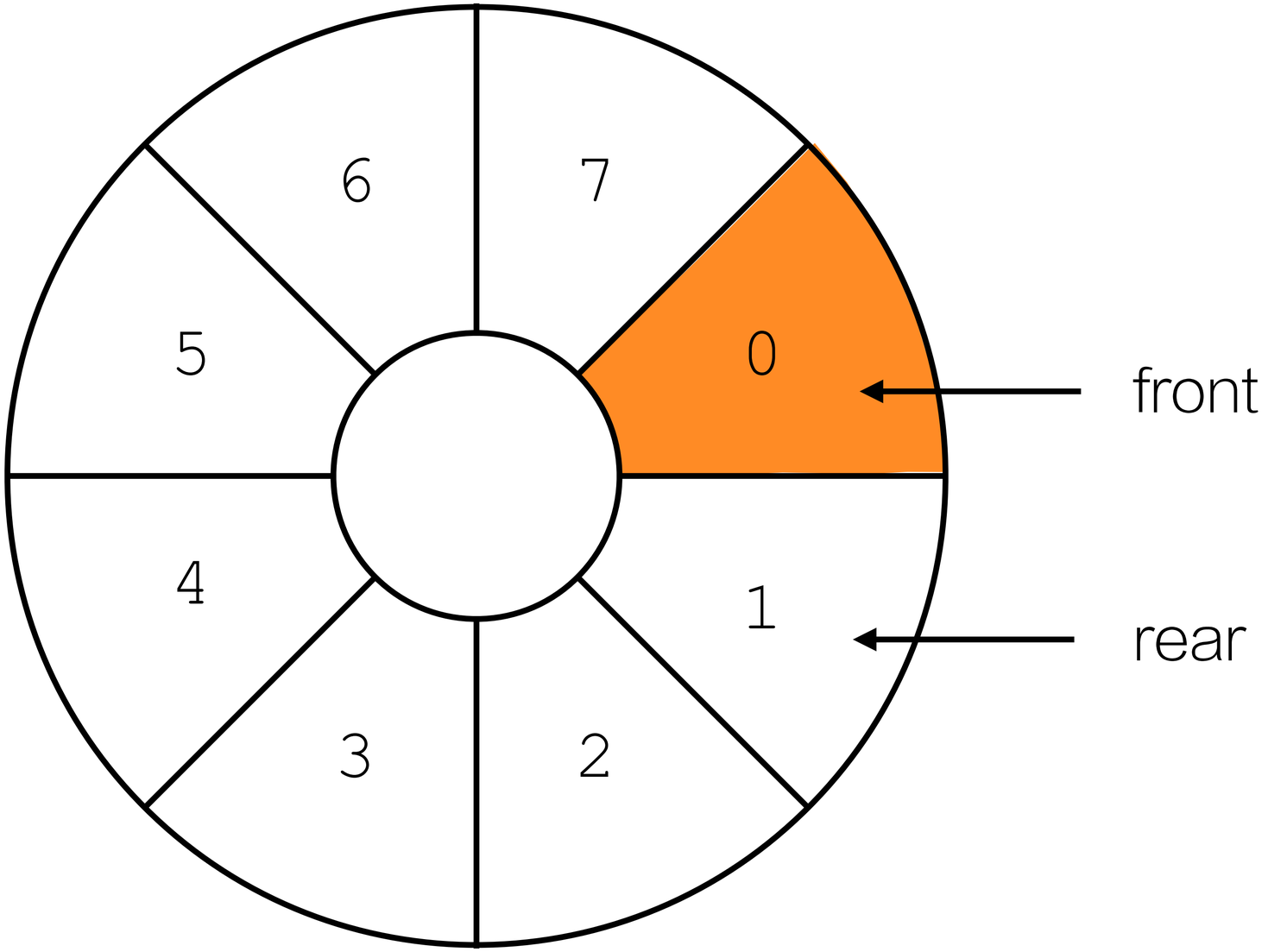
队尾：(rear + 1) % MAXSIZE



队列的顺序结构 – 循环队列

满队列：(rear + 1) % MAXSIZE == front

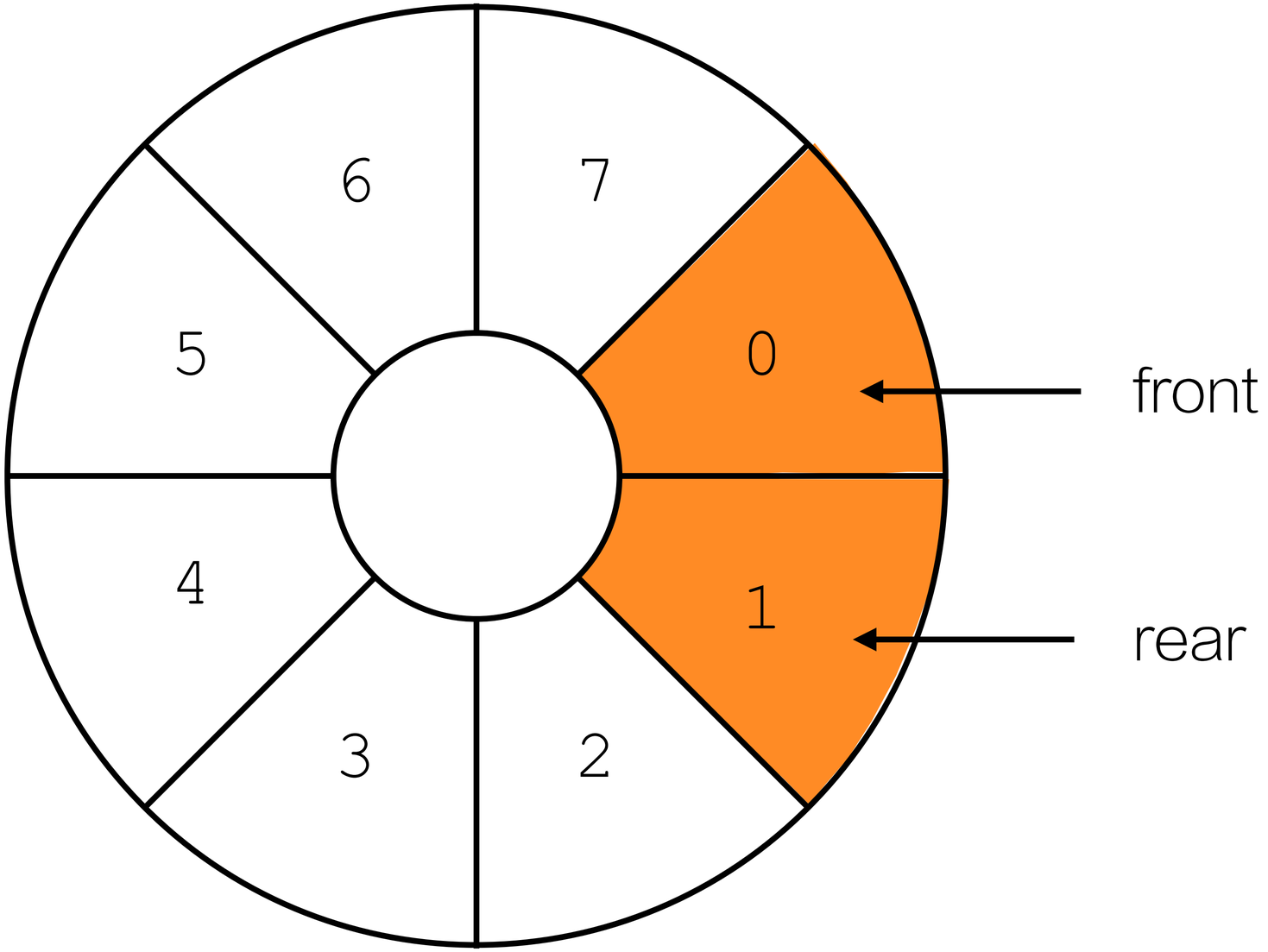
队尾：(rear + 1) % MAXSIZE



队列的顺序结构 – 循环队列

满队列：(rear + 1) % MAXSIZE == front

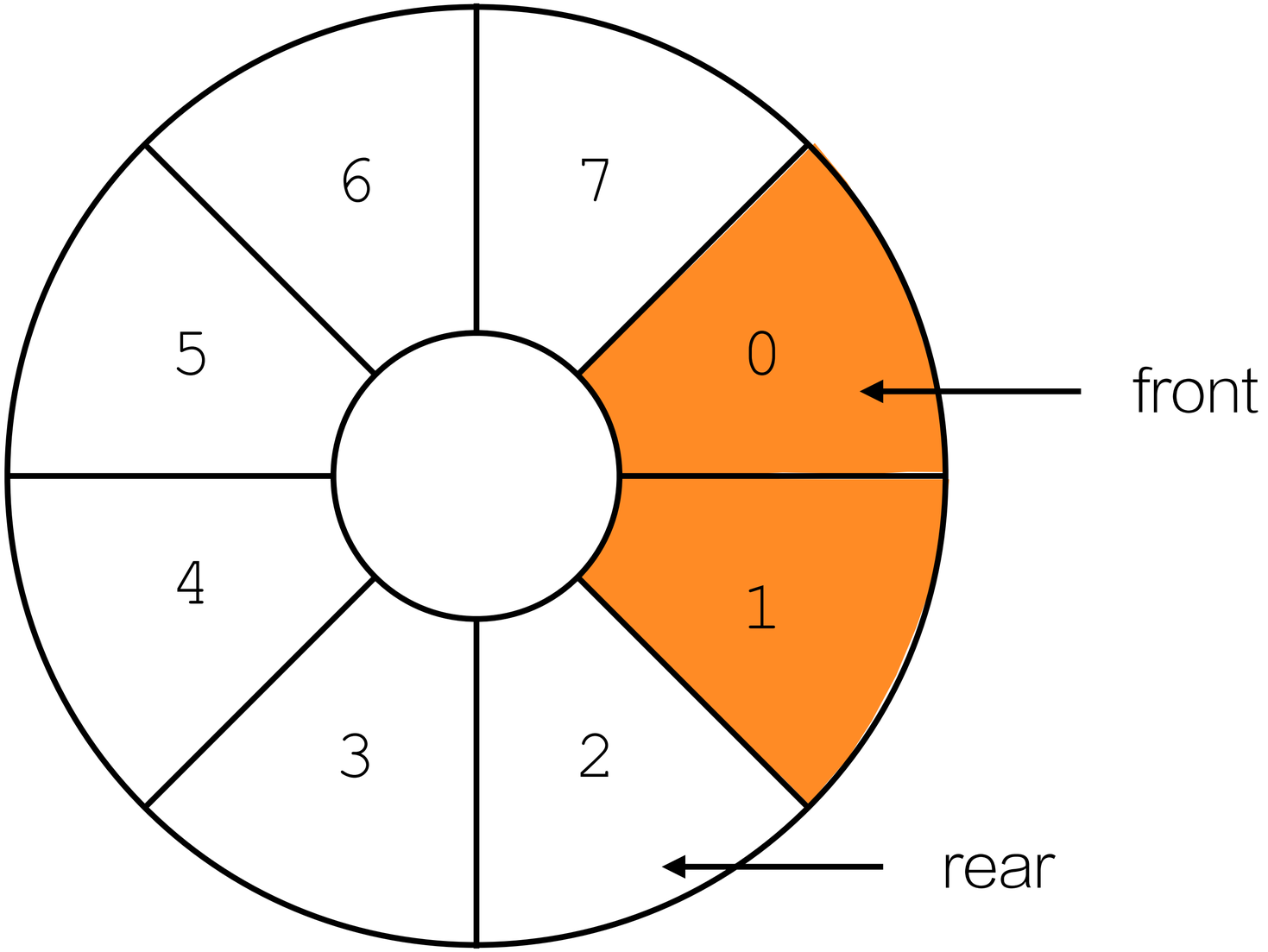
队尾：(rear + 1) % MAXSIZE



队列的顺序结构 – 循环队列

满队列：(rear + 1) % MAXSIZE == front

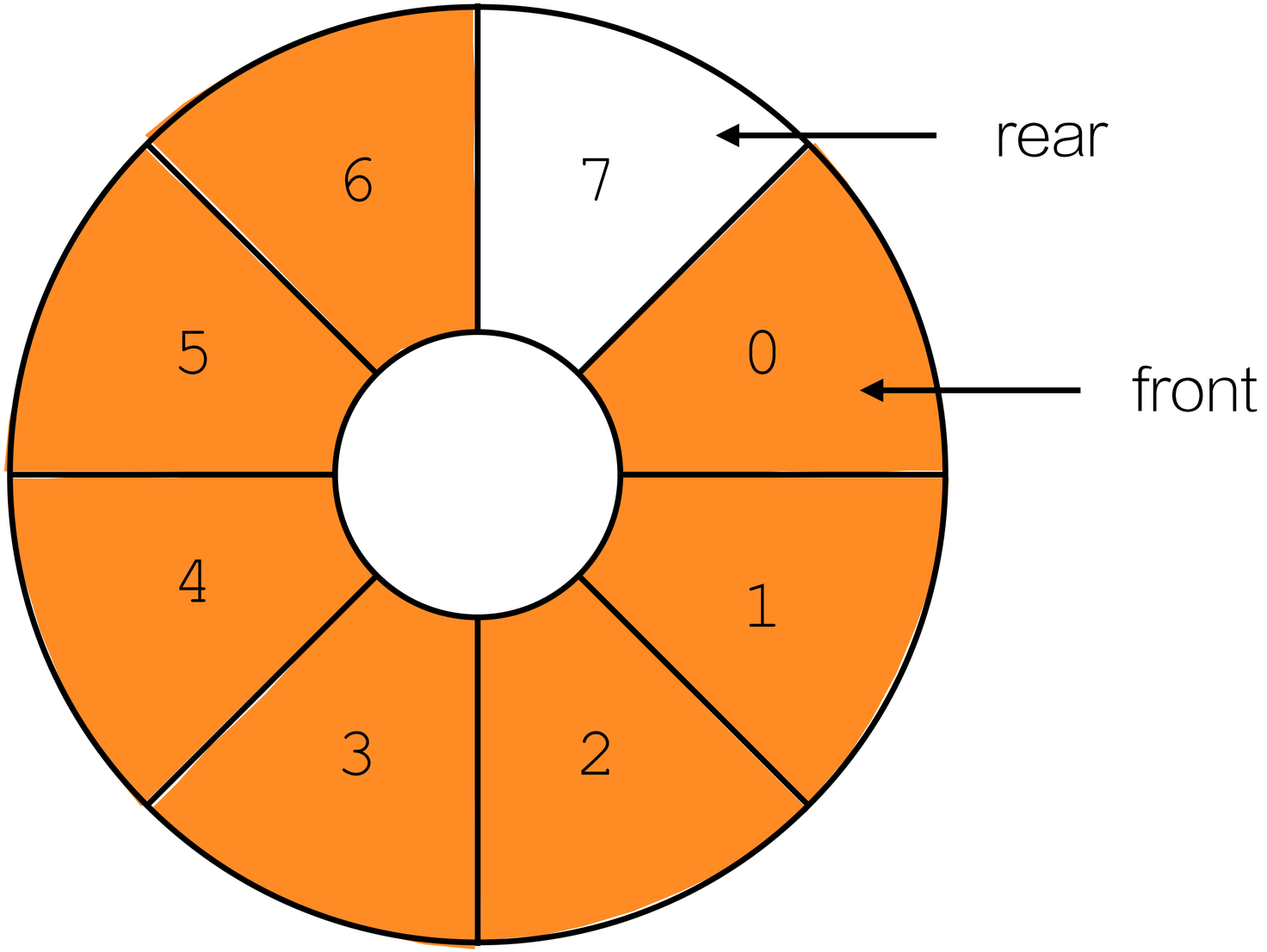
队尾：(rear + 1) % MAXSIZE



队列的顺序结构 – 循环队列

满队列：(rear + 1) % MAXSIZE == front

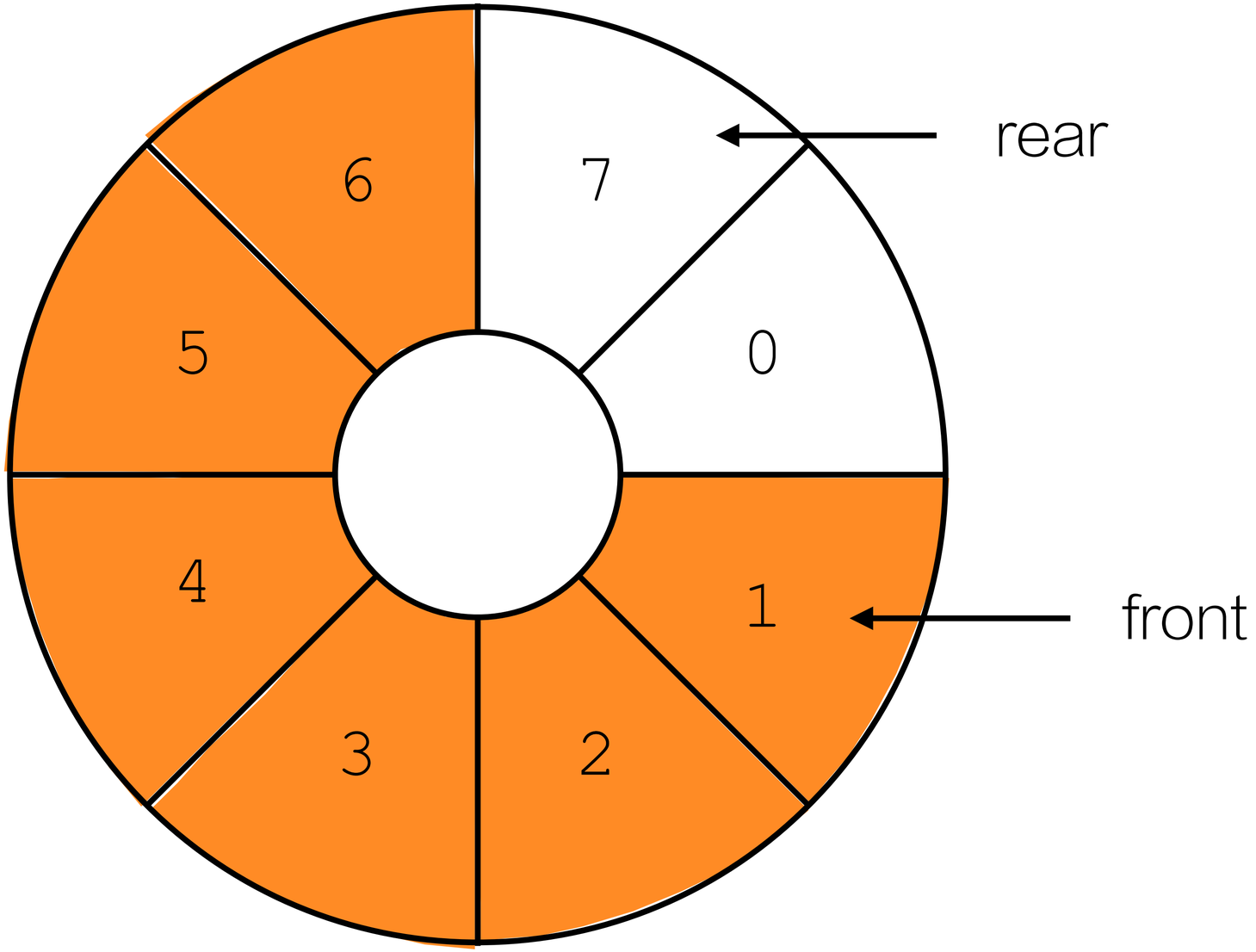
队尾：(rear + 1) % MAXSIZE



队列的顺序结构 – 循环队列

满队列：(rear + 1) % MAXSIZE == front

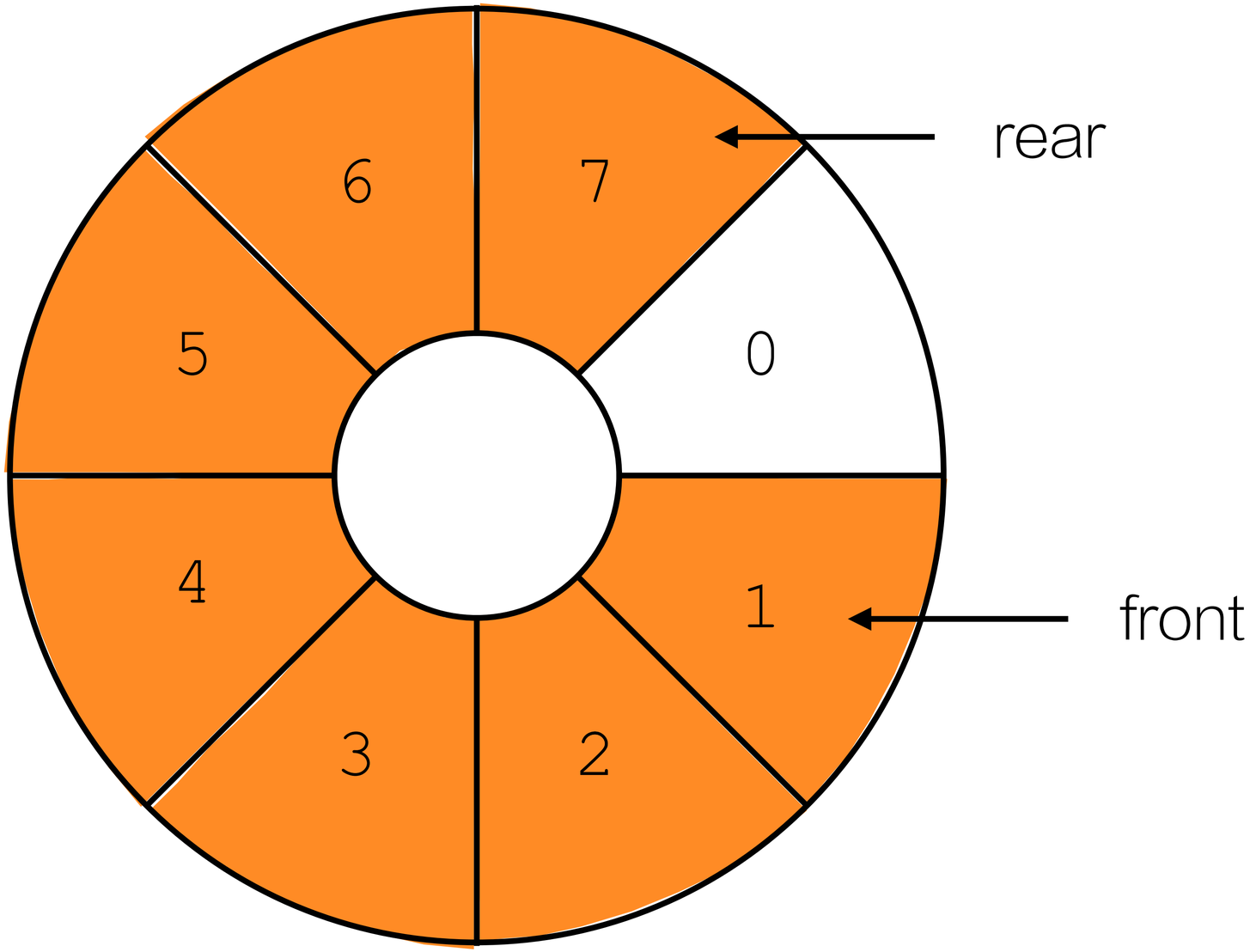
队尾：(rear + 1) % MAXSIZE



队列的顺序结构 – 循环队列

满队列：(rear + 1) % MAXSIZE == front

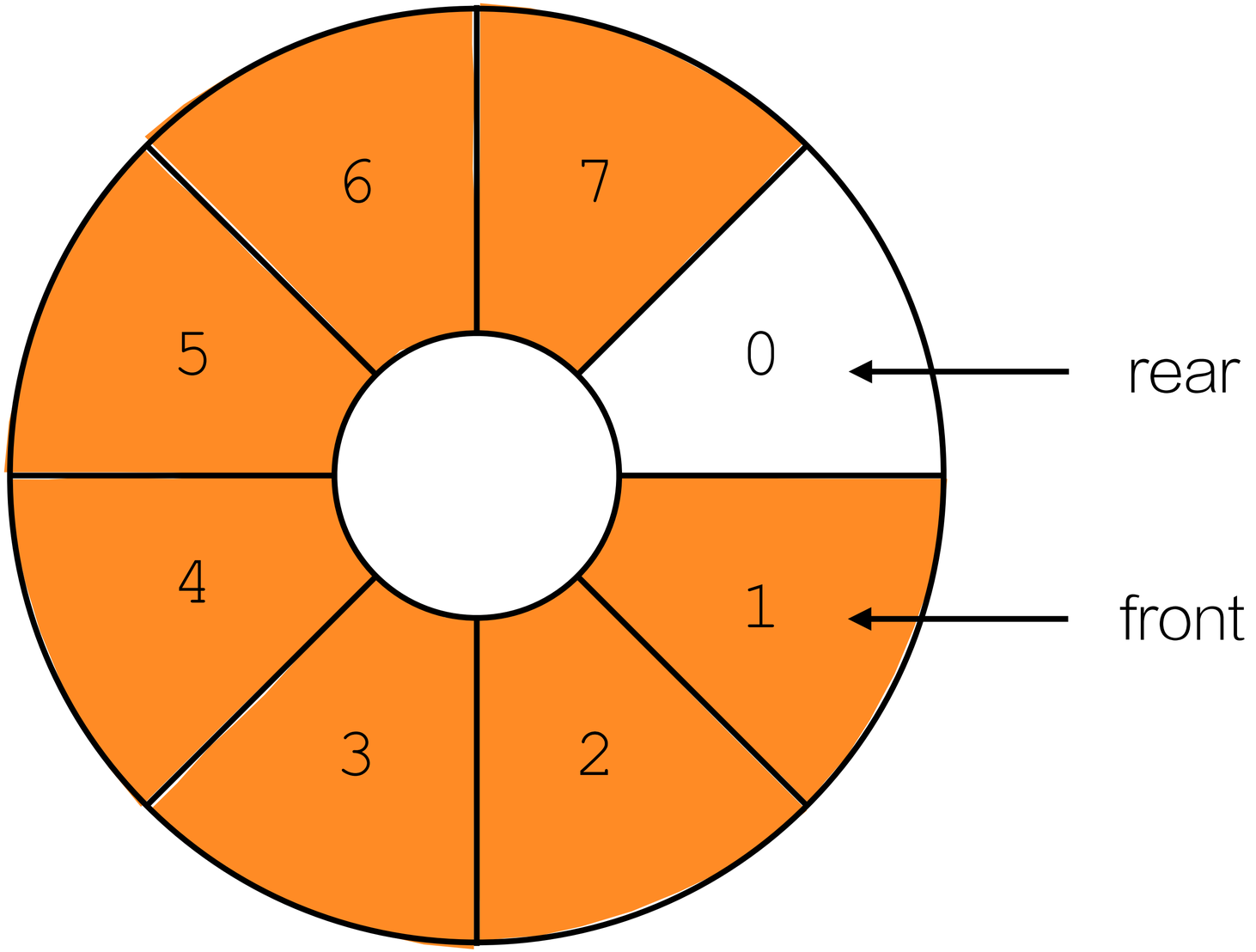
队尾：(rear + 1) % MAXSIZE



队列的顺序结构 – 循环队列

满队列：(rear + 1) % MAXSIZE == front

队尾：(rear + 1) % MAXSIZE



队列的顺序结构 – 循环队列

```
typedef struct
{
    ElemType *data;
    int front;
    int rear;
}Queue;
Queue* initQueue()
{
    Queue *q = (Queue*)malloc(sizeof(Queue));
    q->data = (ElemType*)malloc(sizeof(ElemType) * MAXSIZE);
    q->front = 0;
    q->rear = MAXSIZE - 1;
    return q;
}
```

队列的顺序结构 – 循环队列 – 入队

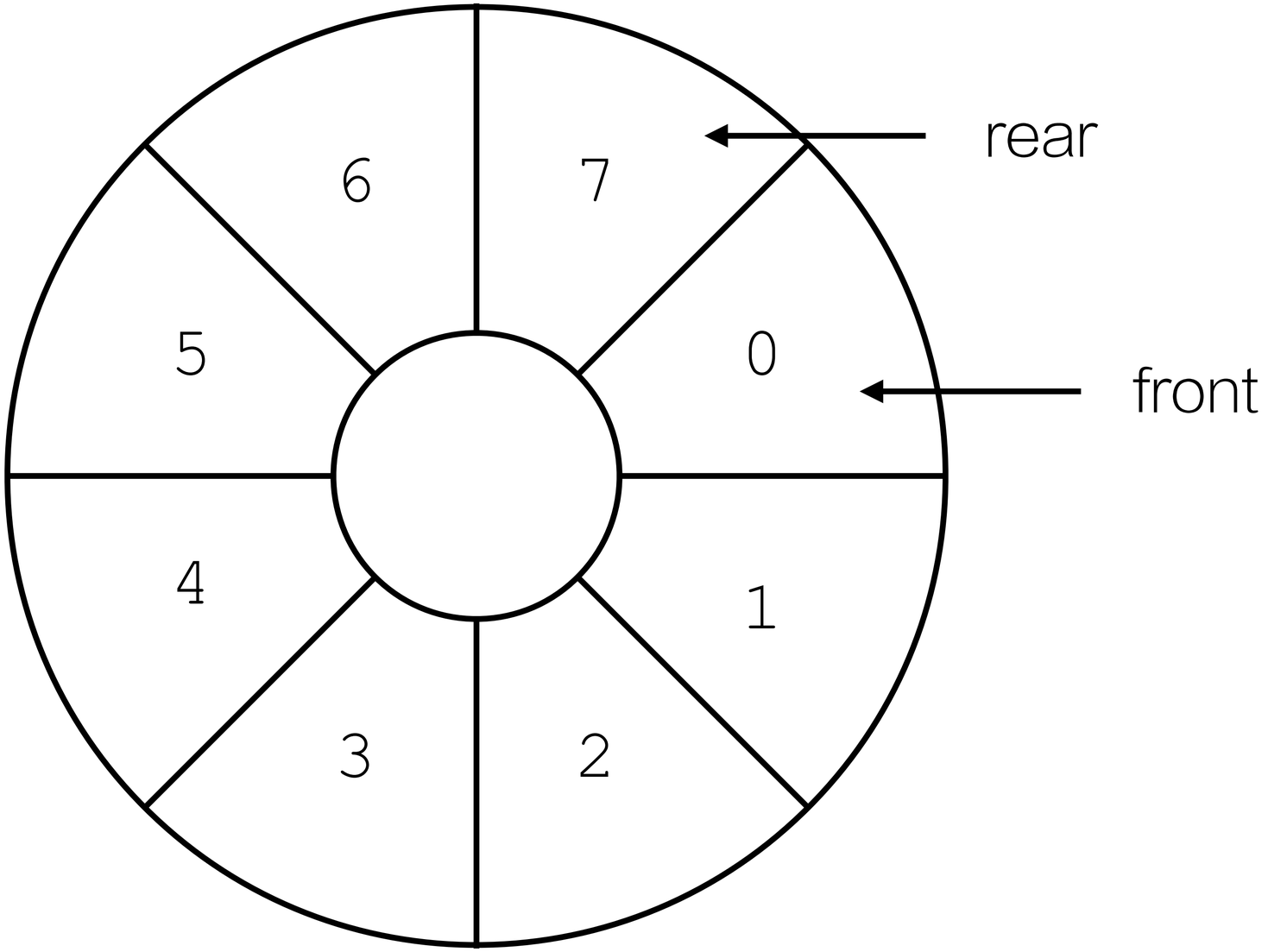
案例：栈与队列 / 队列 / test3.c

```
int equeue(Queue *Q, ElemType e)
{
    Q->rear = (Q->rear + 1) % MAXSIZE;
    if ((Q->rear + 1) % MAXSIZE == Q->front)
    {
        printf("满了\n");
        return 0;
    }
    Q->data[Q->rear] = e;
    return 1;
}
```

队列的顺序结构 – 循环队列

满队列：(rear + 1) % MAXSIZE == front

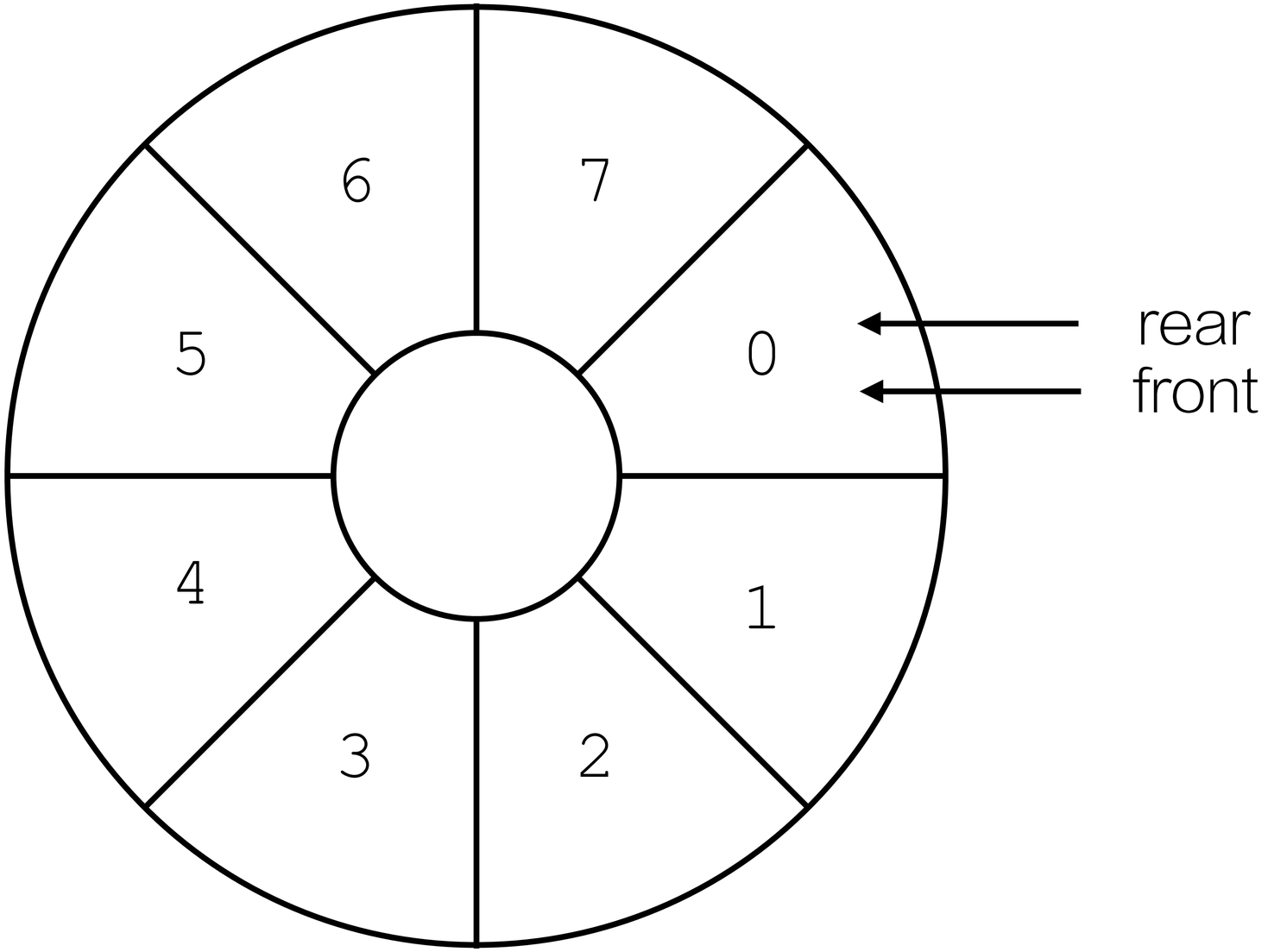
队尾：(rear + 1) % MAXSIZE



队列的顺序结构 – 循环队列

满队列：(rear + 1) % MAXSIZE == front

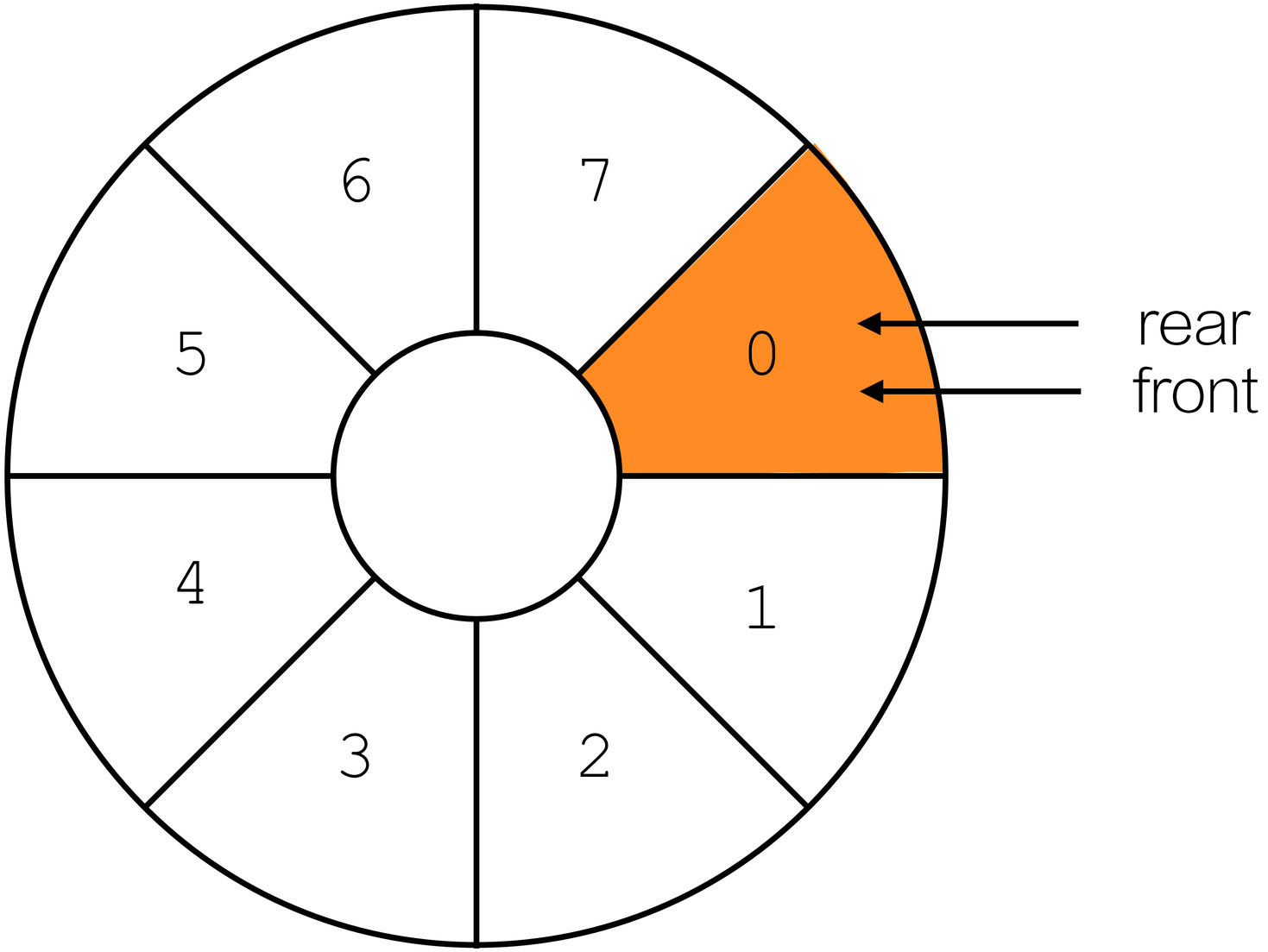
队尾：(rear + 1) % MAXSIZE



队列的顺序结构 – 循环队列

满队列：(rear + 1) % MAXSIZE == front

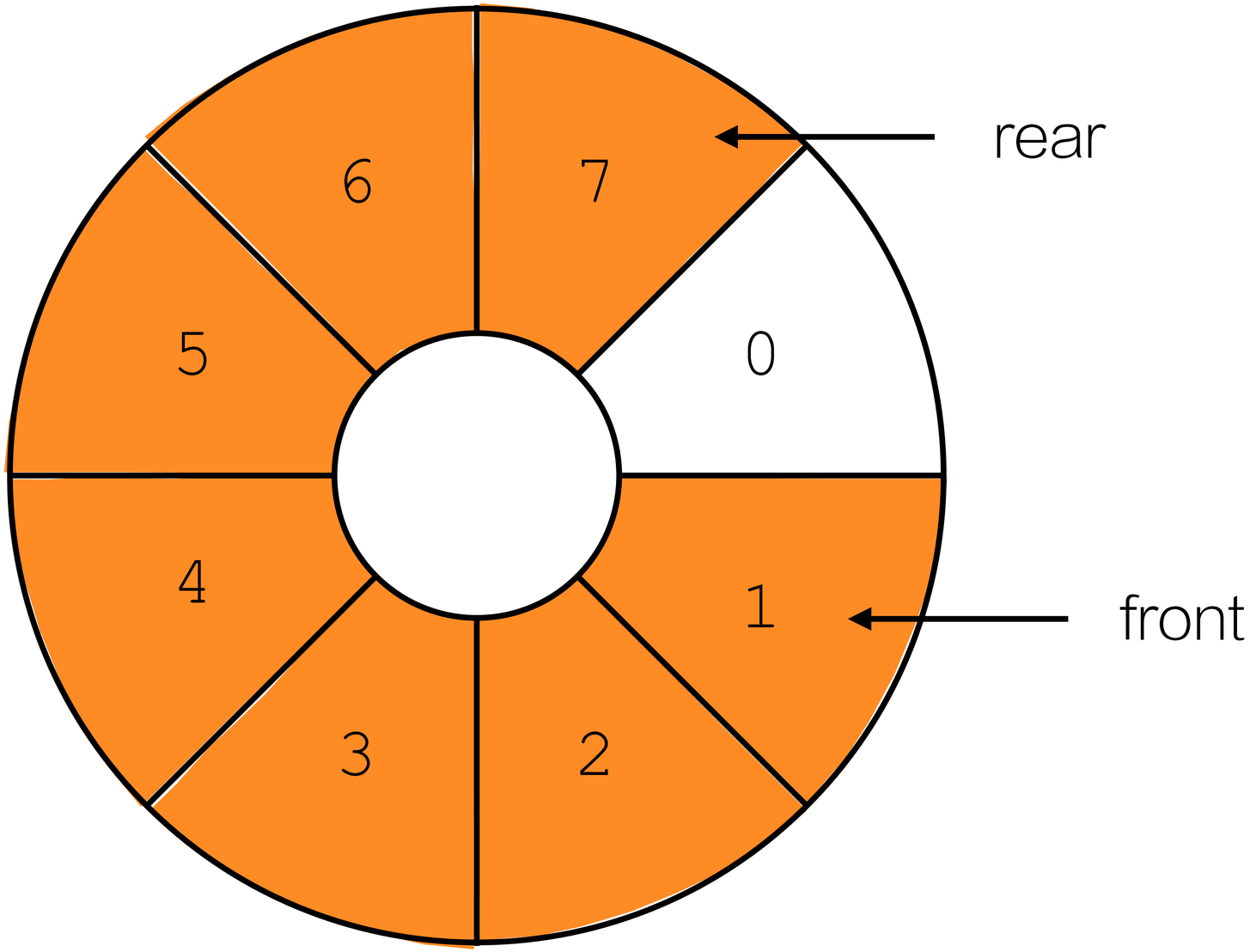
队尾：(rear + 1) % MAXSIZE



队列的顺序结构 – 循环队列

满队列：(rear + 1) % MAXSIZE == front

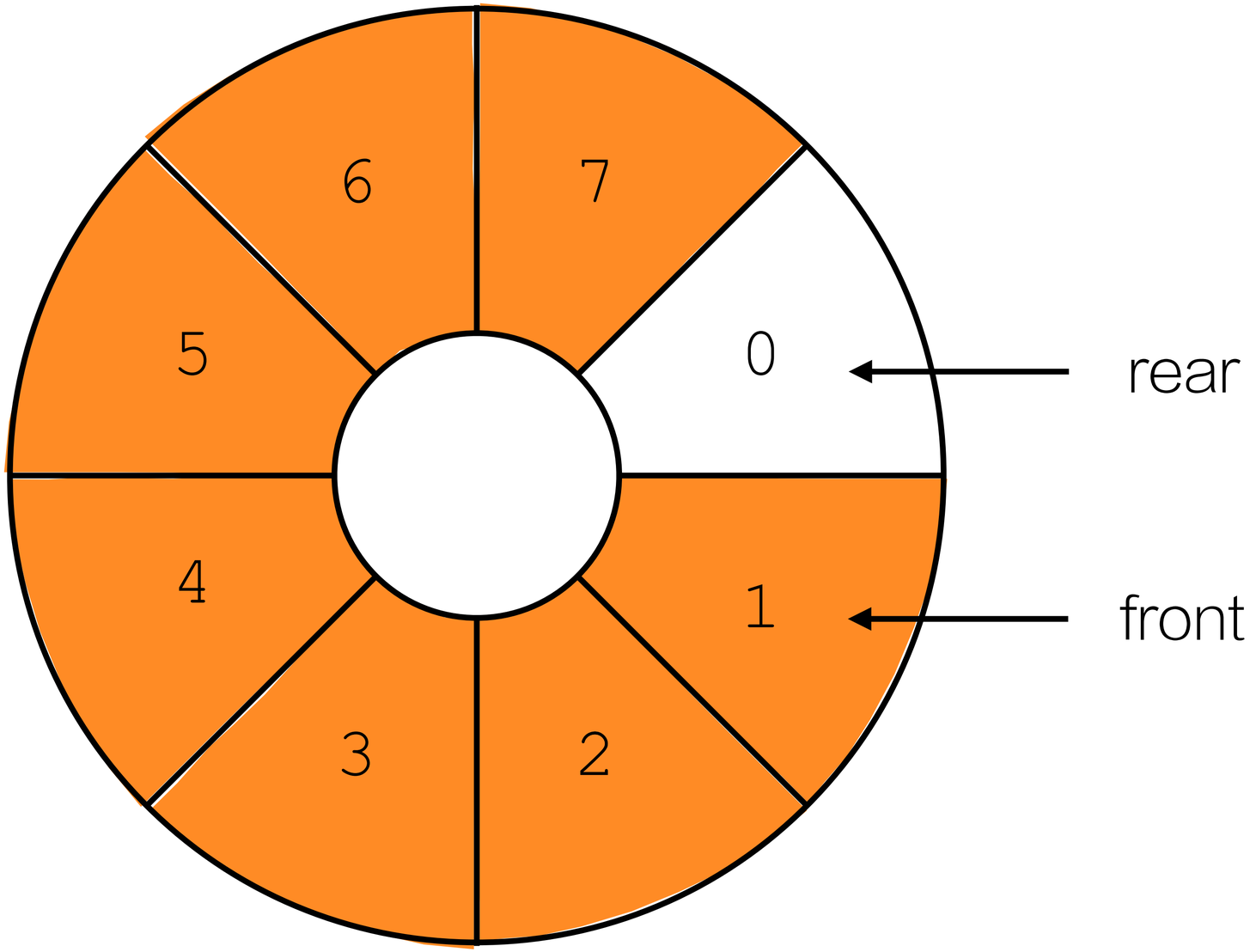
队尾：(rear + 1) % MAXSIZE



队列的顺序结构 – 循环队列

满队列：(rear + 1) % MAXSIZE == front

队尾：(rear + 1) % MAXSIZE



队列的顺序结构 – 循环队列 – 出队

案例：栈与队列 / 队列 / test3.c

```
int dequeue(Queue *Q, ElemType *e)
{
    if (Q->front == Q->rear)
    {
        printf("空的\n");
        return 0;
    }
    *e = Q->data[Q->front];
    Q->front = (Q->front + 1) % MAXSIZE;
    return 1;
}
```

队列的链式结构

```
typedef struct QueueNode
{
    ElemType data;
    struct QueueNode *next;
}QueueNode;

typedef struct
{
    QueueNode *front;
    QueueNode *rear;
}Queue;
```

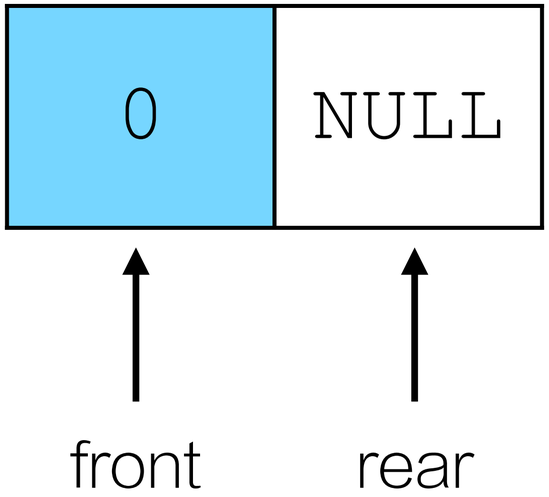
案例：栈与队列 / 队列 / test4.c

队列的链式结构 – 初始化

案例：栈与队列 / 队列 / test4.c

```
Queue* initQueue()
{
    Queue *q = (Queue*)malloc(sizeof(Queue));
    QueueNode *node = (QueueNode*)malloc(sizeof(QueueNode));
    node->data = 0;
    node->next = NULL;
    q->front = node;
    q->rear = node;
    return q;
}
```

队列的链式结构 – 初始化

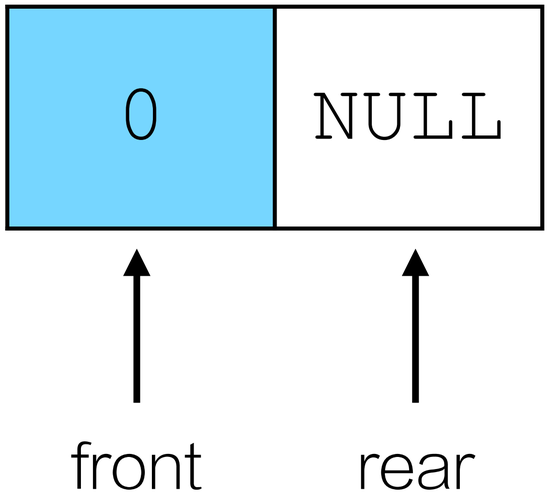


队列的链式结构 – 判断队列是否为空

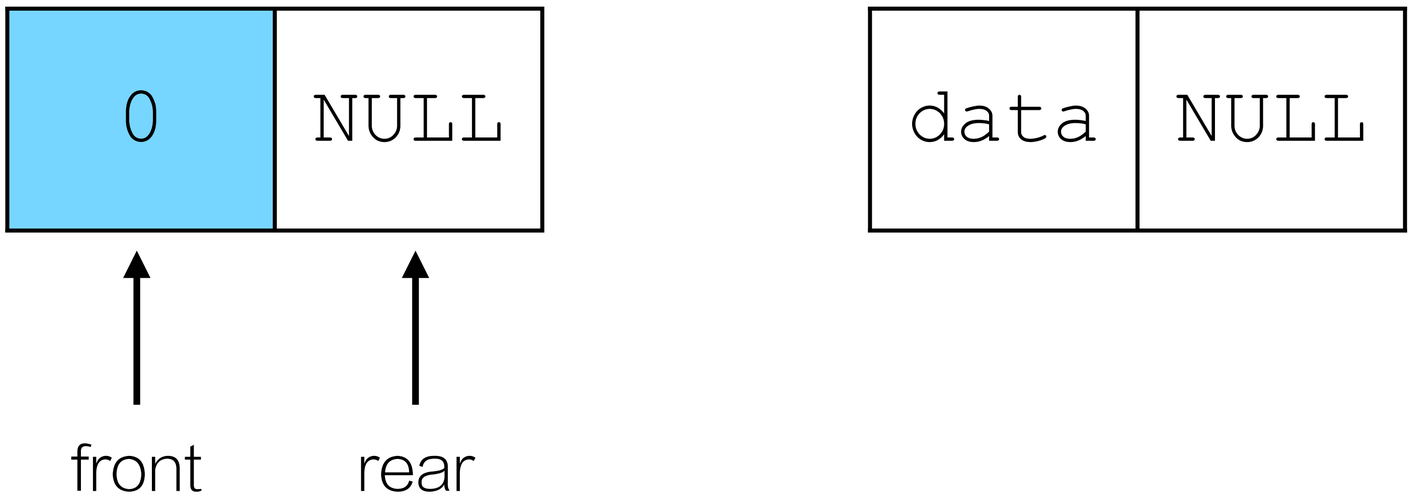
```
int isEmpty(Queue *q)
{
    if (q->front == q->rear)
    {
        return 1;
    }
    else
    {
        return 0;
    }
}
```

案例：栈与队列 / 队列 / test4.c

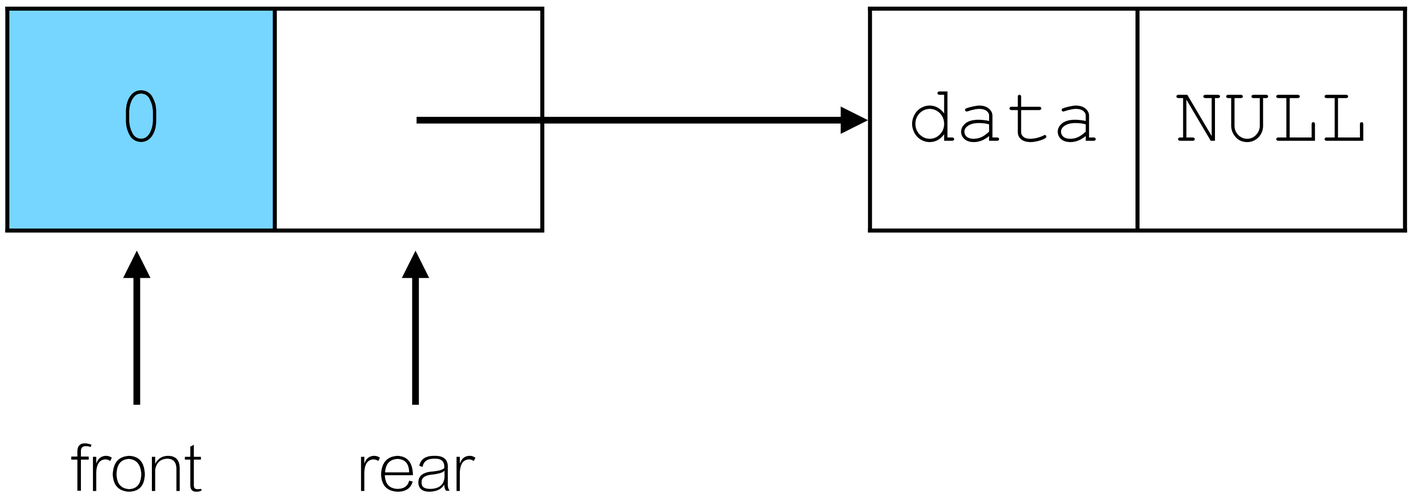
队列的链式结构 – 入队



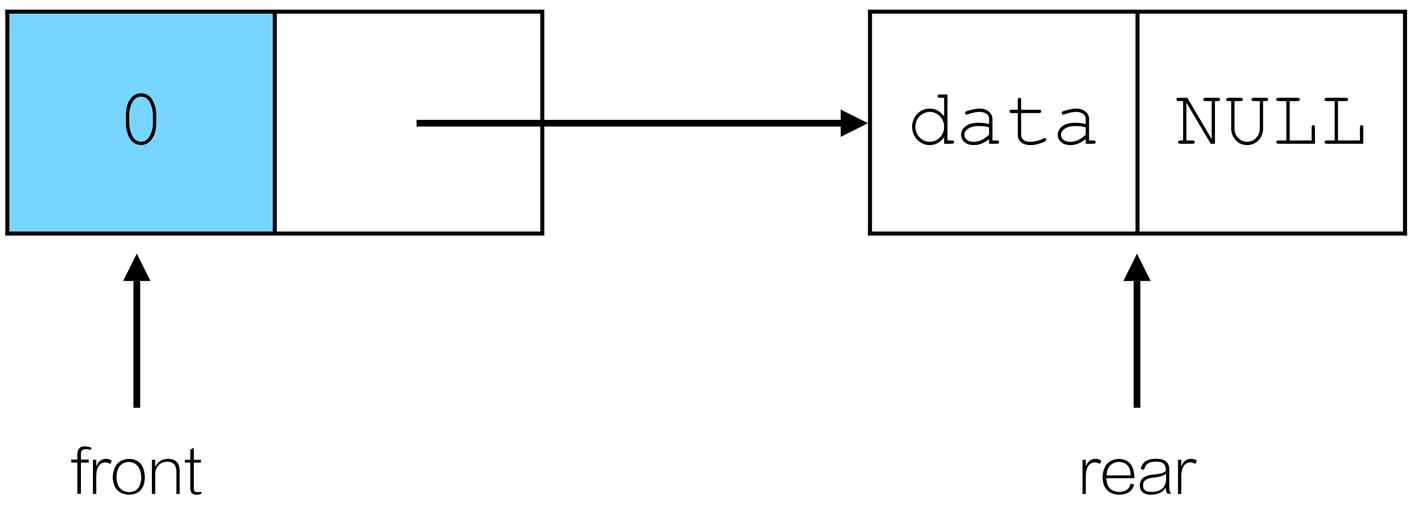
队列的链式结构 – 入队



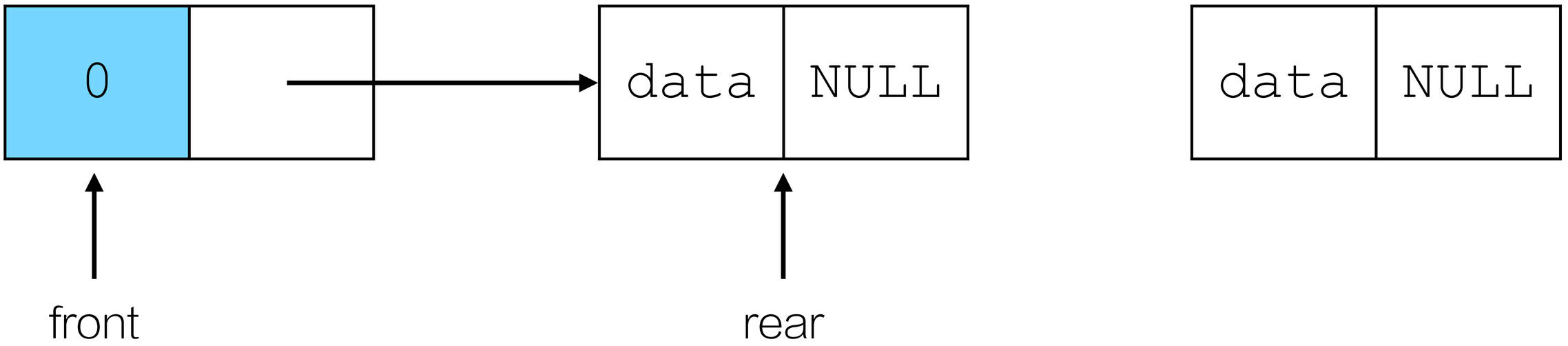
队列的链式结构 – 入队



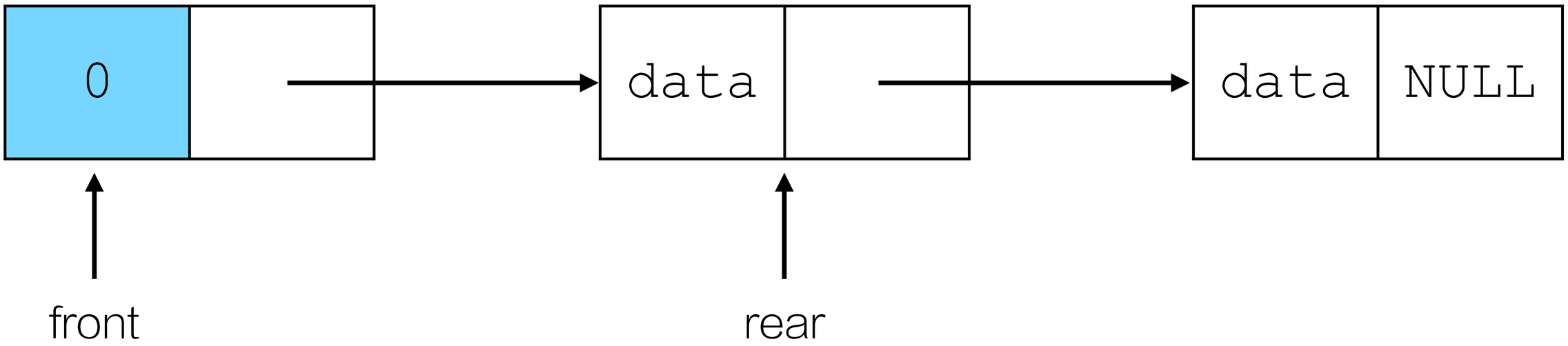
队列的链式结构 – 入队



队列的链式结构 – 入队



队列的链式结构 – 入队



队列的链式结构 – 入队



队列的链式结构 – 入队

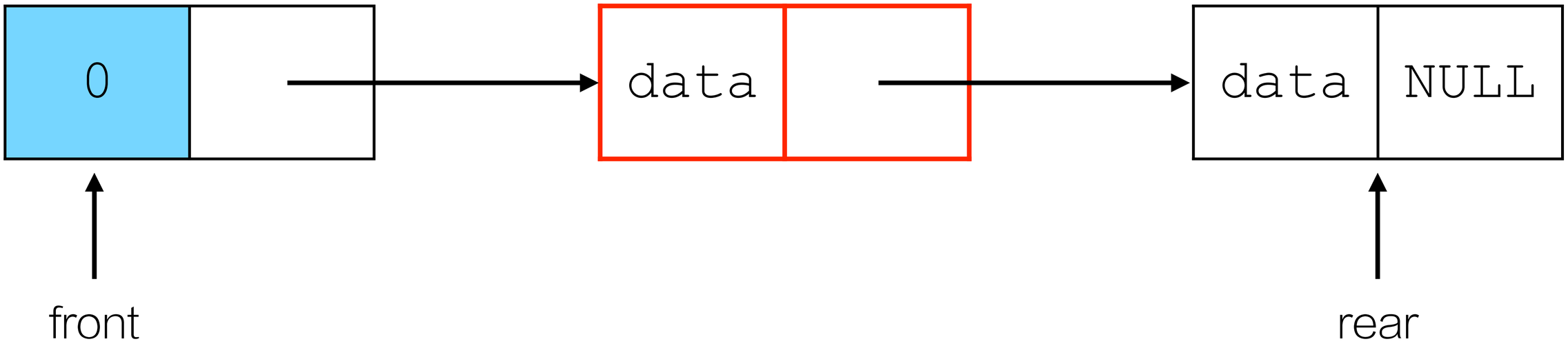
案例：栈与队列 / 队列 / test4.c

```
void equeue(Queue *q, ElemType e)
{
    QueueNode *node = (QueueNode*)malloc(sizeof(QueueNode));
    node->data = e;
    node->next = NULL;
    q->rear->next = node;
    q->rear = node;
}
```

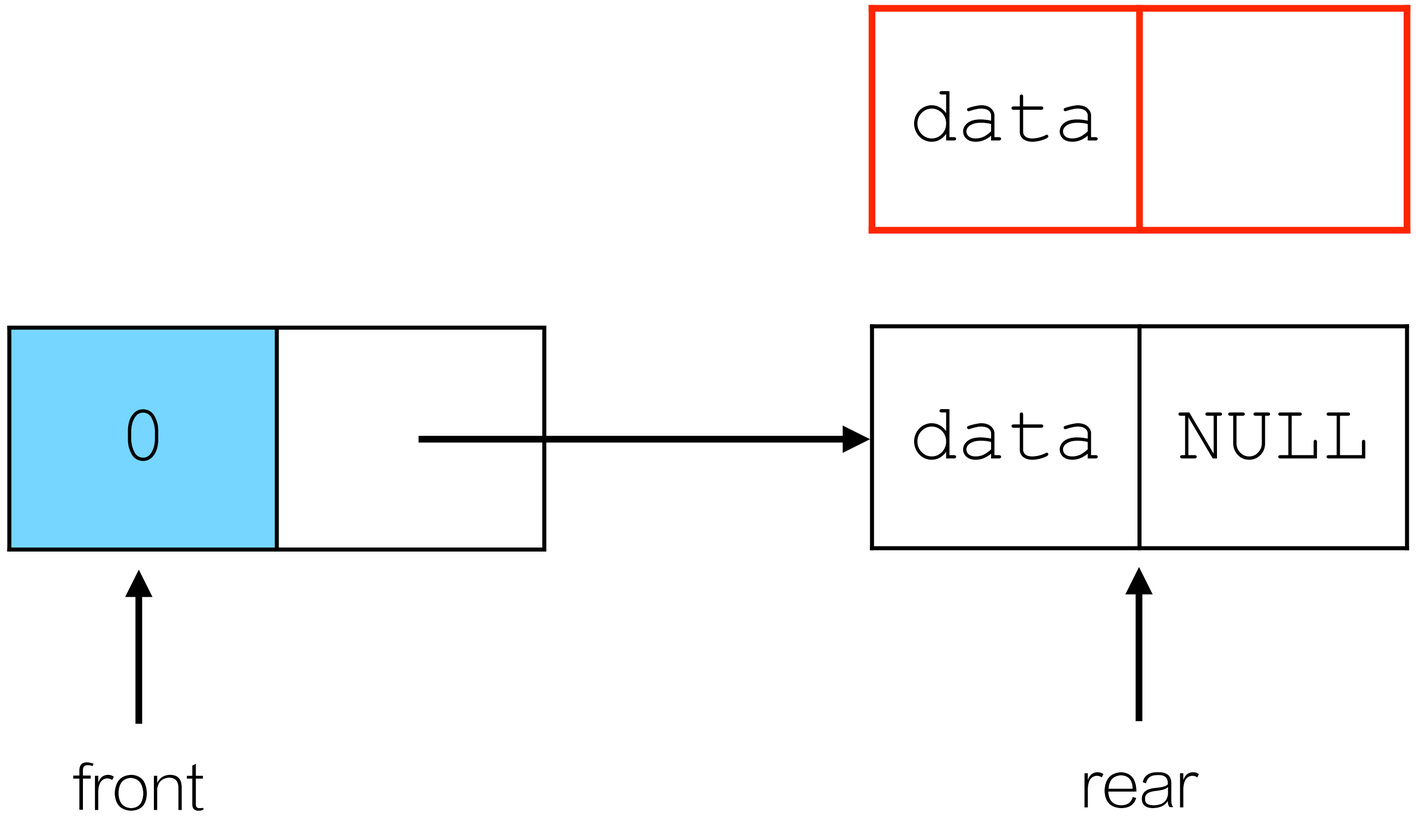
队列的链式结构 – 出队



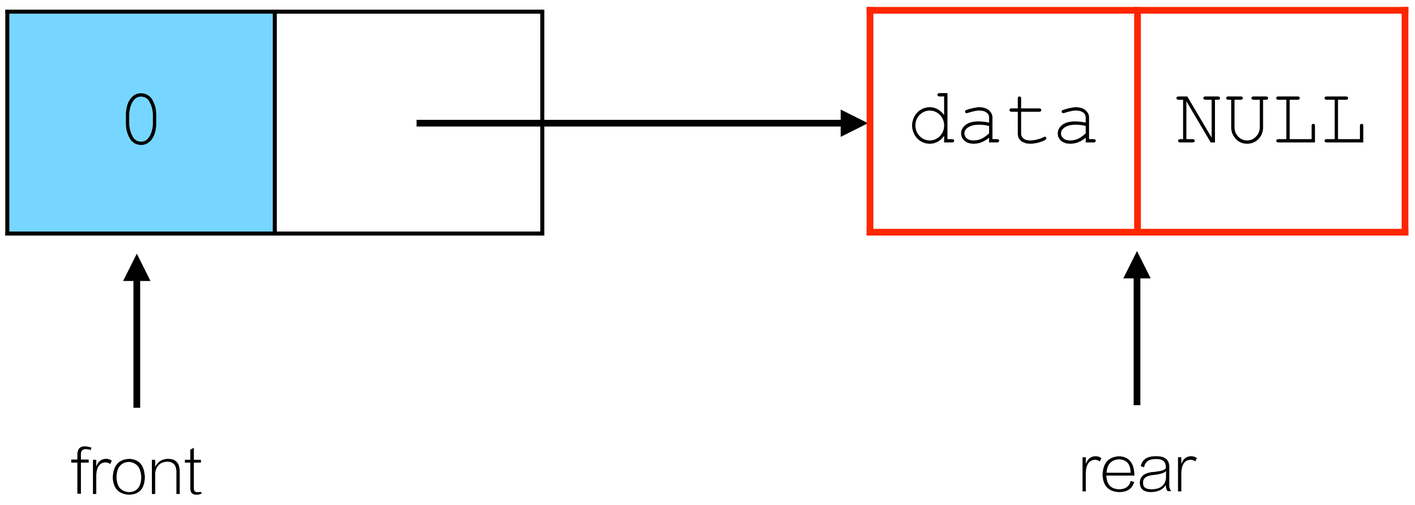
队列的链式结构 – 出队



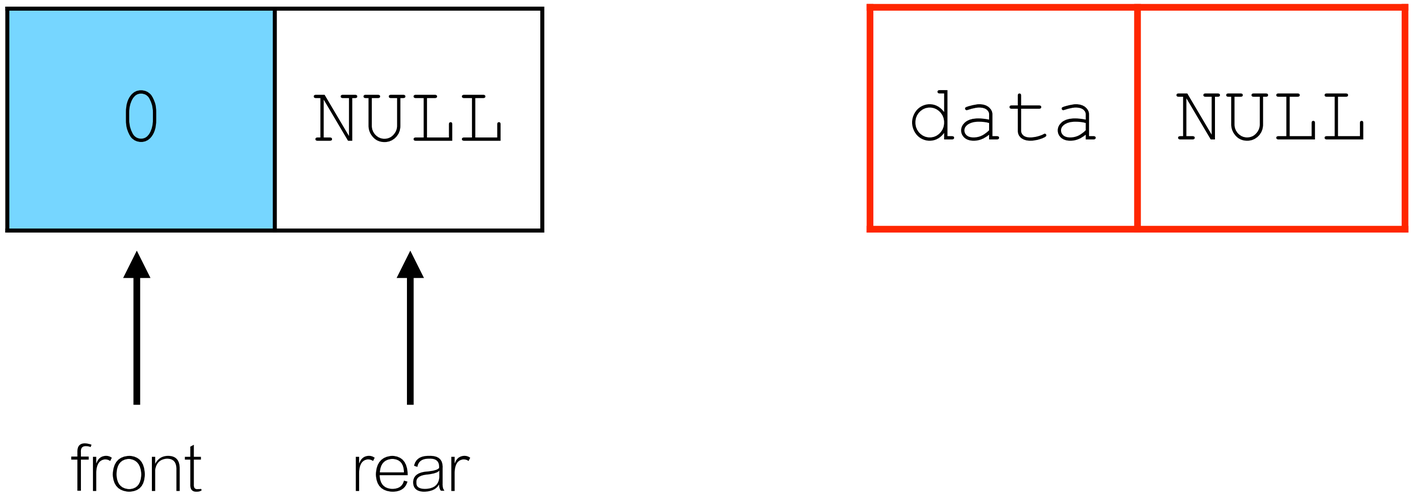
队列的链式结构 – 出队



队列的链式结构 – 出队



队列的链式结构 – 出队



队列的链式结构 – 出队

```
int dequeue(Queue *q, ElemType *e)
{
    QueueNode *node = q->front->next;
    *e = node->data;
    q->front->next = node->next;
    if (q->rear == node)
    {
        q->rear = q->front;
    }
    free(node);
    return 1;
}
```

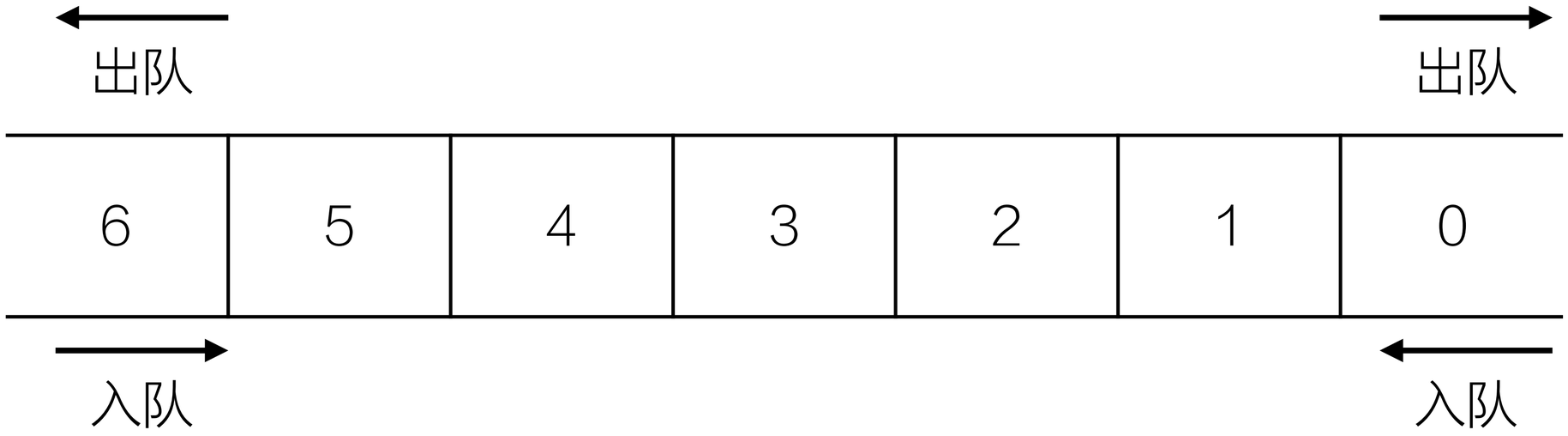
案例：栈与队列 / 队列 / test4.c

队列的链式结构 – 获取队头元素

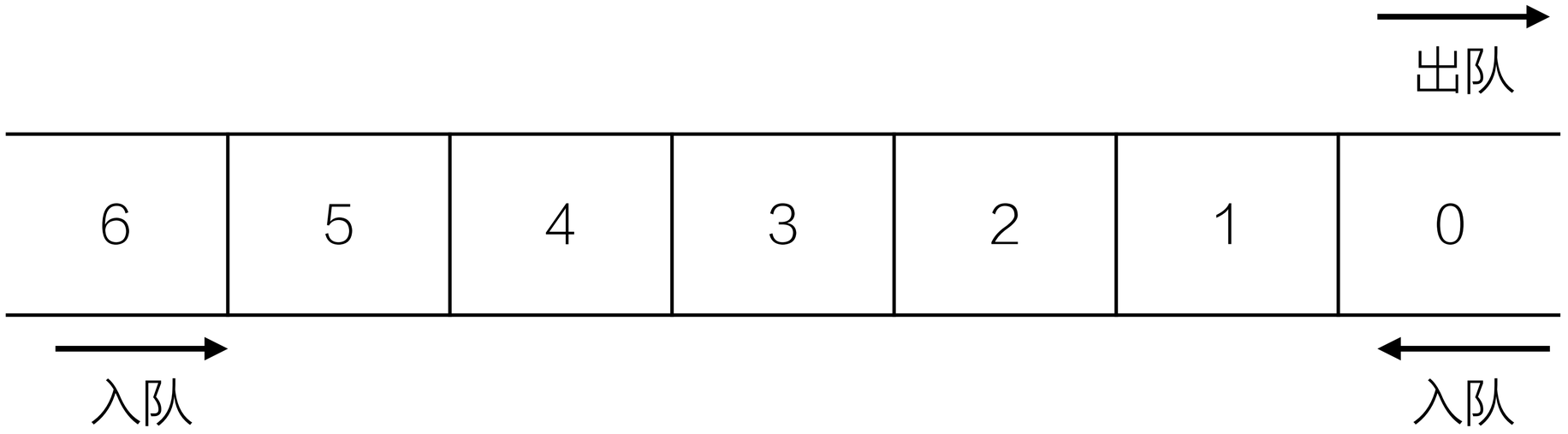
案例：栈与队列 / 队列 / test4.c

```
ElemType getFront(Queue *q)
{
    if (isEmpty(q))
    {
        printf("空的\n");
        return 0;
    }
    return q->front->next->data;
}
```

双端队列



双端队列



栈与队列的应用

[2009] 1. 为解决计算机主机与打印机之间速度不匹配问题，通常设置一个打印数据缓冲区，主机将要输入的数据依次写入该缓冲区，而打印机则依次从该缓冲区中取出数据。该缓冲区的逻辑结构应该是_____。

A. 栈　　B. 队列　　C. 树　　D. 图

栈与队列的应用

[2009] 1. 为解决计算机主机与打印机之间速度不匹配问题，通常设置一个打印数据缓冲区，主机将要输入的数据依次写入该缓冲区，而打印机则依次从该缓冲区中取出数据。该缓冲区的逻辑结构应该是_____。

A. 栈　　B. 队列　　C. 树　　D. 图

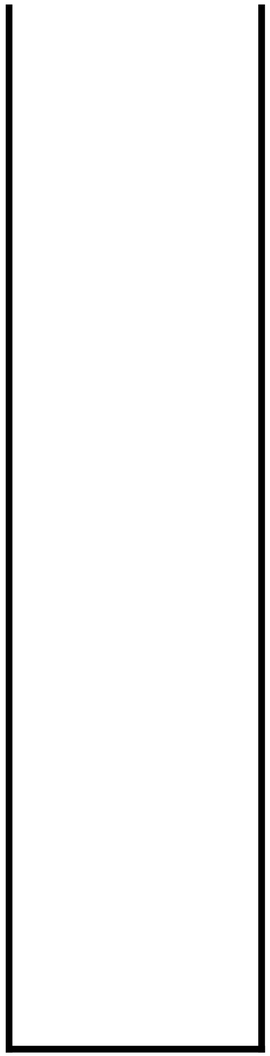
栈与队列的应用

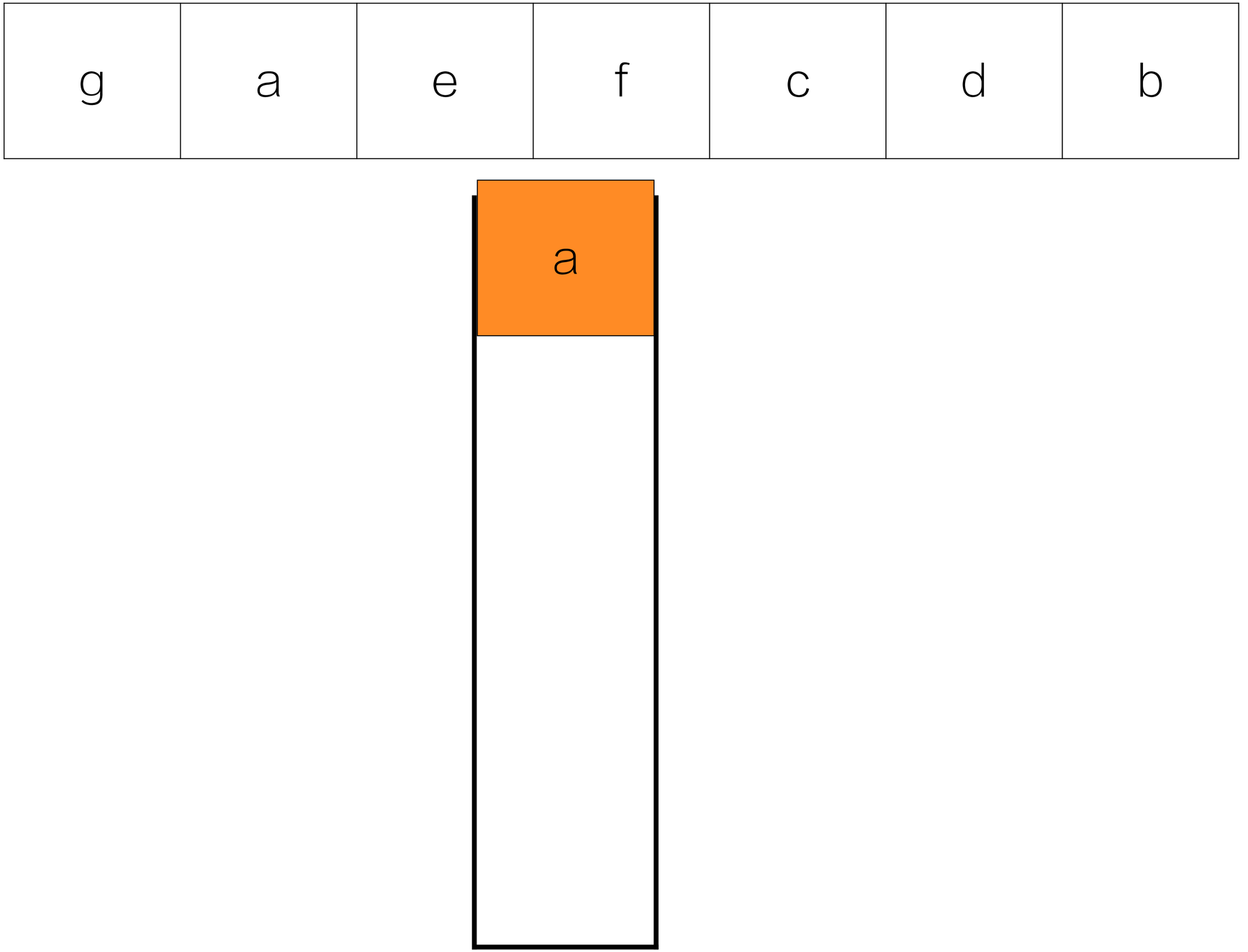
[2009] 2. 设栈 S 和队列 Q 的初始状态均为空，元素 a, b, c, d, e, f, g依次进入栈 S 。若每个元素出栈后立即进入队列 Q，且7个元素出队的顺序是 b, d, c, f, e, a, g，则栈 S 的容量至少是_____。

A. 1　　B. 2　　C. 3　　D. 4

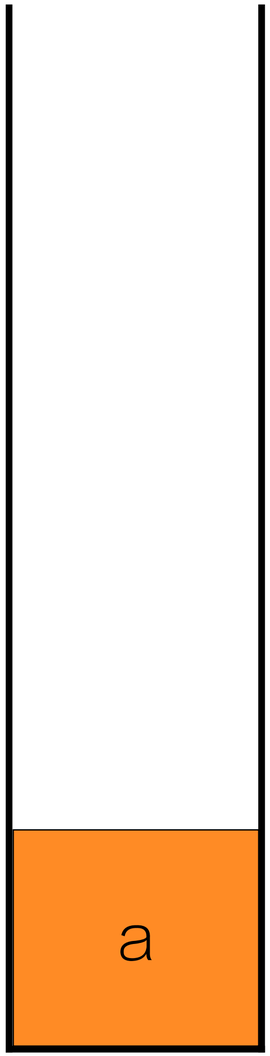
g	a	e	f	c	d	b

g	a	e	f	c	d	b

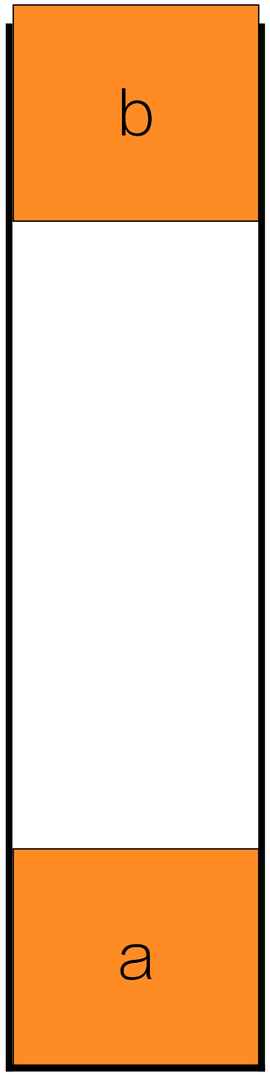



g
a
e
f
c
d
b
a

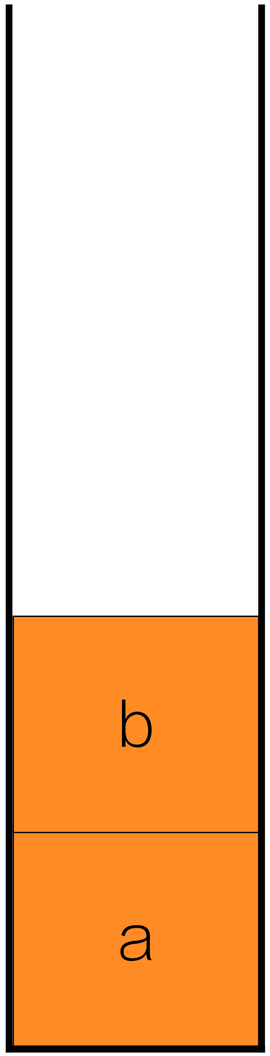
g	a	e	f	c	d	b

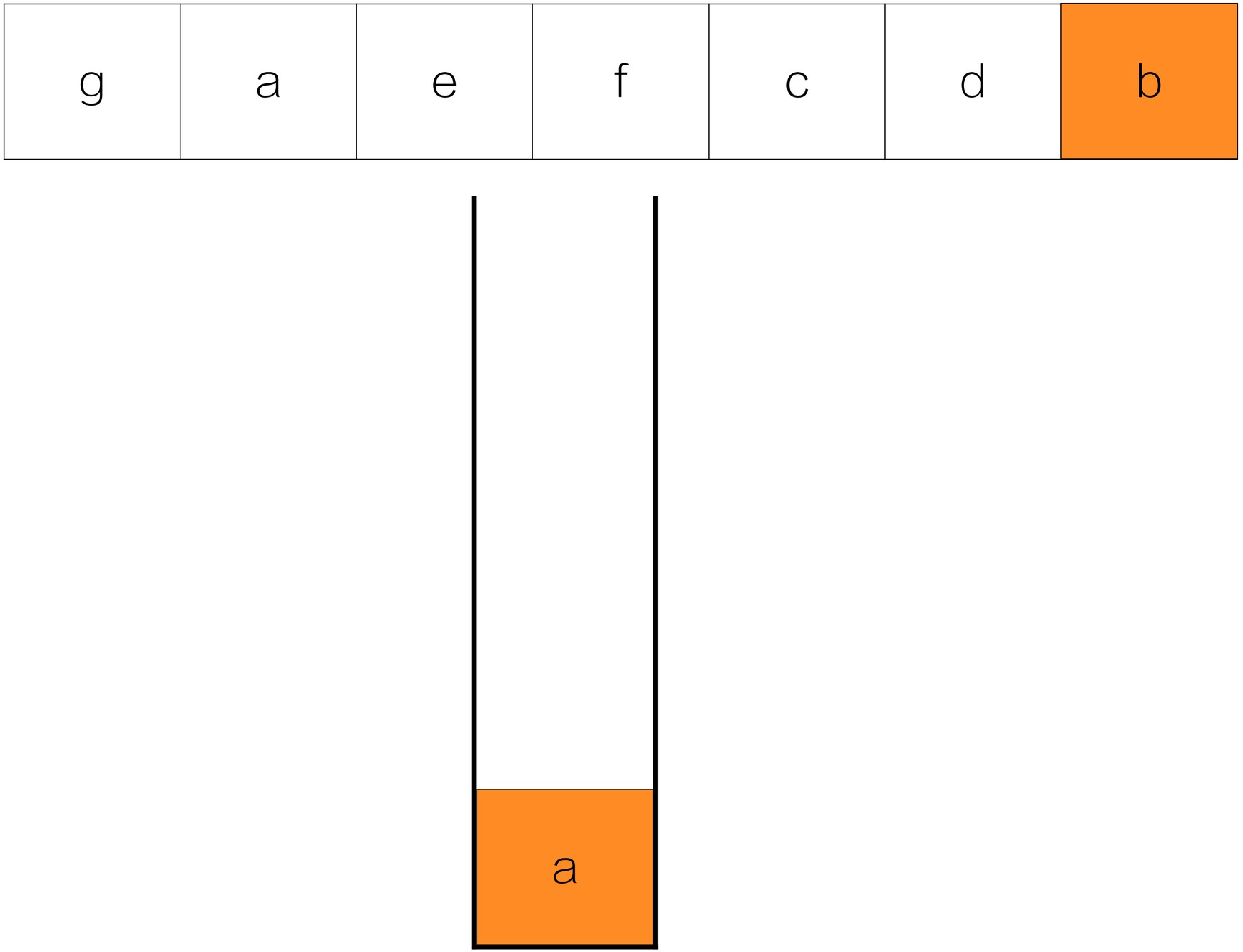


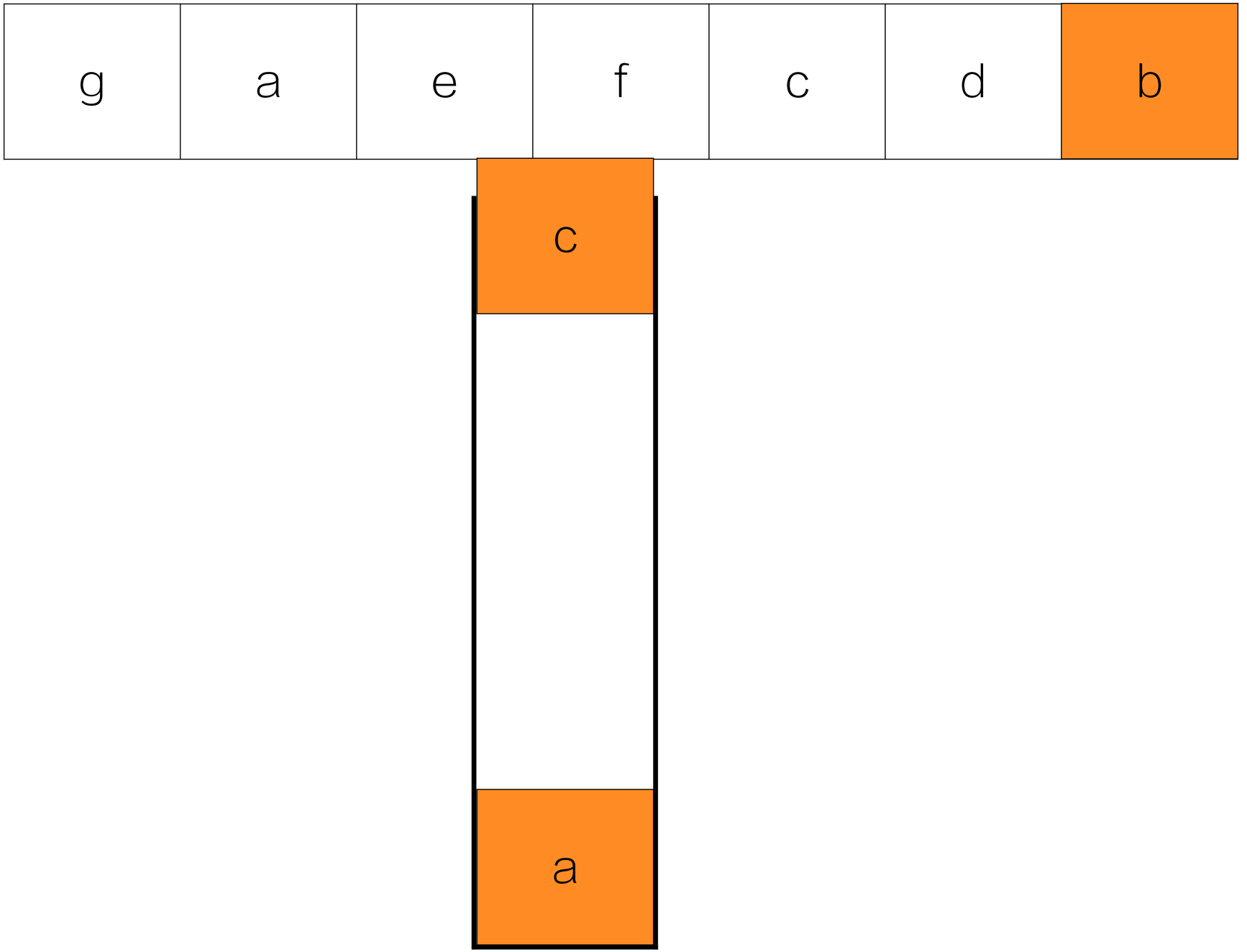
g	a	e	f	c	d	b

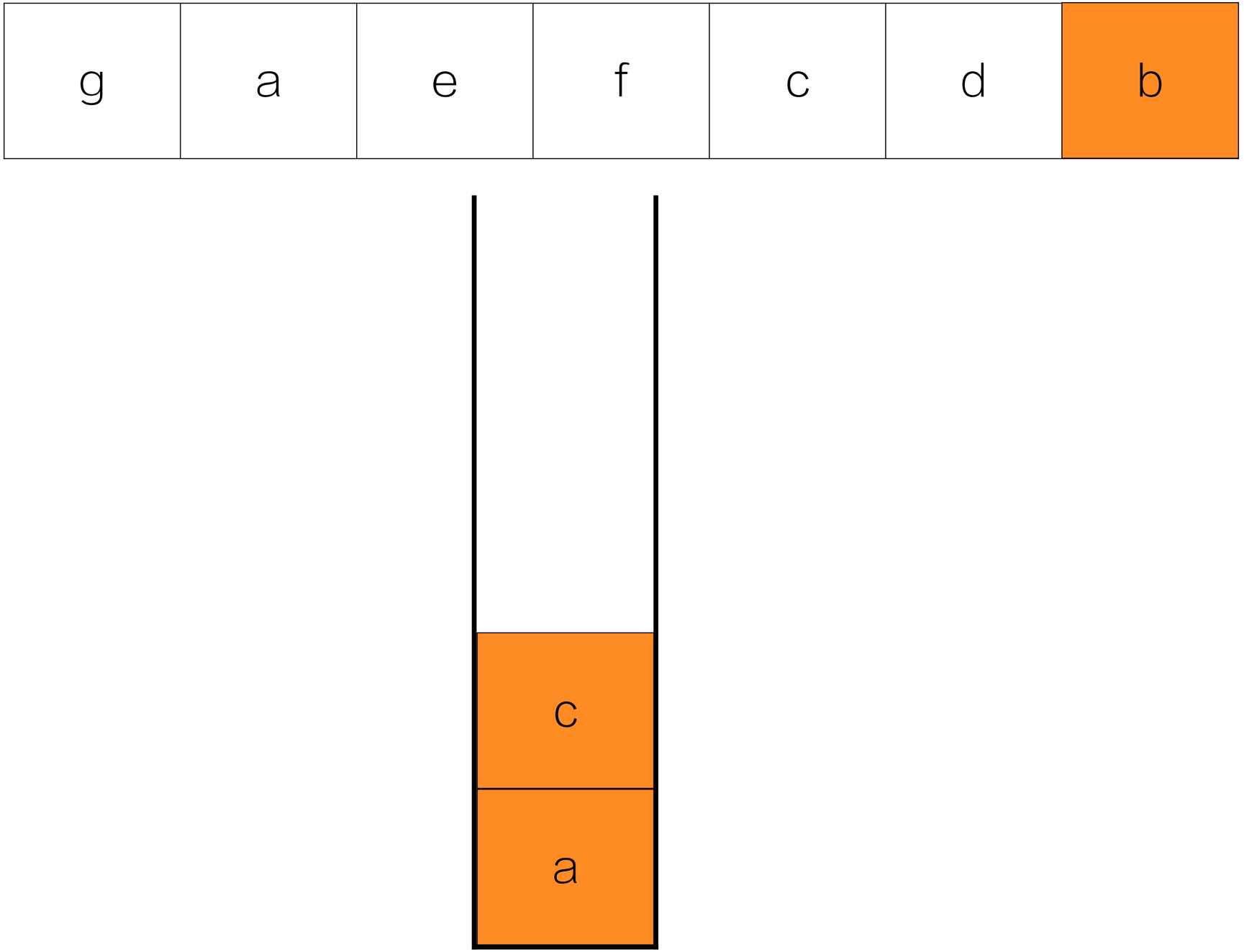


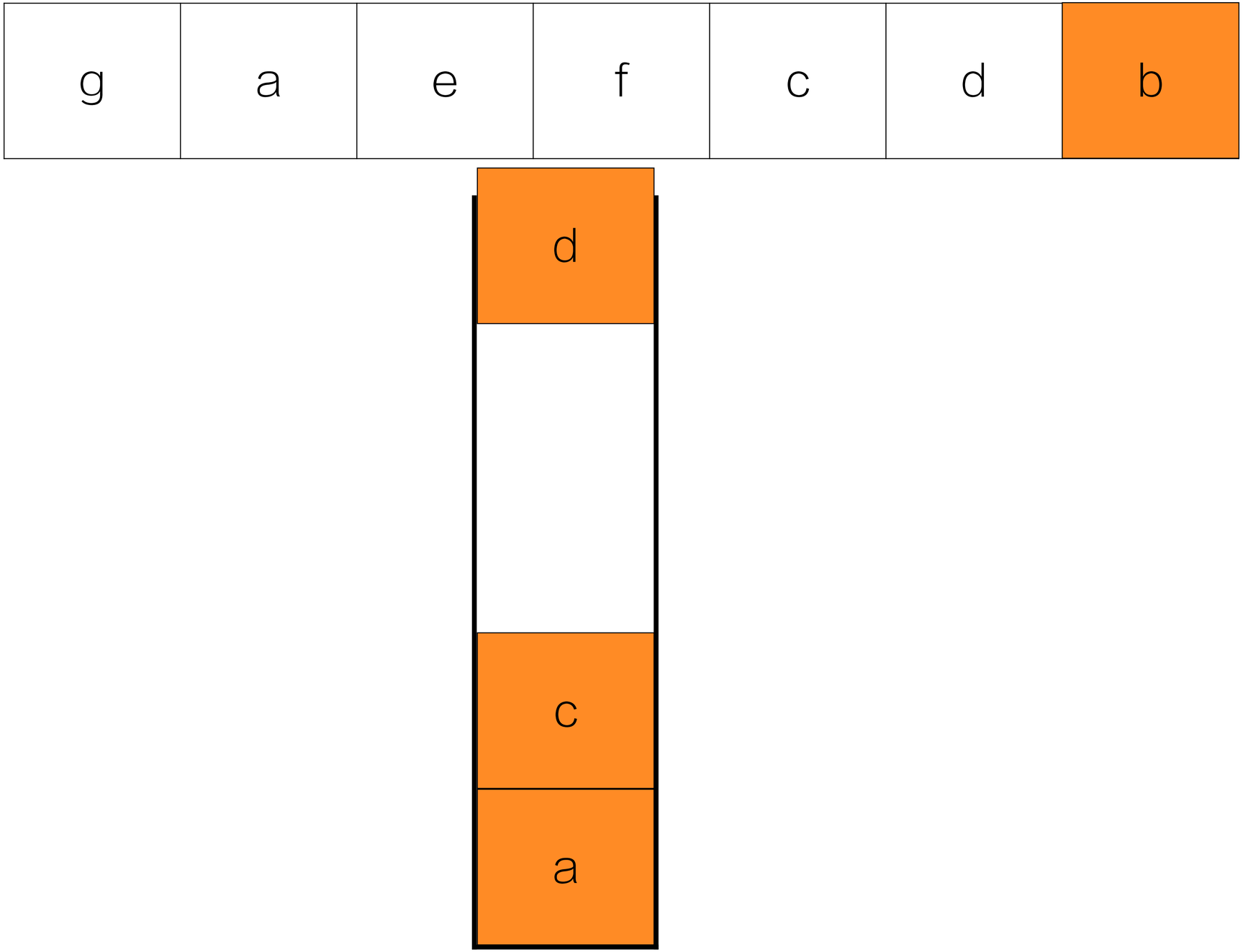
g	a	e	f	c	d	b

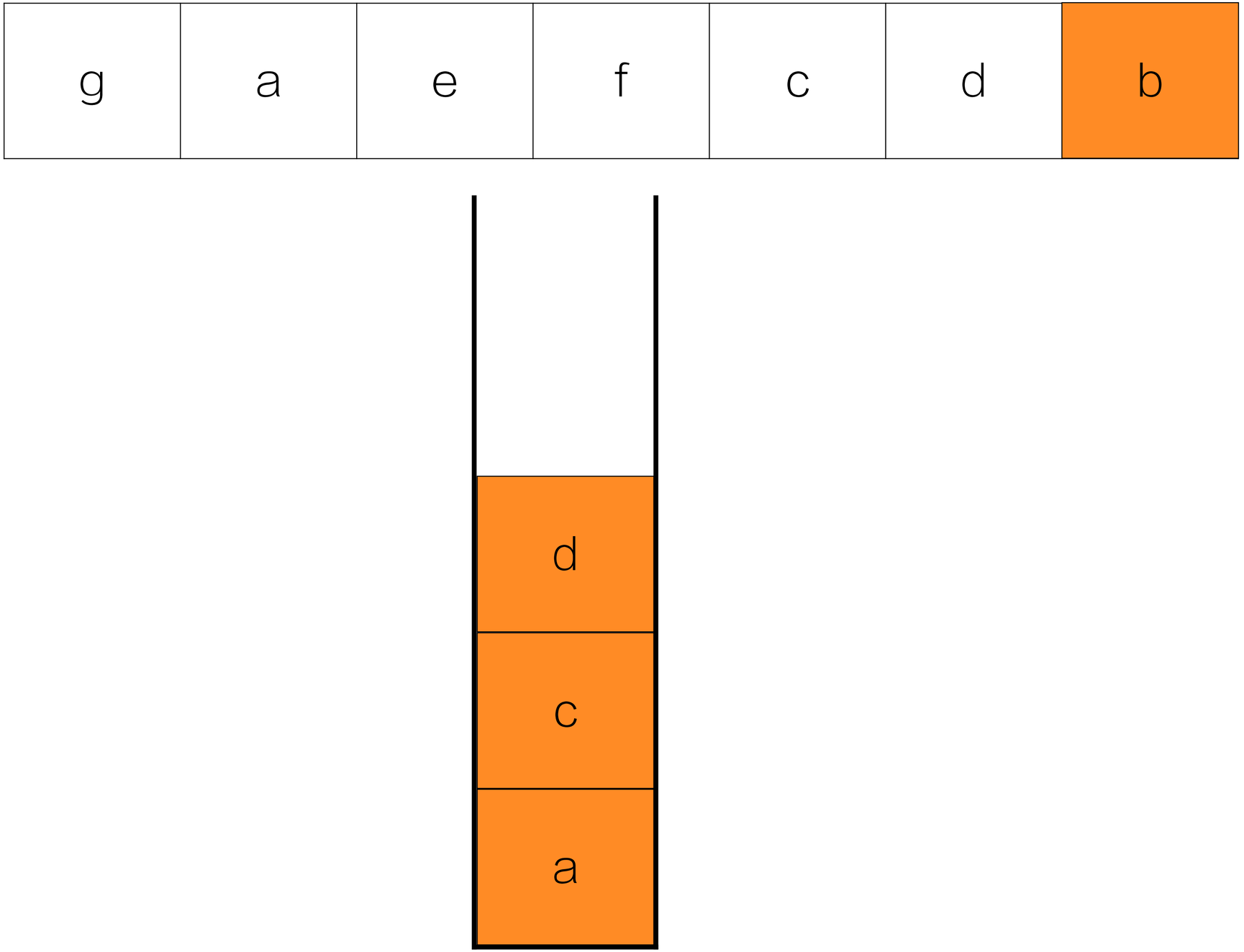


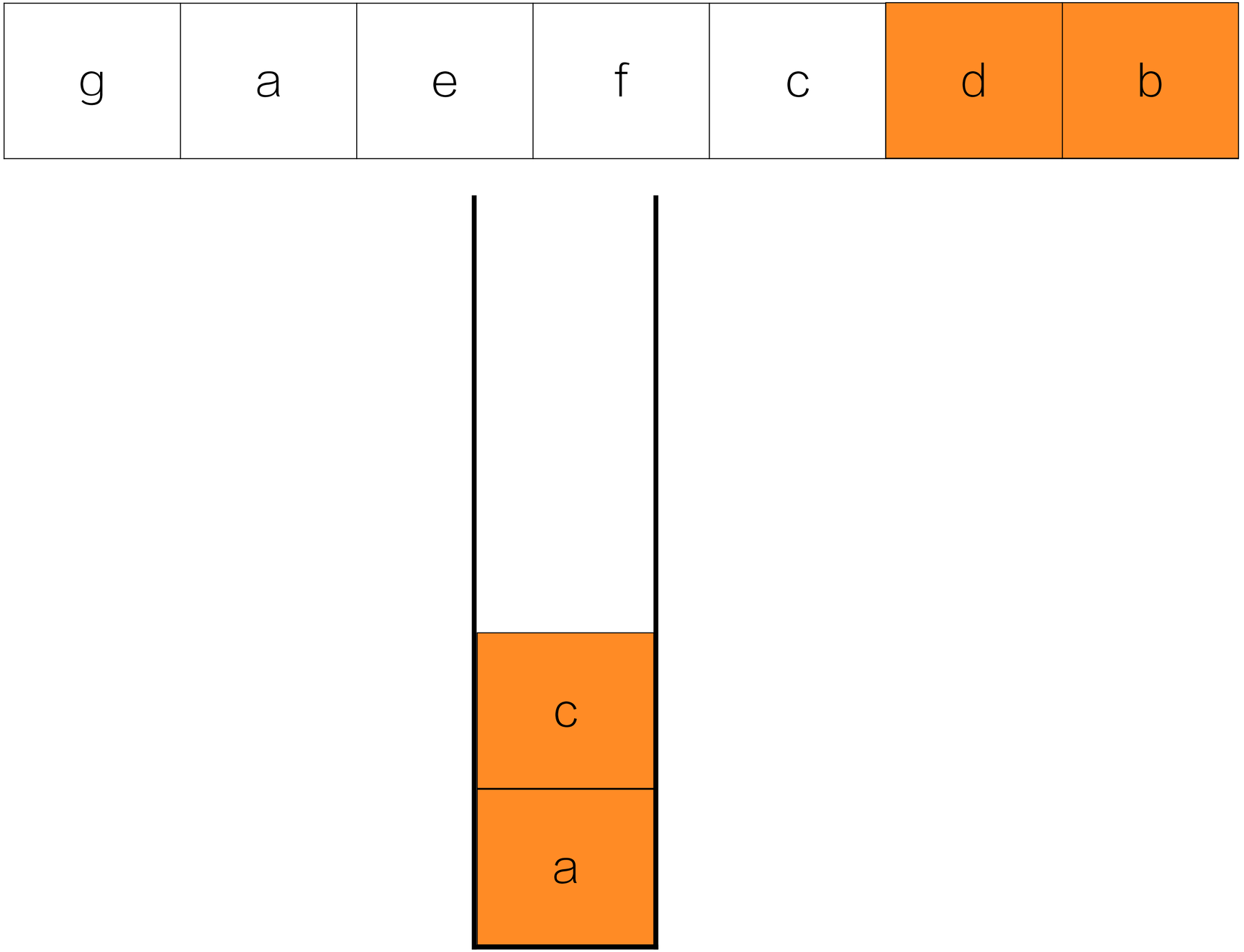

g
a
e
f
c
d
b
a

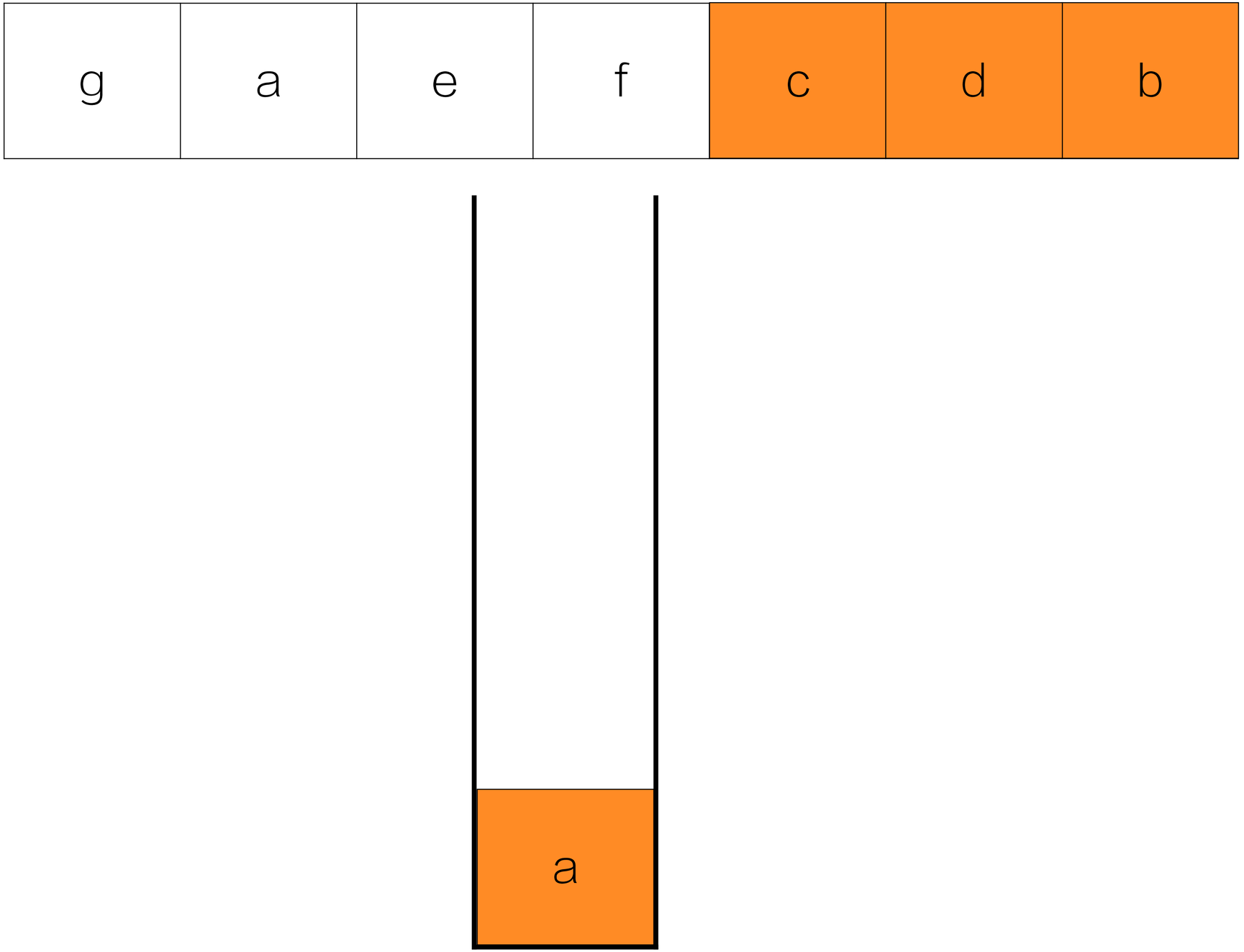

g
a
e
f
c
d
b
c
a

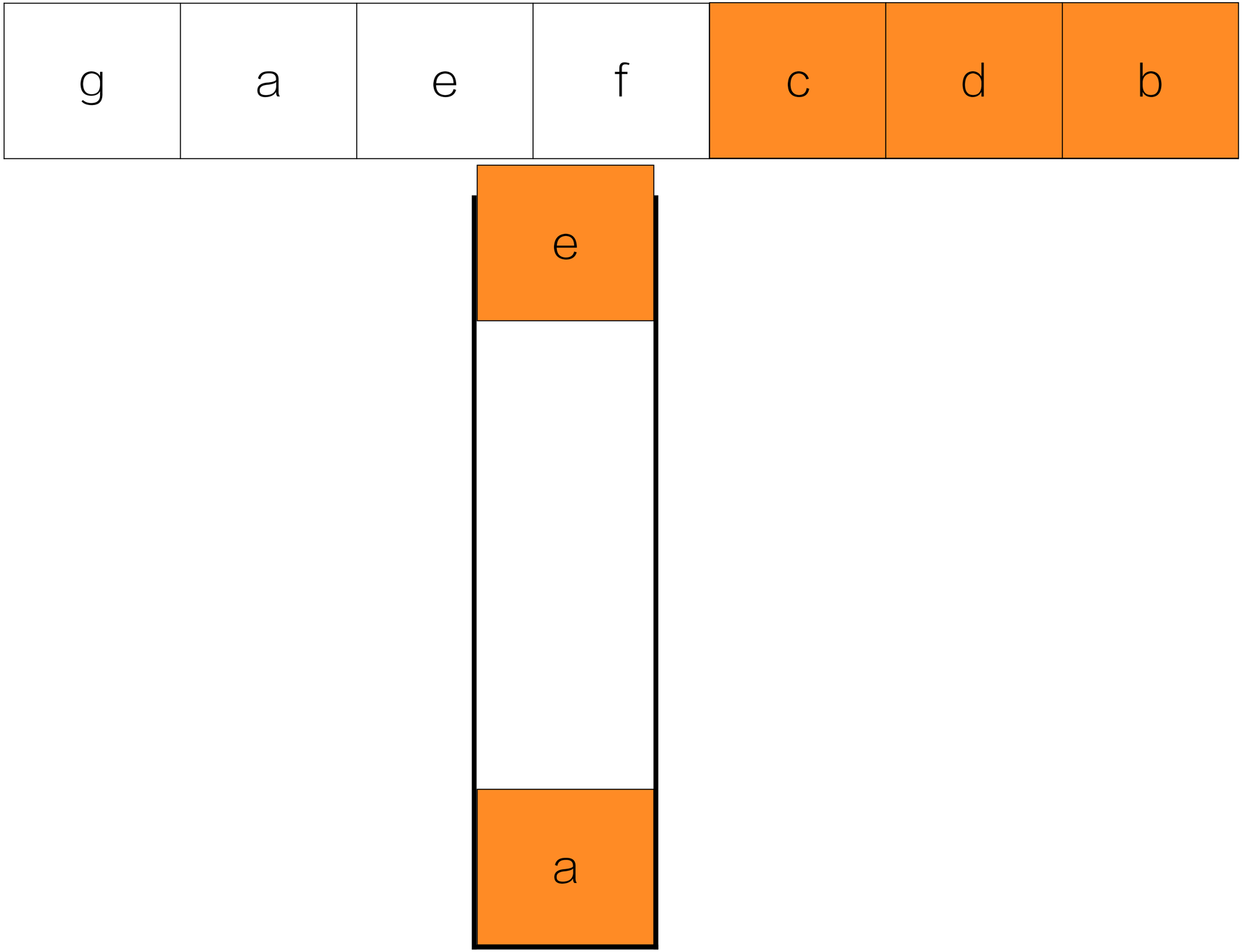

g
a
e
f
c
d
b
c
a

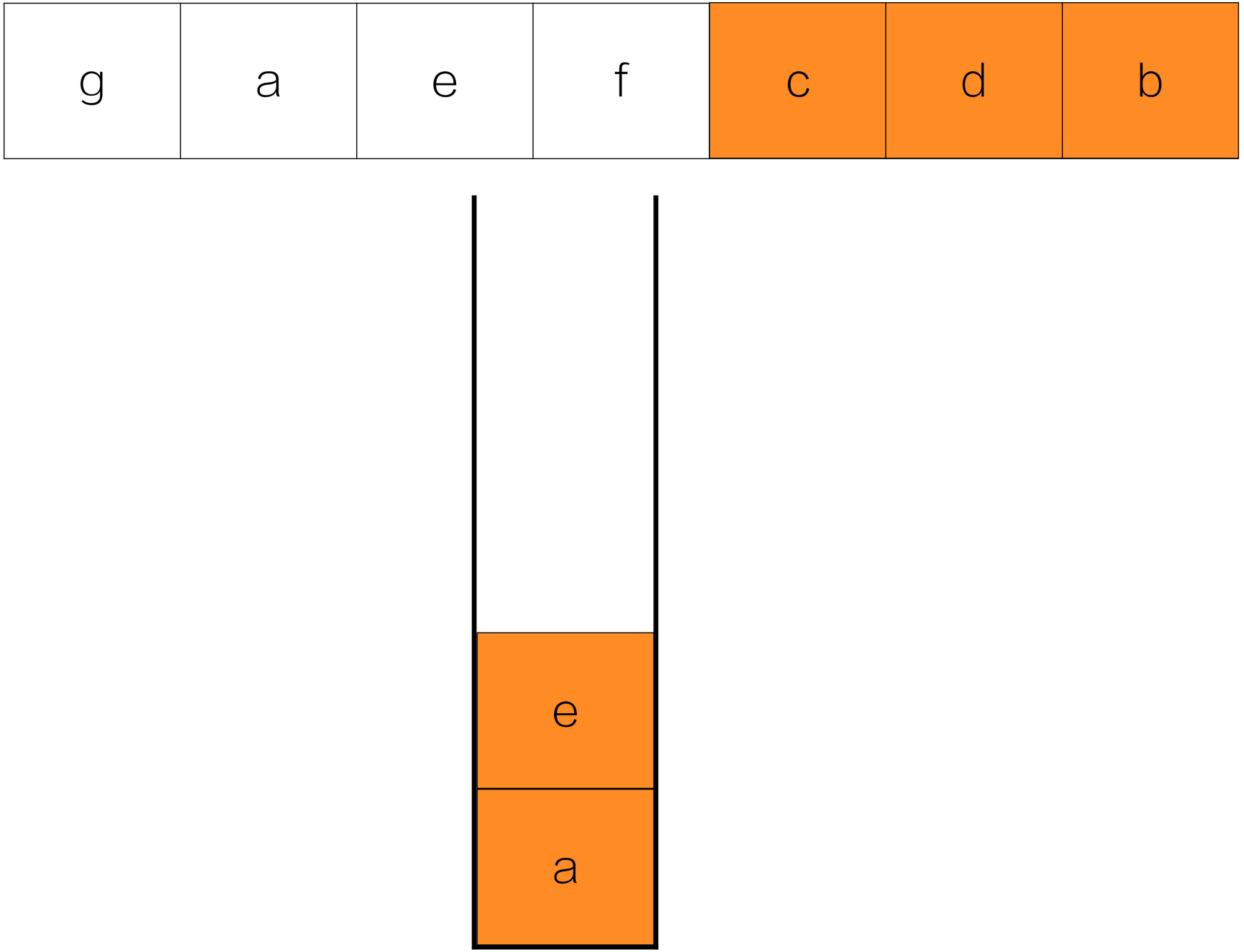

g
a
e
f
c
d
b
d
c
a

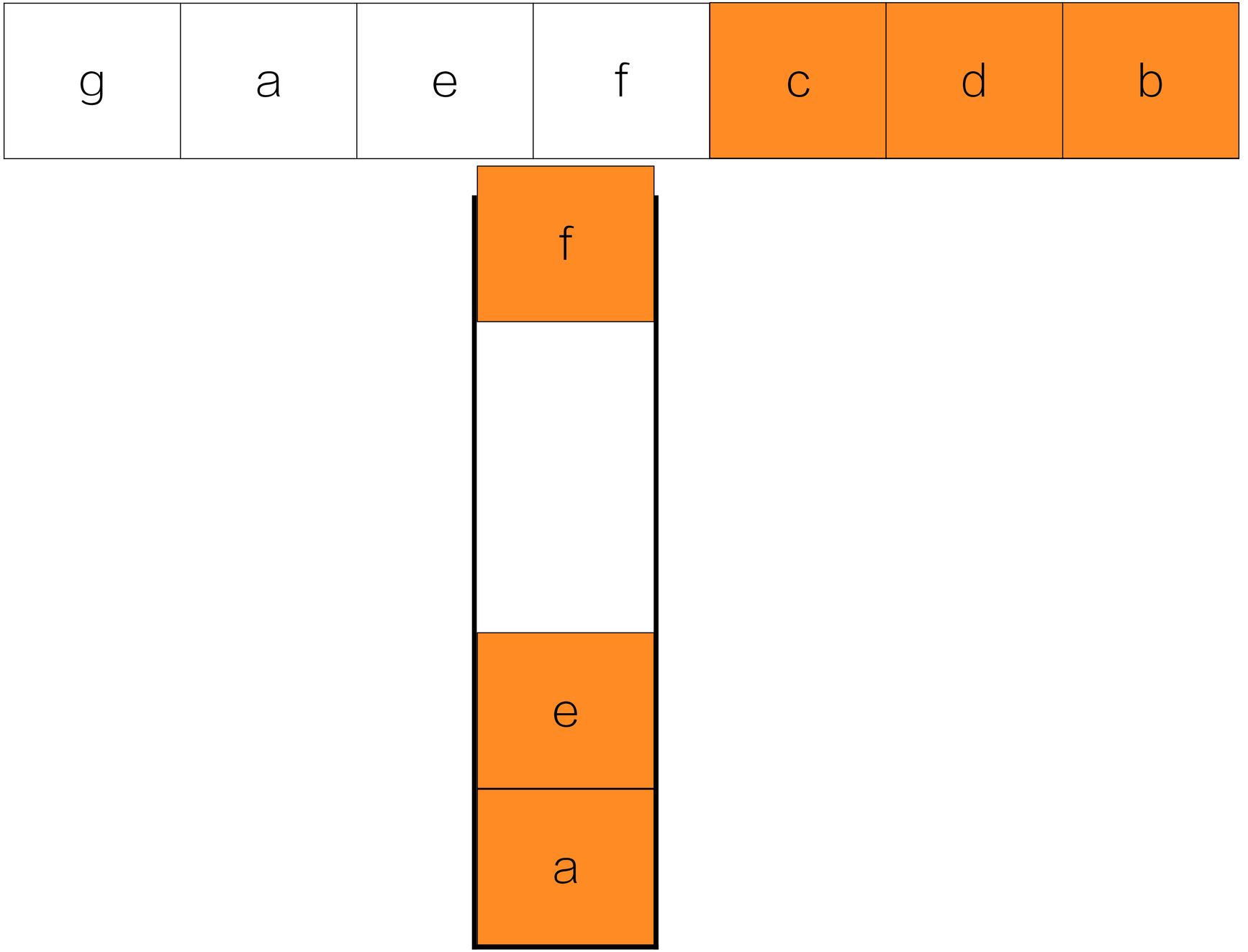

g
a
e
f
c
d
b
d
c
a

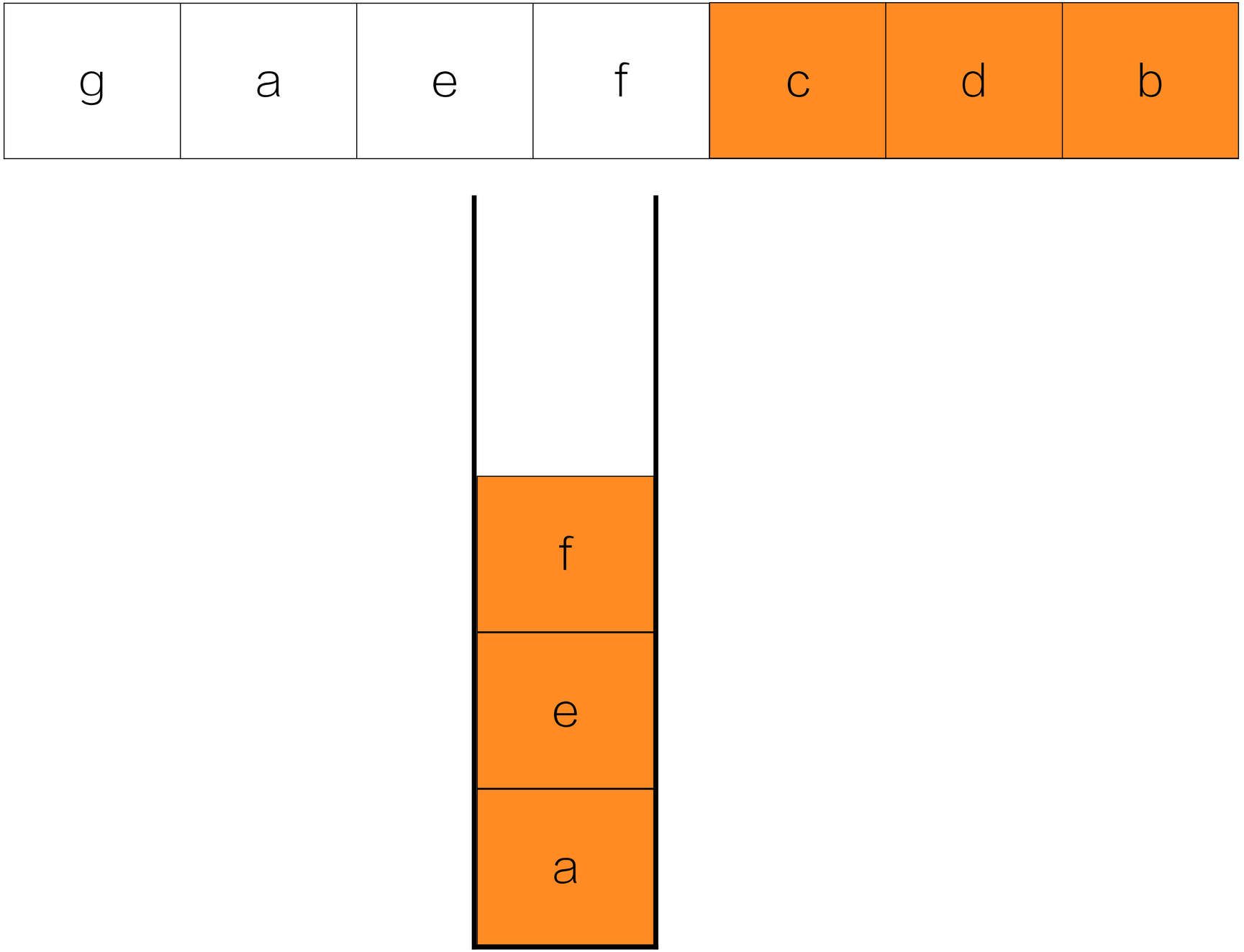

g
a
e
f
c
d
b
c
a

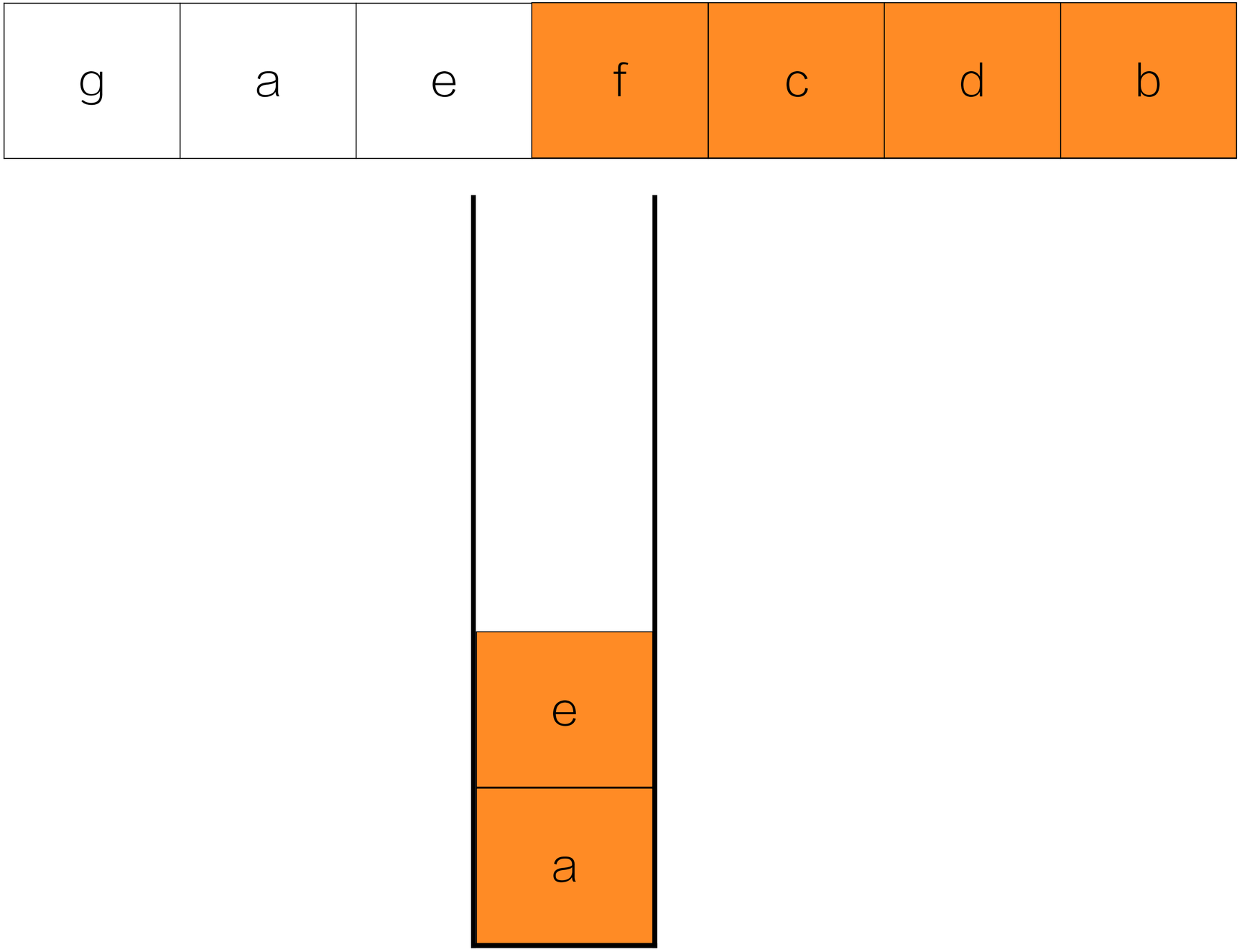

g
a
e
f
c
d
b
a

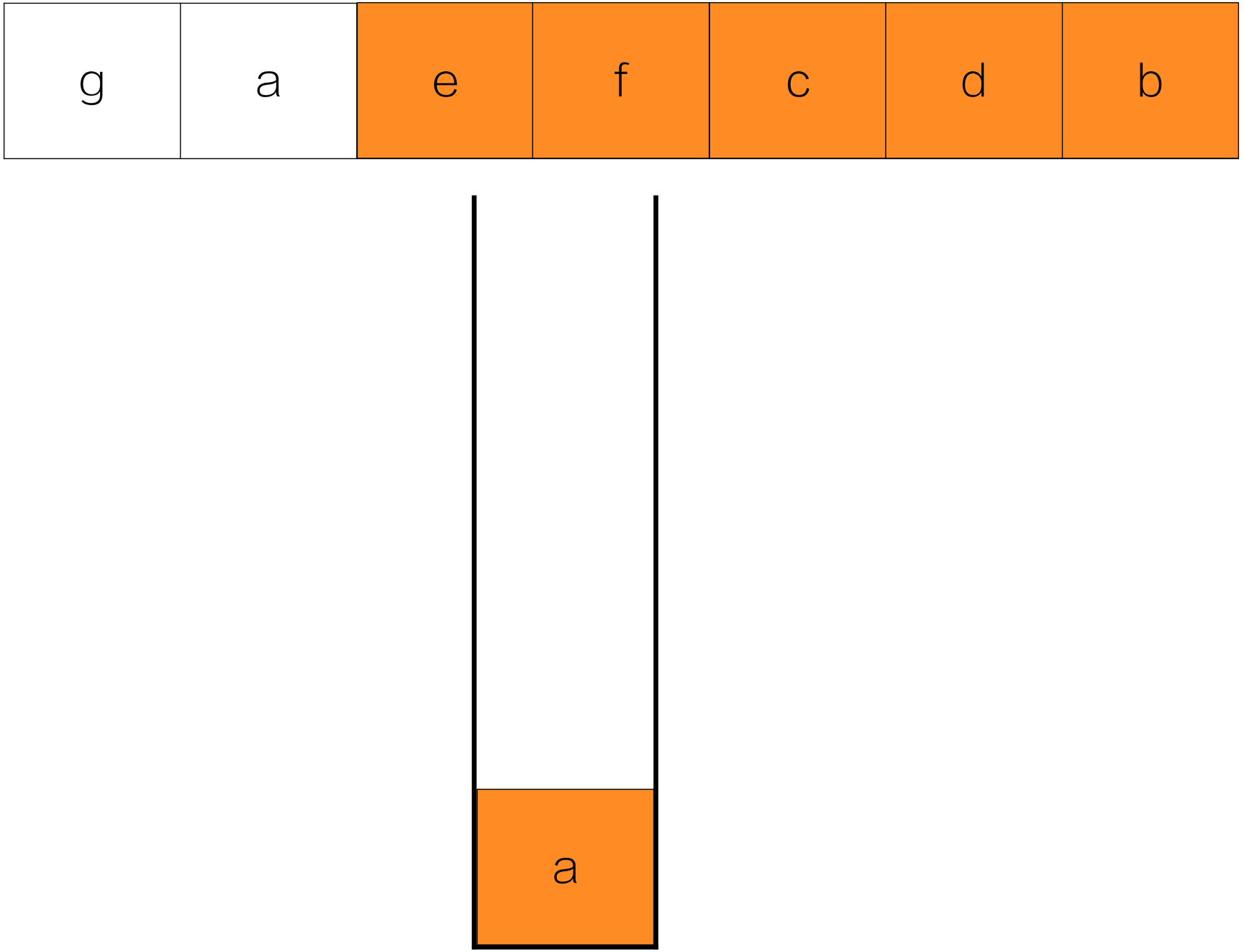

g
a
e
f
c
d
b
e
a

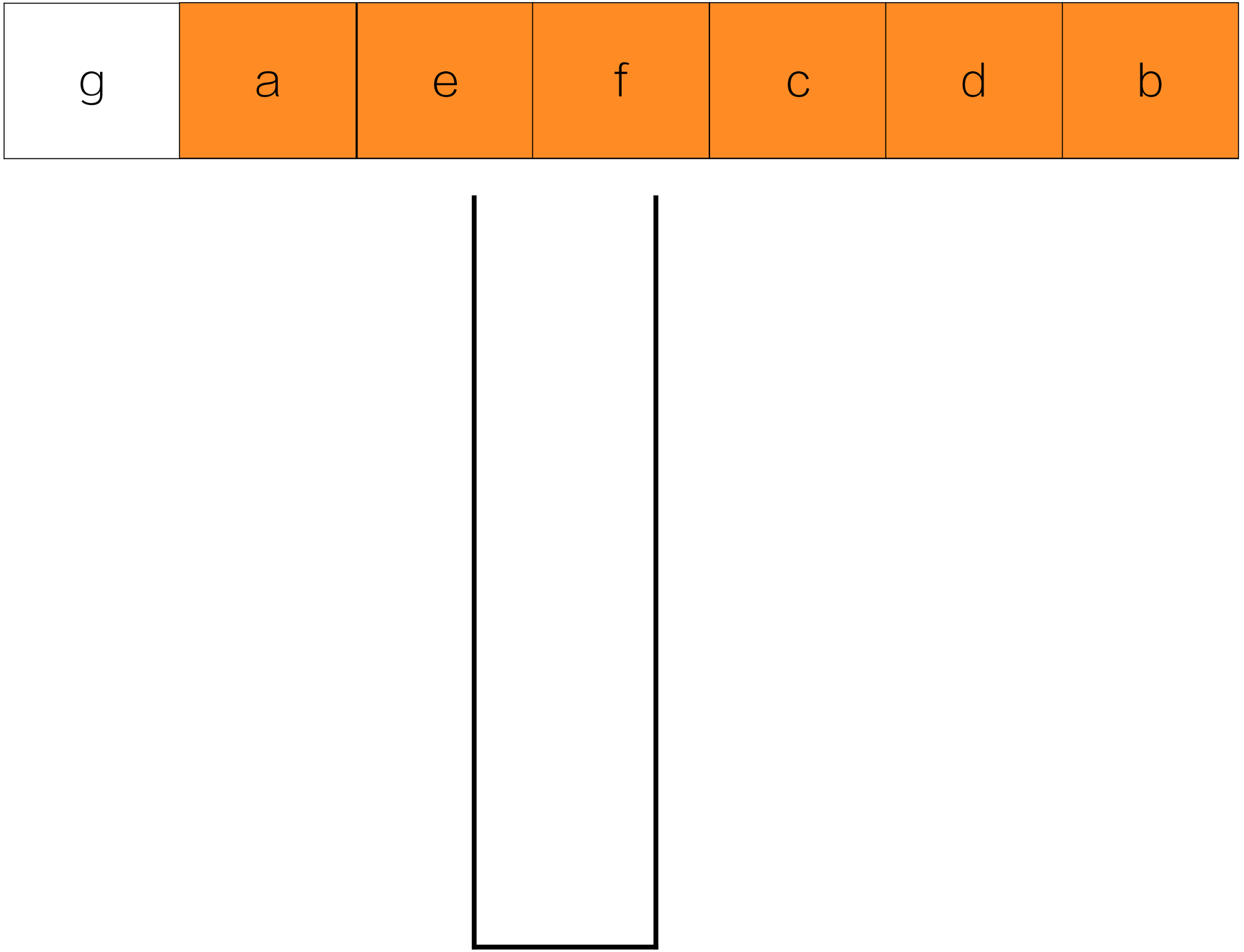

g
a
e
f
c
d
b
e
a

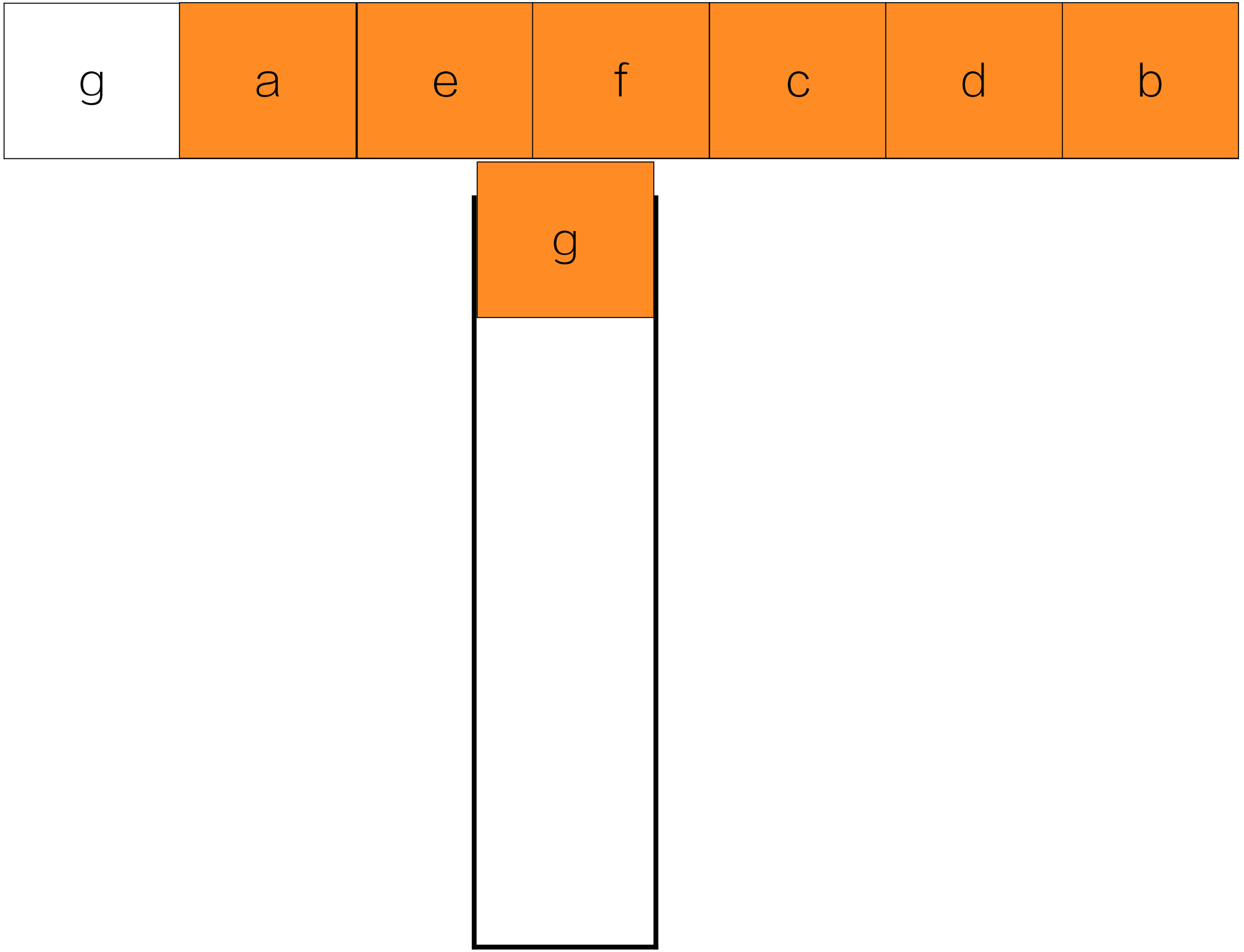

g
a
e
f
c
d
b
f
e
a

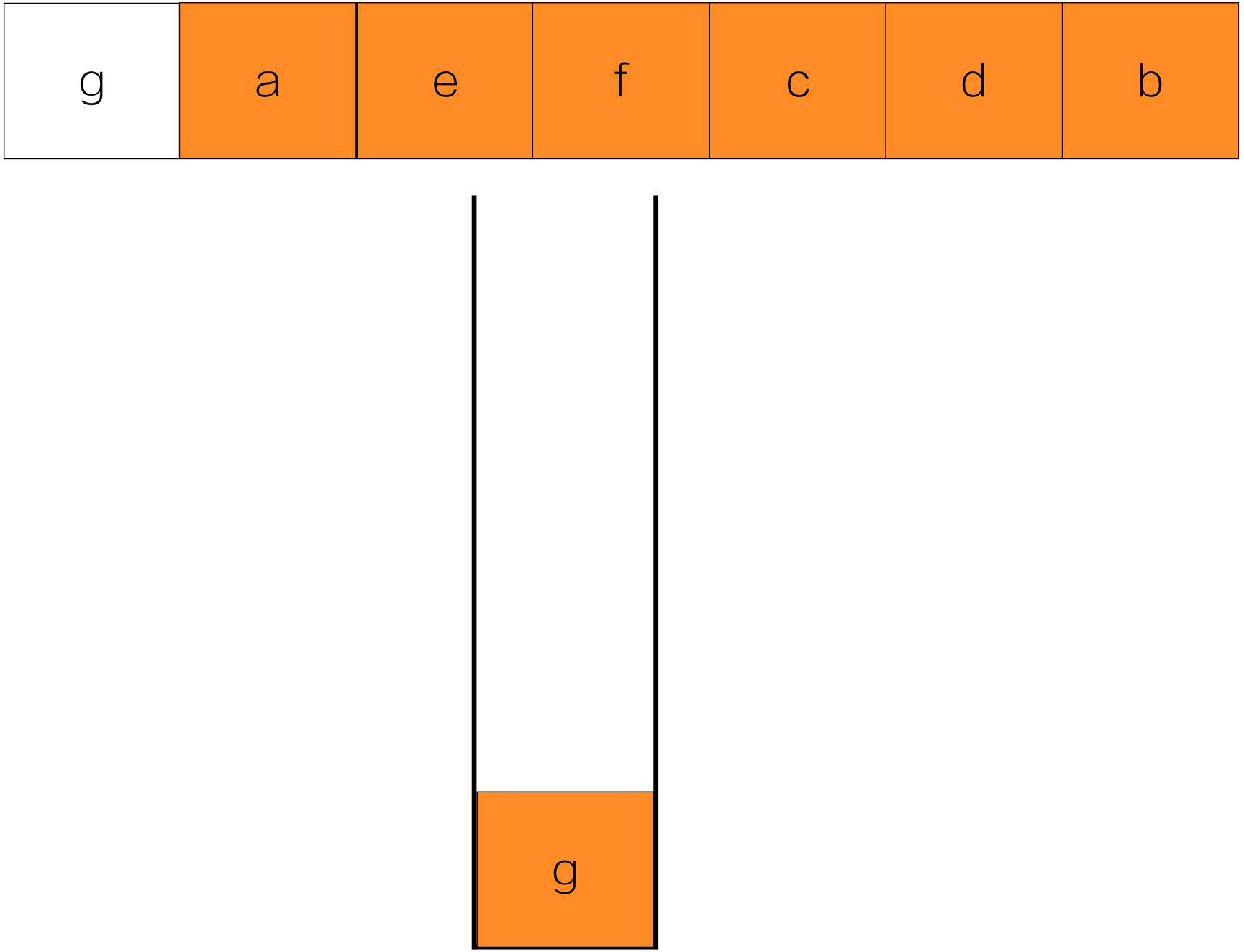

g
a
e
f
c
d
b
f
e
a

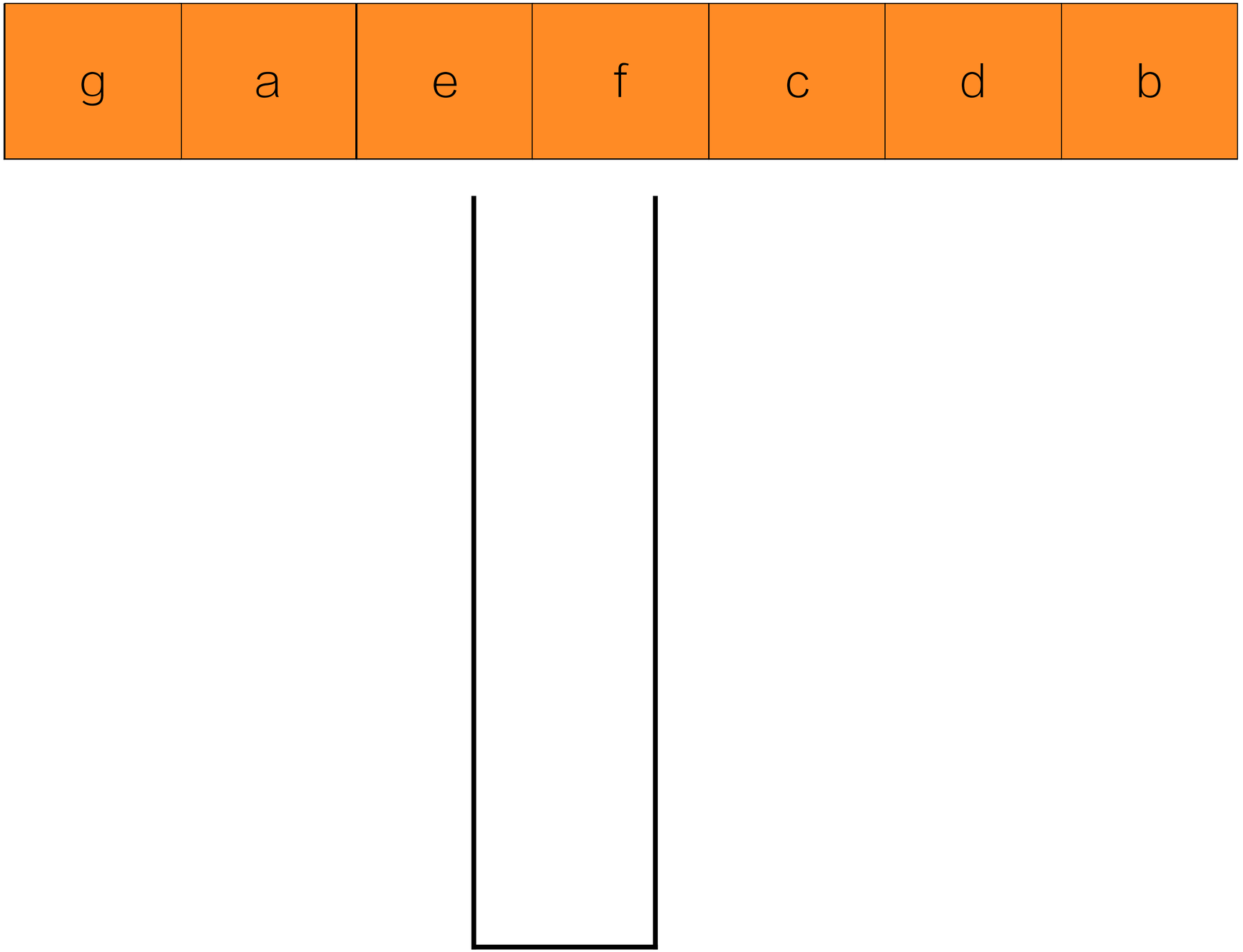

g
a
e
f
c
d
b
e
a


g
a
e
f
c
d
b
a


g
a
e
f
c
d
b


g
a
e
f
c
d
b
g


g
a
e
f
c
d
b
g


g
a
e
f
c
d
b

栈与队列的应用

[2009] 2. 设栈 S 和队列 Q 的初始状态均为空，元素 a, b, c, d, e, f, g依次进入栈 S 。若每个元素出栈后立即进入队列 Q，且7个元素出队的顺序是 b, d, c, f, e, a, g，则栈 S 的容量至少是_____。

A. 1　　B. 2　　C. 3　　D. 4

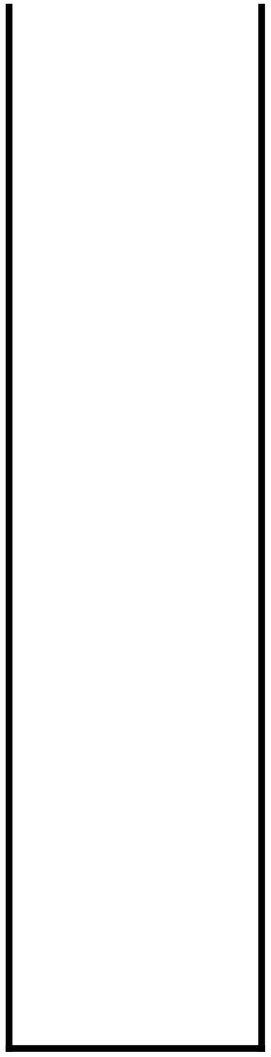
栈与队列的应用

[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b

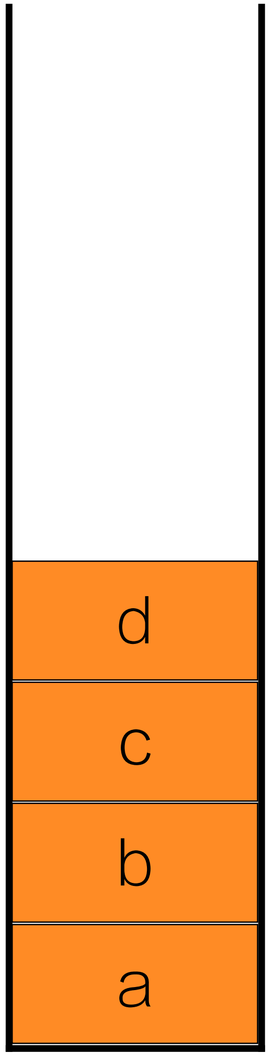
[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b



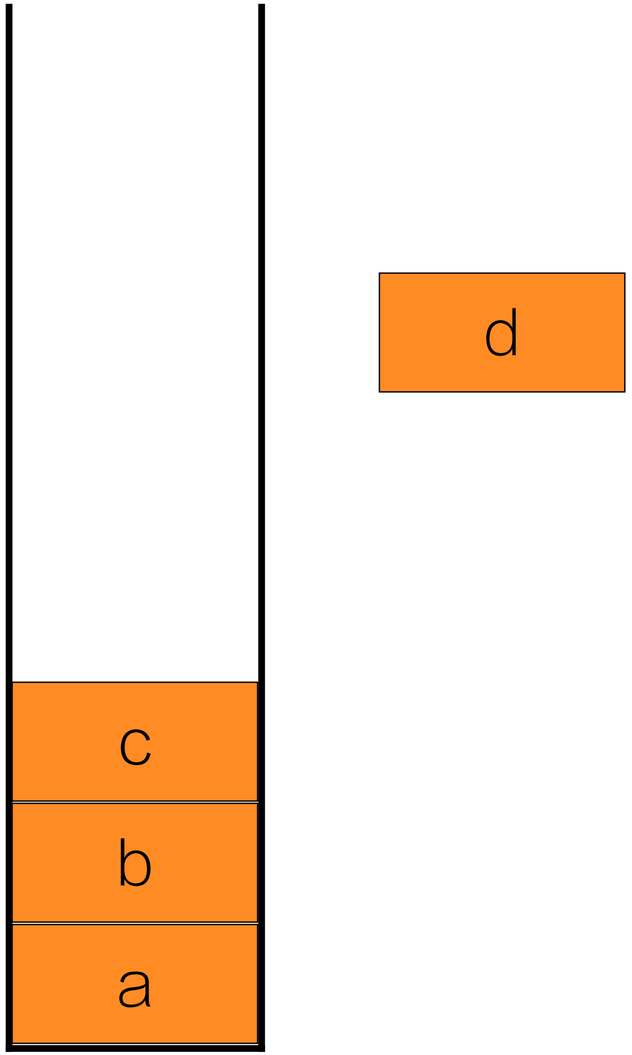
[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b



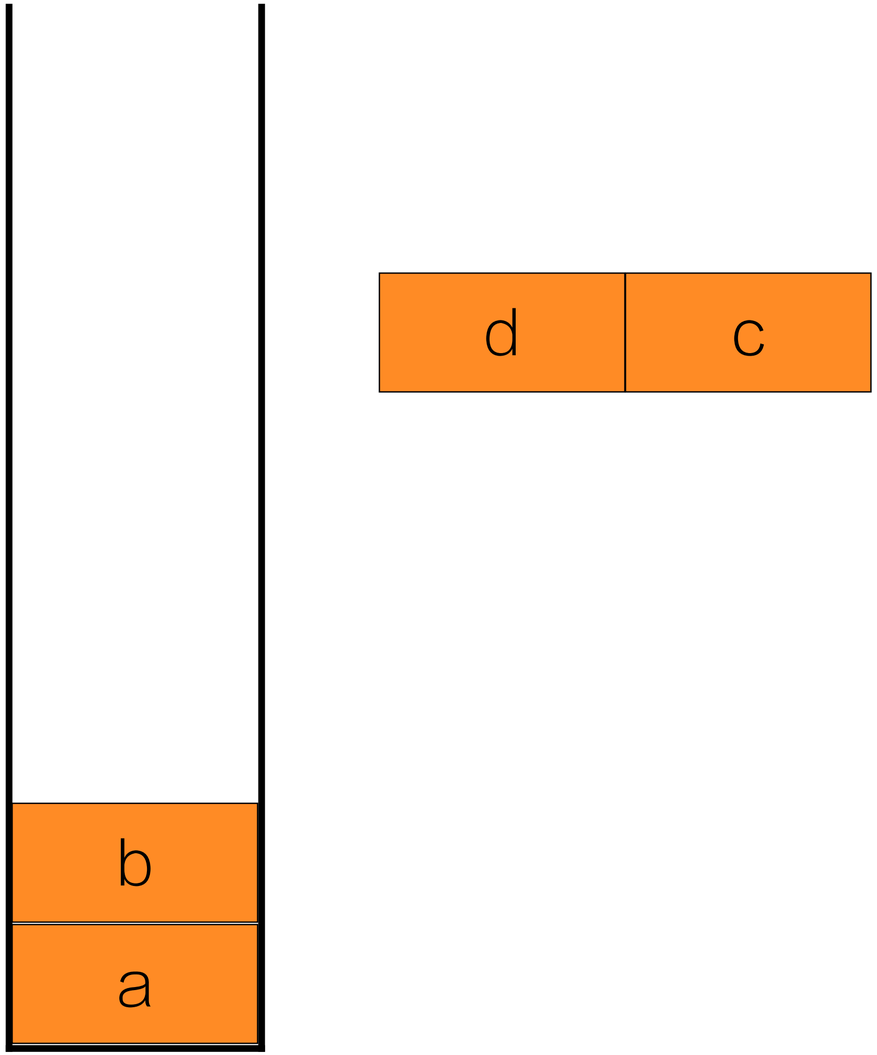
[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b



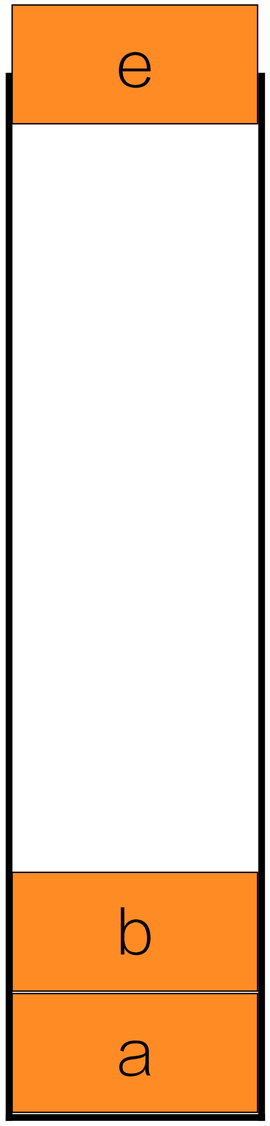
[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

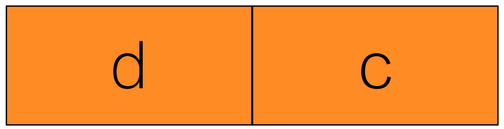
A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b



[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

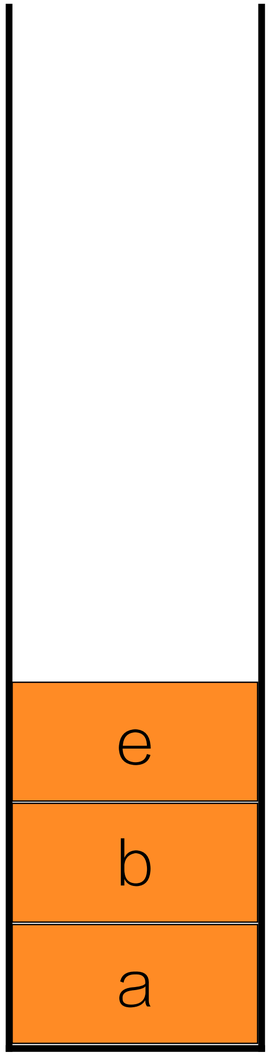
A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b

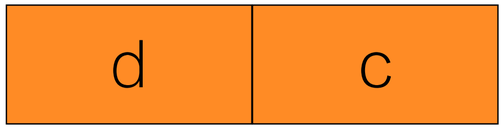




[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

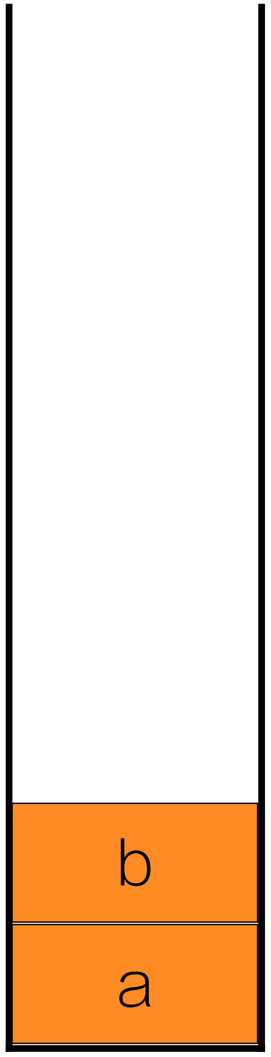
A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b

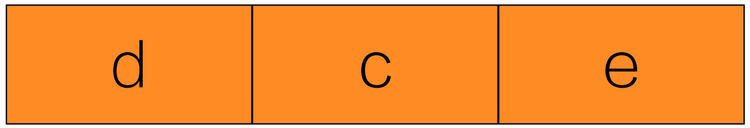




[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

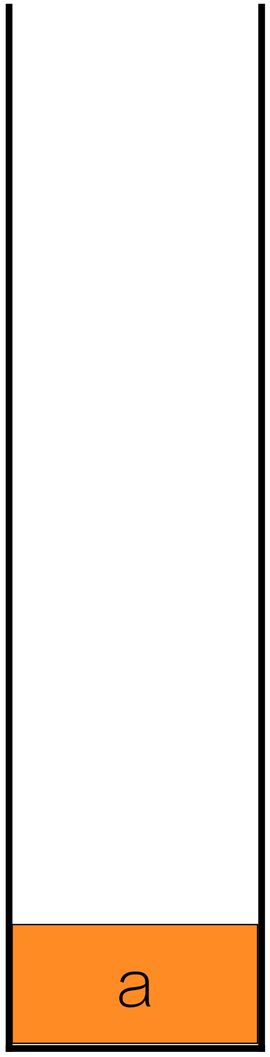
A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b

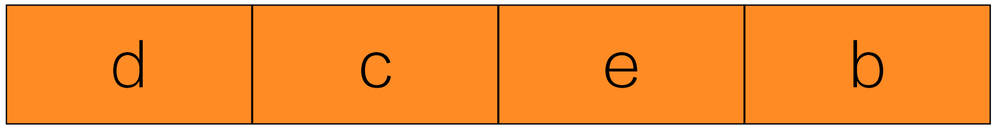




[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

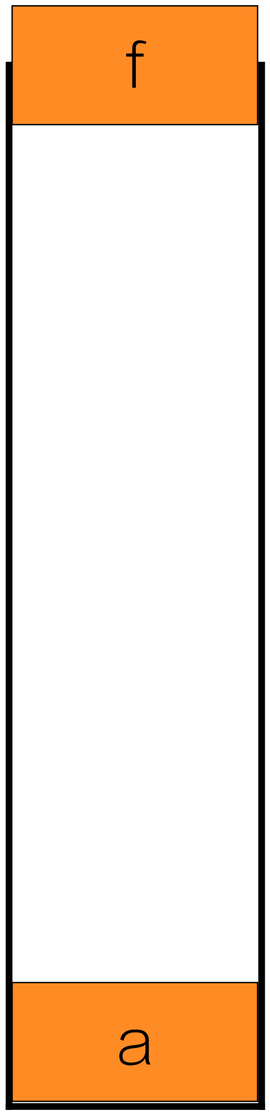
A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b

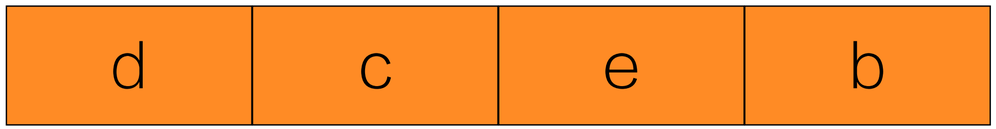




[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

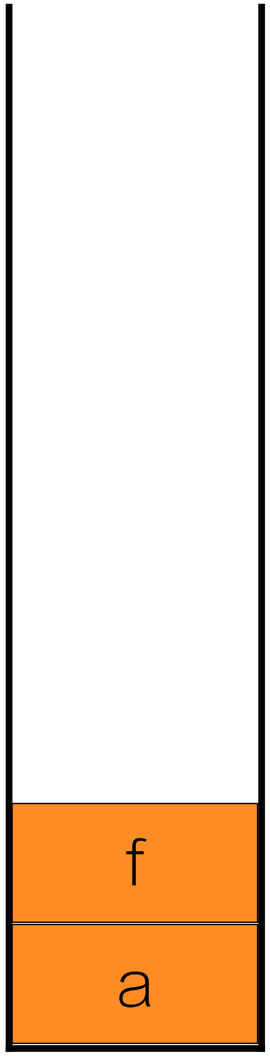
A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b

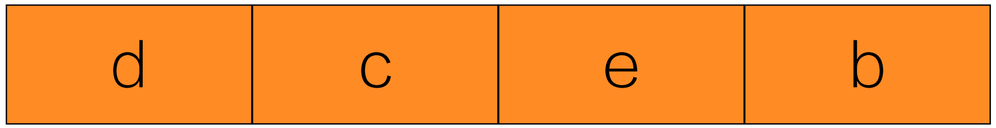




[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

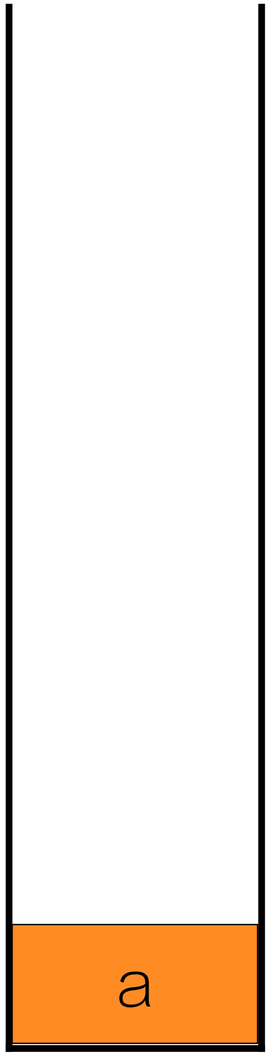
A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b

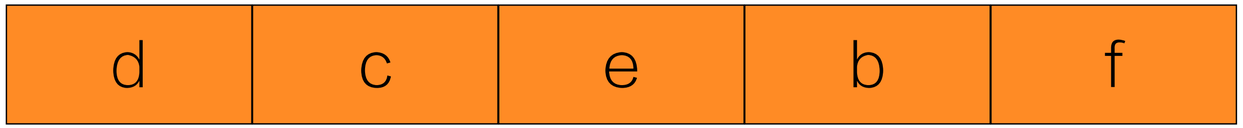




[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

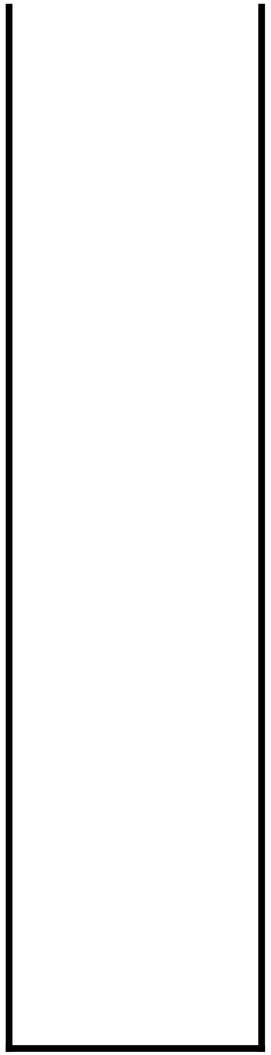
A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b

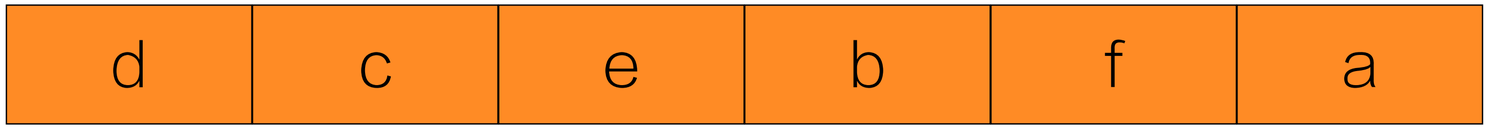




[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

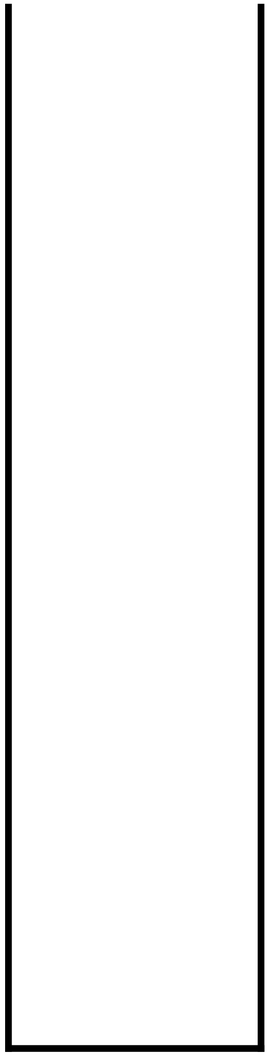
A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b





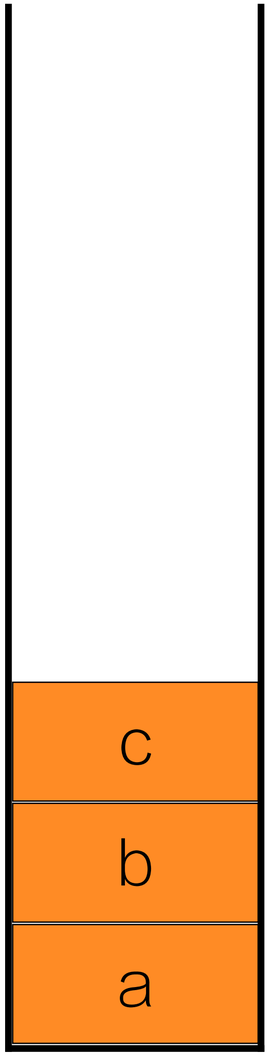
[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b



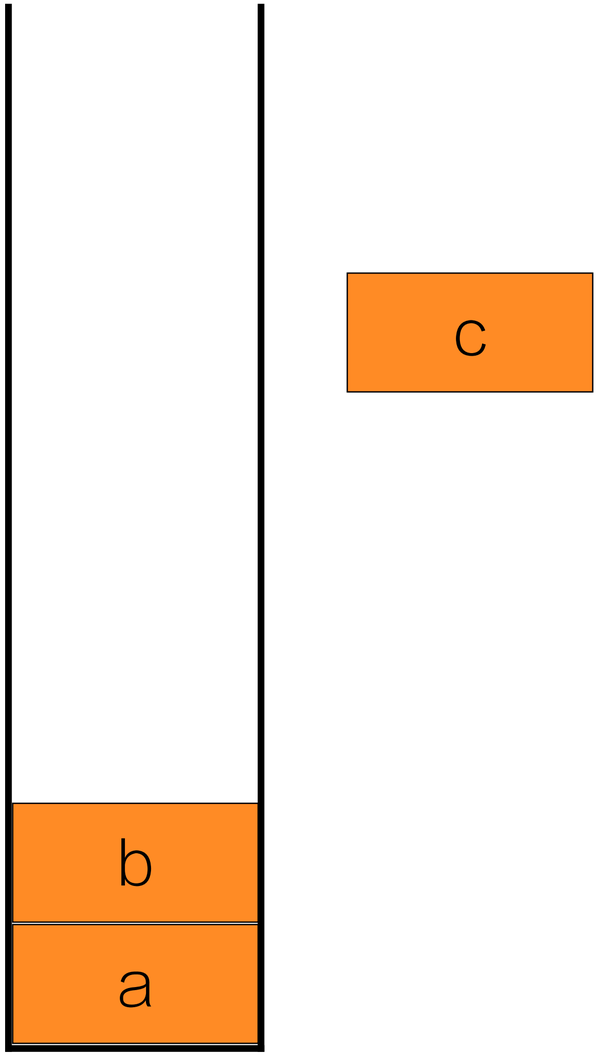
[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b



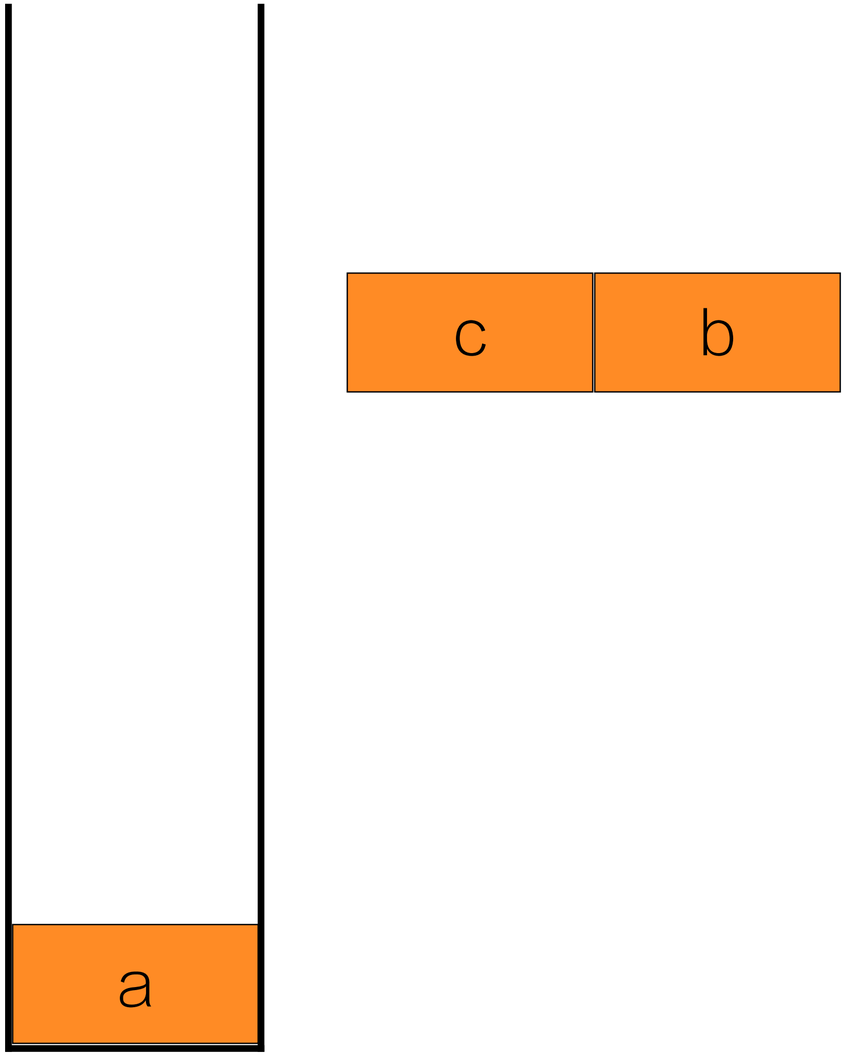
[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b



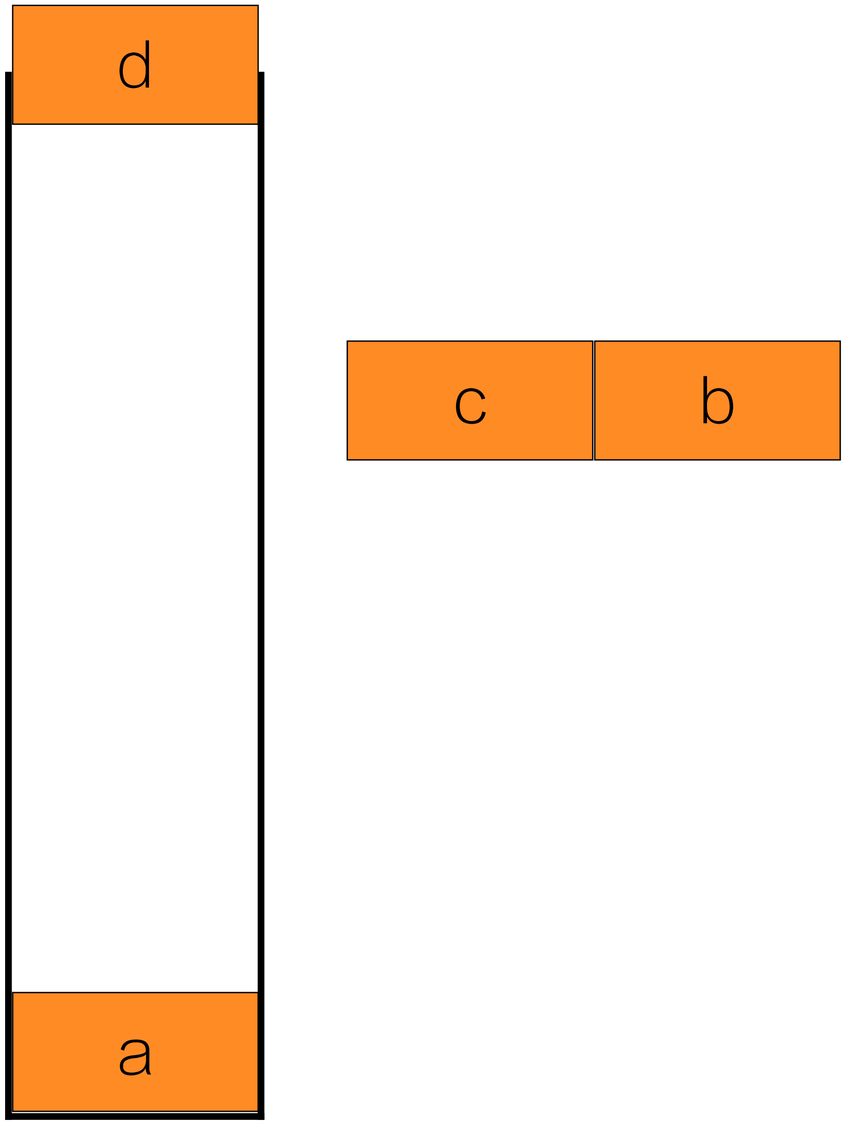
[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b



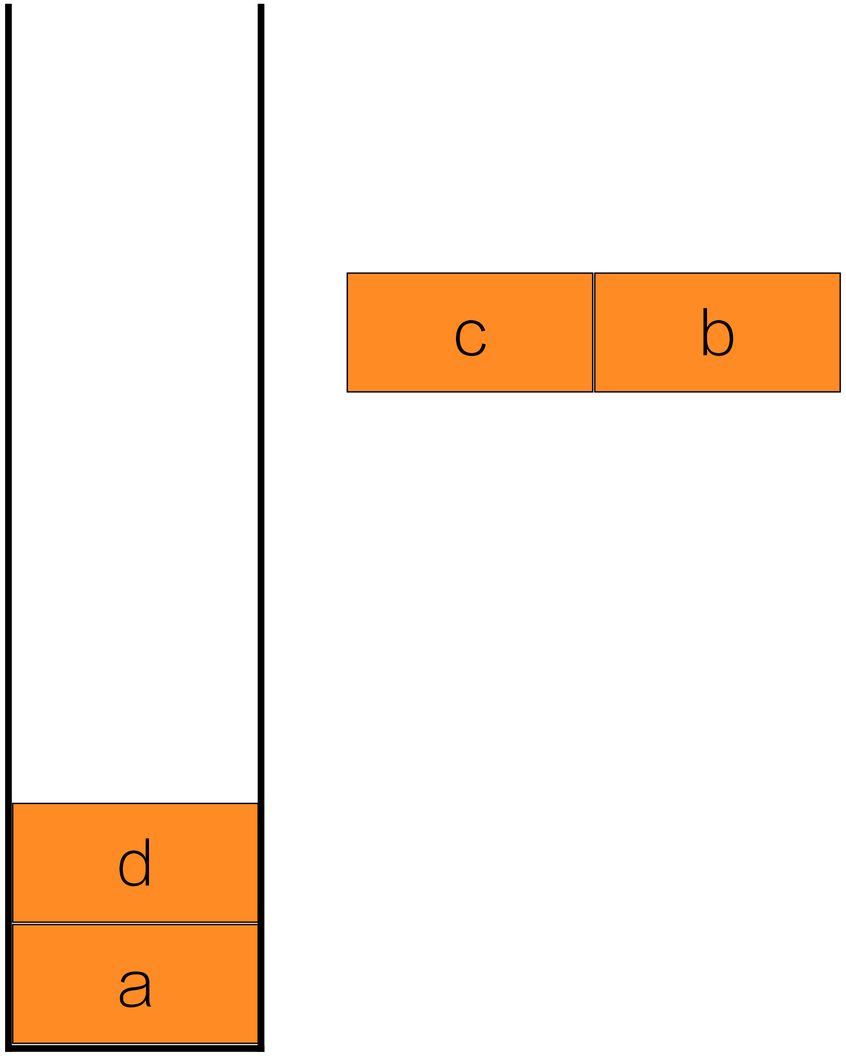
[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b



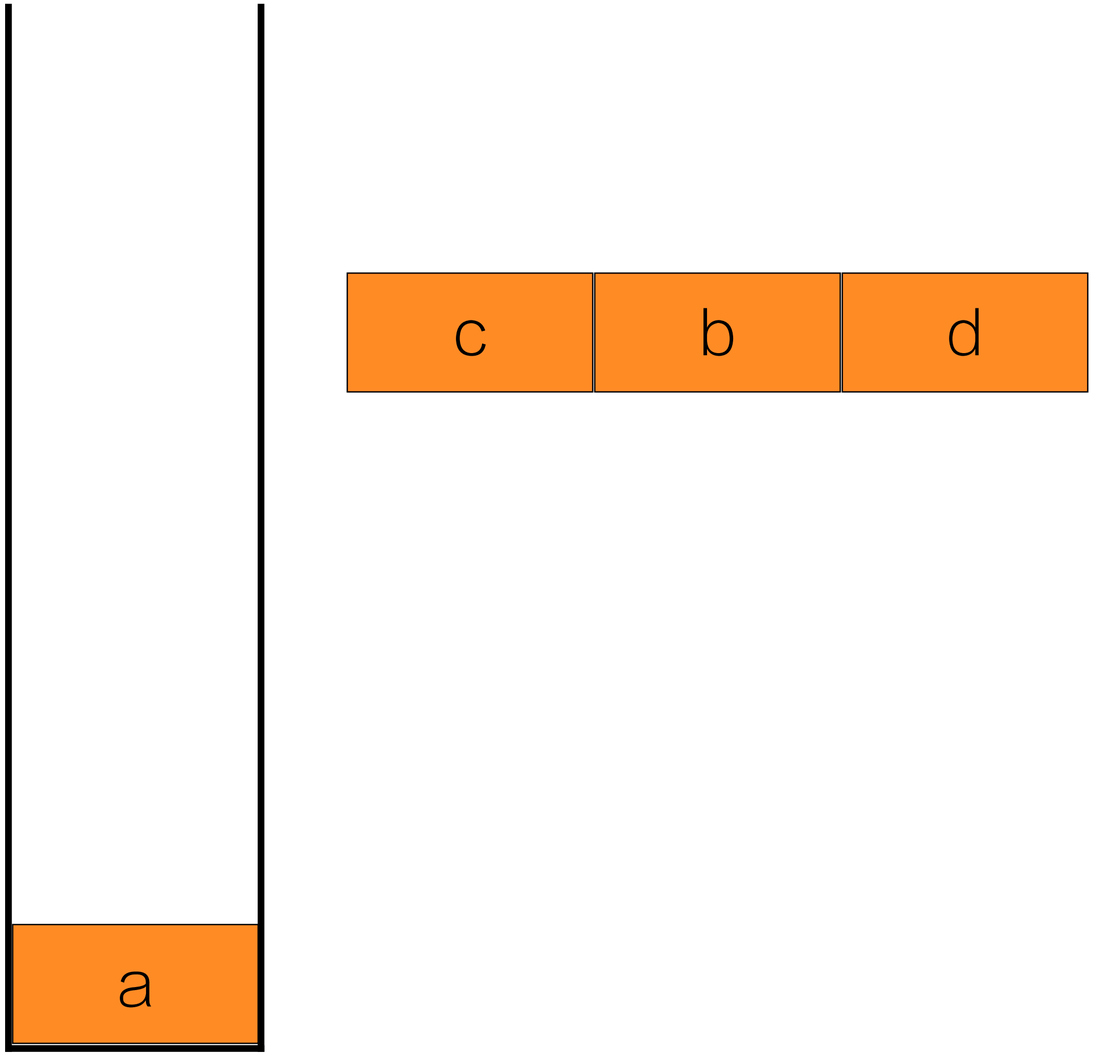
[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b



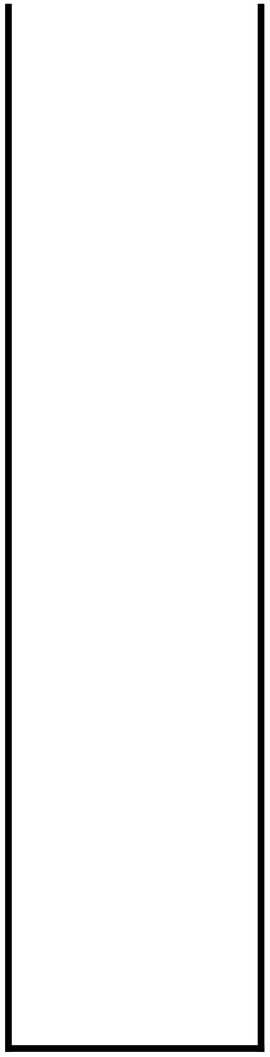
[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

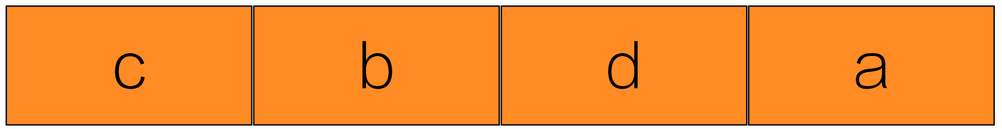
A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b



[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

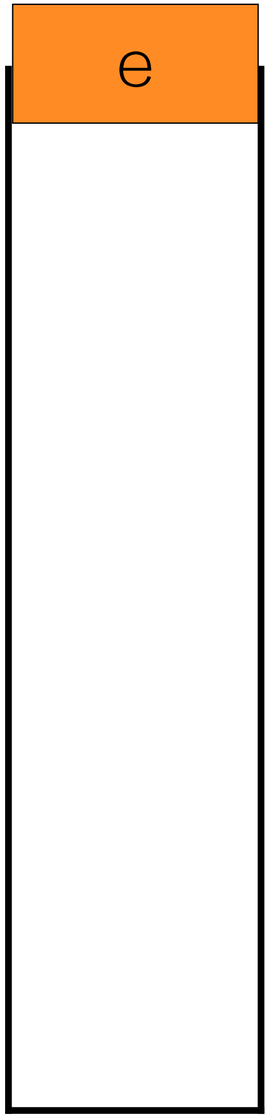
A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b

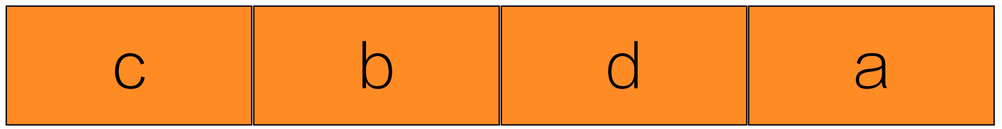




[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

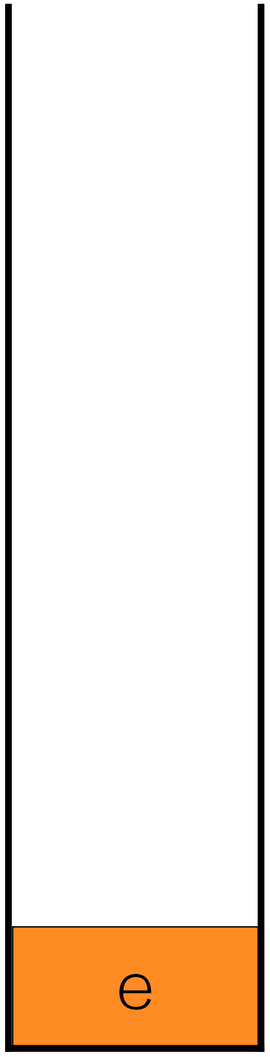
A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b

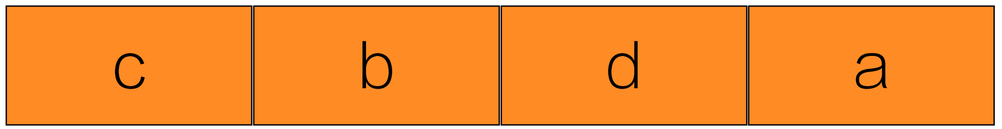




[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

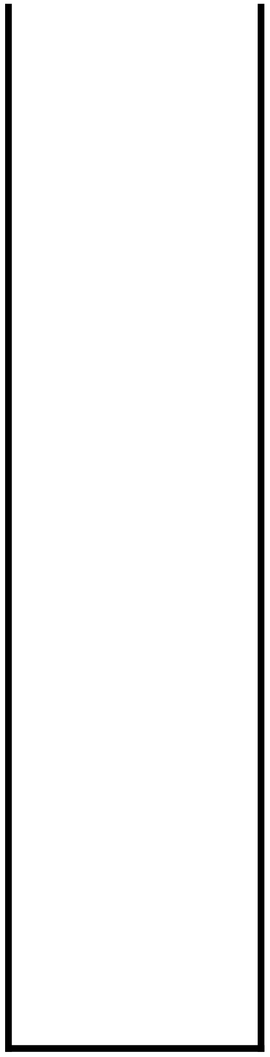
A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b

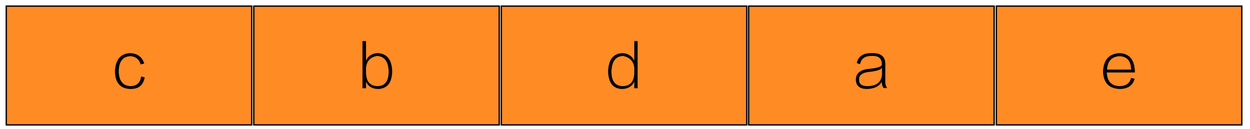




[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

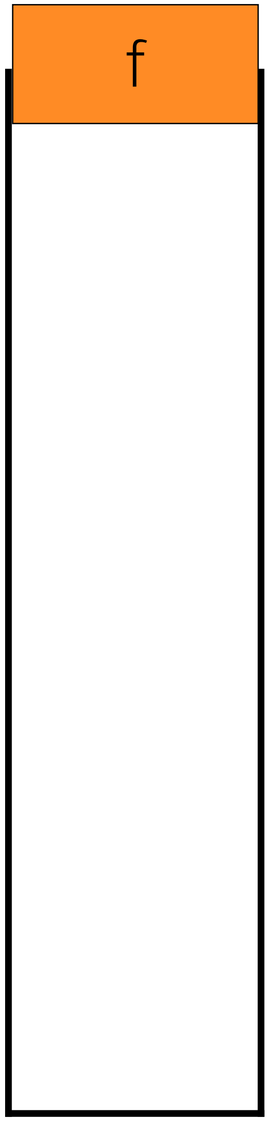
A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b

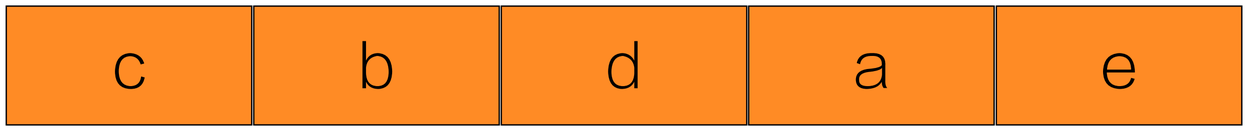




[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

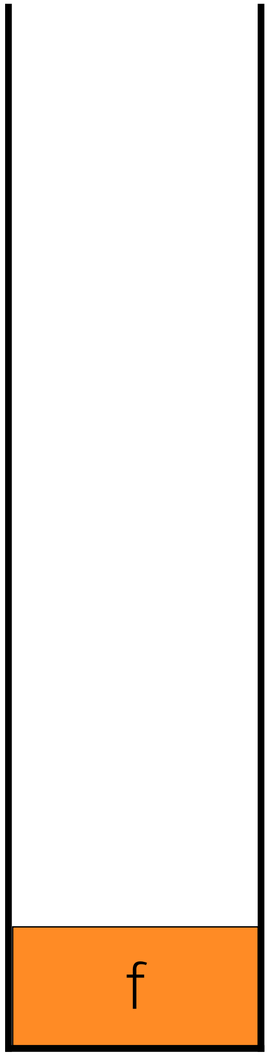
A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b

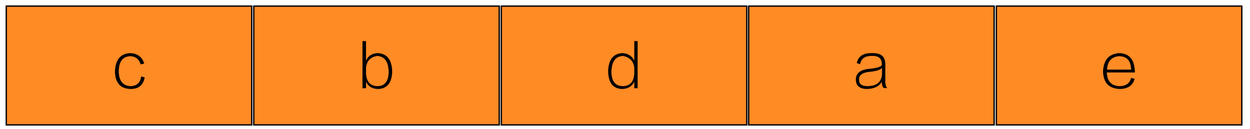




[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

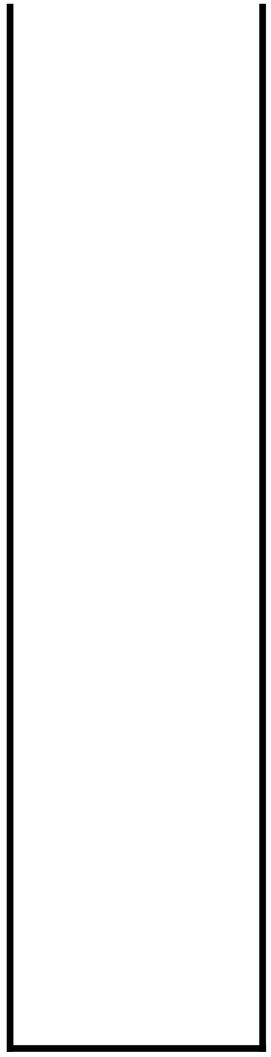
A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b

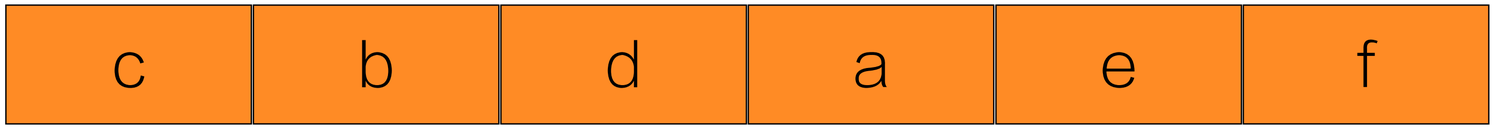




[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

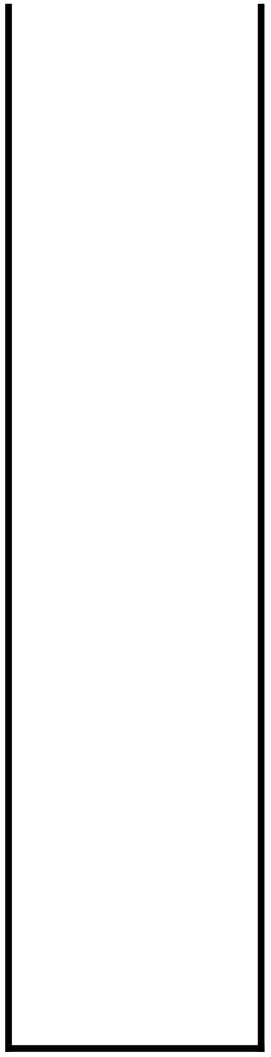
A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b





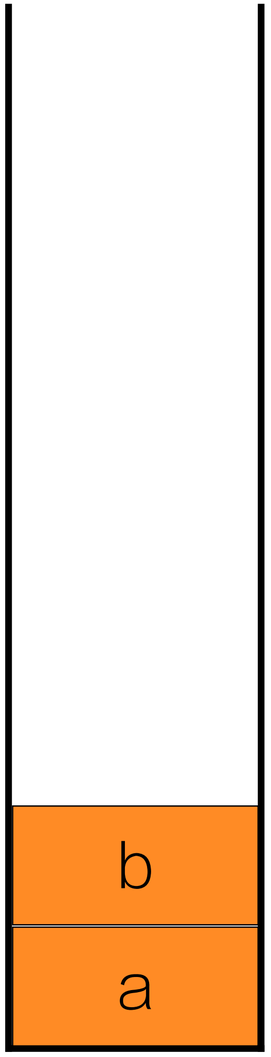
[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b



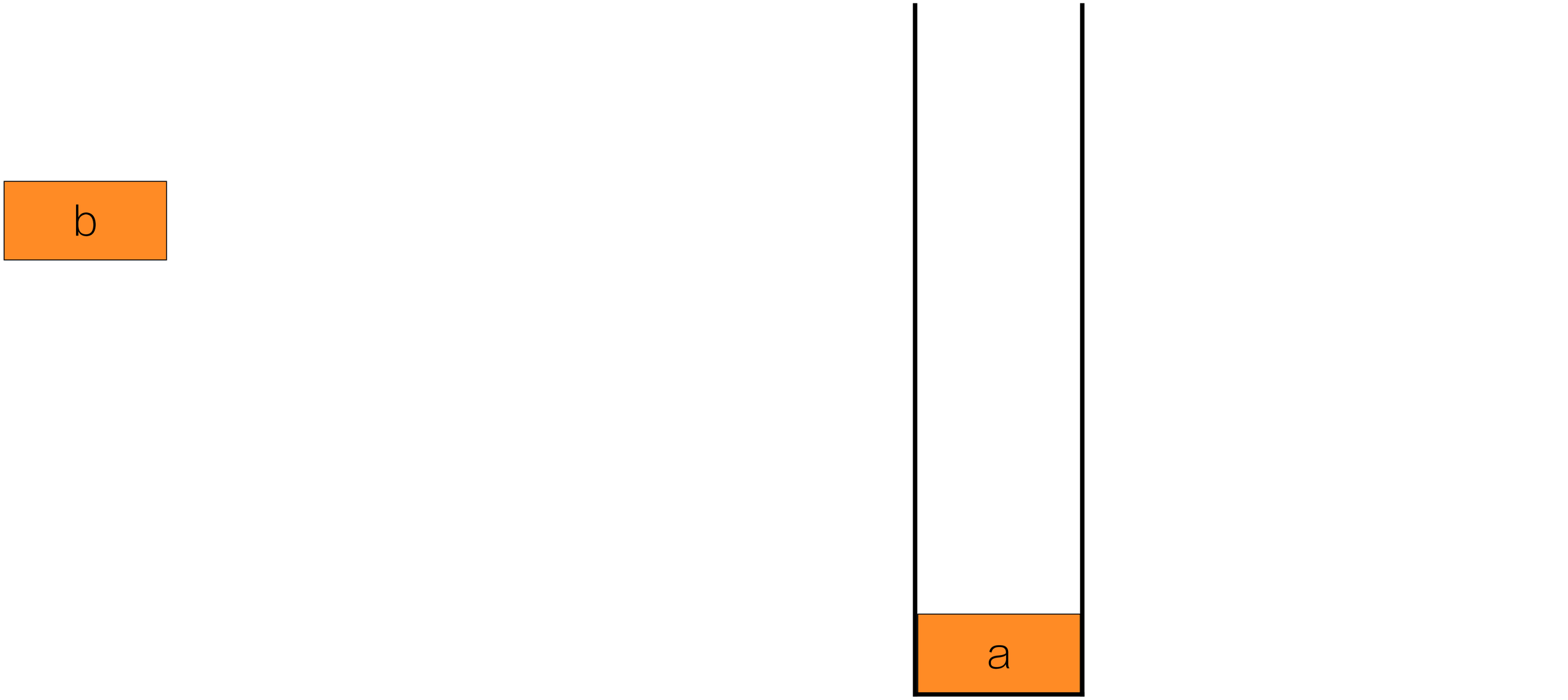
[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b



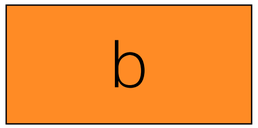
[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

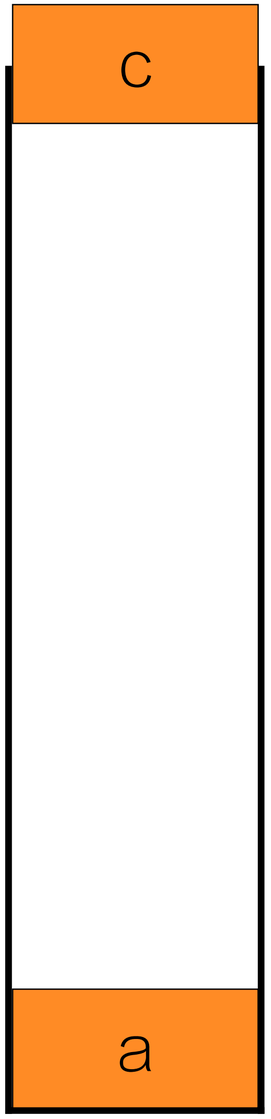
A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b



[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

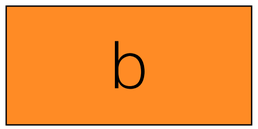
A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b

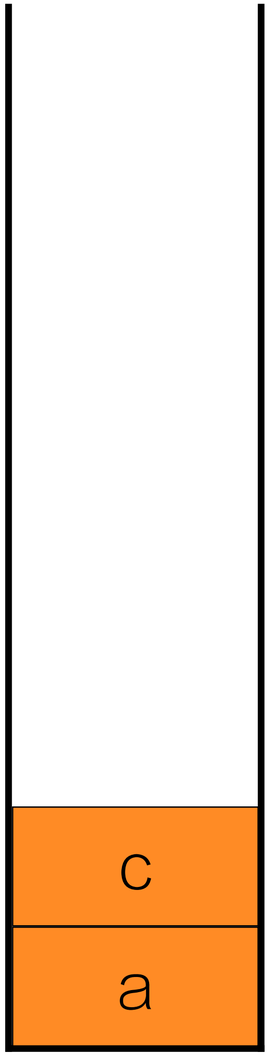




[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

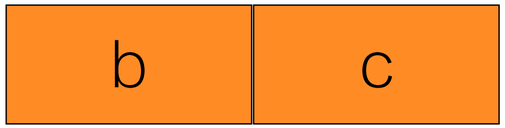
A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b

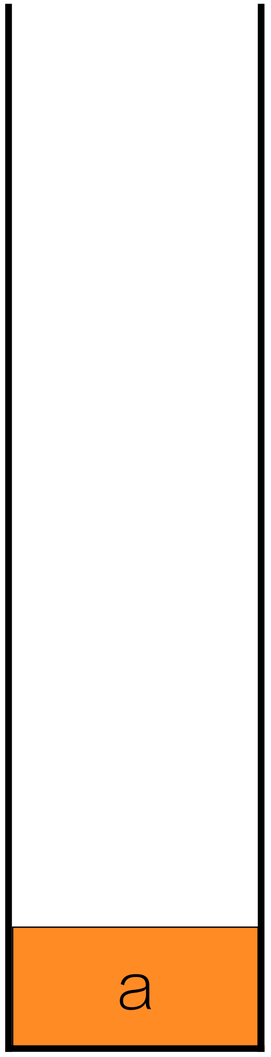




[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

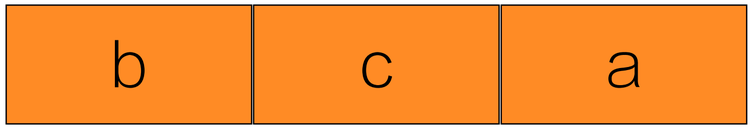
A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b

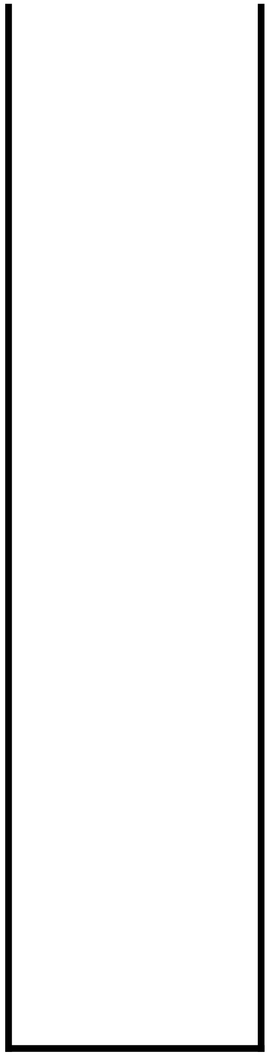




[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

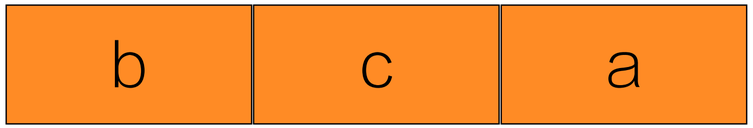
A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b

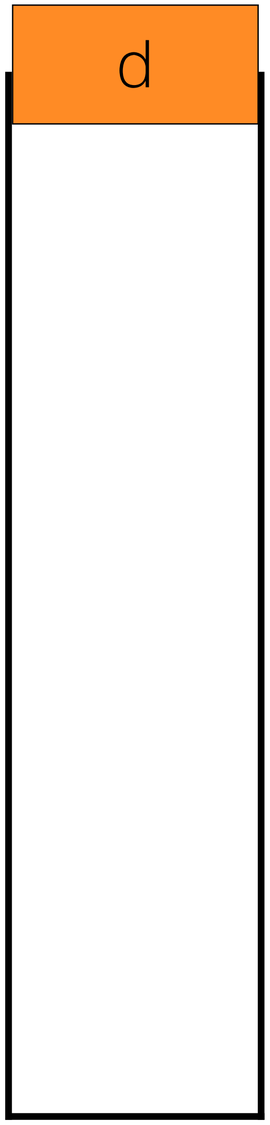




[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

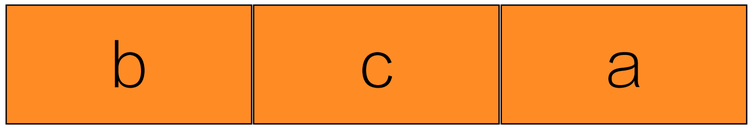
A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b

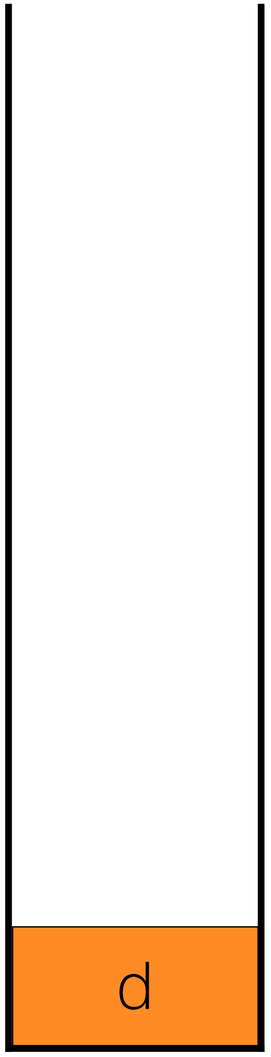




[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

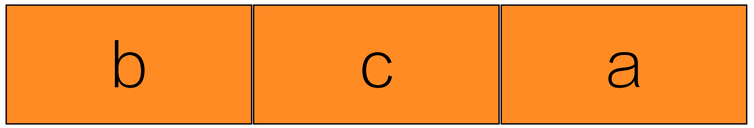
A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b

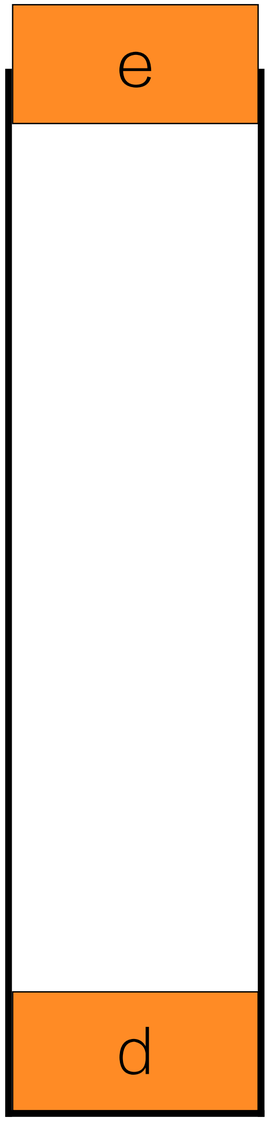




[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

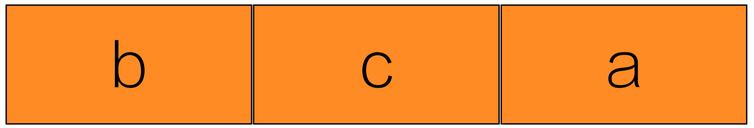
A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b

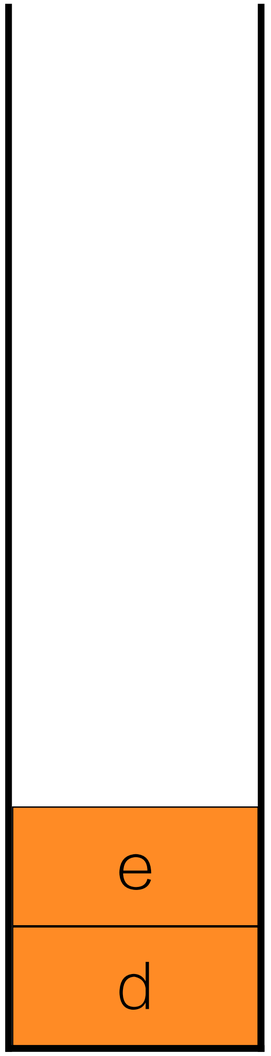




[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

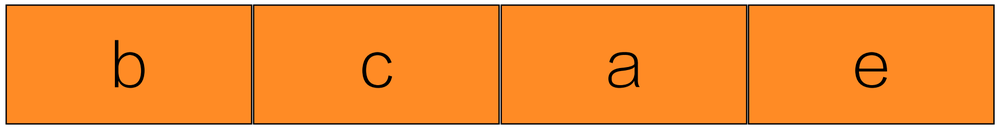
A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b

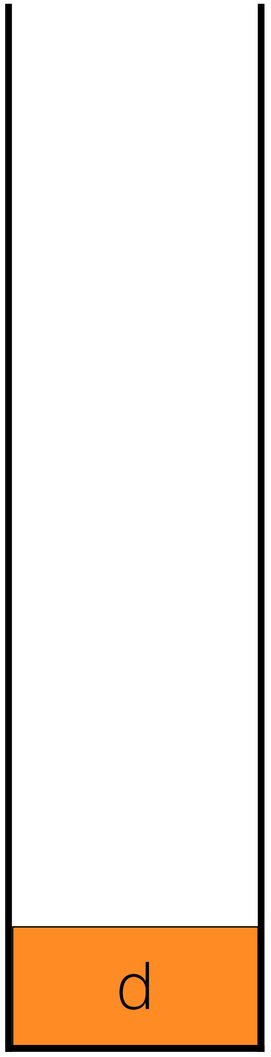




[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

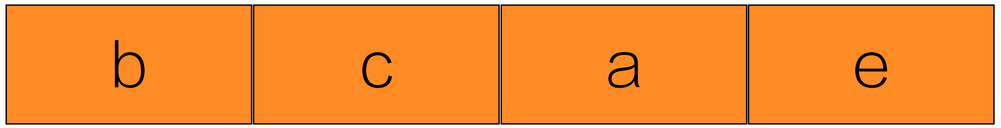
A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b

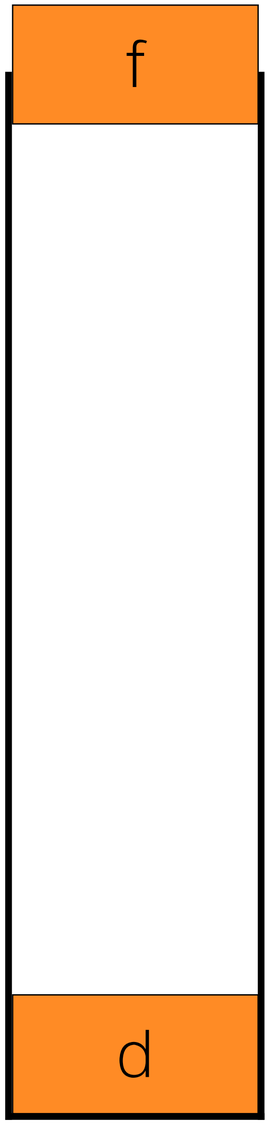




[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

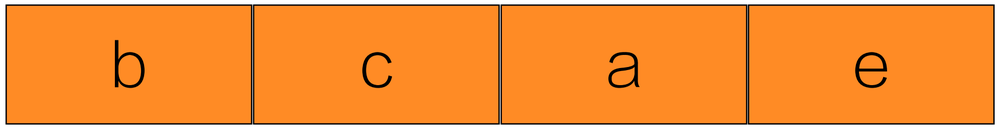
A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b

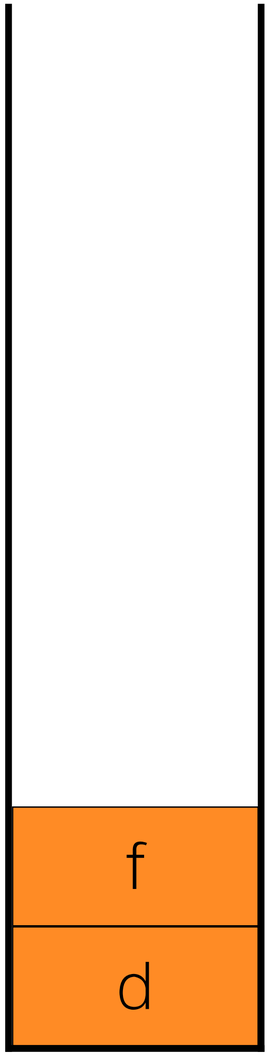




[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

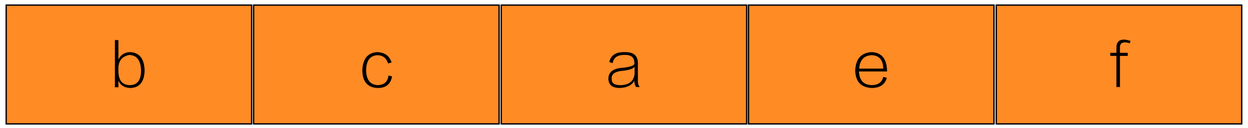
A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b

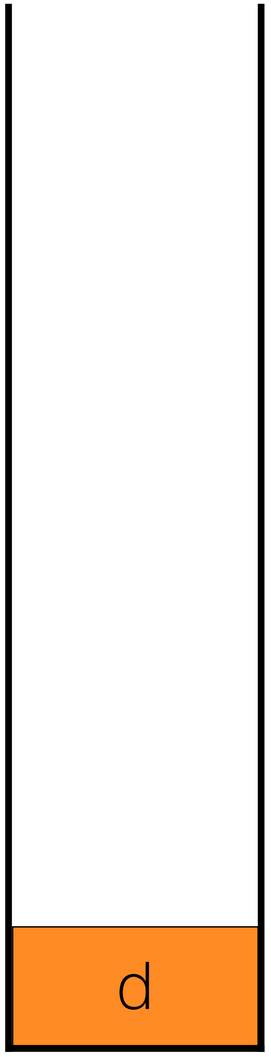




[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

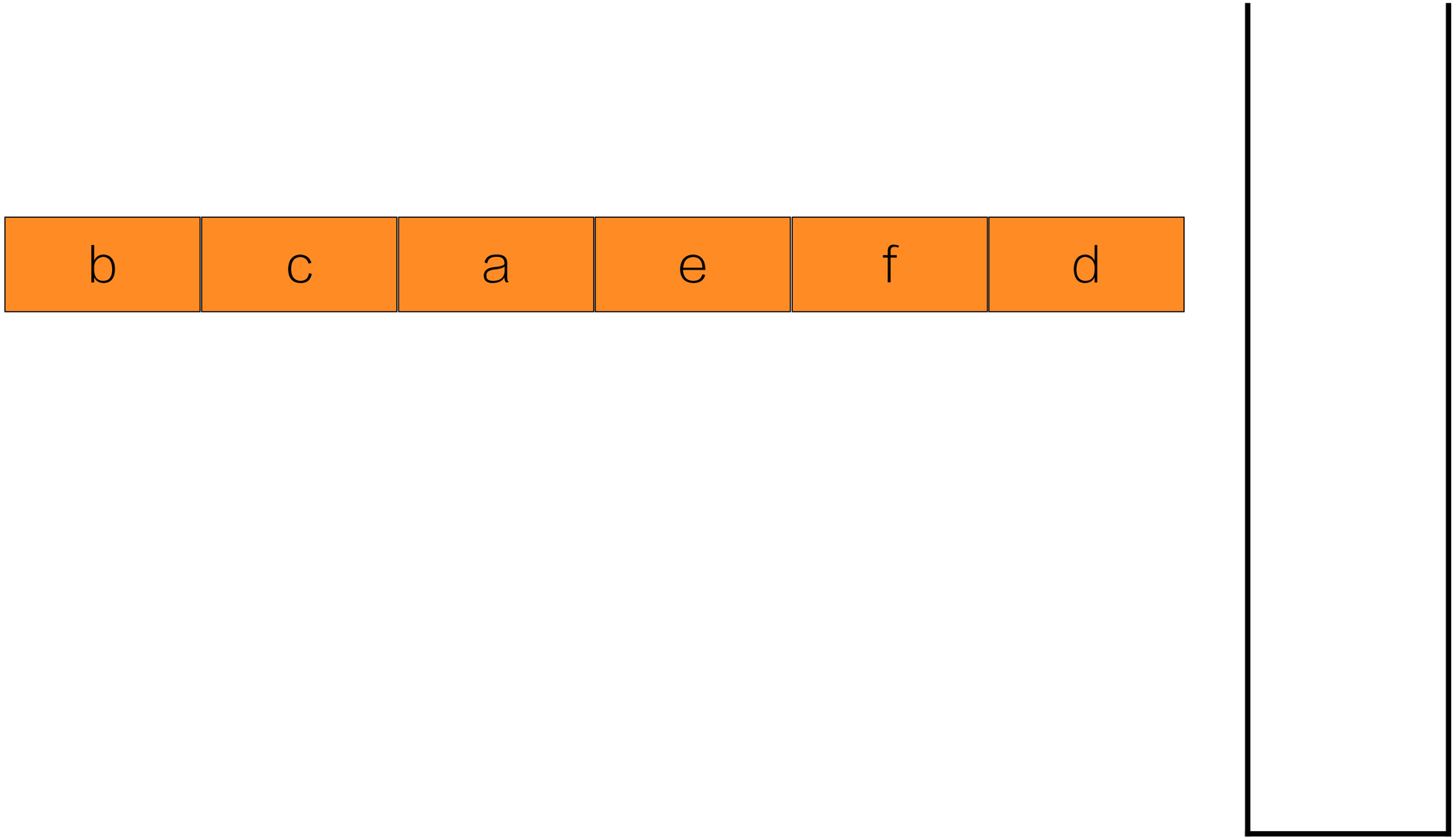
A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b





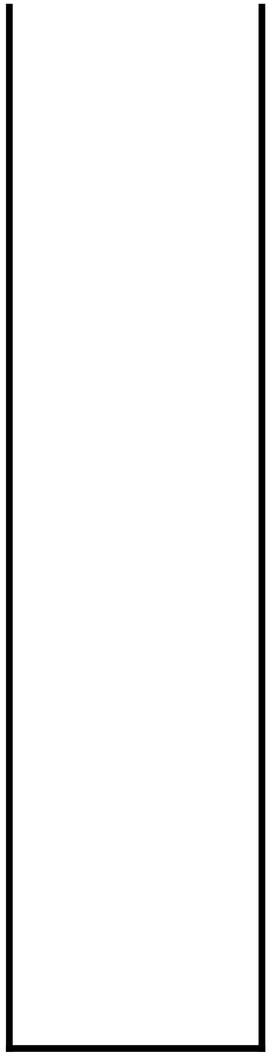
[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b



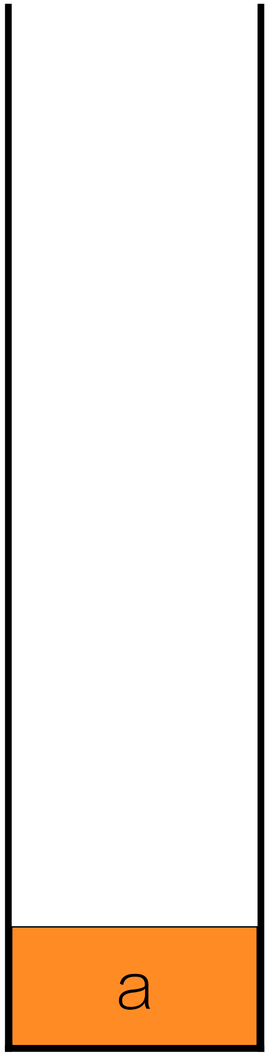
[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b



[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b



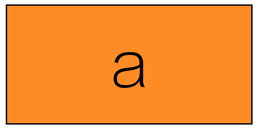
[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

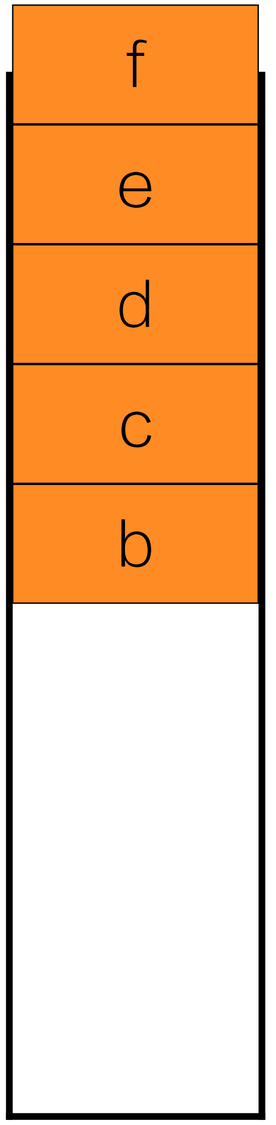
A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b



[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

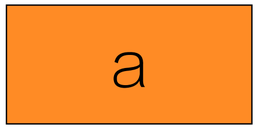
A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b

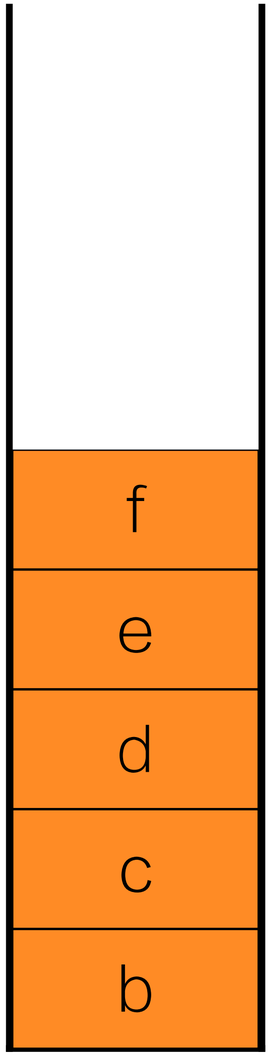




[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

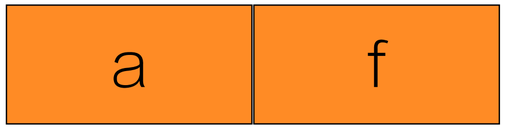
A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b

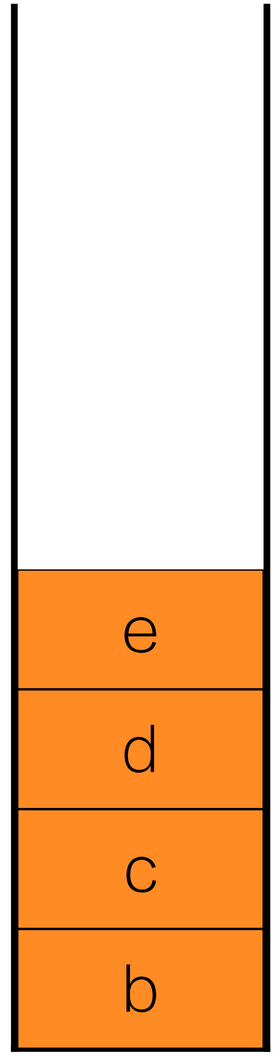




[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b

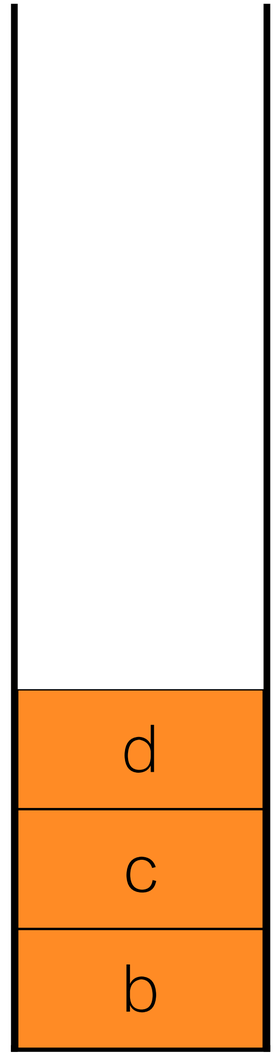




[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

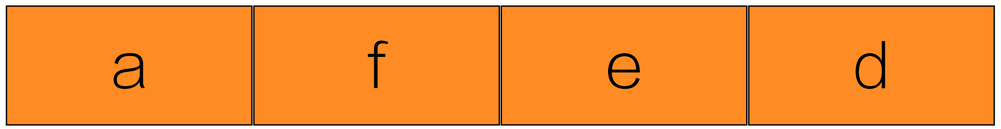
A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b

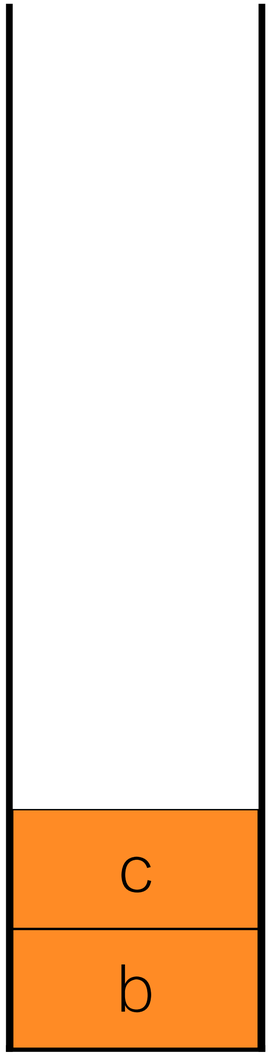




[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

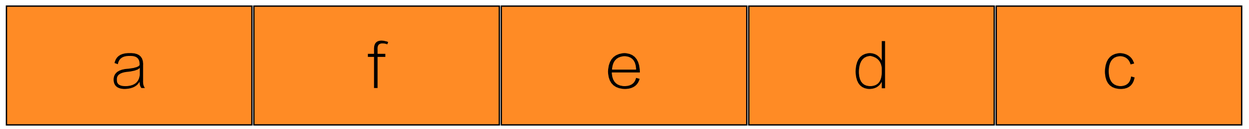
A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b

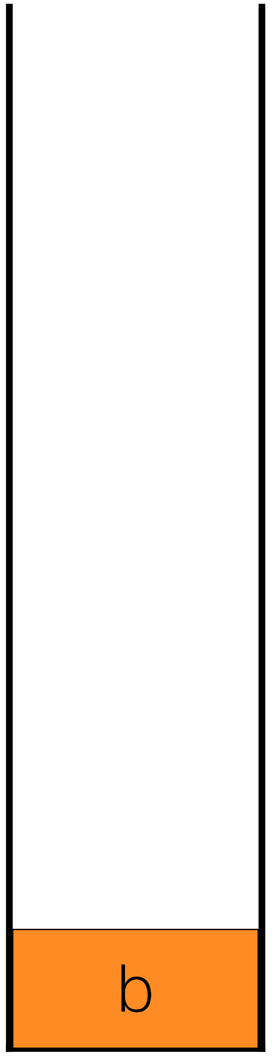




[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

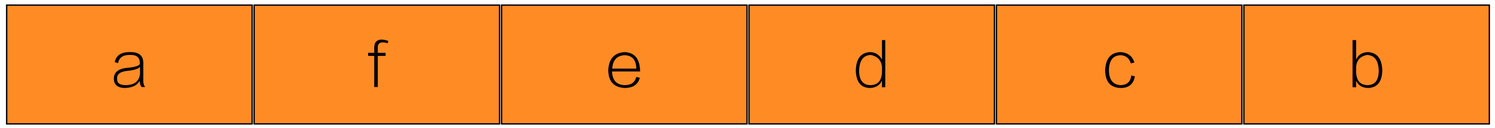
A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b





[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b



栈与队列的应用

[2010] 1. 若元素 a, b, c, d, e, f 依次进栈 ， 允许进栈、退栈交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是_____。

A. d c e b f a　　B. c b d a e f　　C. b c a e f d　　D. a f e d c b

栈与队列的应用

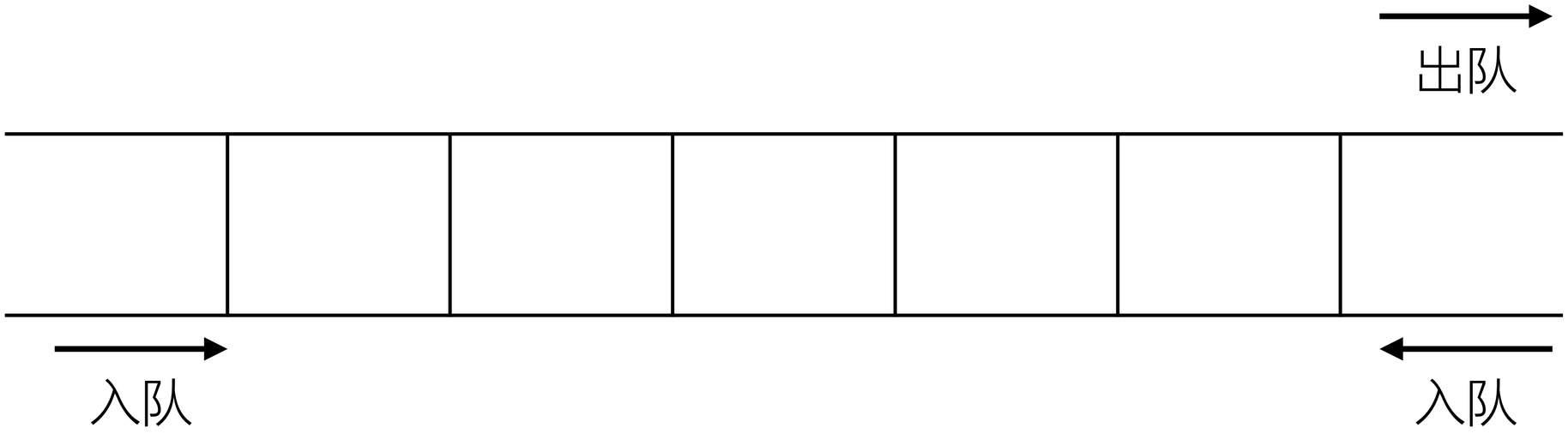
[2010] 2. 某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是_____。

A. b a c d e　　B. d b a c e　　C. d b c a e　　D. e c b a d

栈与队列的应用

[2010] 2. 某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是_____。

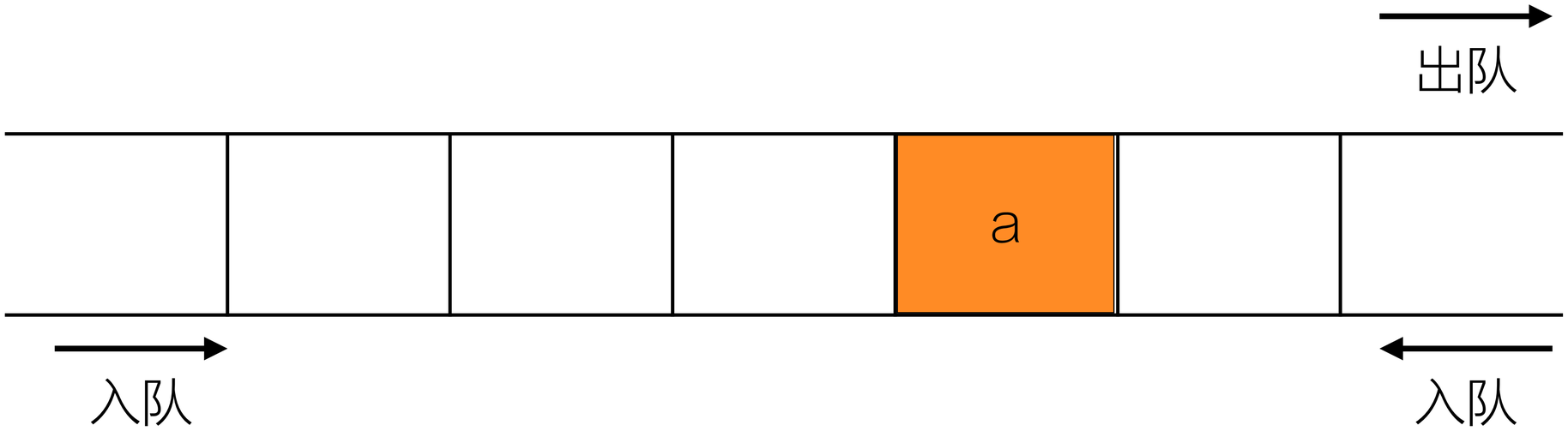
A. b a c d e　　B. d b a c e　　C. d b c a e　　D. e c b a d



栈与队列的应用

[2010] 2. 某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是_____。

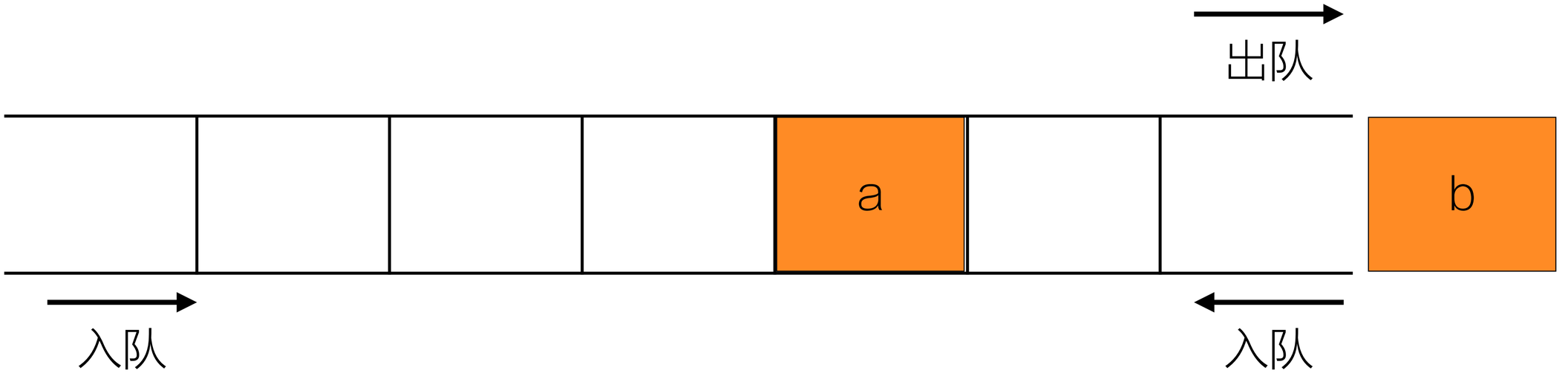
A. b a c d e　　B. d b a c e　　C. d b c a e　　D. e c b a d



栈与队列的应用

[2010] 2. 某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是_____。

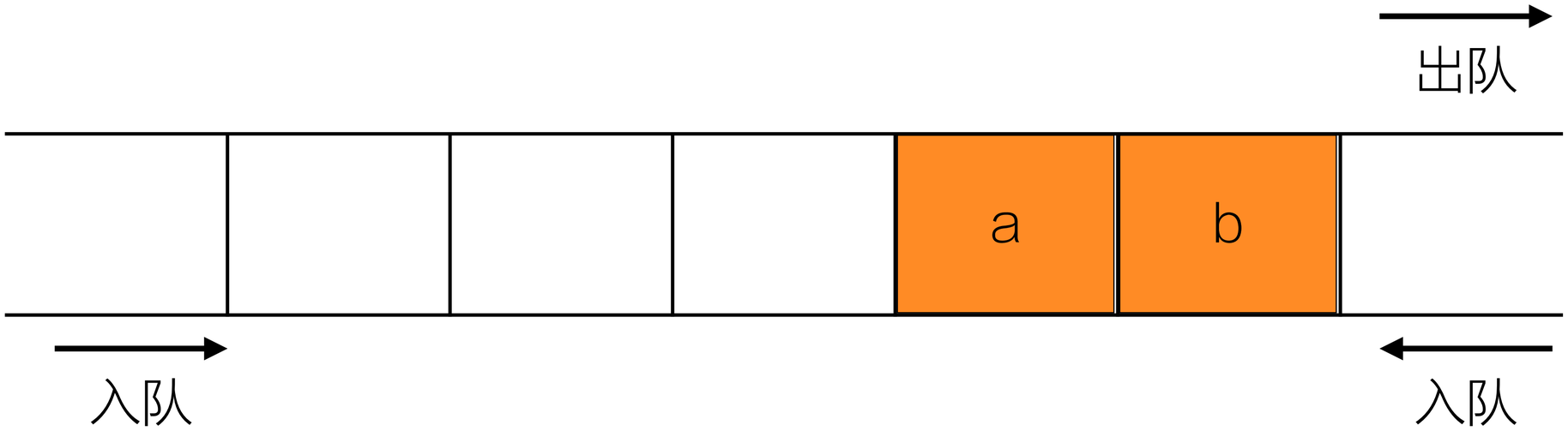
A. b a c d e　　B. d b a c e　　C. d b c a e　　D. e c b a d



栈与队列的应用

[2010] 2. 某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是_____。

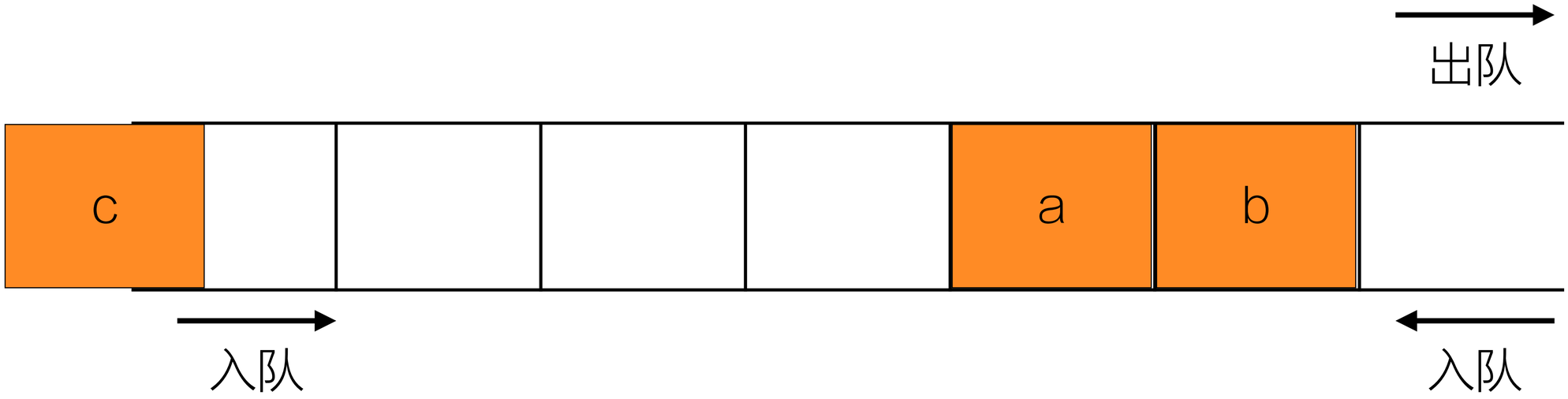
A. b a c d e　　B. d b a c e　　C. d b c a e　　D. e c b a d



栈与队列的应用

[2010] 2. 某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是_____。

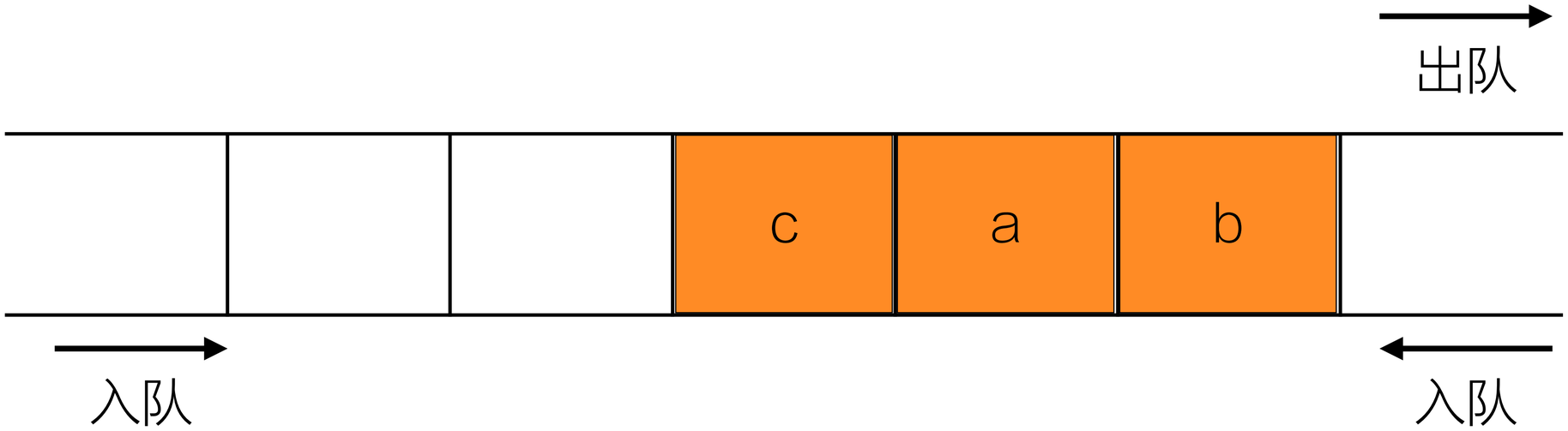
A. b a c d e　　B. d b a c e　　C. d b c a e　　D. e c b a d



栈与队列的应用

[2010] 2. 某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是_____。

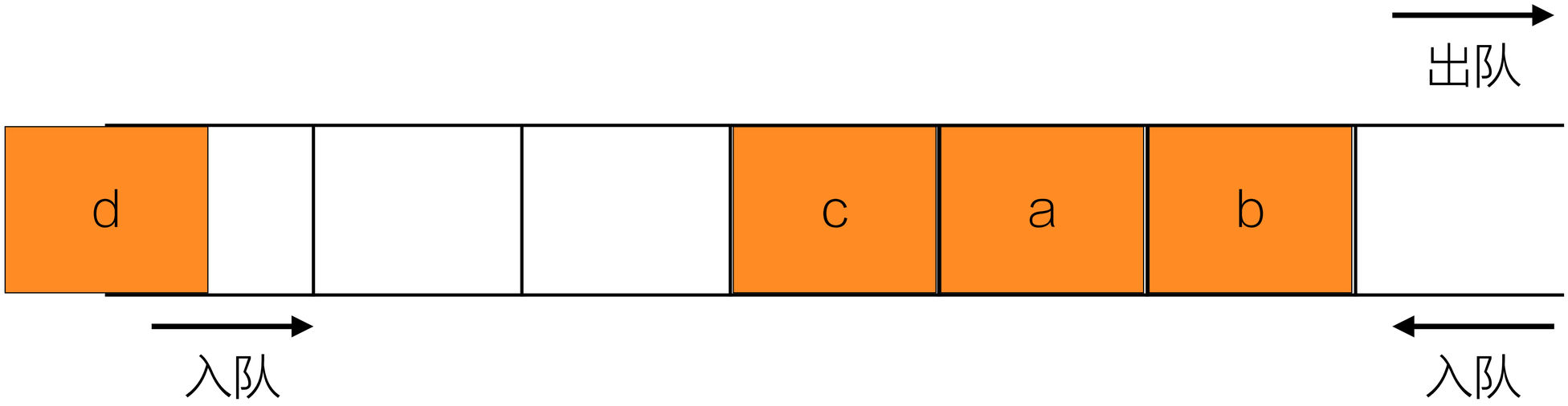
A. b a c d e　　B. d b a c e　　C. d b c a e　　D. e c b a d



栈与队列的应用

[2010] 2. 某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是_____。

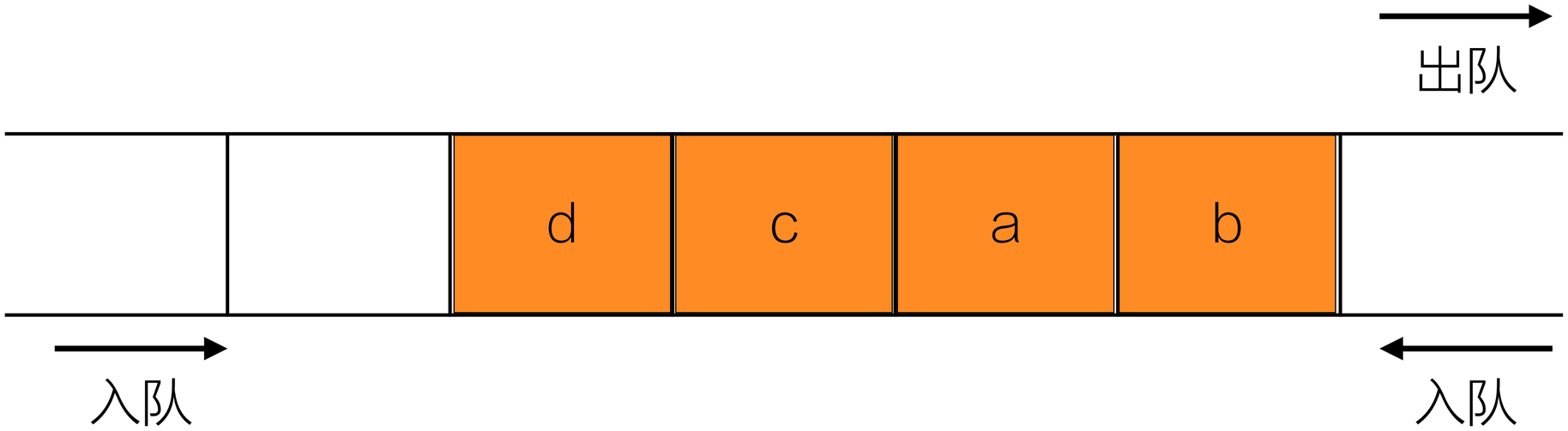
A. b a c d e　　B. d b a c e　　C. d b c a e　　D. e c b a d



栈与队列的应用

[2010] 2. 某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是_____。

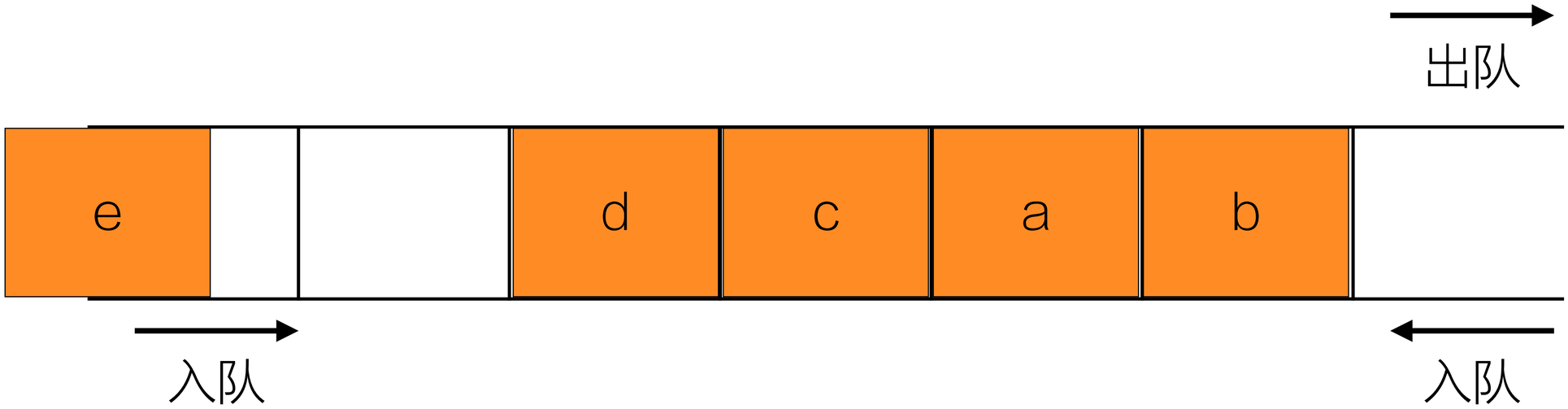
A. b a c d e　　B. d b a c e　　C. d b c a e　　D. e c b a d



栈与队列的应用

[2010] 2. 某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是_____。

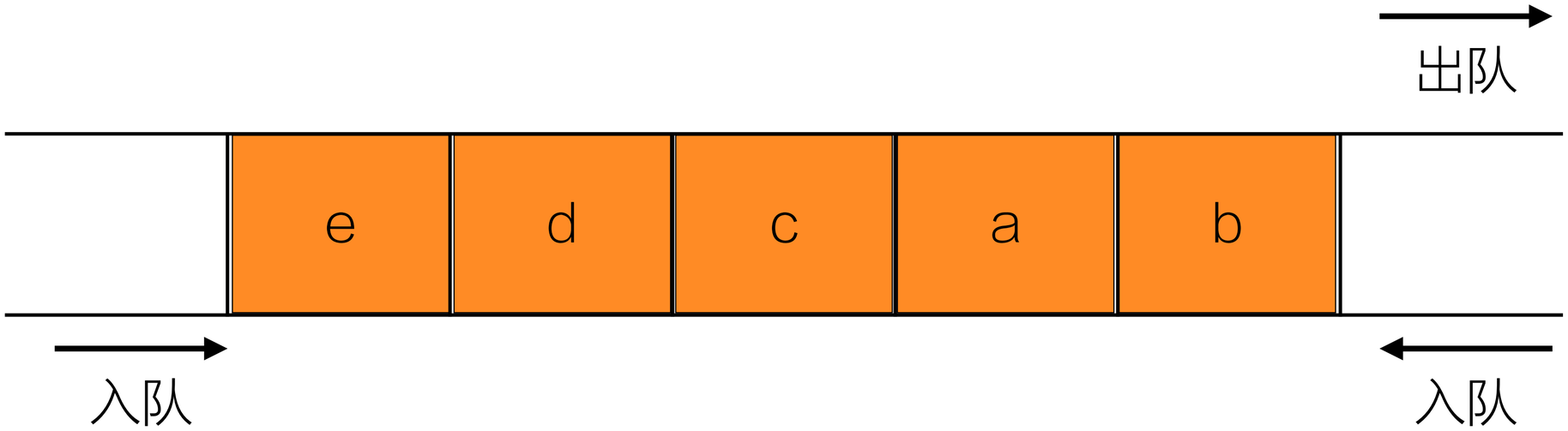
A. b a c d e　　B. d b a c e　　C. d b c a e　　D. e c b a d



栈与队列的应用

[2010] 2. 某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是_____。

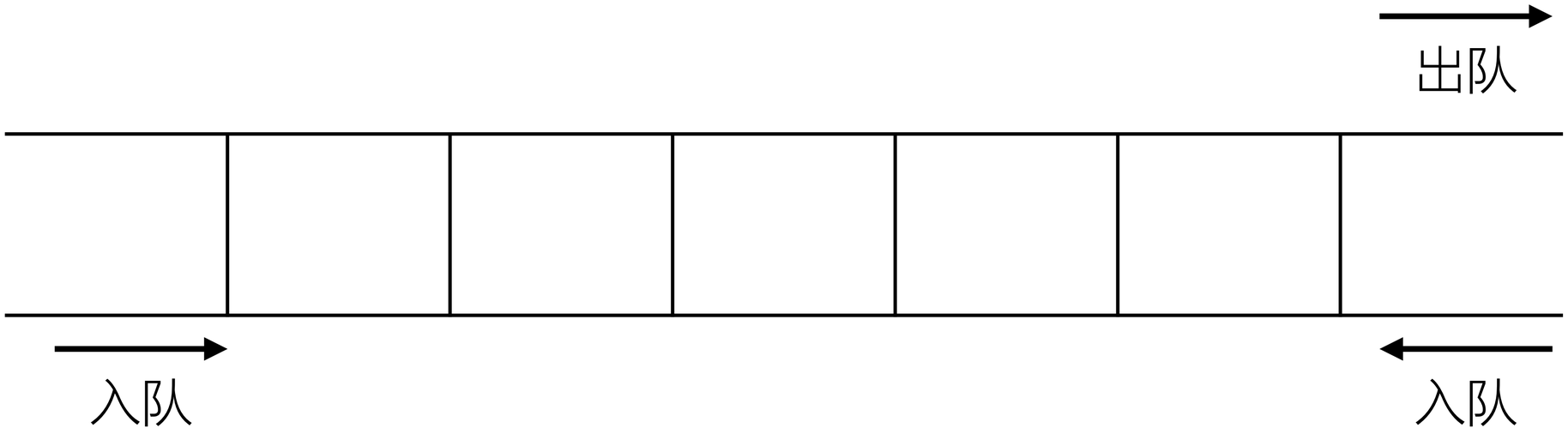
A. b a c d e　　B. d b a c e　　C. d b c a e　　D. e c b a d



栈与队列的应用

[2010] 2. 某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是_____。

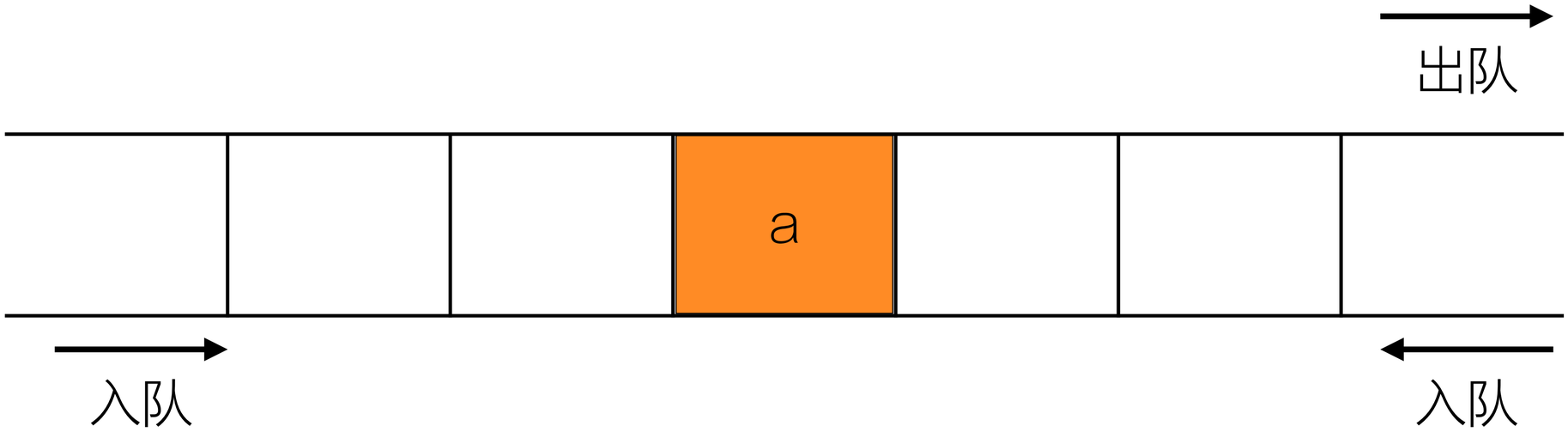
A. b a c d e　　B. d b a c e　　C. d b c a e　　D. e c b a d



栈与队列的应用

[2010] 2. 某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是_____。

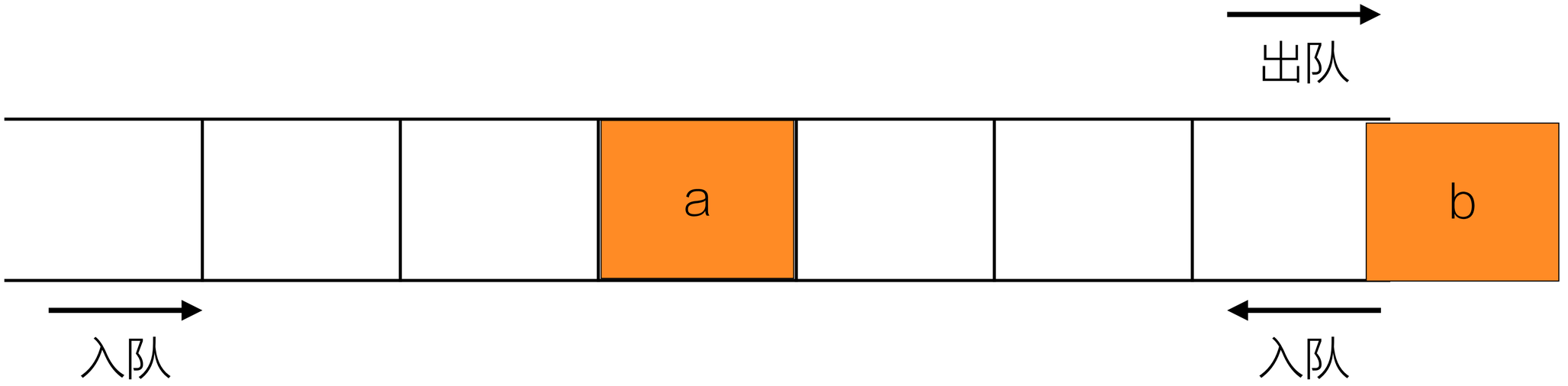
A. b a c d e　　B. d b a c e　　C. d b c a e　　D. e c b a d



栈与队列的应用

[2010] 2. 某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是_____。

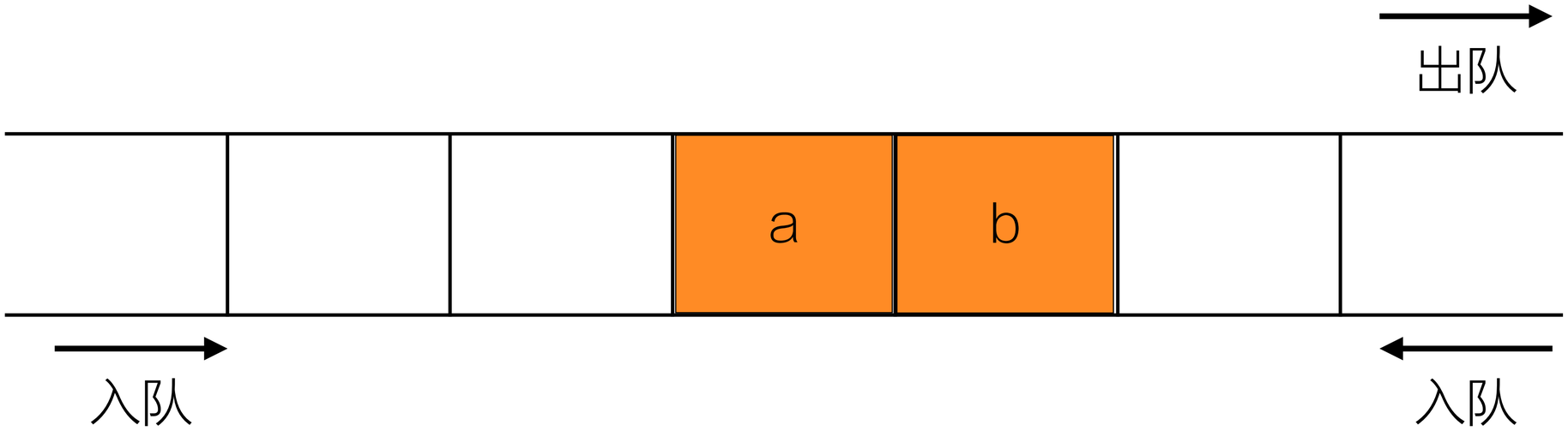
A. b a c d e　　B. d b a c e　　C. d b c a e　　D. e c b a d



栈与队列的应用

[2010] 2. 某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是_____。

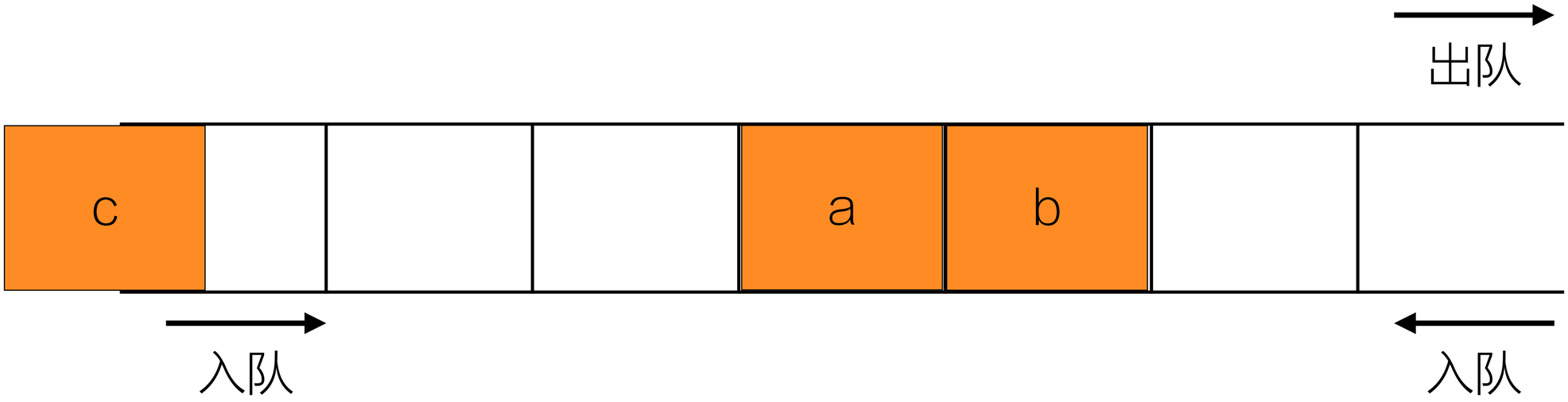
A. b a c d e　　B. d b a c e　　C. d b c a e　　D. e c b a d



栈与队列的应用

[2010] 2. 某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是_____。

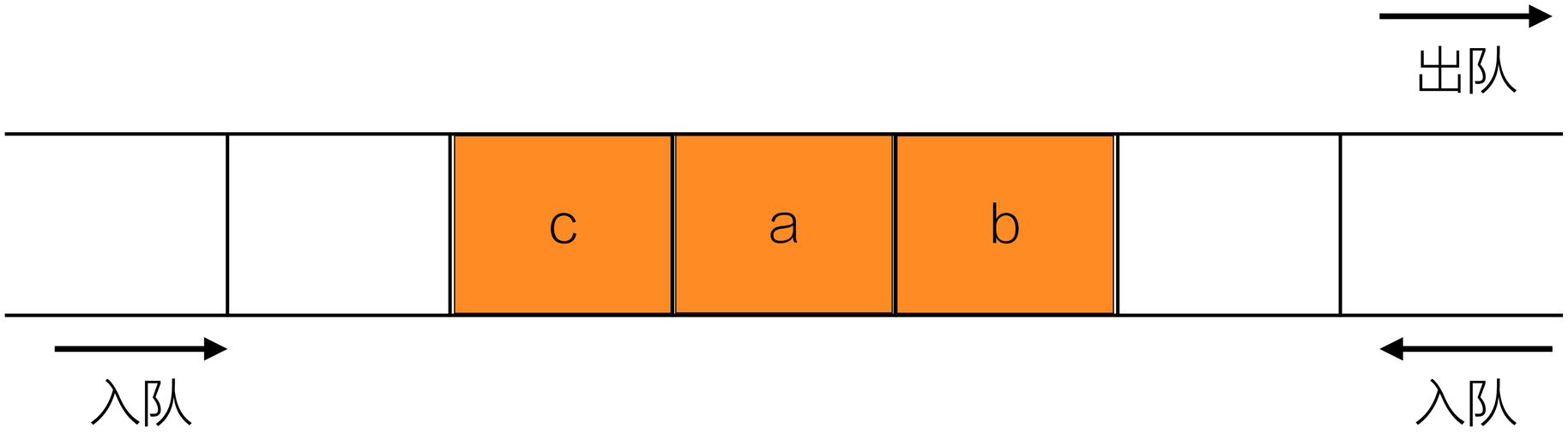
A. b a c d e　　B. d b a c e　　C. d b c a e　　D. e c b a d



栈与队列的应用

[2010] 2. 某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是_____。

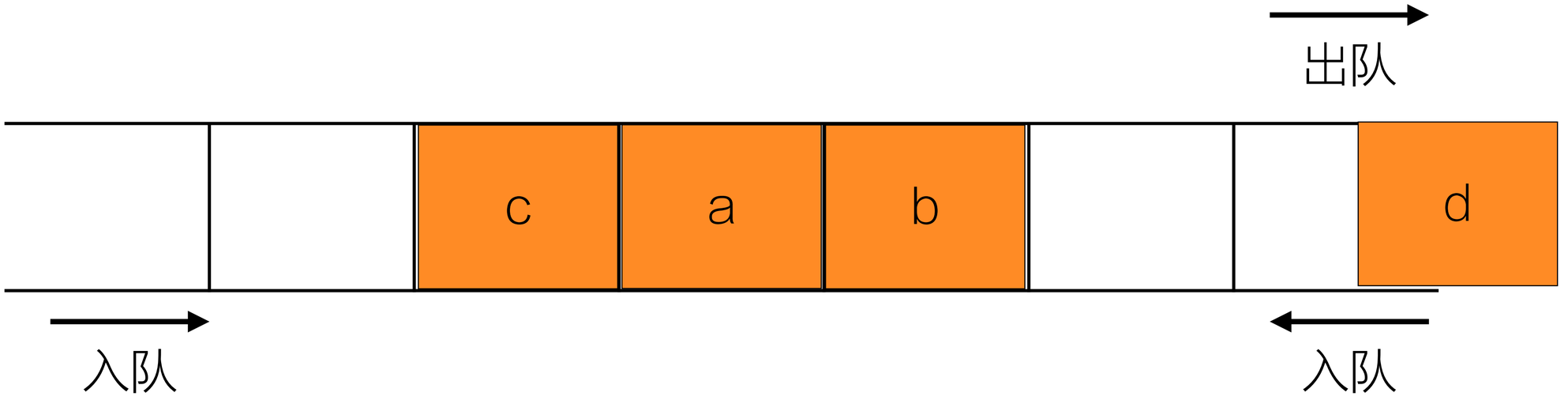
A. b a c d e　　B. d b a c e　　C. d b c a e　　D. e c b a d



栈与队列的应用

[2010] 2. 某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是_____。

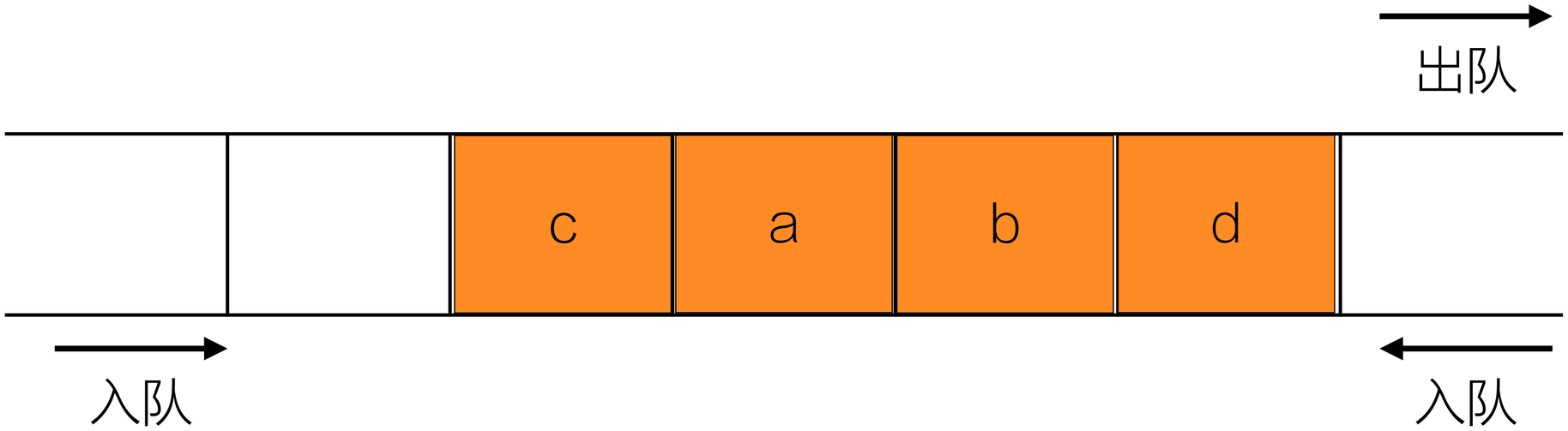
A. b a c d e　　B. d b a c e　　C. d b c a e　　D. e c b a d



栈与队列的应用

[2010] 2. 某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是_____。

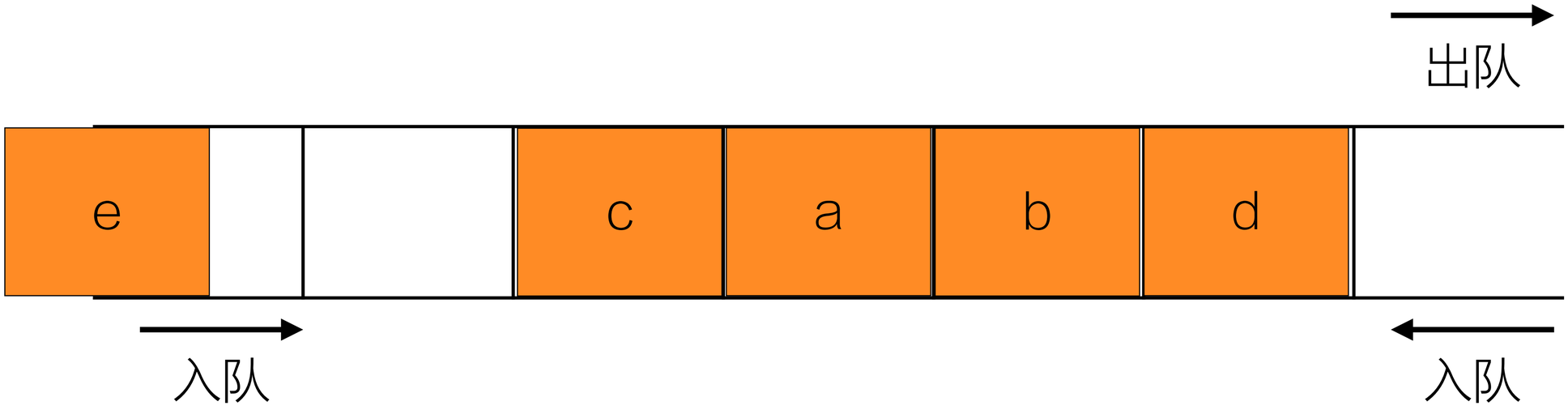
A. b a c d e　　B. d b a c e　　C. d b c a e　　D. e c b a d



栈与队列的应用

[2010] 2. 某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是_____。

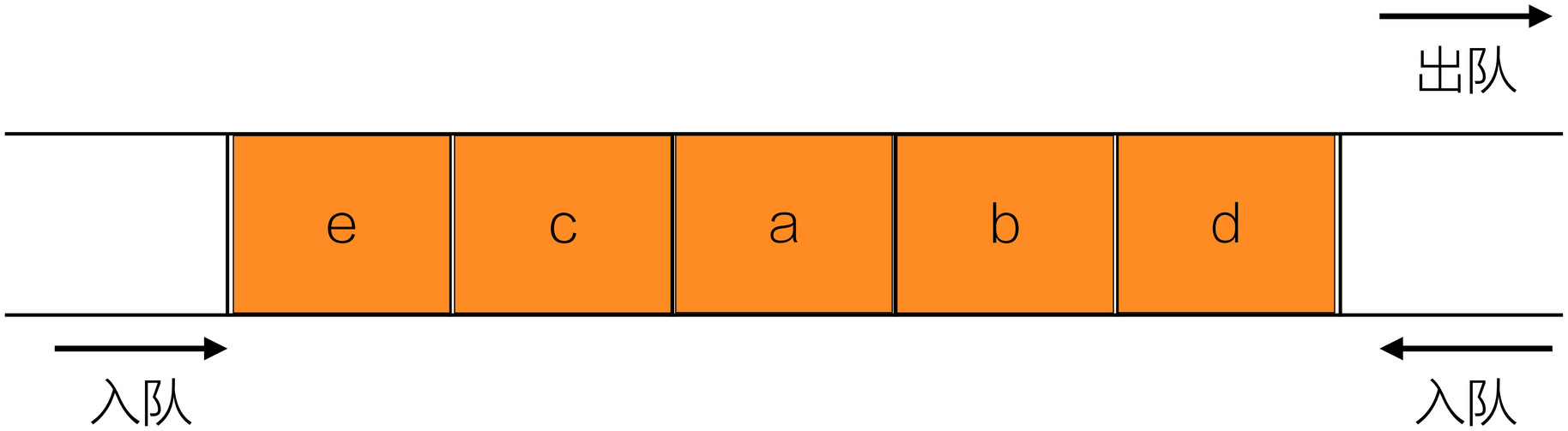
A. b a c d e　　B. d b a c e　　C. d b c a e　　D. e c b a d



栈与队列的应用

[2010] 2. 某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是_____。

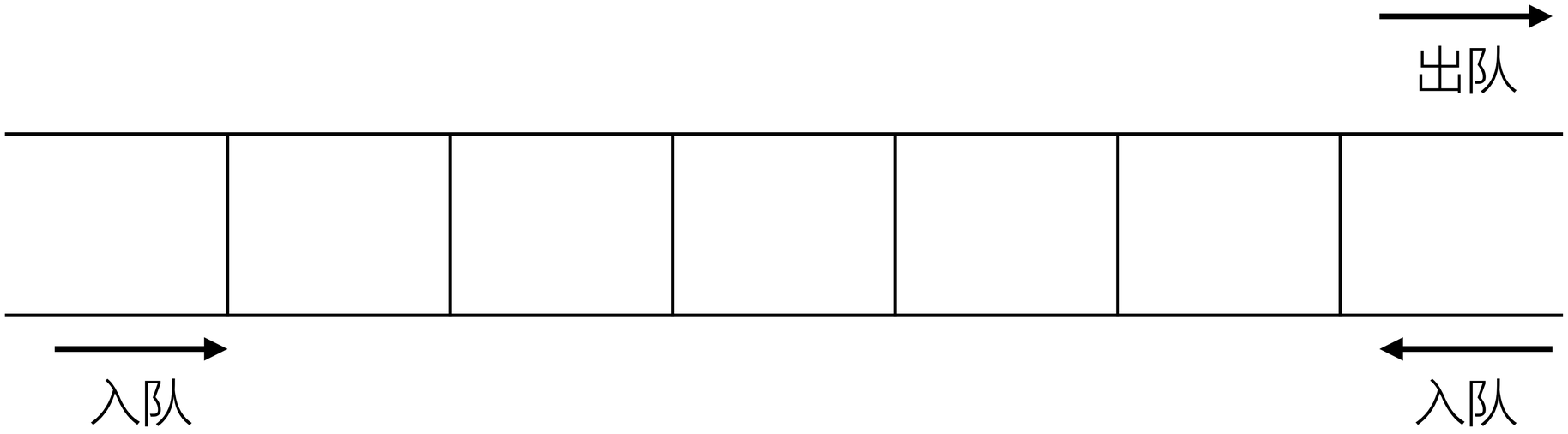
A. b a c d e　　B. d b a c e　　C. d b c a e　　D. e c b a d



栈与队列的应用

[2010] 2. 某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是_____。

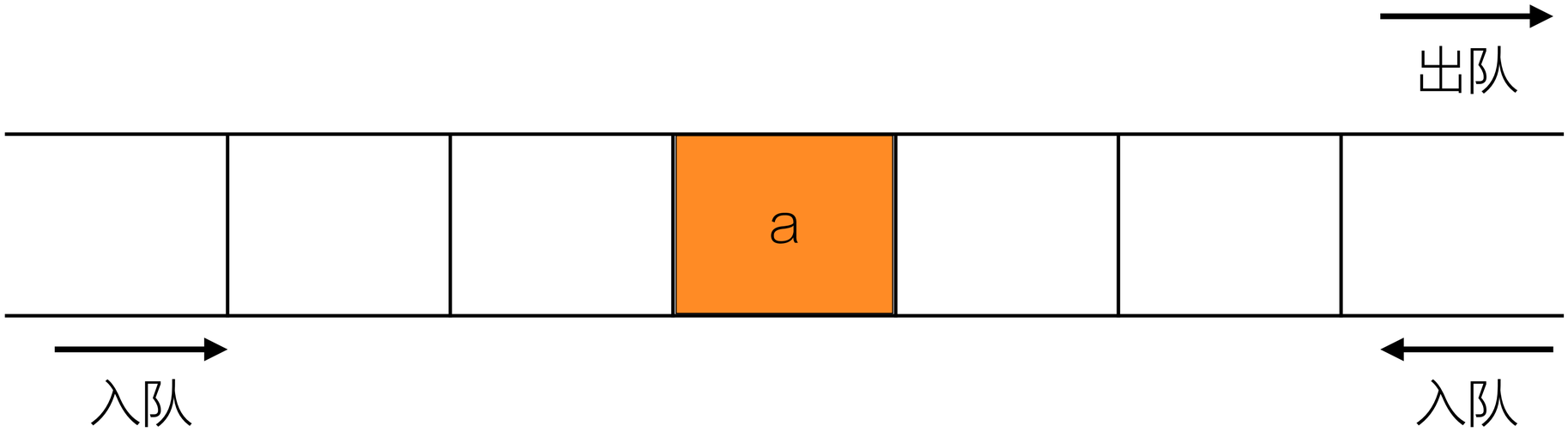
A. b a c d e　　B. d b a c e　　C. d b c a e　　D. e c b a d



栈与队列的应用

[2010] 2. 某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是_____。

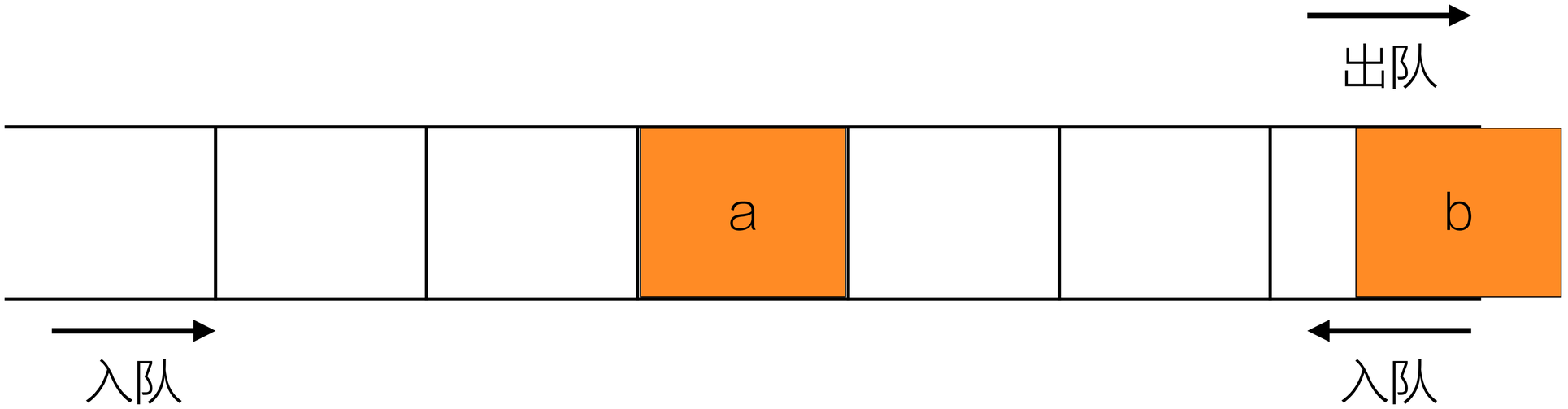
A. b a c d e　　B. d b a c e　　C. d b c a e　　D. e c b a d



栈与队列的应用

[2010] 2. 某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是_____。

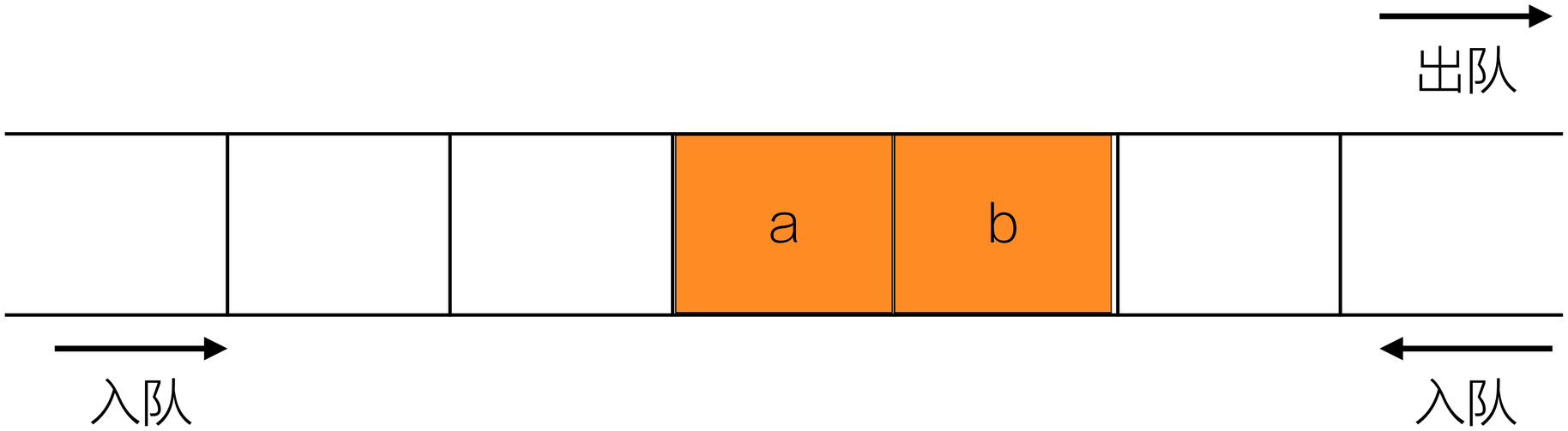
A. b a c d e　　B. d b a c e　　C. d b c a e　　D. e c b a d



栈与队列的应用

[2010] 2. 某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是_____。

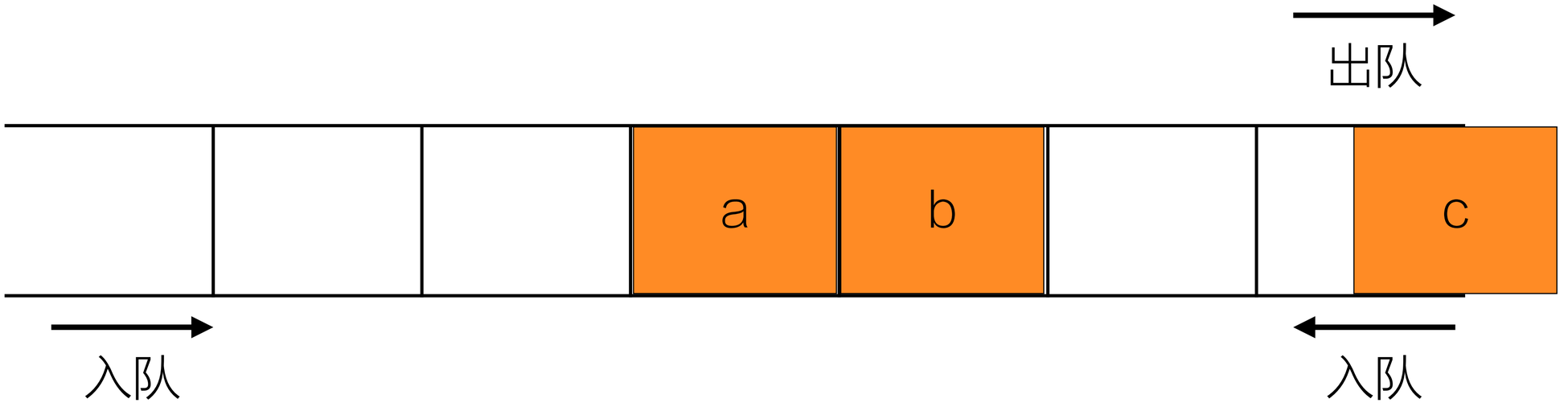
A. b a c d e　　B. d b a c e　　C. d b c a e　　D. e c b a d



栈与队列的应用

[2010] 2. 某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是_____。

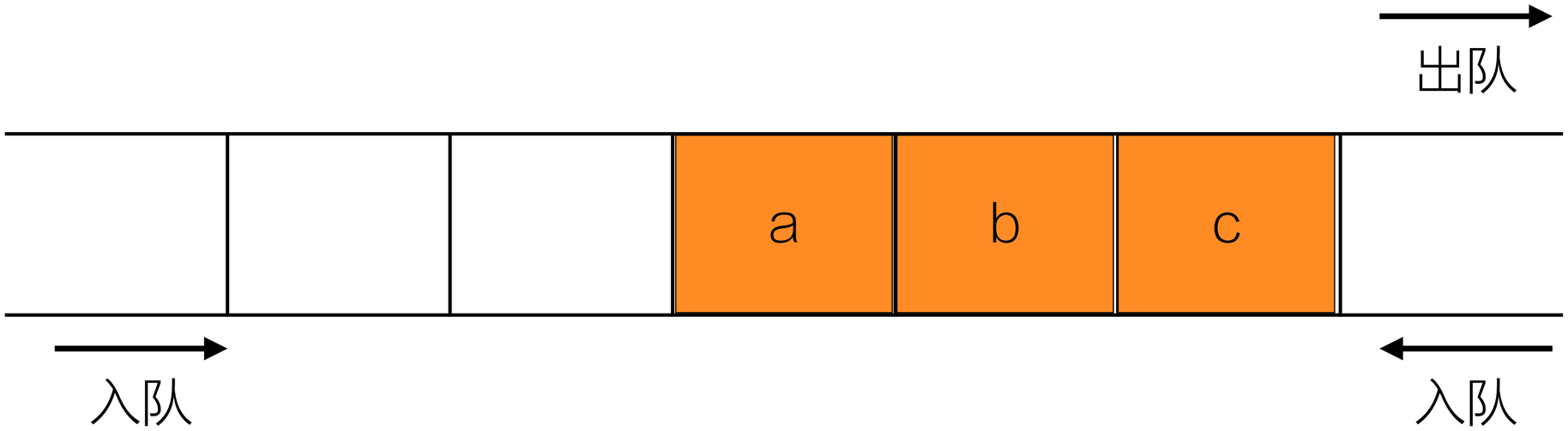
A. b a c d e　　B. d b a c e　　C. d b c a e　　D. e c b a d



栈与队列的应用

[2010] 2. 某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是_____。

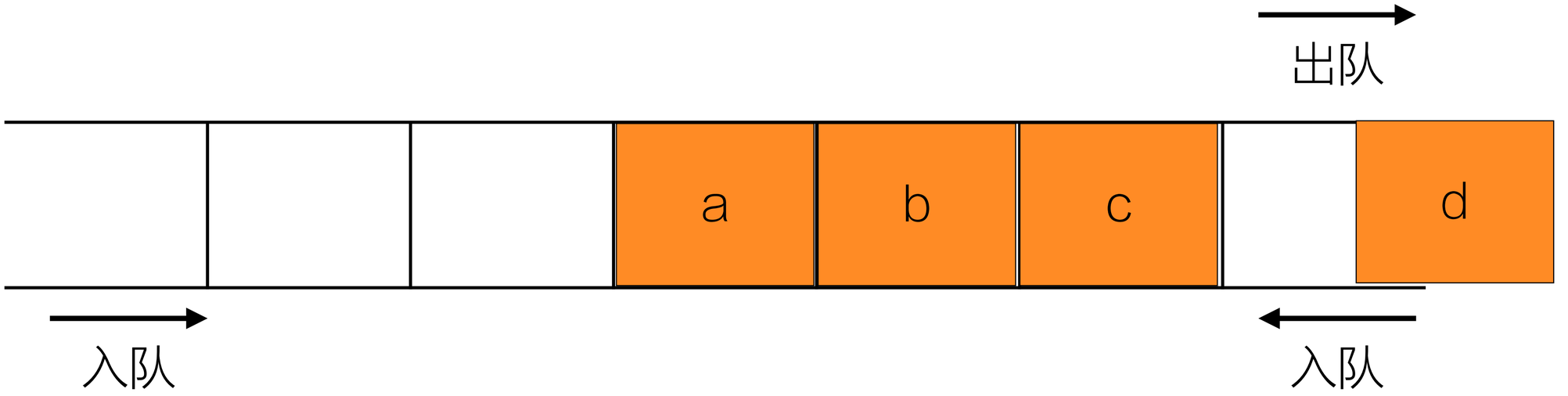
A. b a c d e　　B. d b a c e　　C. d b c a e　　D. e c b a d



栈与队列的应用

[2010] 2. 某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是_____。

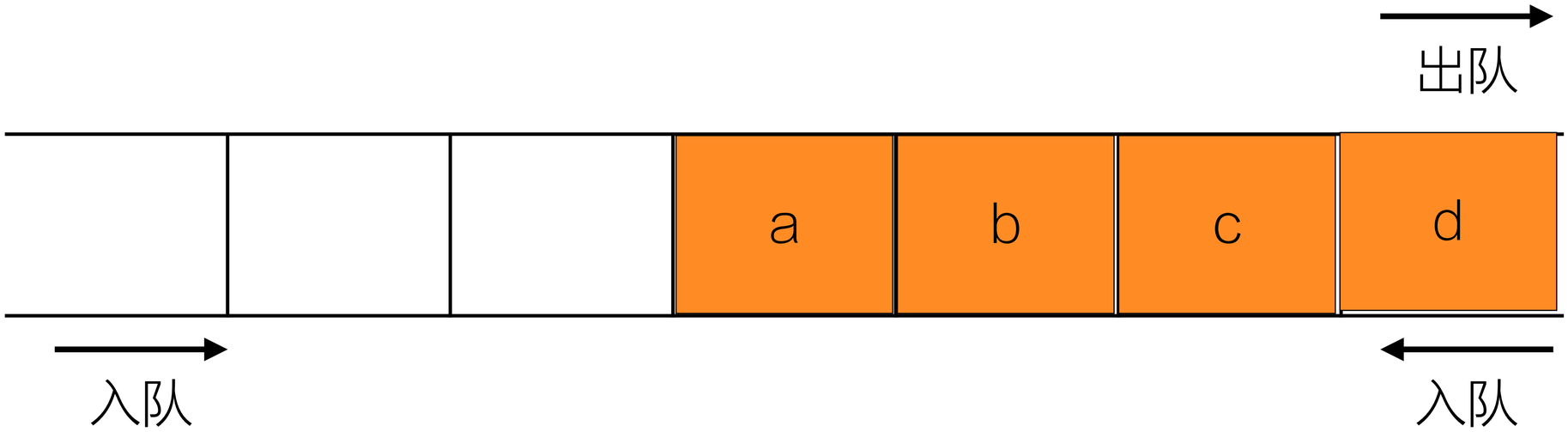
A. b a c d e　　B. d b a c e　　C. d b c a e　　D. e c b a d



栈与队列的应用

[2010] 2. 某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是_____。

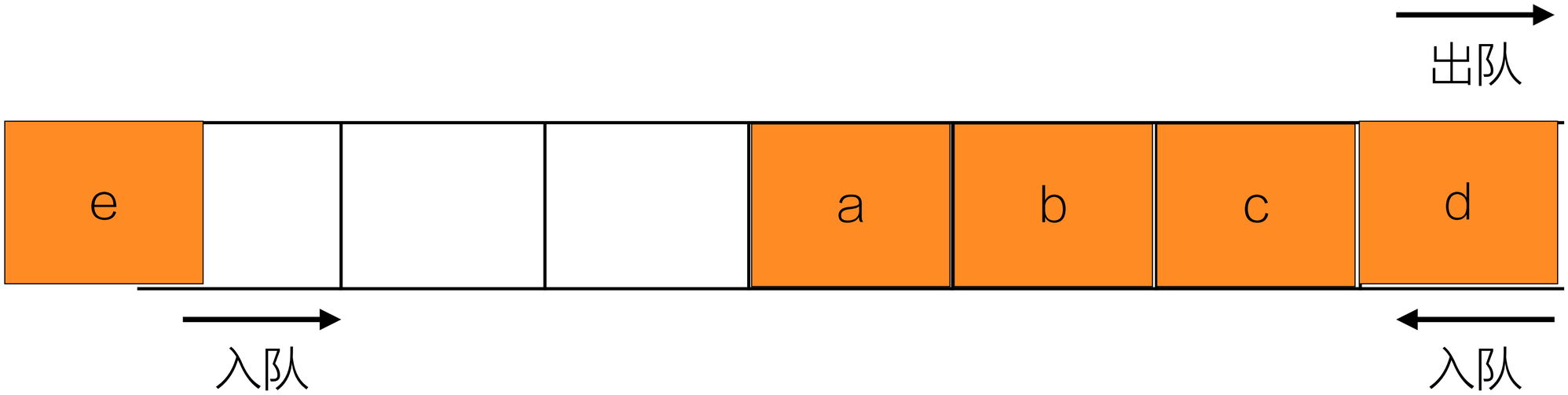
A. b a c d e　　B. d b a c e　　C. d b c a e　　D. e c b a d



栈与队列的应用

[2010] 2. 某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是_____。

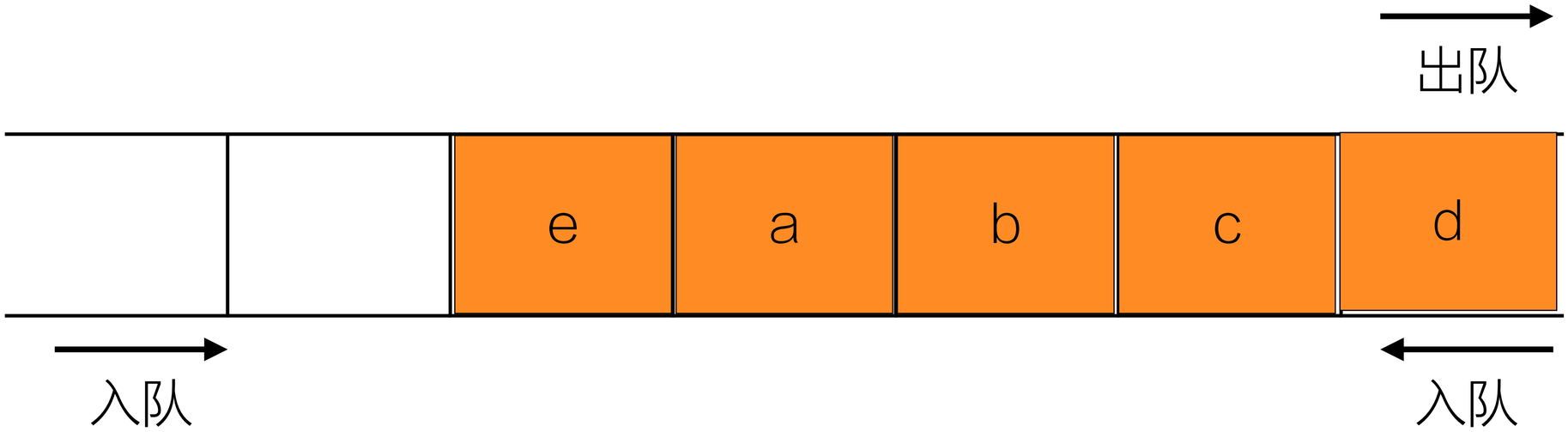
A. b a c d e　　B. d b a c e　　C. d b c a e　　D. e c b a d



栈与队列的应用

[2010] 2. 某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是_____。

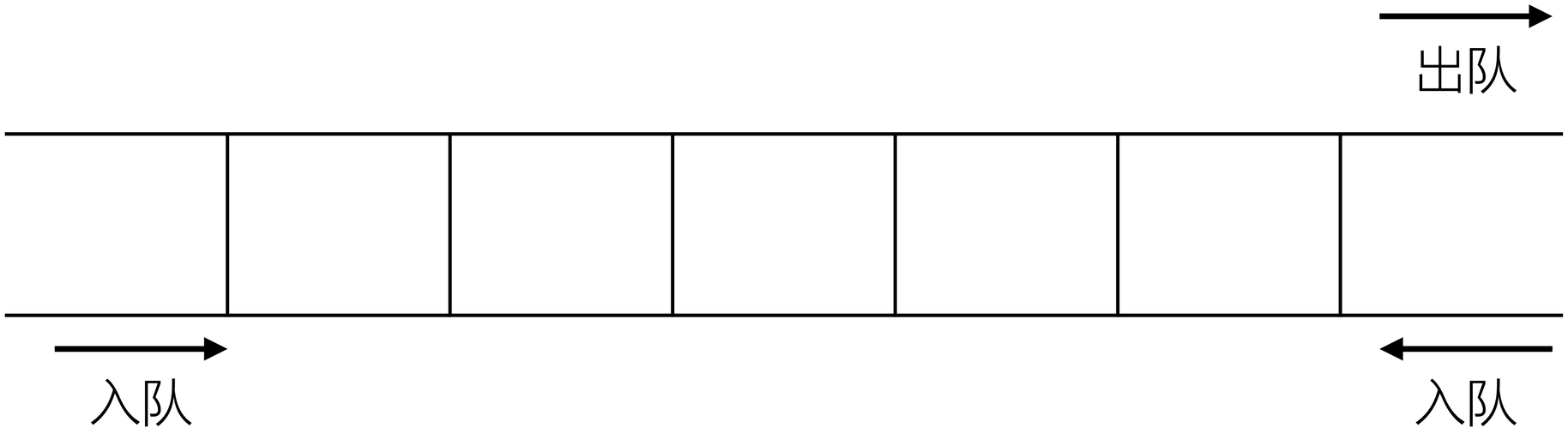
A. b a c d e　　B. d b a c e　　C. d b c a e　　D. e c b a d



栈与队列的应用

[2010] 2. 某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是_____。

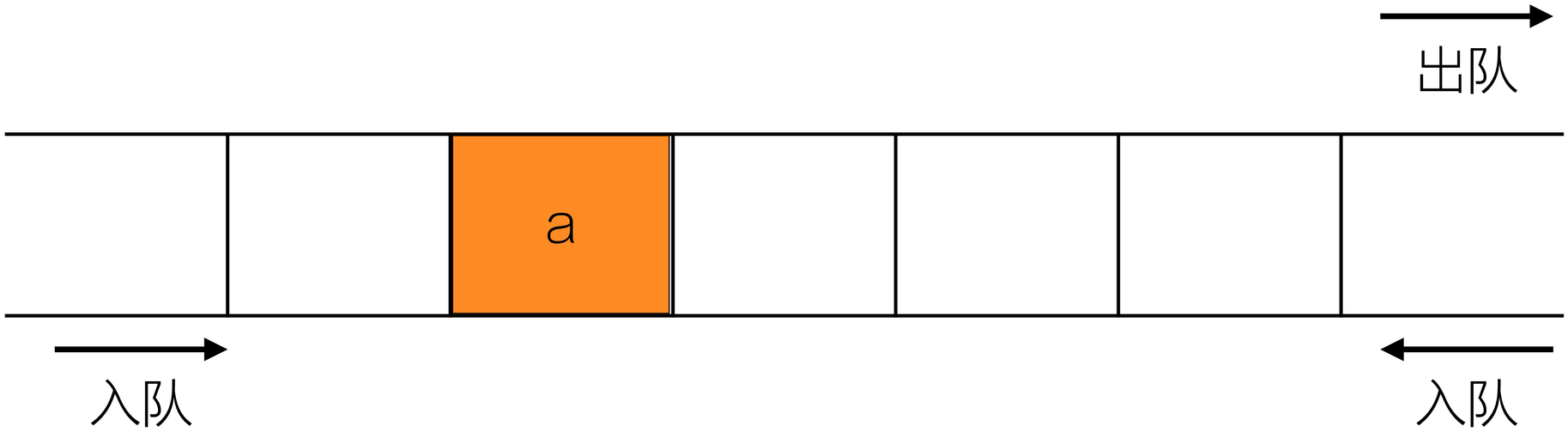
A. b a c d e　　B. d b a c e　　C. d b c a e　　D. e c b a d



栈与队列的应用

[2010] 2. 某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是_____。

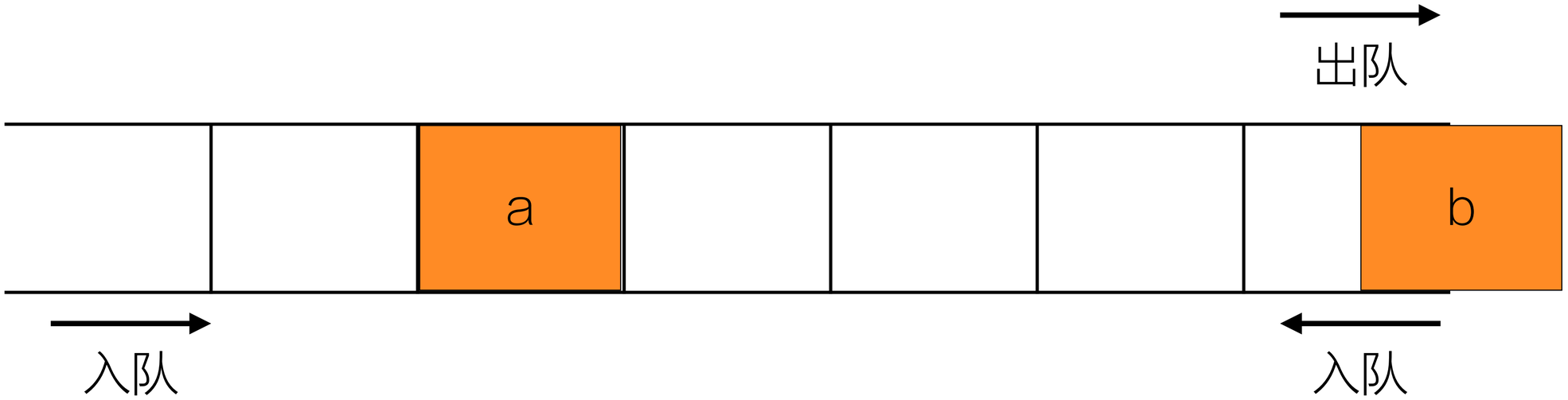
A. b a c d e　　B. d b a c e　　C. d b c a e　　D. e c b a d



栈与队列的应用

[2010] 2. 某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是_____。

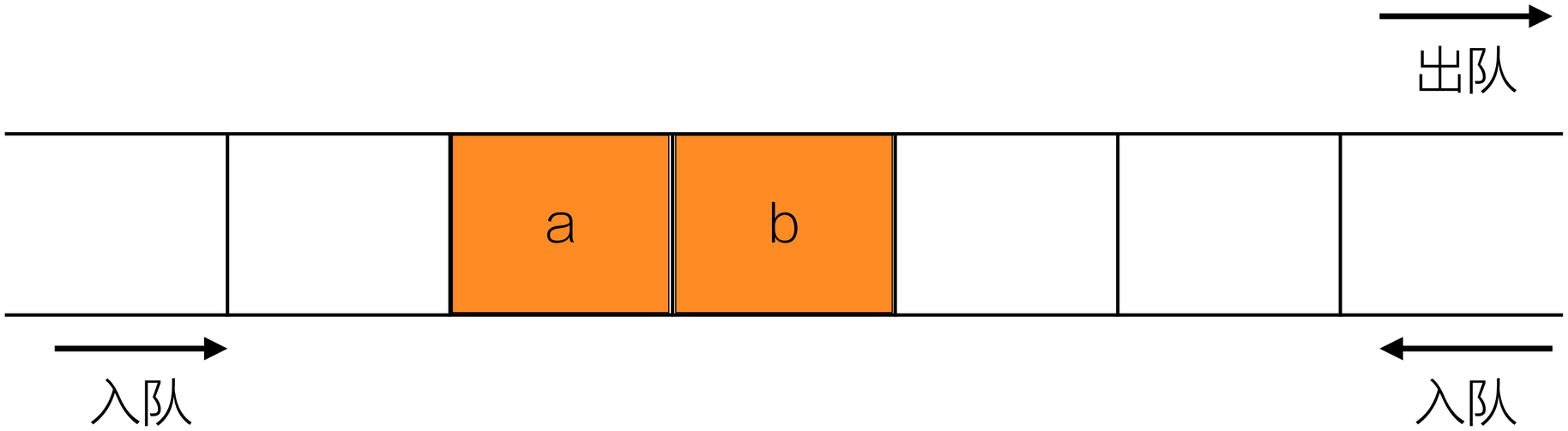
A. b a c d e　　B. d b a c e　　C. d b c a e　　D. e c b a d



栈与队列的应用

[2010] 2. 某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是_____。

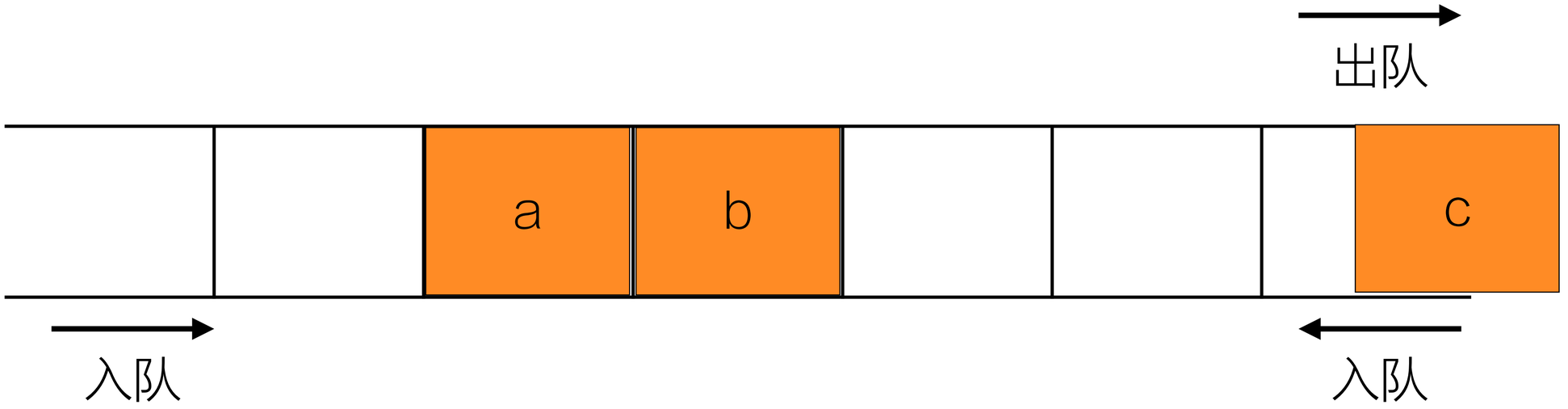
A. b a c d e　　B. d b a c e　　C. d b c a e　　D. e c b a d



栈与队列的应用

[2010] 2. 某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是_____。

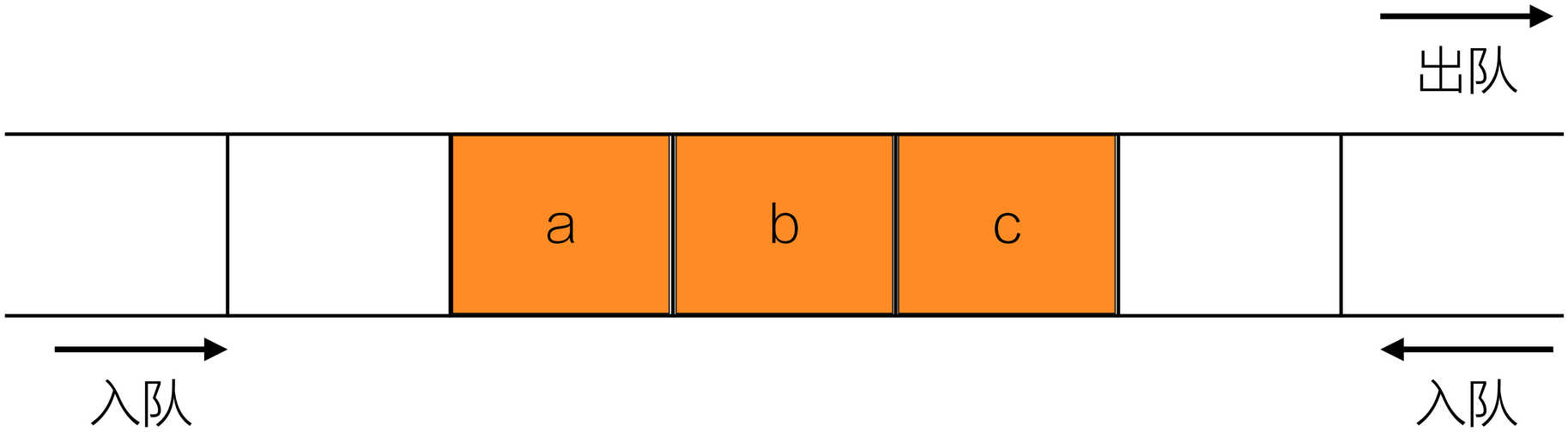
A. b a c d e　　B. d b a c e　　C. d b c a e　　D. e c b a d



栈与队列的应用

[2010] 2. 某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是_____。

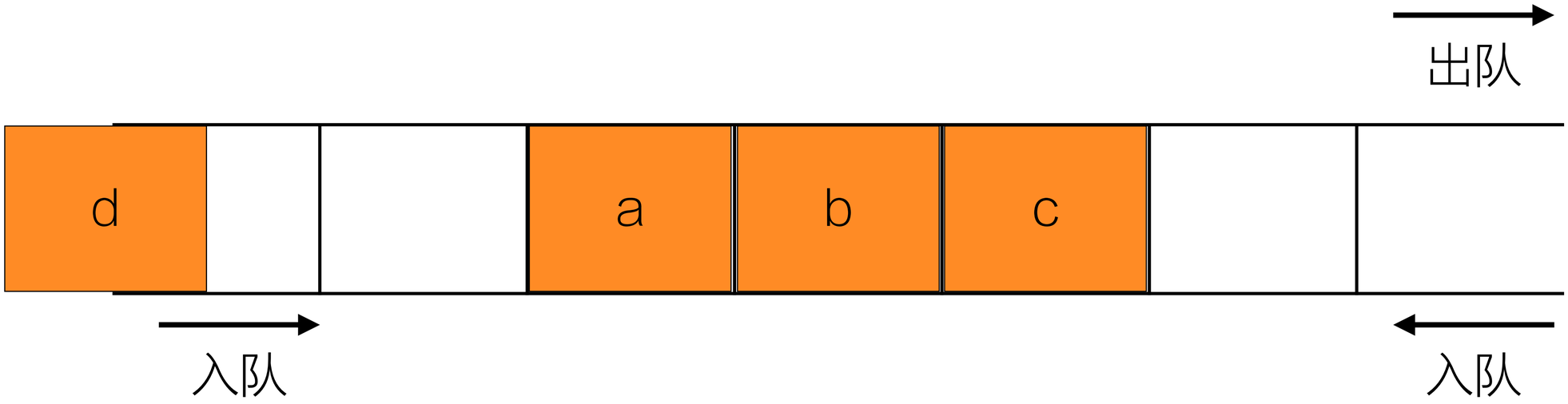
A. b a c d e　　B. d b a c e　　C. d b c a e　　D. e c b a d



栈与队列的应用

[2010] 2. 某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是_____。

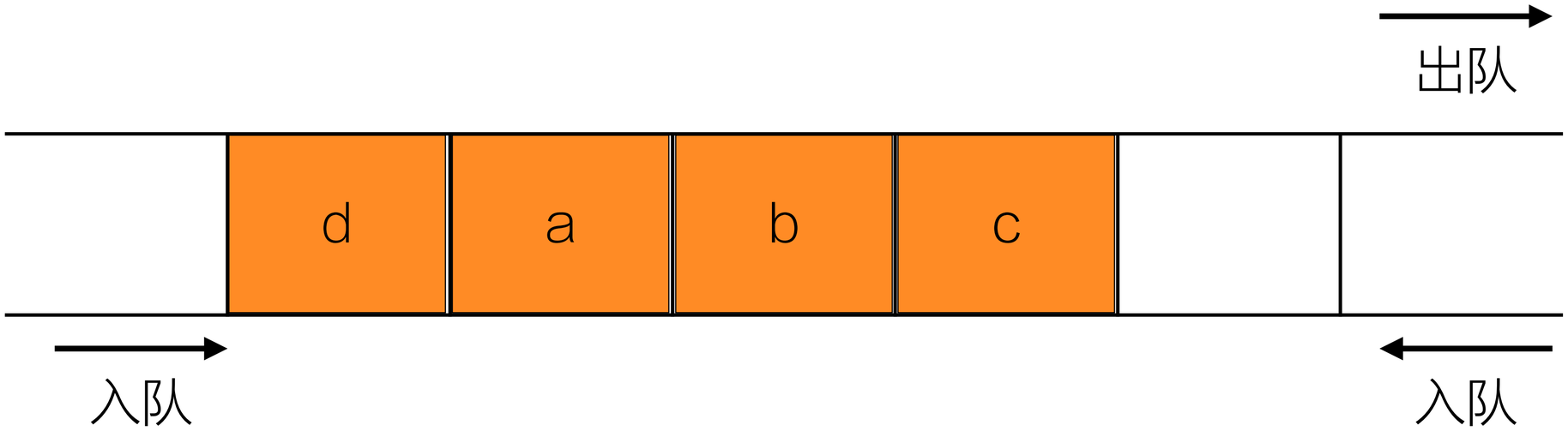
A. b a c d e　　B. d b a c e　　C. d b c a e　　D. e c b a d



栈与队列的应用

[2010] 2. 某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是_____。

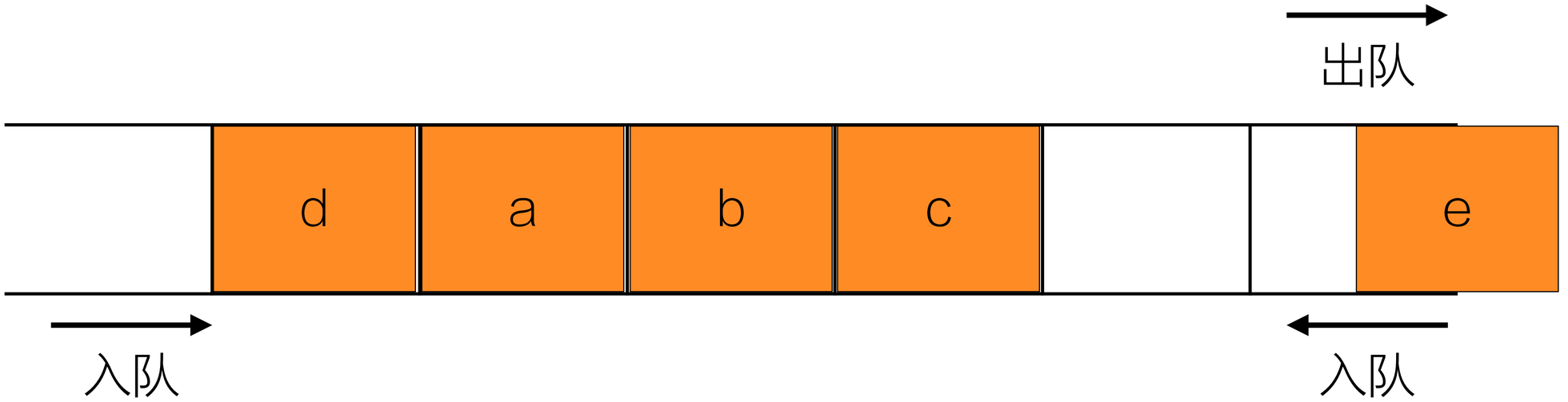
A. b a c d e　　B. d b a c e　　C. d b c a e　　D. e c b a d



栈与队列的应用

[2010] 2. 某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是_____。

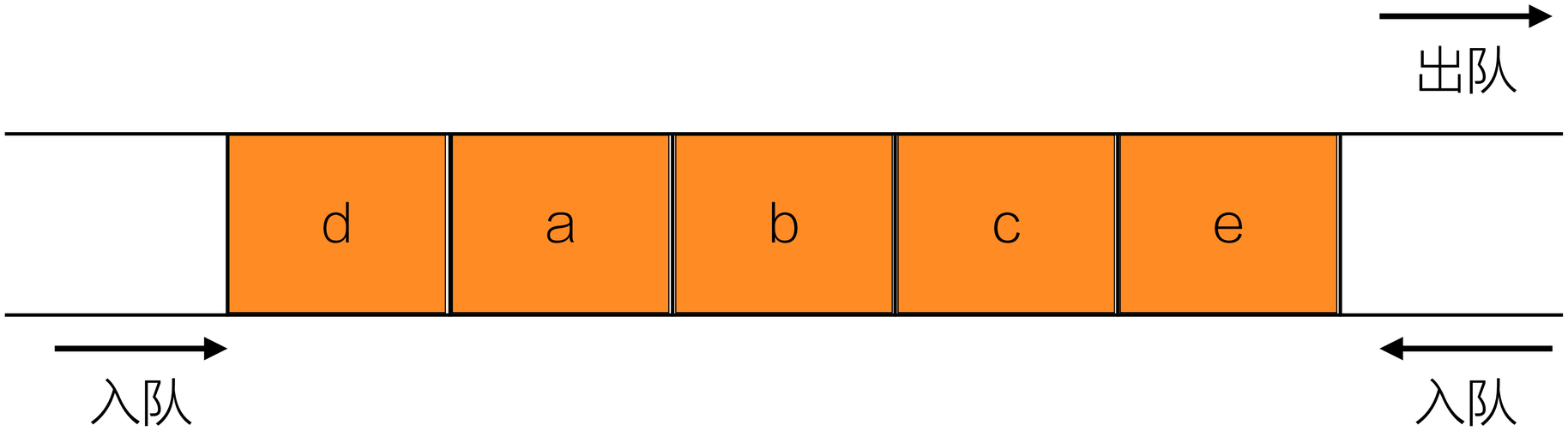
A. b a c d e　　B. d b a c e　　C. d b c a e　　D. e c b a d



栈与队列的应用

[2010] 2. 某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是_____。

A. b a c d e　　B. d b a c e　　C. d b c a e　　D. e c b a d



栈与队列的应用

[2010] 2. 某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是_____。

A. b a c d e　　B. d b a c e　　C. d b c a e　　D. e c b a d

栈与队列的应用

[2010] 2. 某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是_____。

A. b a c d e　　B. d b a c e　　C. d b c a e　　D. e c b a d

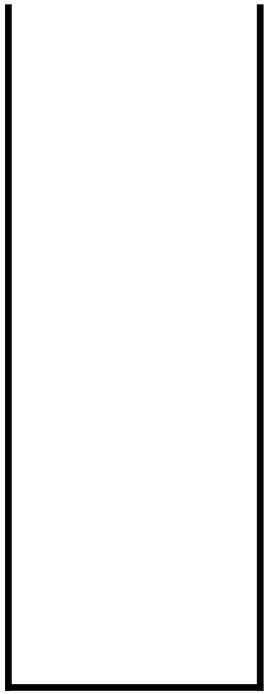
栈与队列的应用

[2011] 2. 元素 a, b, c, d, e 依次进入初始为空的栈中，若元素进栈后可停留、可出栈，直到所有元素都出栈，则在所有可能的出栈序列中，以元素 d 开头的序列个数是____。

A. 3　　B. 4　　C. 5　　D. 6

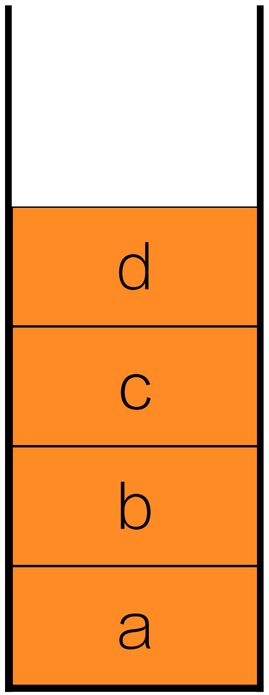
[2011] 2. 元素 a, b, c, d, e 依次进入初始为空的栈中，若元素进栈后可停留、可出栈，直到所有元素都出栈，则在所有可能的出栈序列中，以元素 d 开头的序列个数是____。

A. 3　　B. 4　　C. 5　　D. 6



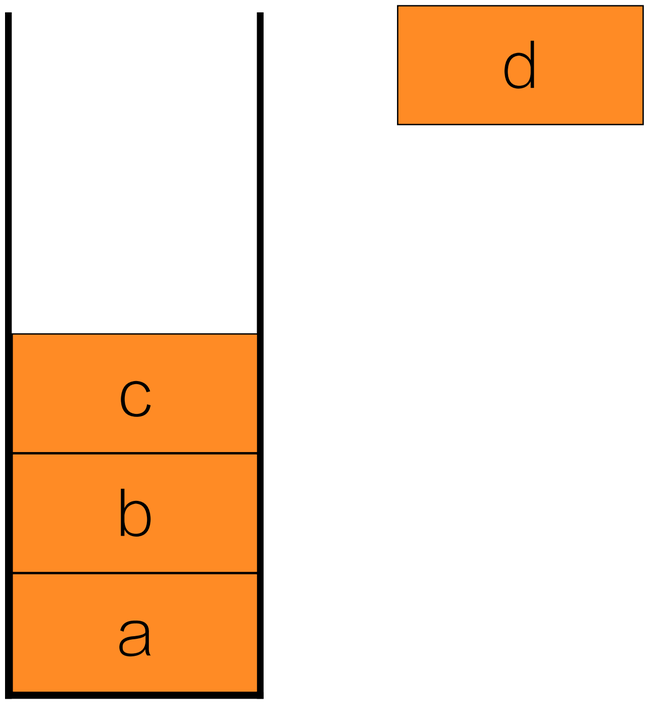
[2011] 2. 元素 a, b, c, d, e 依次进入初始为空的栈中，若元素进栈后可停留、可出栈，直到所有元素都出栈，则在所有可能的出栈序列中，以元素 d 开头的序列个数是____。

A. 3　　B. 4　　C. 5　　D. 6



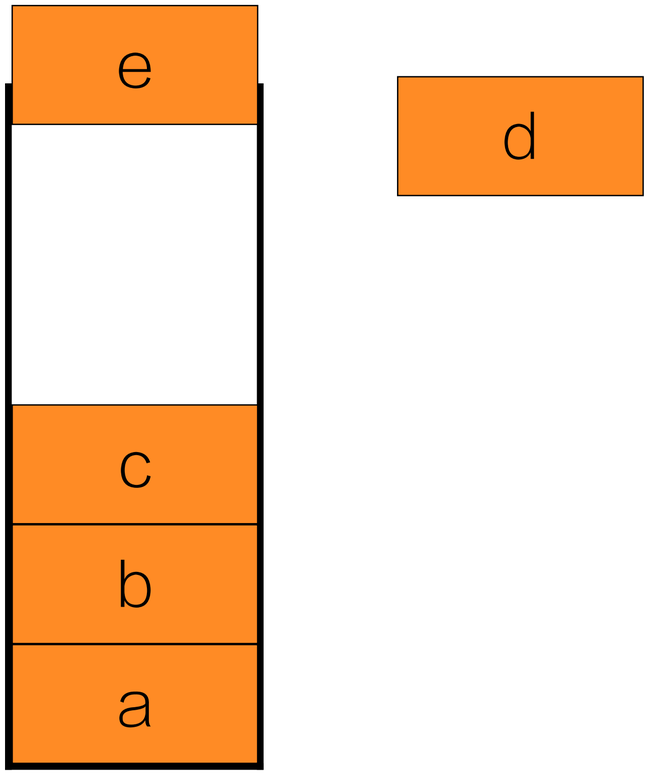
[2011] 2. 元素 a, b, c, d, e 依次进入初始为空的栈中，若元素进栈后可停留、可出栈，直到所有元素都出栈，则在所有可能的出栈序列中，以元素 d 开头的序列个数是____。

A. 3　　B. 4　　C. 5　　D. 6



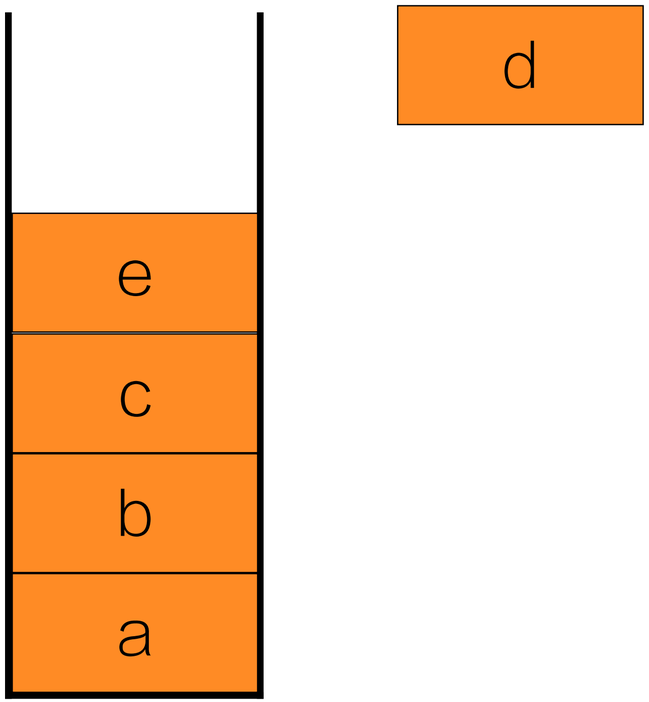
[2011] 2. 元素 a, b, c, d, e 依次进入初始为空的栈中，若元素进栈后可停留、可出栈，直到所有元素都出栈，则在所有可能的出栈序列中，以元素 d 开头的序列个数是____。

A. 3　　B. 4　　C. 5　　D. 6



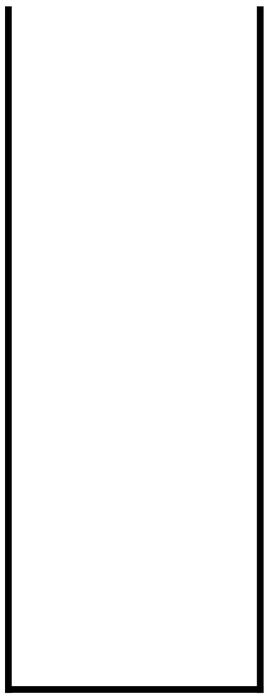
[2011] 2. 元素 a, b, c, d, e 依次进入初始为空的栈中，若元素进栈后可停留、可出栈，直到所有元素都出栈，则在所有可能的出栈序列中，以元素 d 开头的序列个数是____。

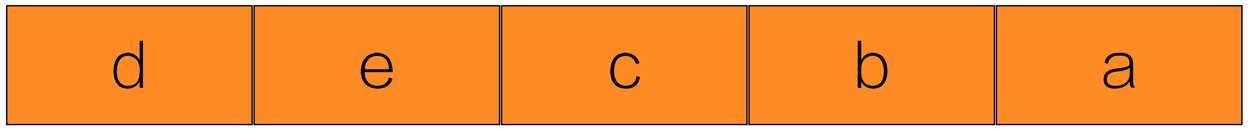
A. 3　　B. 4　　C. 5　　D. 6



[2011] 2. 元素 a, b, c, d, e 依次进入初始为空的栈中，若元素进栈后可停留、可出栈，直到所有元素都出栈，则在所有可能的出栈序列中，以元素 d 开头的序列个数是____。

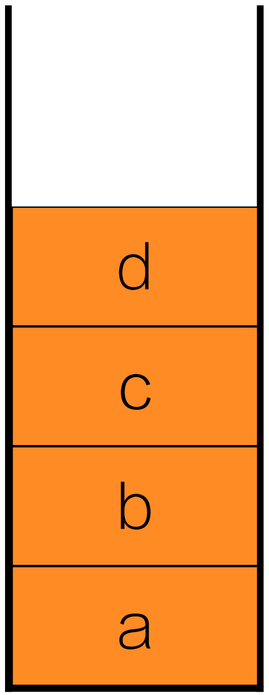
A. 3　　B. 4　　C. 5　　D. 6

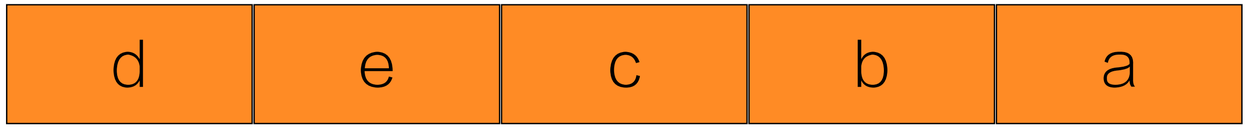




[2011] 2. 元素 a, b, c, d, e 依次进入初始为空的栈中，若元素进栈后可停留、可出栈，直到所有元素都出栈，则在所有可能的出栈序列中，以元素 d 开头的序列个数是____。

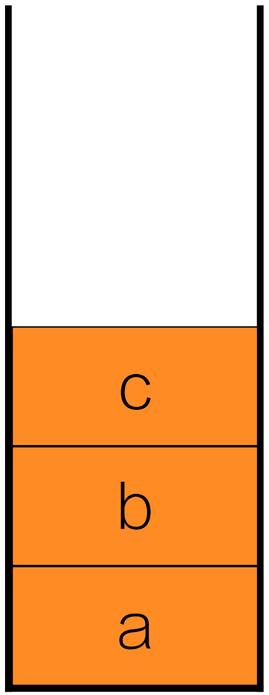
A. 3　　B. 4　　C. 5　　D. 6

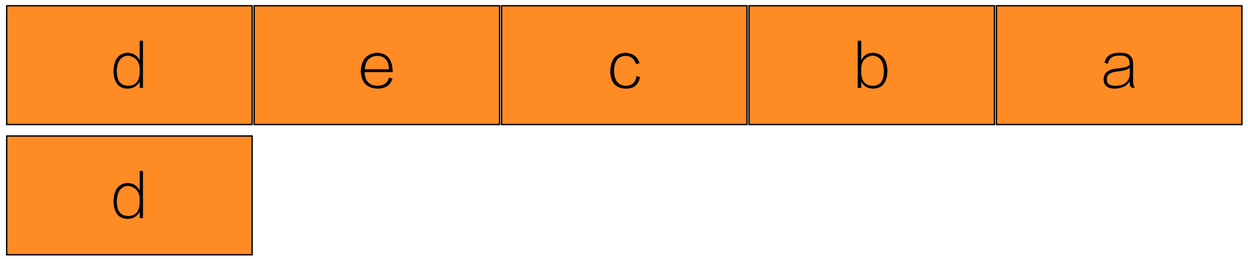




[2011] 2. 元素 a, b, c, d, e 依次进入初始为空的栈中，若元素进栈后可停留、可出栈，直到所有元素都出栈，则在所有可能的出栈序列中，以元素 d 开头的序列个数是____。

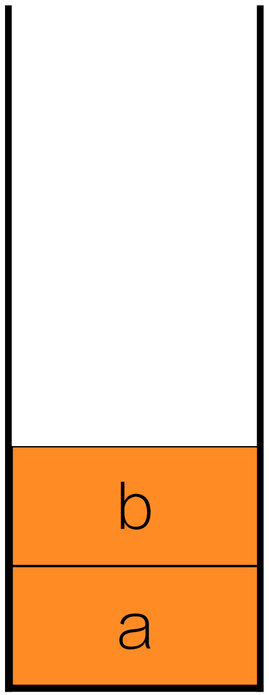
A. 3　　B. 4　　C. 5　　D. 6

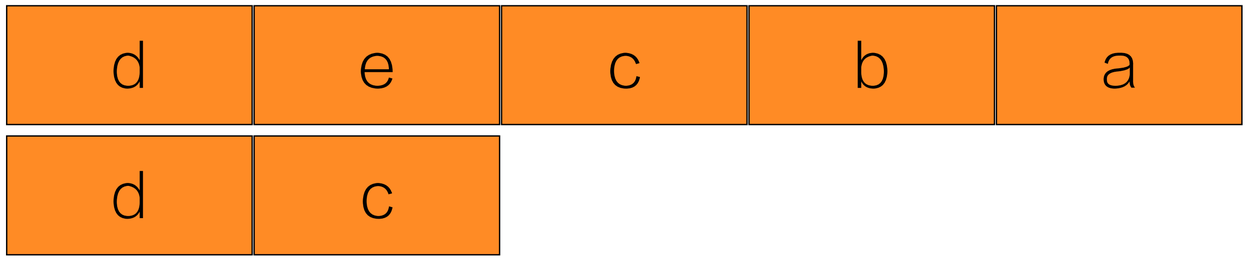




[2011] 2. 元素 a, b, c, d, e 依次进入初始为空的栈中，若元素进栈后可停留、可出栈，直到所有元素都出栈，则在所有可能的出栈序列中，以元素 d 开头的序列个数是____。

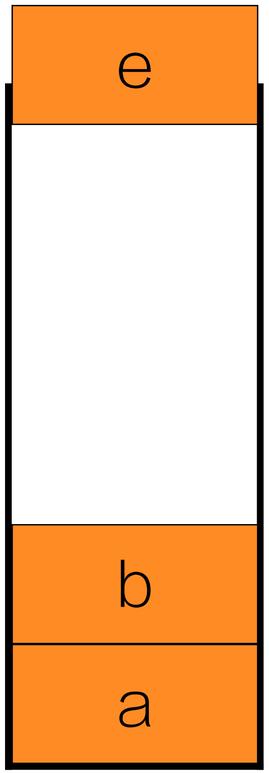
A. 3　　B. 4　　C. 5　　D. 6

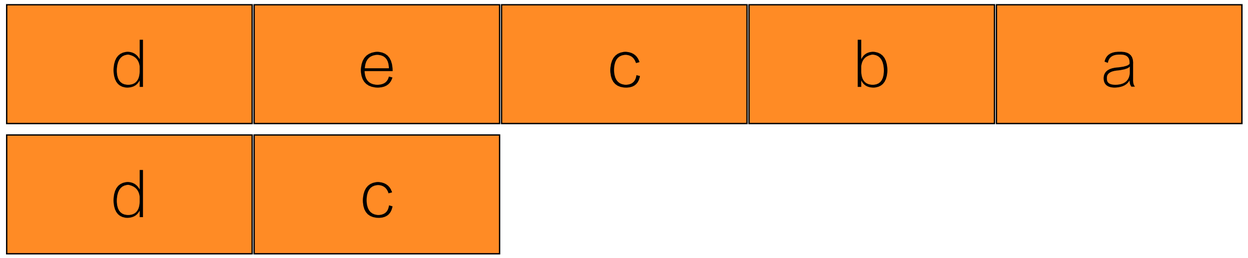




[2011] 2. 元素 a, b, c, d, e 依次进入初始为空的栈中，若元素进栈后可停留、可出栈，直到所有元素都出栈，则在所有可能的出栈序列中，以元素 d 开头的序列个数是____。

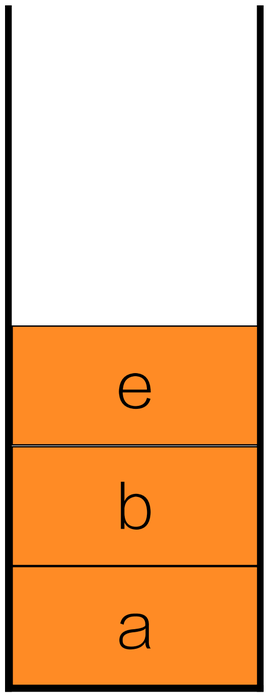
A. 3　　B. 4　　C. 5　　D. 6

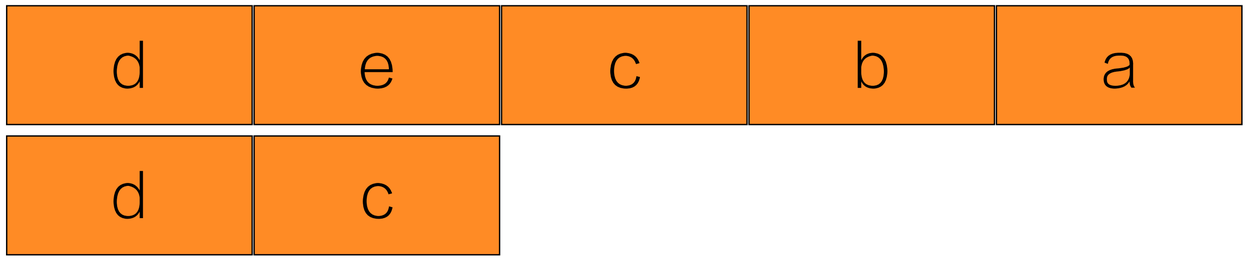




[2011] 2. 元素 a, b, c, d, e 依次进入初始为空的栈中，若元素进栈后可停留、可出栈，直到所有元素都出栈，则在所有可能的出栈序列中，以元素 d 开头的序列个数是____。

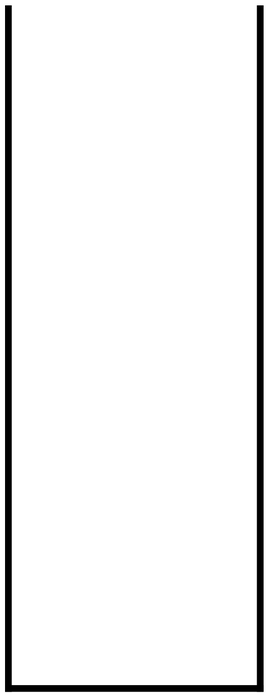
A. 3　　B. 4　　C. 5　　D. 6





[2011] 2. 元素 a, b, c, d, e 依次进入初始为空的栈中，若元素进栈后可停留、可出栈，直到所有元素都出栈，则在所有可能的出栈序列中，以元素 d 开头的序列个数是____。

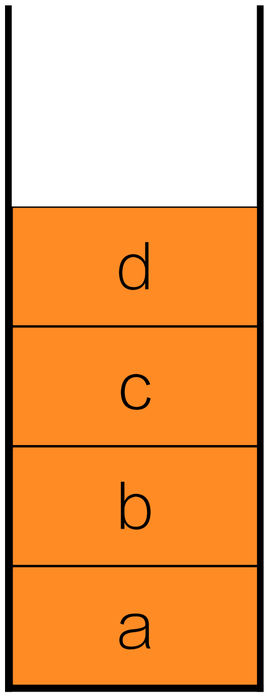
A. 3　　B. 4　　C. 5　　D. 6



d	e	c	b	a
d	c	e	b	a

[2011] 2. 元素 a, b, c, d, e 依次进入初始为空的栈中，若元素进栈后可停留、可出栈，直到所有元素都出栈，则在所有可能的出栈序列中，以元素 d 开头的序列个数是____。

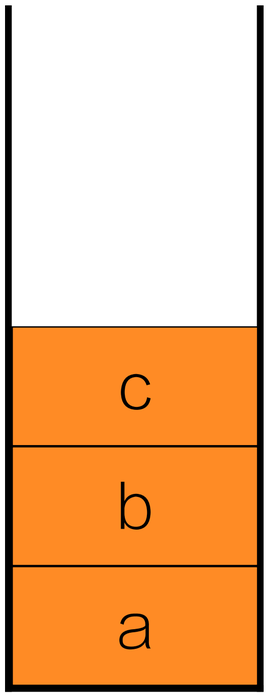
A. 3　　B. 4　　C. 5　　D. 6

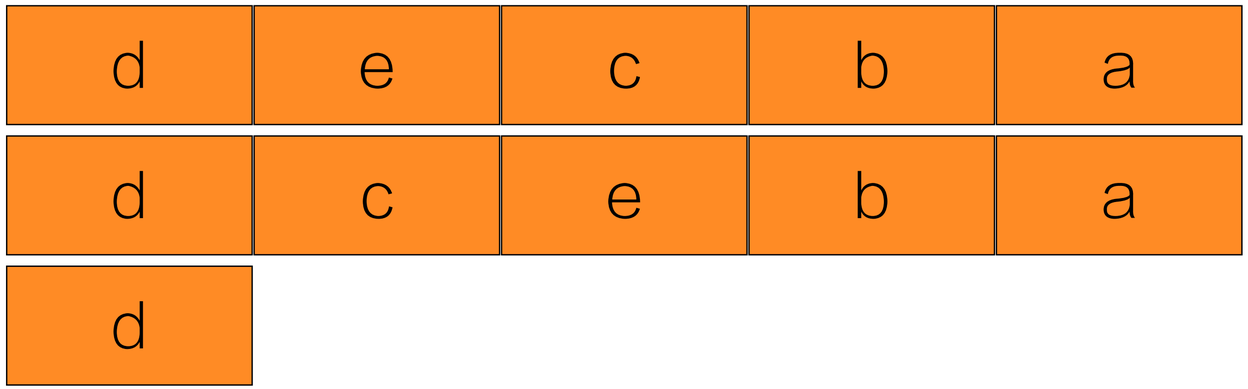


d	e	c	b	a
d	c	e	b	a

[2011] 2. 元素 a, b, c, d, e 依次进入初始为空的栈中，若元素进栈后可停留、可出栈，直到所有元素都出栈，则在所有可能的出栈序列中，以元素 d 开头的序列个数是____。

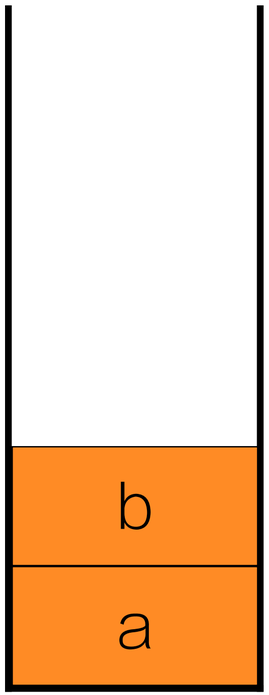
A. 3　　B. 4　　C. 5　　D. 6

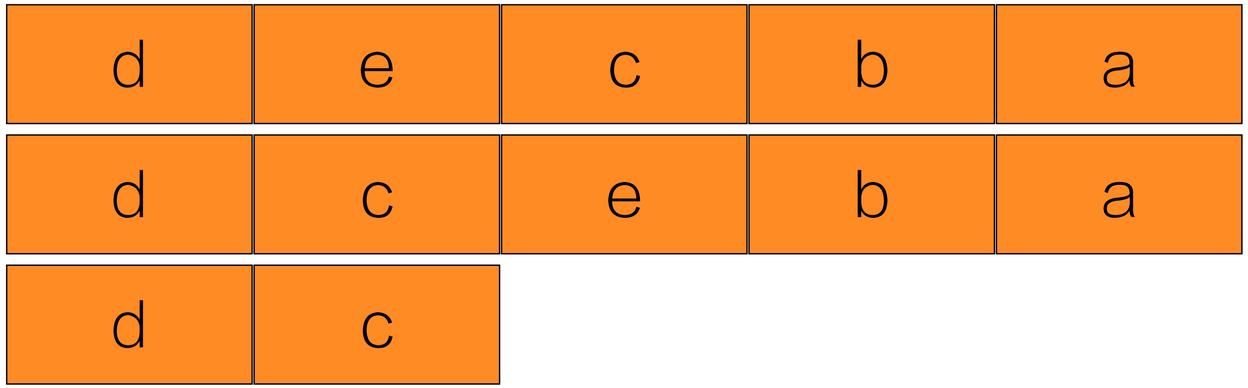




[2011] 2. 元素 a, b, c, d, e 依次进入初始为空的栈中，若元素进栈后可停留、可出栈，直到所有元素都出栈，则在所有可能的出栈序列中，以元素 d 开头的序列个数是____。

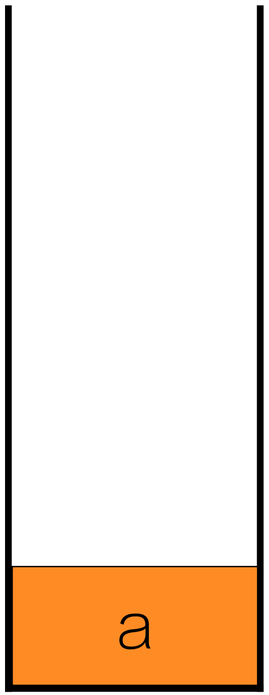
A. 3　　B. 4　　C. 5　　D. 6





[2011] 2. 元素 a, b, c, d, e 依次进入初始为空的栈中，若元素进栈后可停留、可出栈，直到所有元素都出栈，则在所有可能的出栈序列中，以元素 d 开头的序列个数是____。

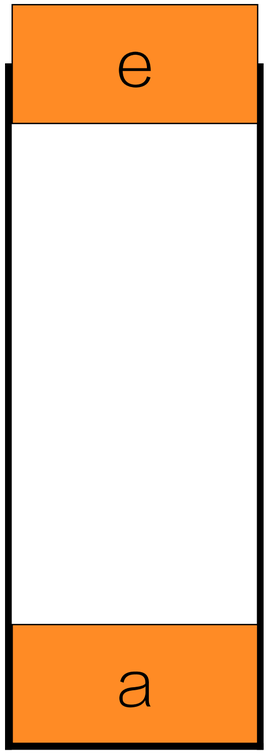
A. 3　　B. 4　　C. 5　　D. 6

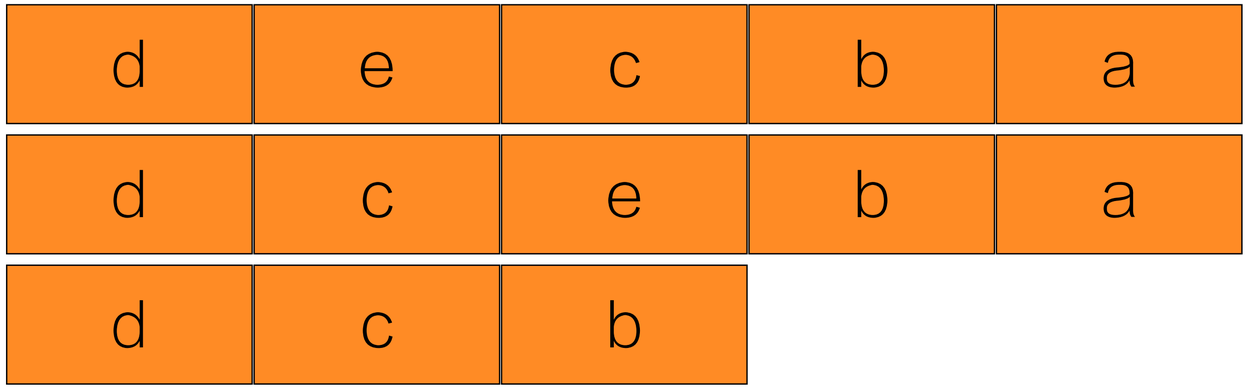


d	e	c	b	a
d	c	e	b	a
d	c	b		

[2011] 2. 元素 a, b, c, d, e 依次进入初始为空的栈中，若元素进栈后可停留、可出栈，直到所有元素都出栈，则在所有可能的出栈序列中，以元素 d 开头的序列个数是____。

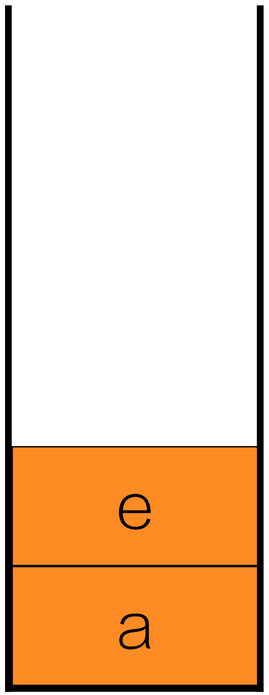
A. 3　　B. 4　　C. 5　　D. 6

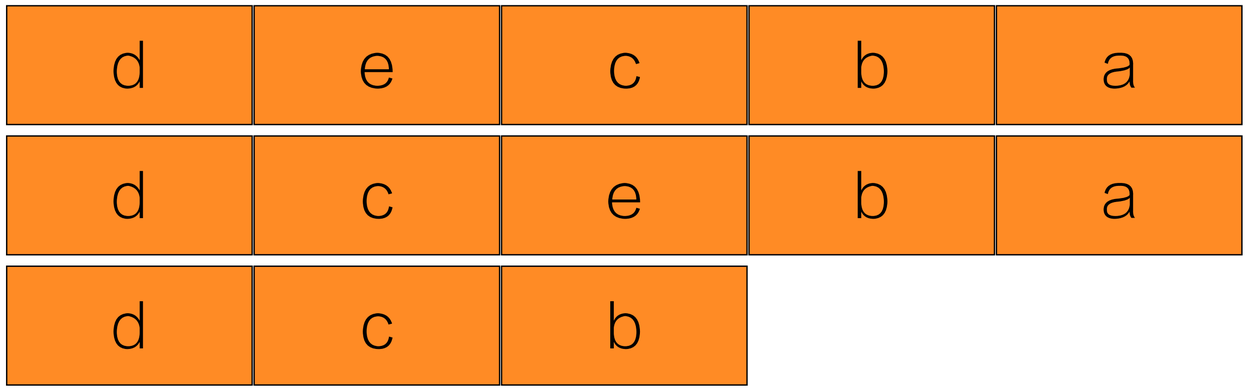




[2011] 2. 元素 a, b, c, d, e 依次进入初始为空的栈中，若元素进栈后可停留、可出栈，直到所有元素都出栈，则在所有可能的出栈序列中，以元素 d 开头的序列个数是____。

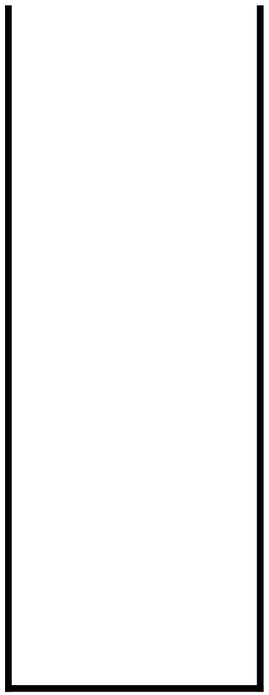
A. 3　　B. 4　　C. 5　　D. 6





[2011] 2. 元素 a, b, c, d, e 依次进入初始为空的栈中，若元素进栈后可停留、可出栈，直到所有元素都出栈，则在所有可能的出栈序列中，以元素 d 开头的序列个数是____。

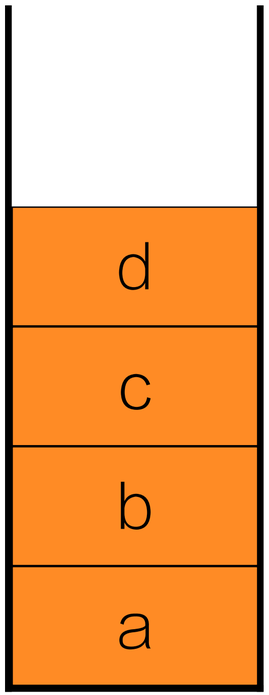
A. 3　　B. 4　　C. 5　　D. 6



d	e	c	b	a
d	c	e	b	a
d	c	b	e	a

[2011] 2. 元素 a, b, c, d, e 依次进入初始为空的栈中，若元素进栈后可停留、可出栈，直到所有元素都出栈，则在所有可能的出栈序列中，以元素 d 开头的序列个数是____。

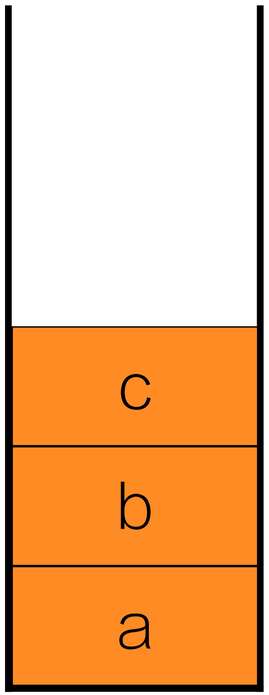
A. 3　　B. 4　　C. 5　　D. 6



d	e	c	b	a
d	c	e	b	a
d	c	b	e	a

[2011] 2. 元素 a, b, c, d, e 依次进入初始为空的栈中，若元素进栈后可停留、可出栈，直到所有元素都出栈，则在所有可能的出栈序列中，以元素 d 开头的序列个数是____。

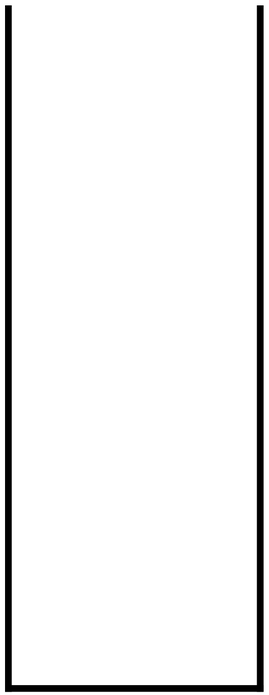
A. 3　　B. 4　　C. 5　　D. 6



d	e	c	b	a
d	c	e	b	a
d	c	b	e	a
d				

[2011] 2. 元素 a, b, c, d, e 依次进入初始为空的栈中，若元素进栈后可停留、可出栈，直到所有元素都出栈，则在所有可能的出栈序列中，以元素 d 开头的序列个数是____。

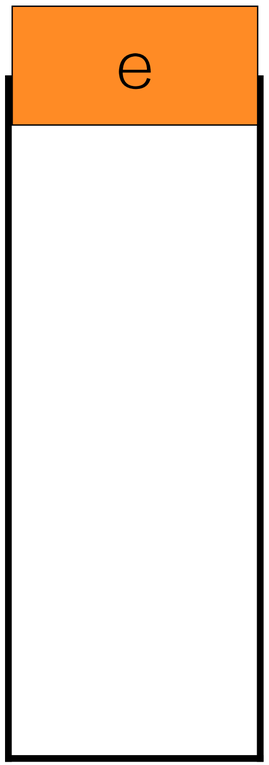
A. 3　　B. 4　　C. 5　　D. 6



d	e	c	b	a
d	c	e	b	a
d	c	b	e	a
d	c	b	a	

[2011] 2. 元素 a, b, c, d, e 依次进入初始为空的栈中，若元素进栈后可停留、可出栈，直到所有元素都出栈，则在所有可能的出栈序列中，以元素 d 开头的序列个数是____。

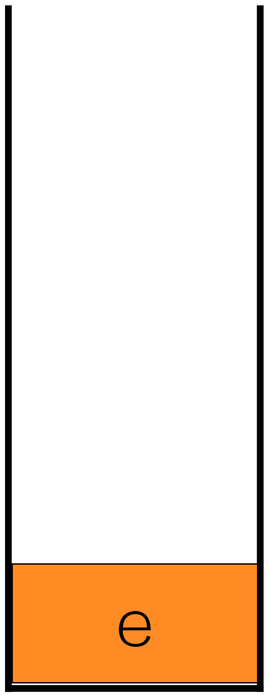
A. 3　　B. 4　　C. 5　　D. 6



d	e	c	b	a
d	c	e	b	a
d	c	b	e	a
d	c	b	a	

[2011] 2. 元素 a, b, c, d, e 依次进入初始为空的栈中，若元素进栈后可停留、可出栈，直到所有元素都出栈，则在所有可能的出栈序列中，以元素 d 开头的序列个数是____。

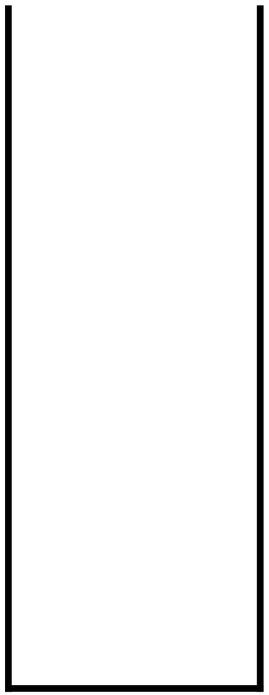
A. 3　　B. 4　　C. 5　　D. 6



d	e	c	b	a
d	c	e	b	a
d	c	b	e	a
d	c	b	a	

[2011] 2. 元素 a, b, c, d, e 依次进入初始为空的栈中，若元素进栈后可停留、可出栈，直到所有元素都出栈，则在所有可能的出栈序列中，以元素 d 开头的序列个数是____。

A. 3　　B. 4　　C. 5　　D. 6



d	e	c	b	a
d	c	e	b	a
d	c	b	e	a
d	c	b	a	e

栈与队列的应用

[2011] 2. 元素 a, b, c, d, e 依次进入初始为空的栈中，若元素进栈后可停留、可出栈，直到所有元素都出栈，则在所有可能的出栈序列中，以元素 d 开头的序列个数是____。

A. 3　　B. 4　　C. 5　　D. 6

栈与队列的应用

[2011] 3. 已知循环队列存储在一维数组A[0 … n−1] 中，且队列非空时 front 和 rear 分别指向队头元素和队尾元素。若初始队列为空，且要求第 1 个进入队列的元素在 A[0] 处，则初始时 front 和 rear 的值分别是_____。

A. 0, 0　　B. 0, n−1　　C. n−1, 0　　D. n−1, n−1

栈与队列的应用

[2011] 3. 已知循环队列存储在一维数组A[0 … n−1] 中，且队列非空时 front 和 rear 分别指向队头元素和队尾元素。若初始队列为空，且要求第 1 个进入队列的元素在 A[0] 处，则初始时 front 和 rear 的值分别是_____。

A. 0, 0　　B. 0, n−1　　C. n−1, 0　　D. n−1, n−1

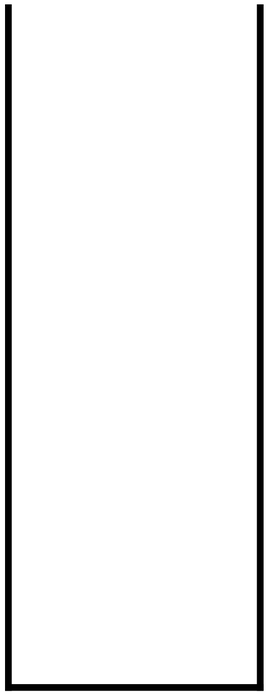
栈与队列的应用

[2013] 2. 一个栈的入栈序列为 1, 2, 3, …, n，其中出栈序列是 p_1, p_2, p_3, …, p_n。若 p_2=3，则 p_3 可能取值的个数是_____。

A. n-3　　B. n-2　　C. n-1　　D. 无法确定

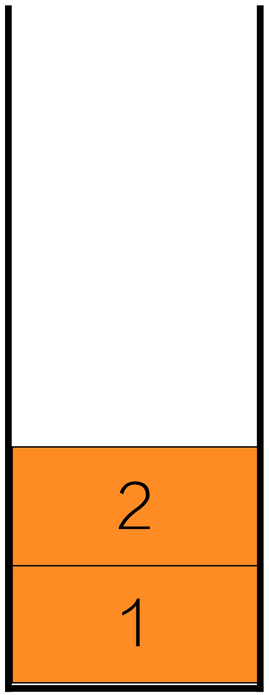
[2013] 2. 一个栈的入栈序列为 1, 2, 3, …, n，其中出栈序列是 p_1, p_2, p_3, …, p_n。若 p_2=3，则 p_3 可能取值的个数是_____。

A. n-3　　B. n-2　　C. n-1　　D. 无法确定



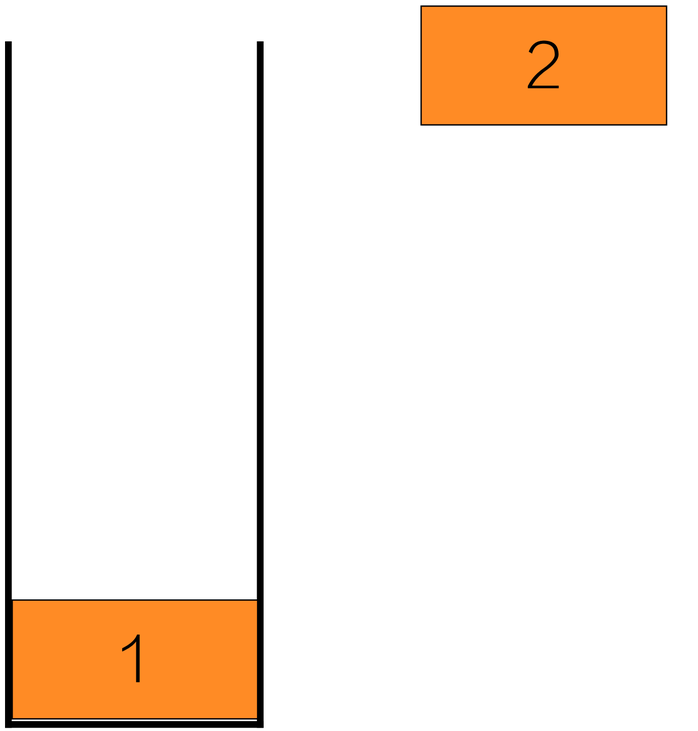
[2013] 2. 一个栈的入栈序列为 1, 2, 3, …, n，其中出栈序列是 p_1, p_2, p_3, …, p_n。若 p_2=3，则 p_3 可能取值的个数是_____。

A. n-3　　B. n-2　　C. n-1　　D. 无法确定



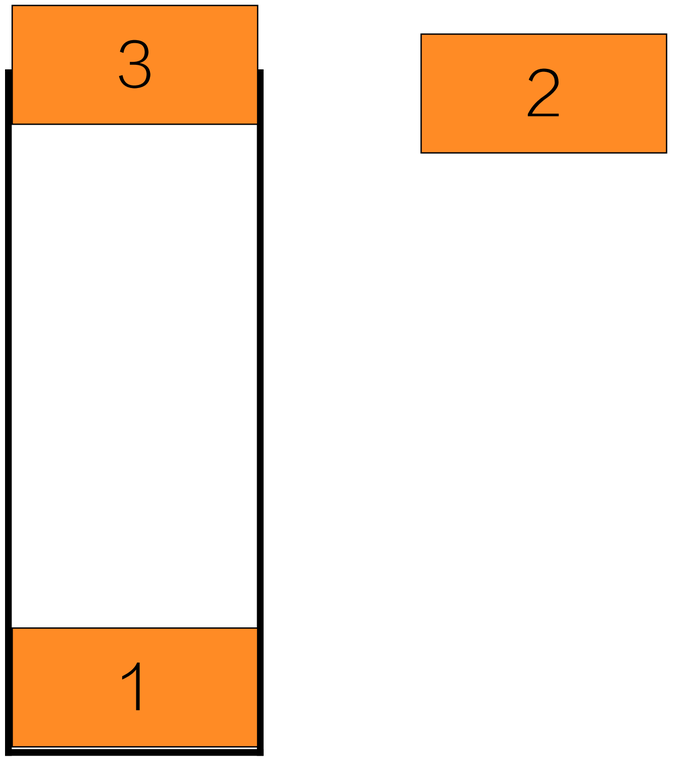
[2013] 2. 一个栈的入栈序列为 1, 2, 3, …, n，其中出栈序列是 p_1, p_2, p_3, …, p_n。若 p_2=3，则 p_3 可能取值的个数是_____。

A. n-3　　B. n-2　　C. n-1　　D. 无法确定



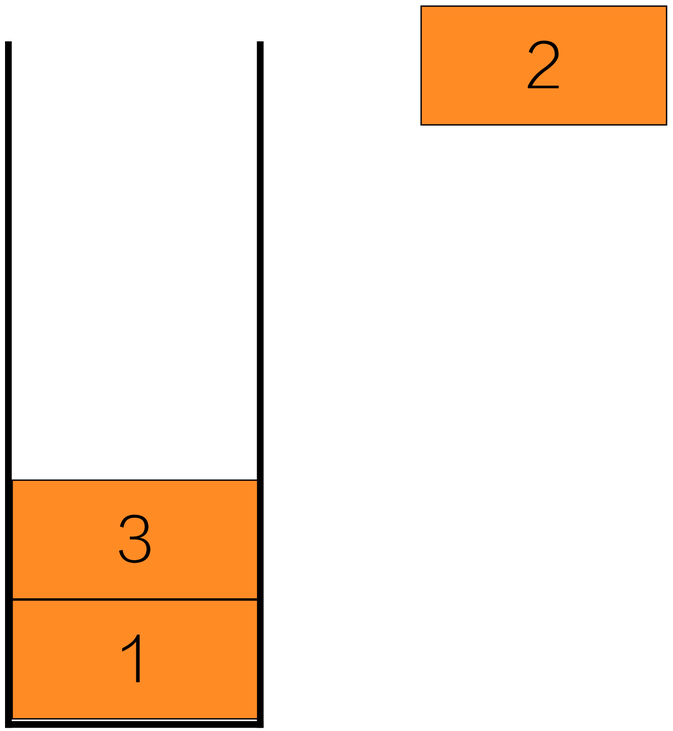
[2013] 2. 一个栈的入栈序列为 1, 2, 3, …, n，其中出栈序列是 p_1, p_2, p_3, …, p_n。若 p_2=3，则 p_3 可能取值的个数是_____。

A. n-3　　B. n-2　　C. n-1　　D. 无法确定



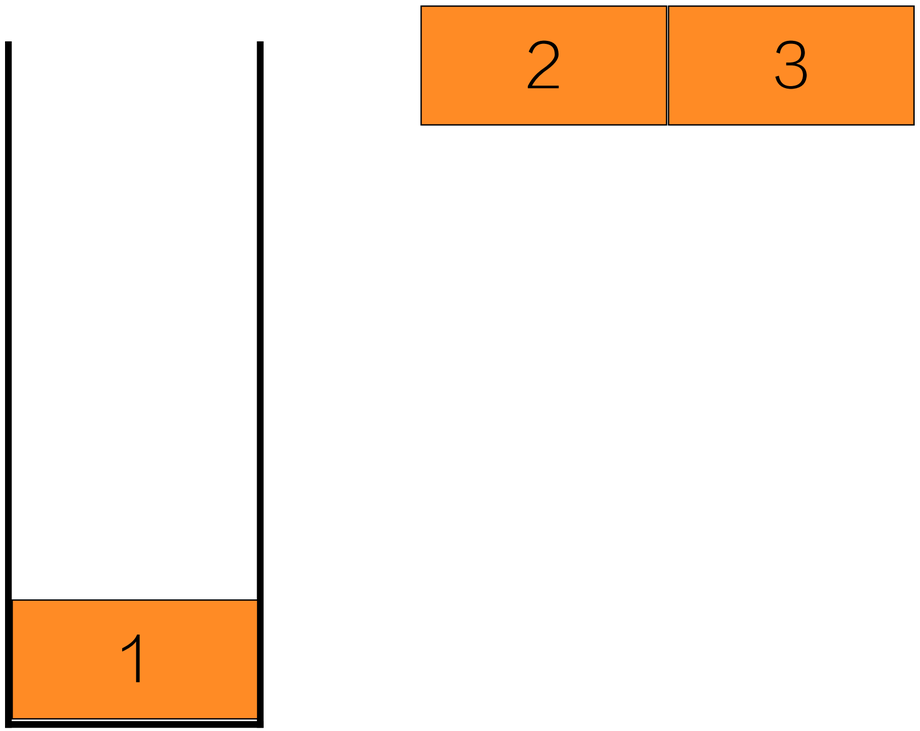
[2013] 2. 一个栈的入栈序列为 1, 2, 3, …, n，其中出栈序列是 p_1, p_2, p_3, …, p_n。若 p_2=3，则 p_3 可能取值的个数是_____。

A. n-3　　B. n-2　　C. n-1　　D. 无法确定



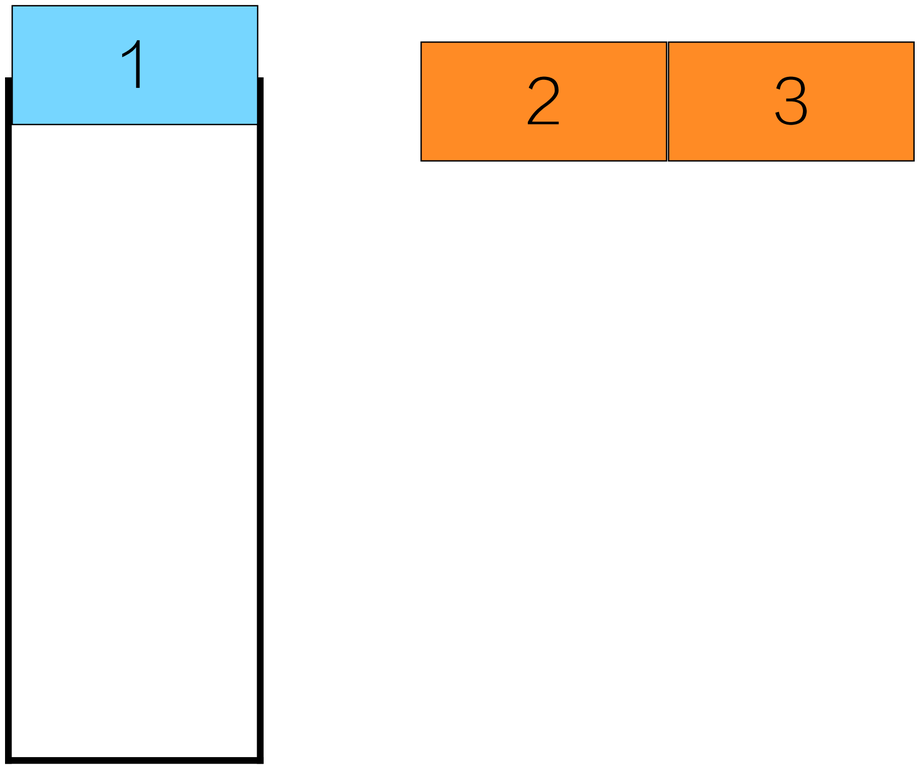
[2013] 2. 一个栈的入栈序列为 1, 2, 3, …, n，其中出栈序列是 p_1, p_2, p_3, …, p_n。若 p_2=3，则 p_3 可能取值的个数是_____。

A. n-3　　B. n-2　　C. n-1　　D. 无法确定



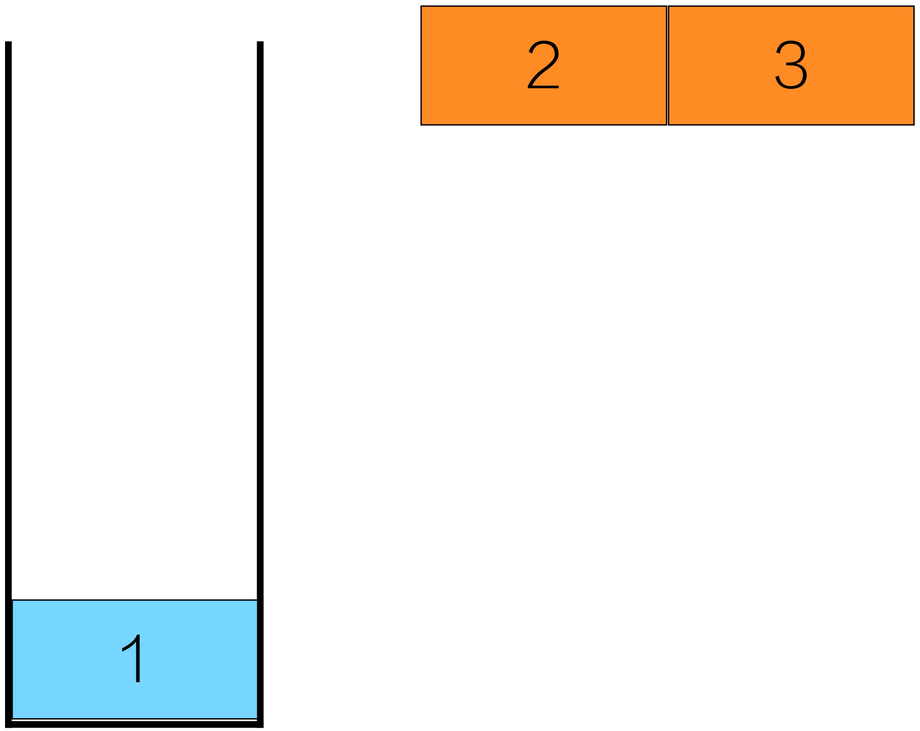
[2013] 2. 一个栈的入栈序列为 1, 2, 3, …, n，其中出栈序列是 p_1, p_2, p_3, …, p_n。若 p_2=3，则 p_3 可能取值的个数是_____。

A. n-3　　B. n-2　　C. n-1　　D. 无法确定



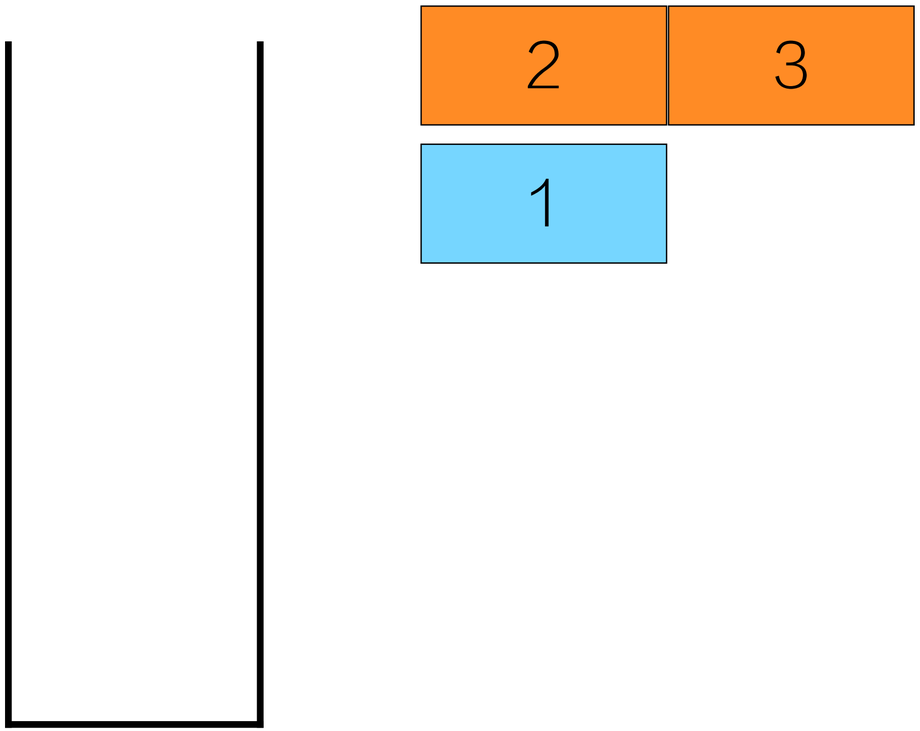
[2013] 2. 一个栈的入栈序列为 1, 2, 3, …, n，其中出栈序列是 p_1, p_2, p_3, …, p_n。若 p_2=3，则 p_3 可能取值的个数是_____。

A. n-3　　B. n-2　　C. n-1　　D. 无法确定



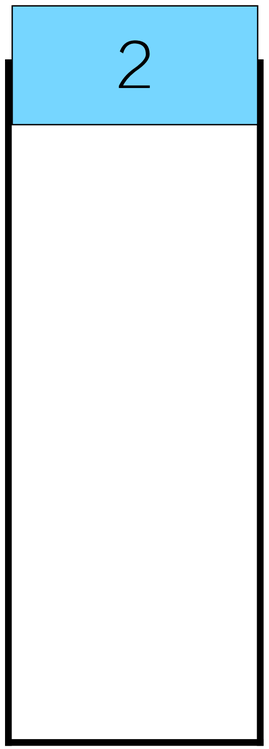
[2013] 2. 一个栈的入栈序列为 1, 2, 3, …, n，其中出栈序列是 p_1, p_2, p_3, …, p_n。若 p_2=3，则 p_3 可能取值的个数是_____。

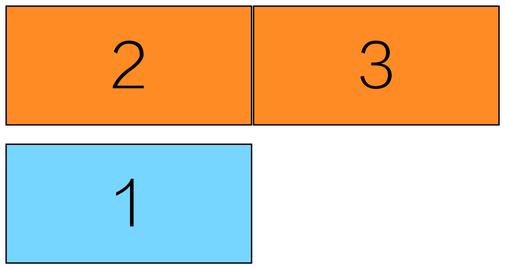
A. n-3　　B. n-2　　C. n-1　　D. 无法确定



[2013] 2. 一个栈的入栈序列为 1, 2, 3, …, n，其中出栈序列是 p_1, p_2, p_3, …, p_n。若 p_2=3，则 p_3 可能取值的个数是_____。

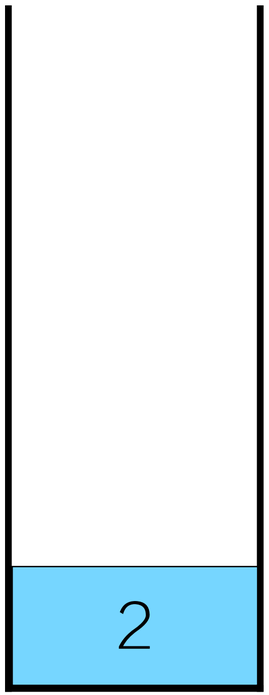
A. n-3　　B. n-2　　C. n-1　　D. 无法确定

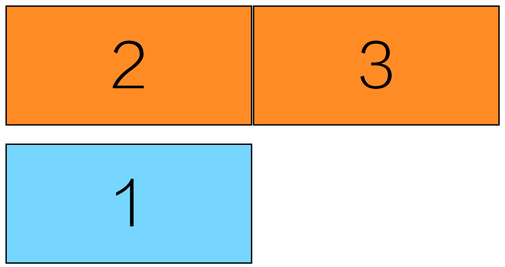




[2013] 2. 一个栈的入栈序列为 1, 2, 3, …, n，其中出栈序列是 p_1, p_2, p_3, …, p_n。若 p_2=3，则 p_3 可能取值的个数是_____。

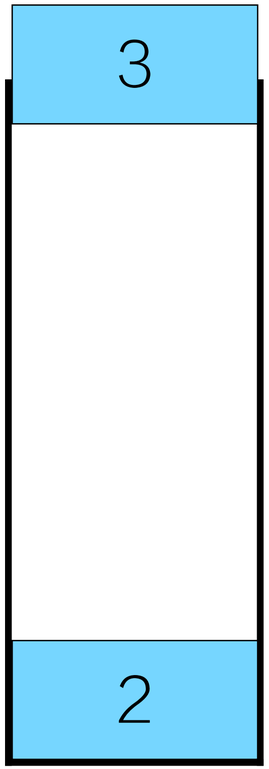
A. n-3　　B. n-2　　C. n-1　　D. 无法确定

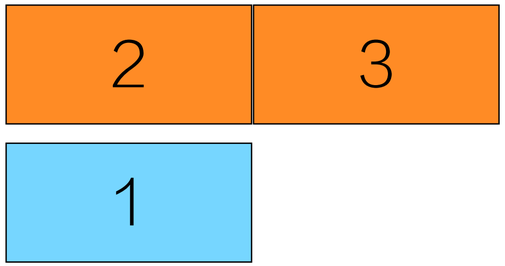




[2013] 2. 一个栈的入栈序列为 1, 2, 3, …, n，其中出栈序列是 p_1, p_2, p_3, …, p_n。若 p_2=3，则 p_3 可能取值的个数是_____。

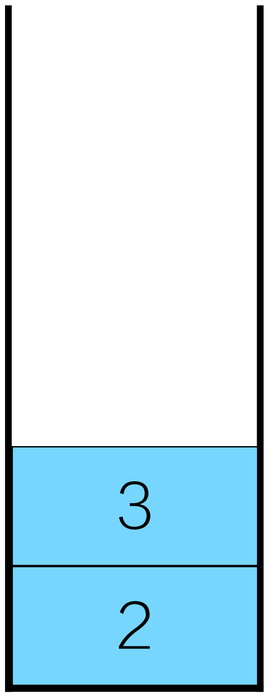
A. n-3　　B. n-2　　C. n-1　　D. 无法确定

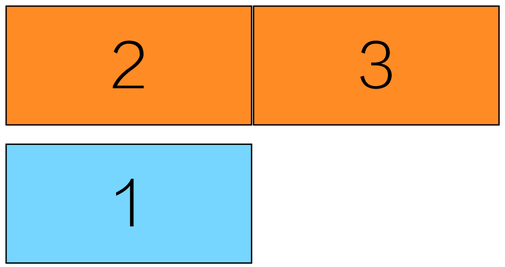




[2013] 2. 一个栈的入栈序列为 1, 2, 3, …, n，其中出栈序列是 p_1, p_2, p_3, …, p_n。若 p_2=3，则 p_3 可能取值的个数是_____。

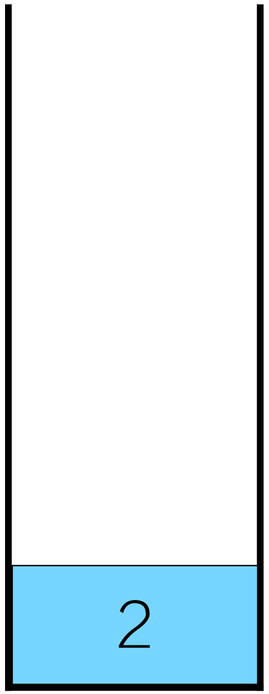
A. n-3　　B. n-2　　C. n-1　　D. 无法确定

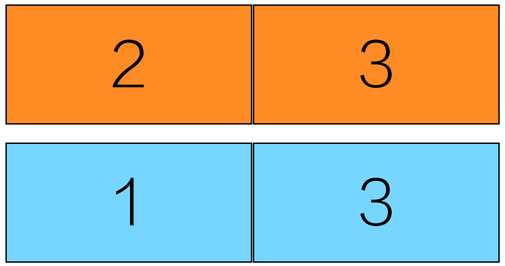




[2013] 2. 一个栈的入栈序列为 1, 2, 3, …, n，其中出栈序列是 p_1, p_2, p_3, …, p_n。若 p_2=3，则 p_3 可能取值的个数是_____。

A. n-3　　B. n-2　　C. n-1　　D. 无法确定





栈与队列的应用

[2013] 2. 一个栈的入栈序列为 1, 2, 3, …, n，其中出栈序列是 p_1, p_2, p_3, …, p_n。若 p_2=3，则 p_3 可能取值的个数是_____。

A. n-3　　B. n-2　　C. n-1　　D. 无法确定

栈与队列的应用

[2014] 3. 循环队列放在一维数组 A[0 … M−1] 中，end1 指向队头元素，end2 指向队尾元素的后一个位置。假设队列两端均可进行入队和出队操作，队列中最多能容纳 M−1 个元素。初始时为空，下列判断队空和队满的条件中，正确的是_____。

A. 队空：end1 == end2;　　队满：end1 == (end2 + 1) mod M;

B. 队空：end1 == end2;　　队满：end1 == (end2 + 1) mod (M − 1);

C. 队空：end2 == (end1 + 1) mod M;　　队满：end1 == (end2 + 1) mod M;

D. 队空：end1 == (end2 + 1) mod M;　　队满：end2 == (end1 + 1) mod (M − 1);

栈与队列的应用

[2014] 3. 循环队列放在一维数组 A[0 … M−1] 中，end1 指向队头元素，end2 指向队尾元素的后一个位置。假设队列两端均可进行入队和出队操作，队列中最多能容纳 M−1 个元素。初始时为空，下列判断队空和队满的条件中，正确的是_____。

A. 队空：end1 == end2;　　队满：end1 == (end2 + 1) mod M;

B. 队空：end1 == end2;　　队满：end1 == (end2 + 1) mod (M − 1);

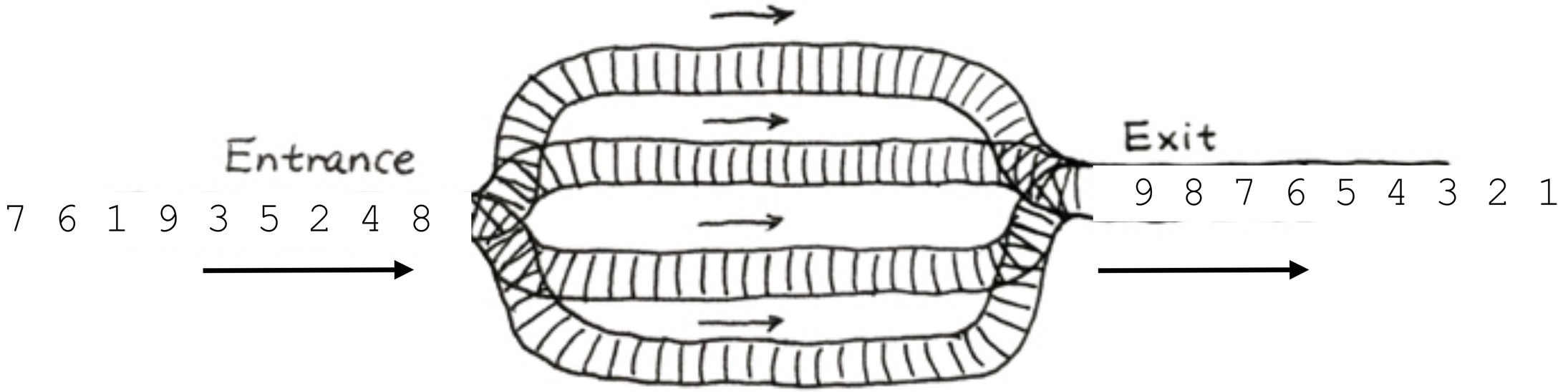
C. 队空：end2 == (end1 + 1) mod M;　　队满：end1 == (end2 + 1) mod M;

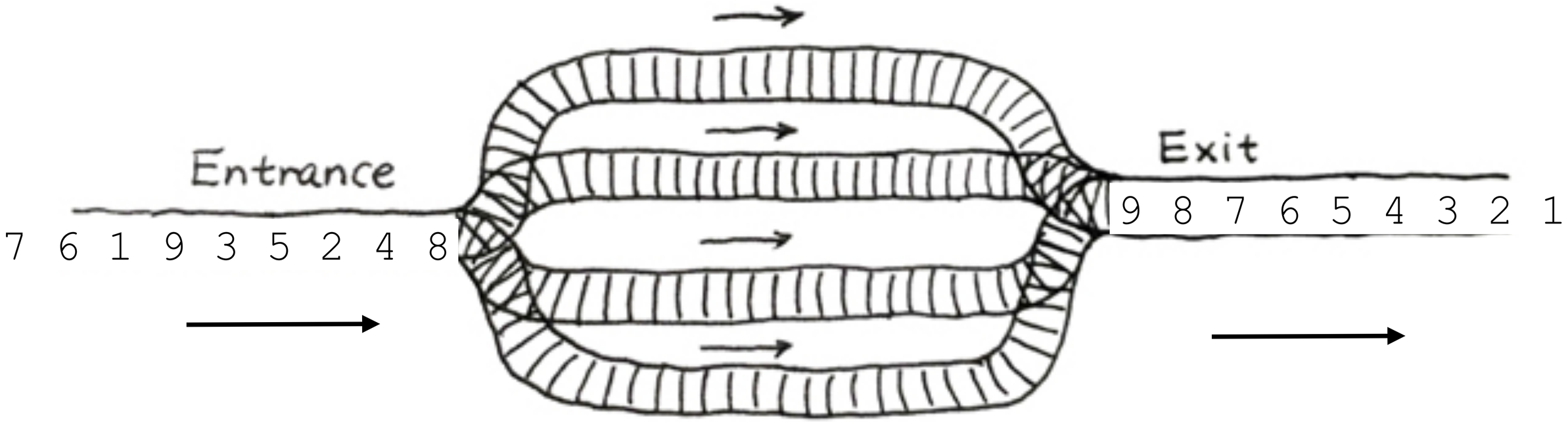
D. 队空：end1 == (end2 + 1) mod M;　　队满：end2 == (end1 + 1) mod (M − 1);

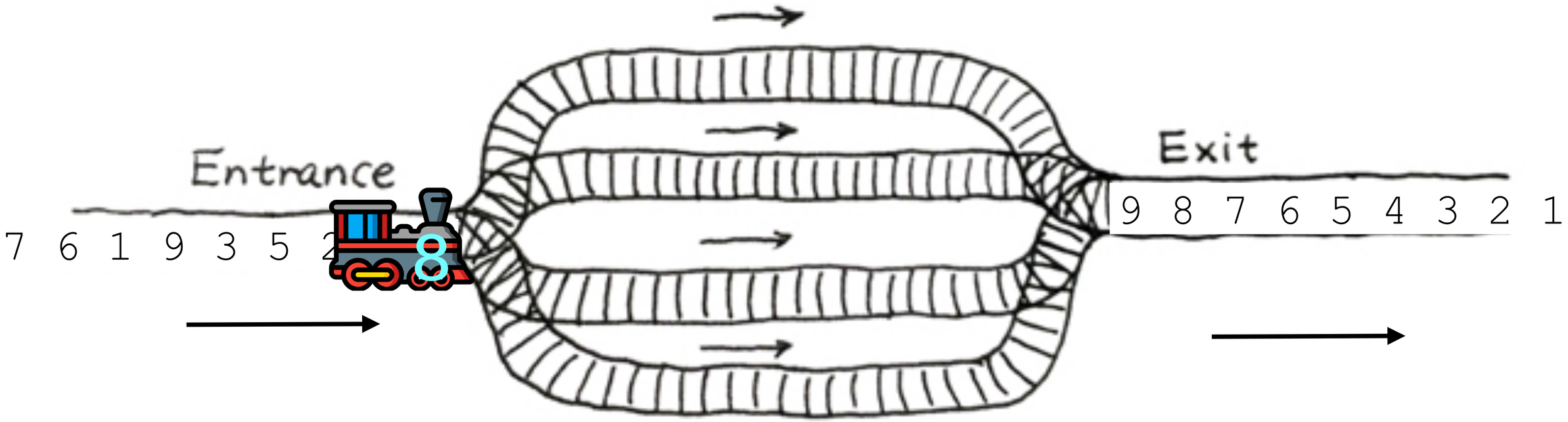
栈与队列的应用

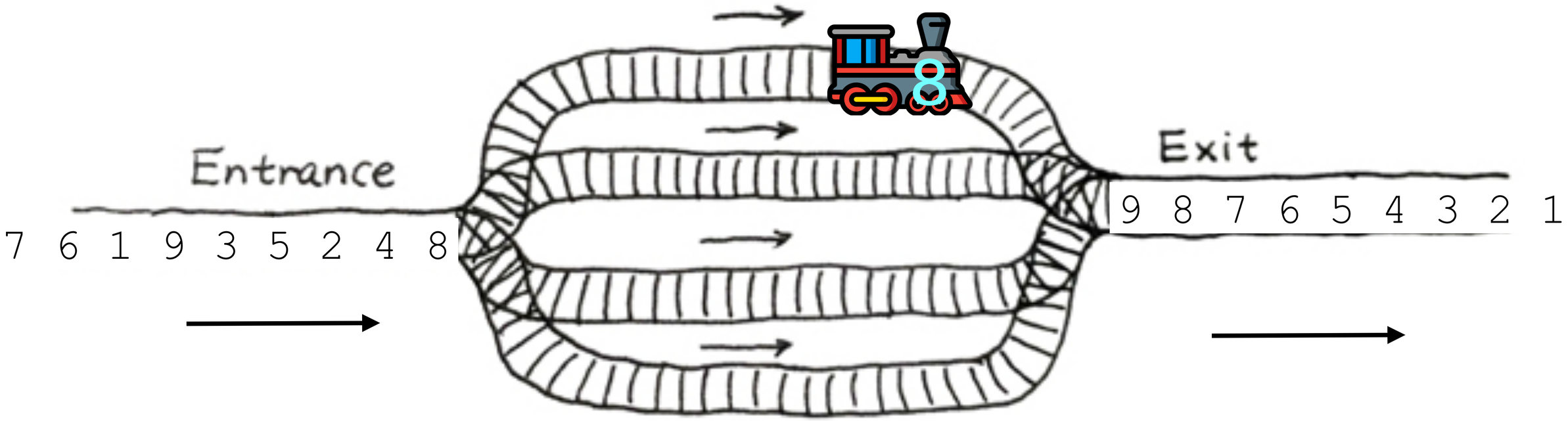
[2016] 3. 设有下图所示的火车车轨，入口到出口之间有 n 条轨道，列车的行进方向均为从左至右，列车可驶入任意一条轨道。现有编号为 1～9 的 9 列列车，驶入的次序依次是 8, 4, 2, 5, 3, 9, 1, 6, 7。若期望驶出的次序依次为 1~9，则 n 至少是_____。

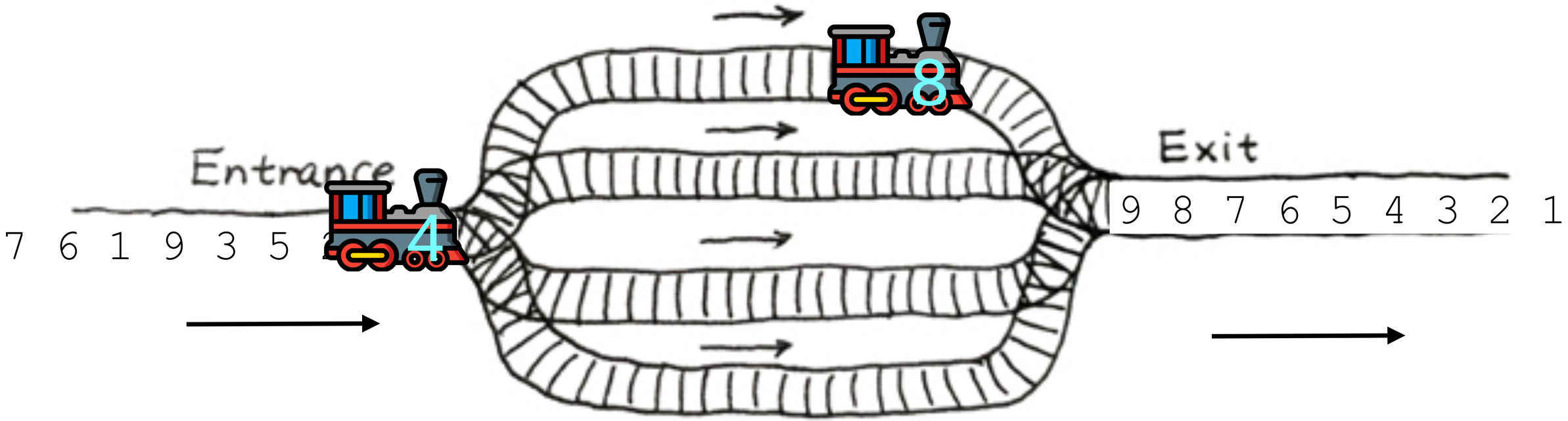
A. 2　　B. 3　　C. 4　　D. 5

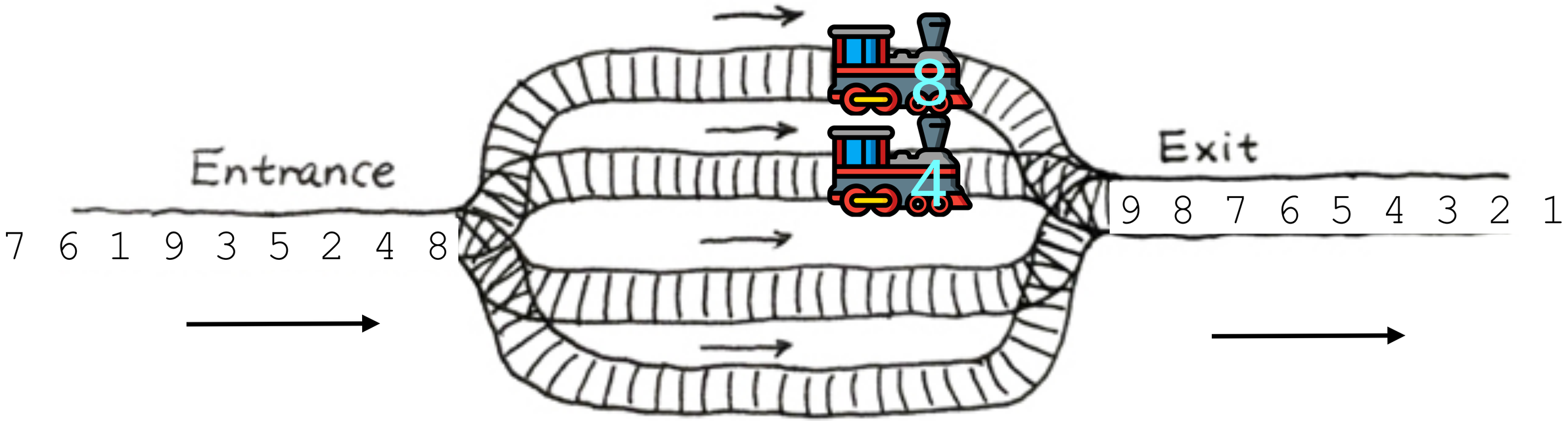


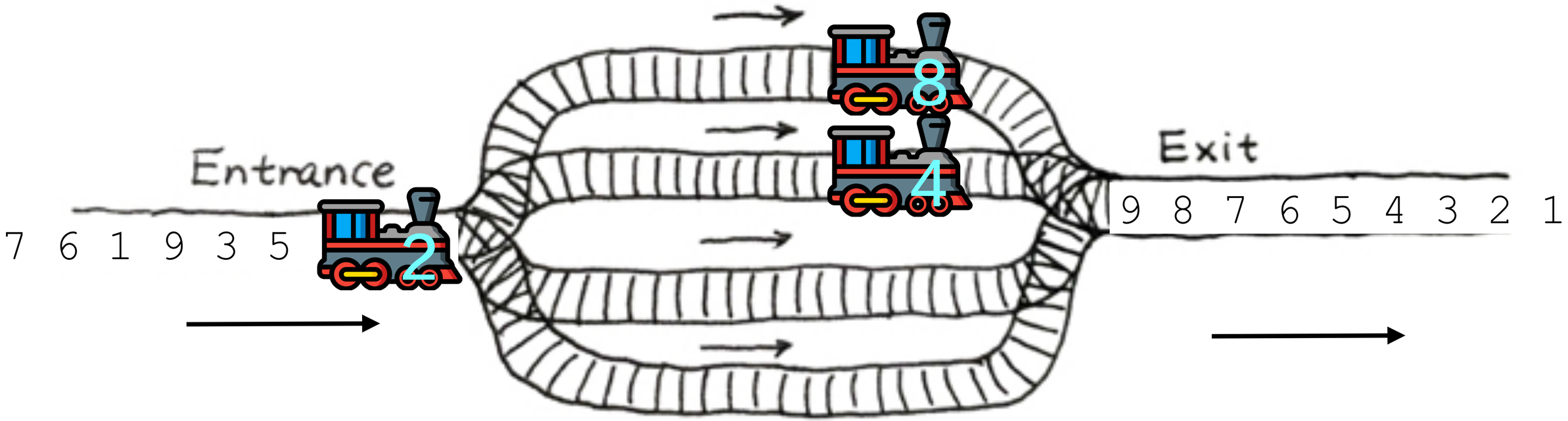

Entrance
7 6 1 9 3 5 2 4 8
Exit
9 8 7 6 5 4 3 2 1

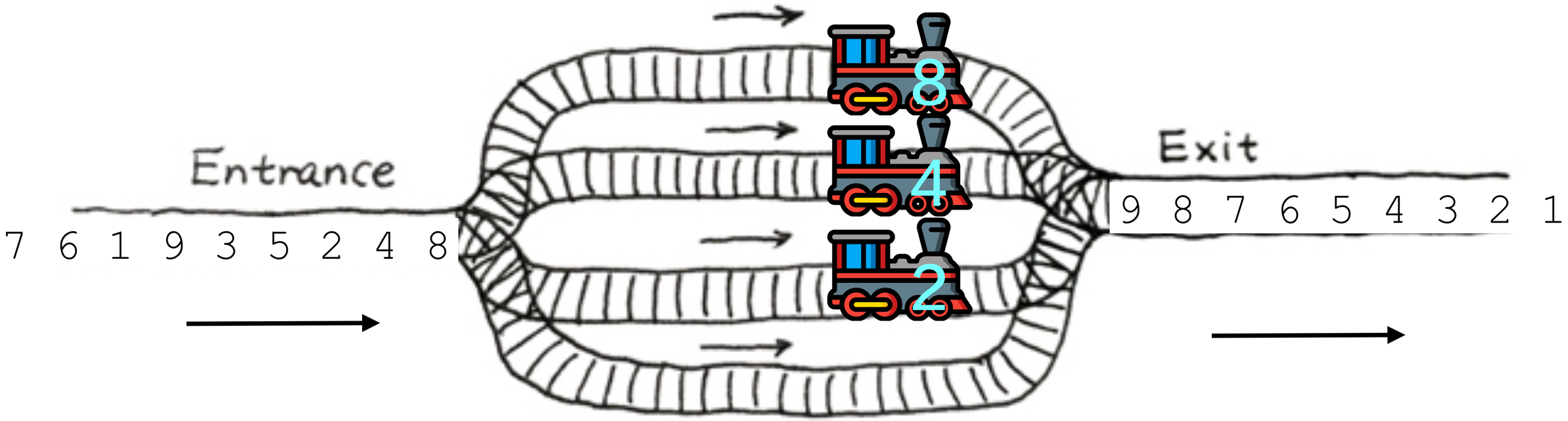

Entrance
7 6 1 9 3 5 2
8
Exit
9 8 7 6 5 4 3 2 1

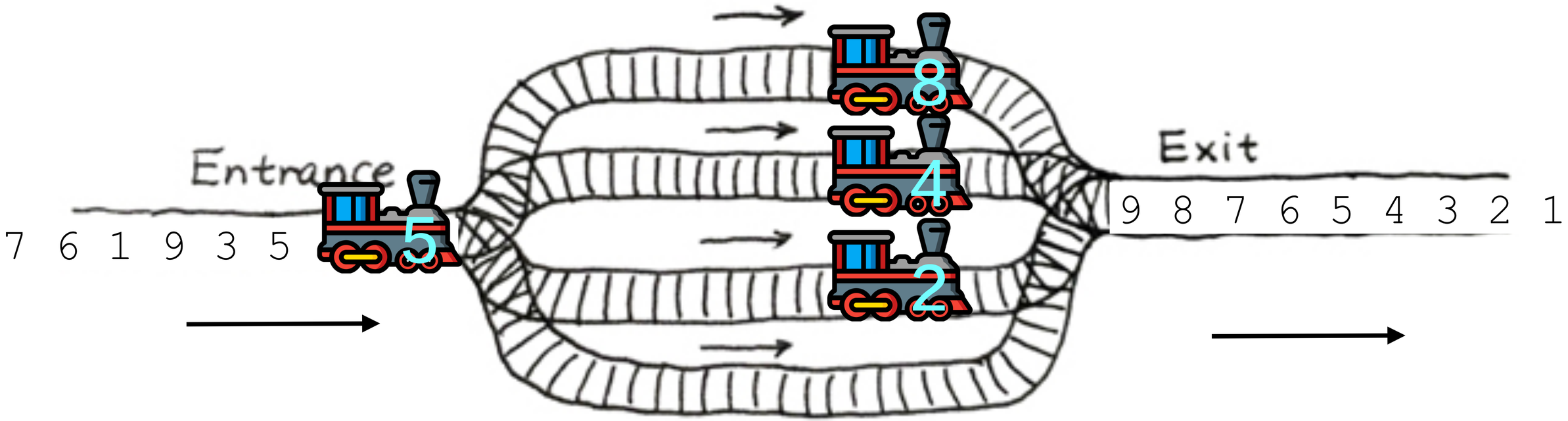

Entrance
7 6 1 9 3 5 2 4 8
8
Exit
9 8 7 6 5 4 3 2 1

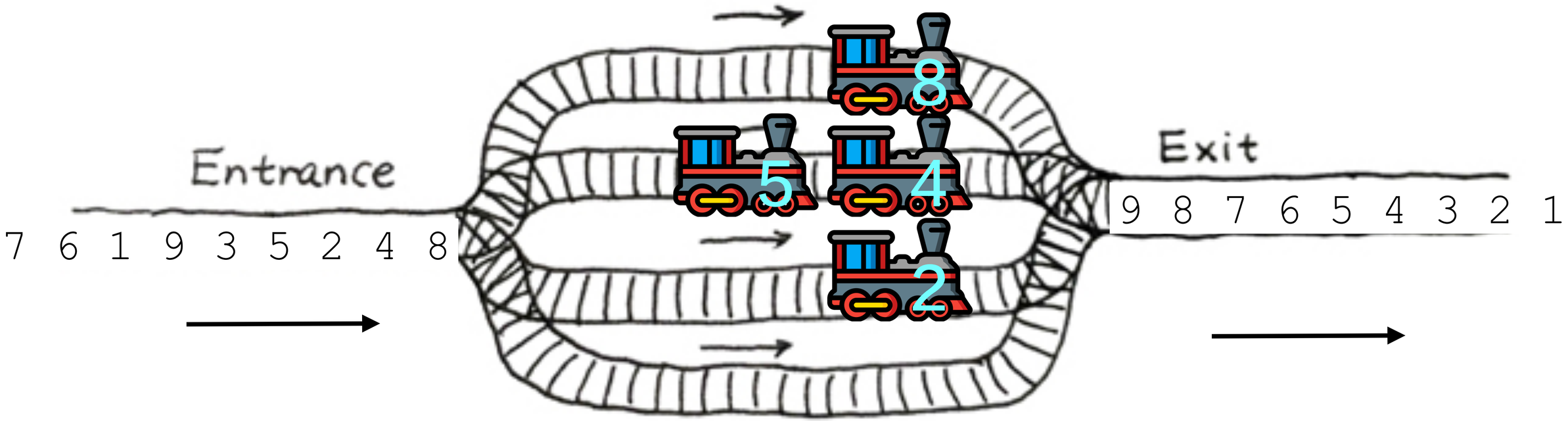

Entrance
7 6 1 9 3 5
4
8
Exit
9 8 7 6 5 4 3 2 1

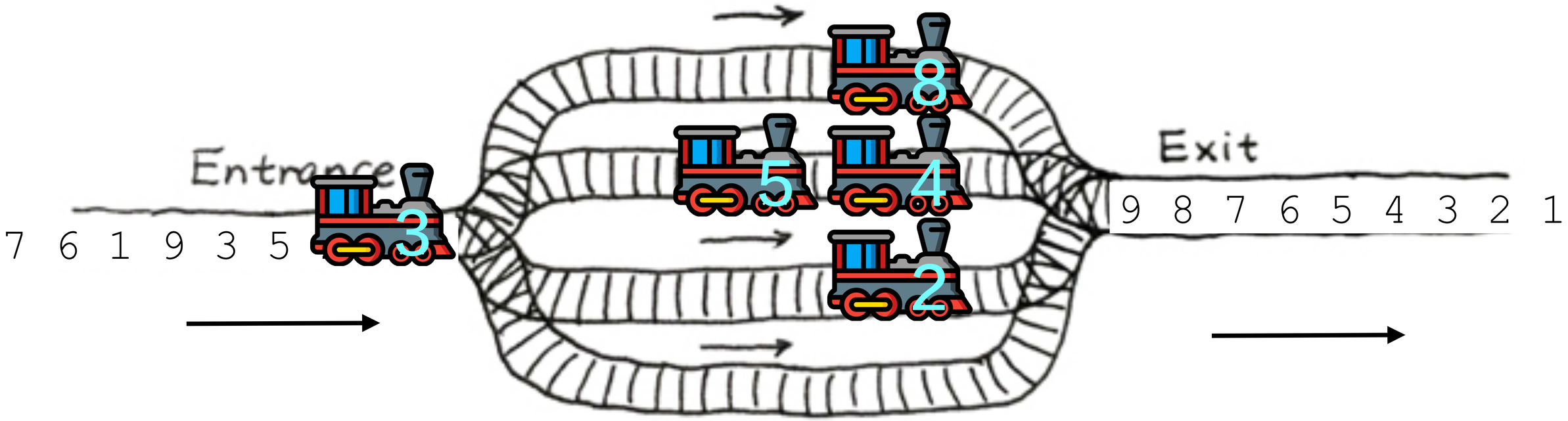

Entrance
7 6 1 9 3 5 2 4 8
8
4
Exit
9 8 7 6 5 4 3 2 1

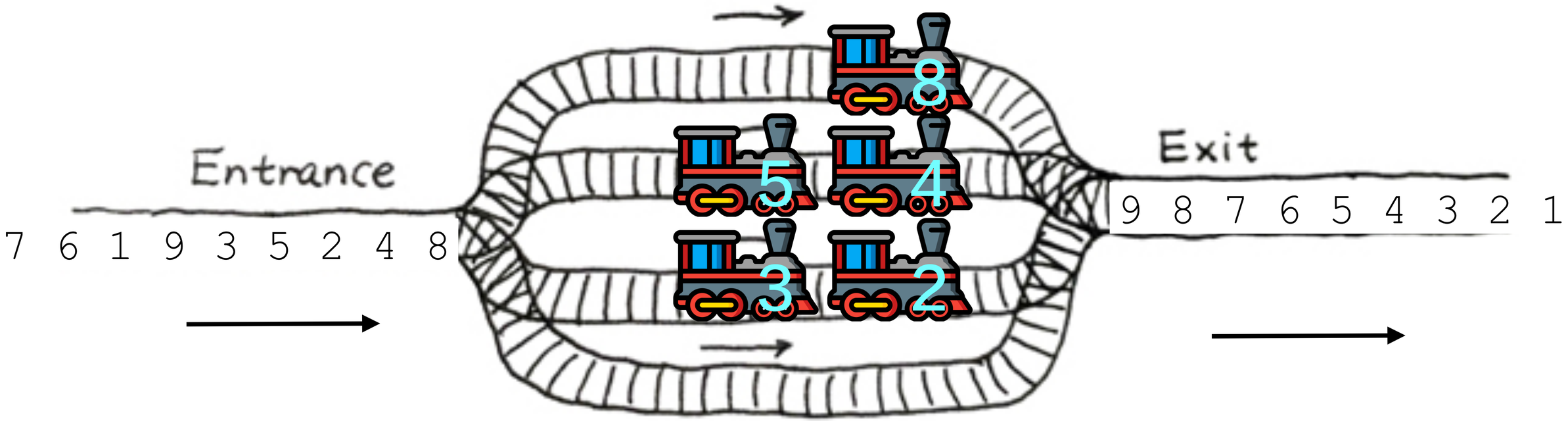

Entrance
7 6 1 9 3 5
2
8
4
Exit
9 8 7 6 5 4 3 2 1

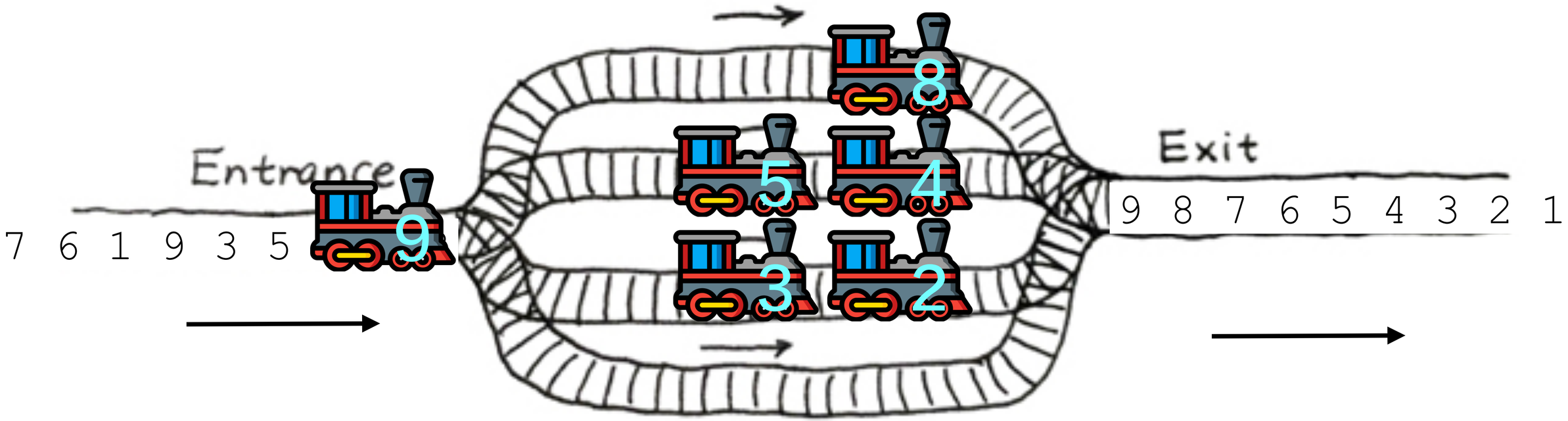

Entrance
7 6 1 9 3 5 2 4 8
8
4
2
Exit
9 8 7 6 5 4 3 2 1

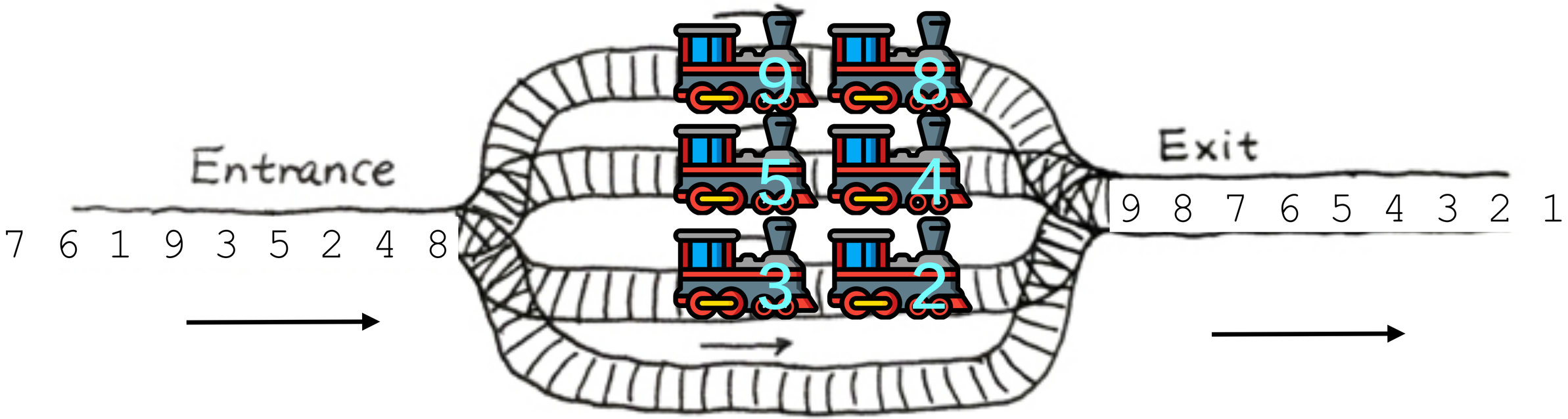

Entrance
7 6 1 9 3 5
5
8
4
2
Exit
9 8 7 6 5 4 3 2 1

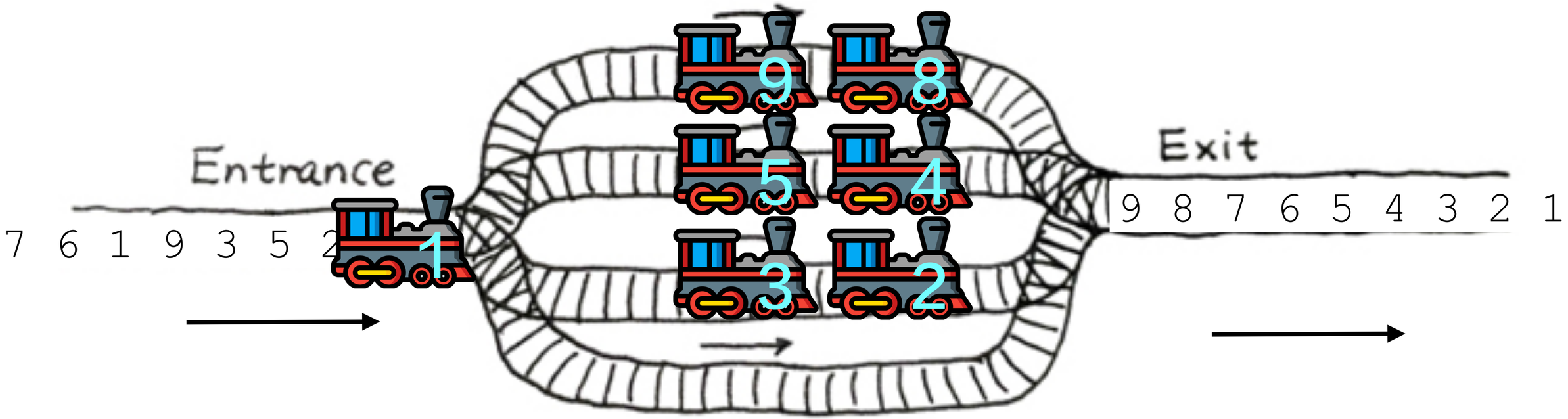

Entrance
7 6 1 9 3 5 2 4 8
8
5
4
2
Exit
9 8 7 6 5 4 3 2 1

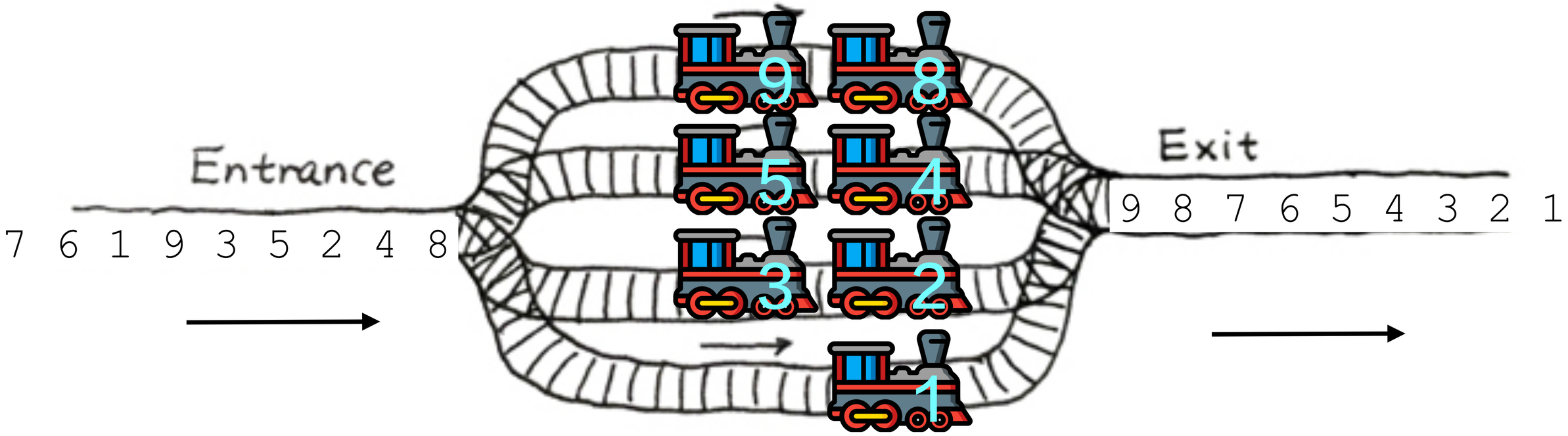

Entrance
7 6 1 9 3 5
3
5
8
4
2
Exit
9 8 7 6 5 4 3 2 1

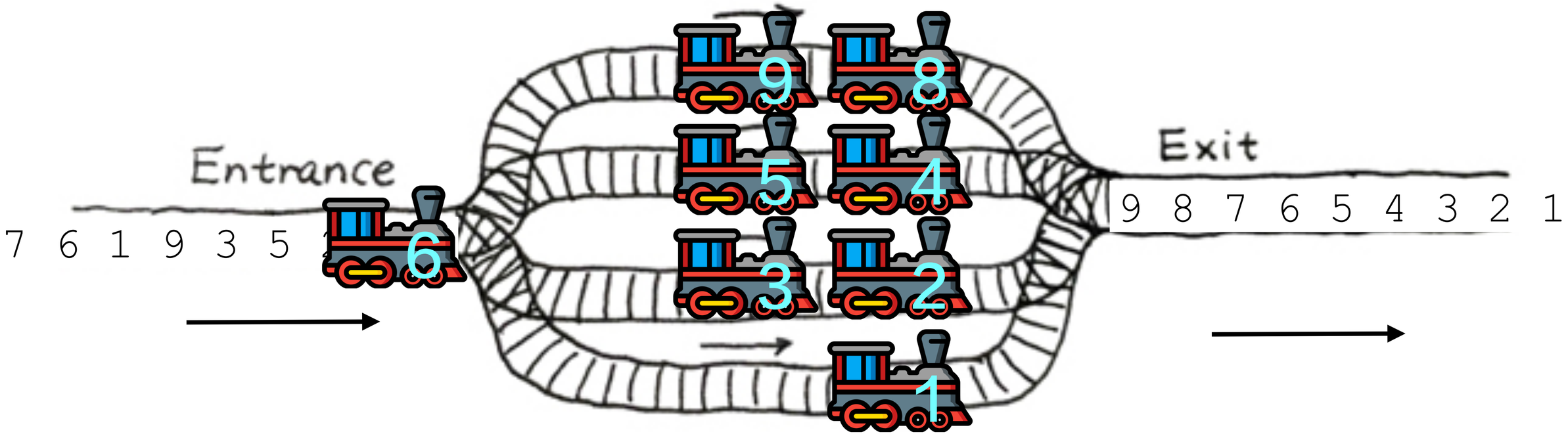

Entrance
7 6 1 9 3 5 2 4 8
8
5
4
3
2
Exit
9 8 7 6 5 4 3 2 1

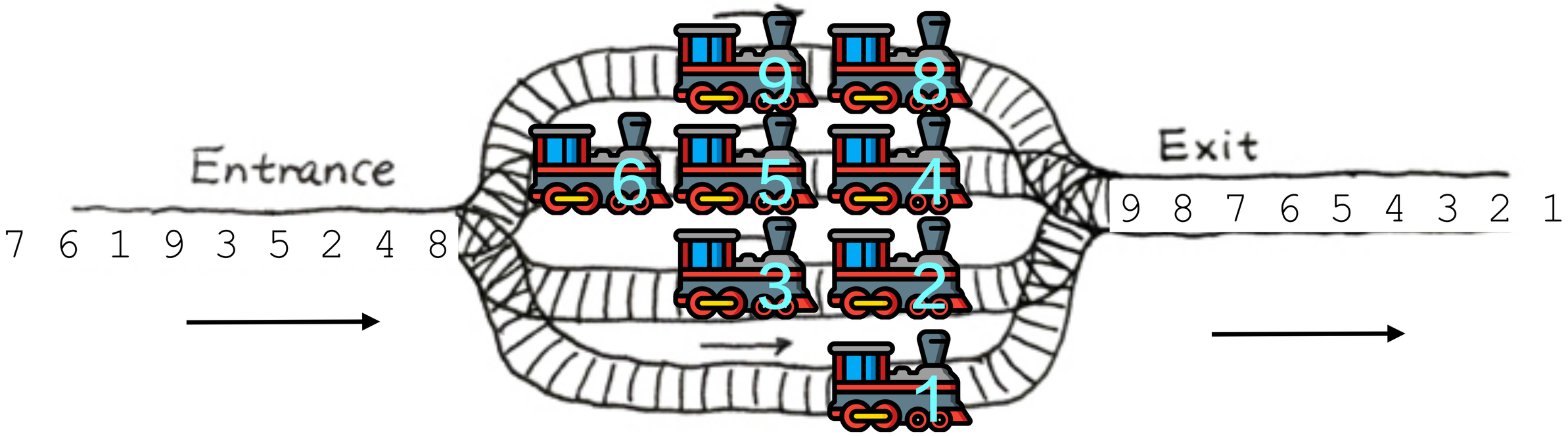

Entrance
7 6 1 9 3 5
9
8
5
4
3
2
Exit
9 8 7 6 5 4 3 2 1

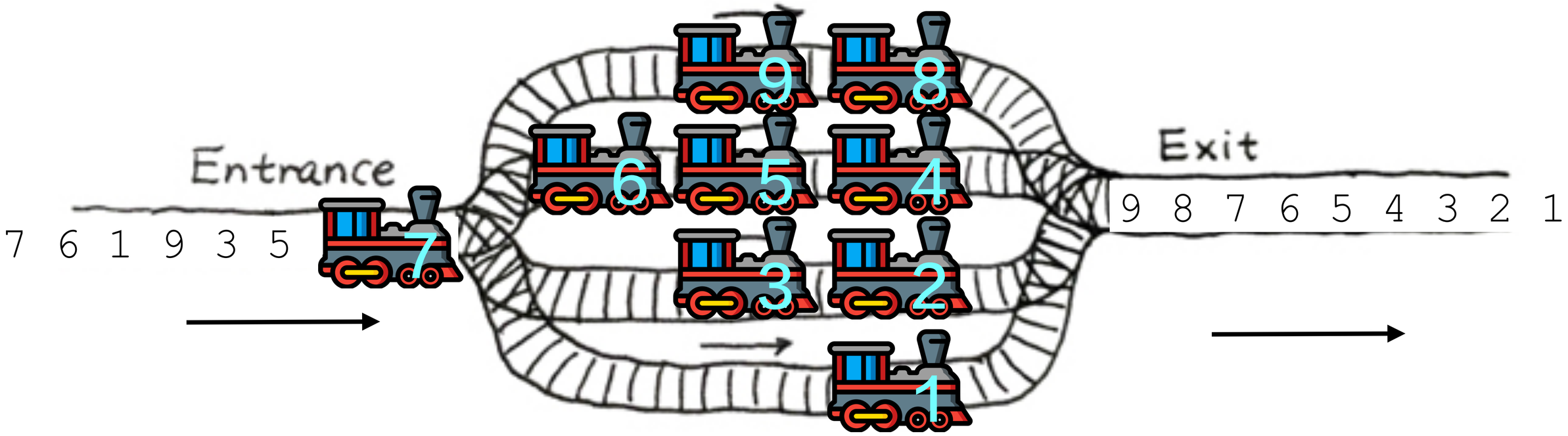

Entrance
7 6 1 9 3 5 2 4 8
9
8
5
4
3
2
Exit
9 8 7 6 5 4 3 2 1

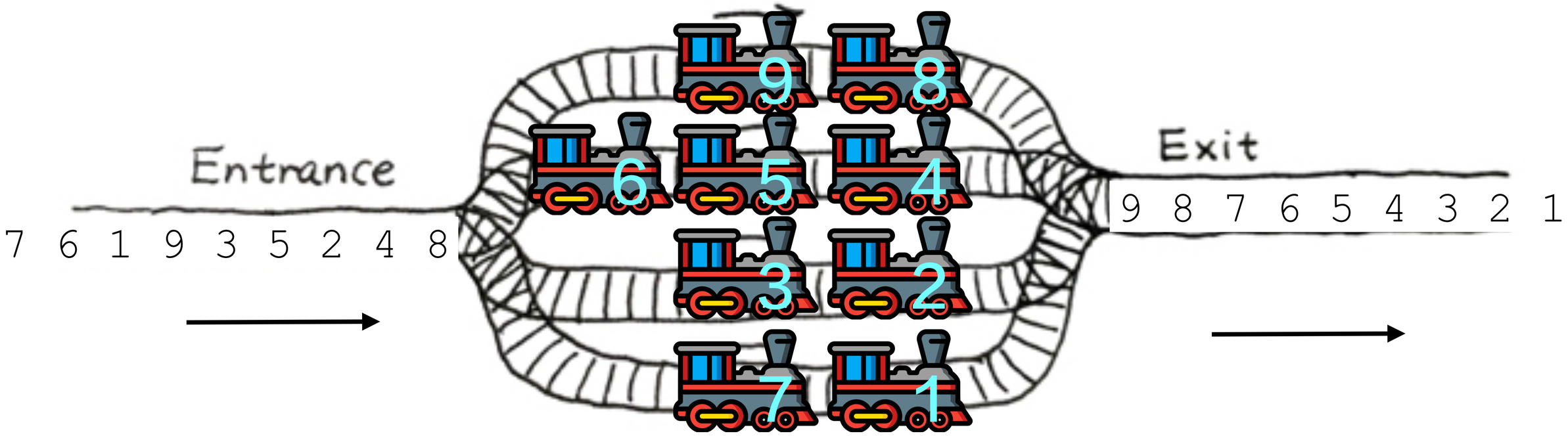

Entrance
7 6 1 9 3 5 2
1
9
8
5
4
3
2
Exit
9 8 7 6 5 4 3 2 1

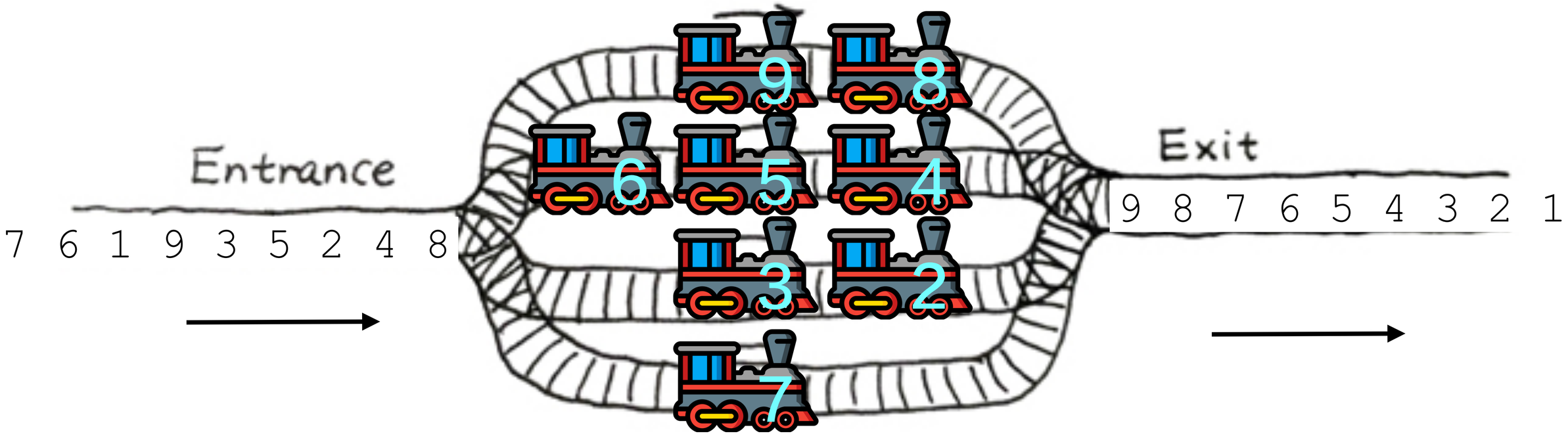

Entrance
7 6 1 9 3 5 2 4 8
9
8
5
4
3
2
1
Exit
9 8 7 6 5 4 3 2 1

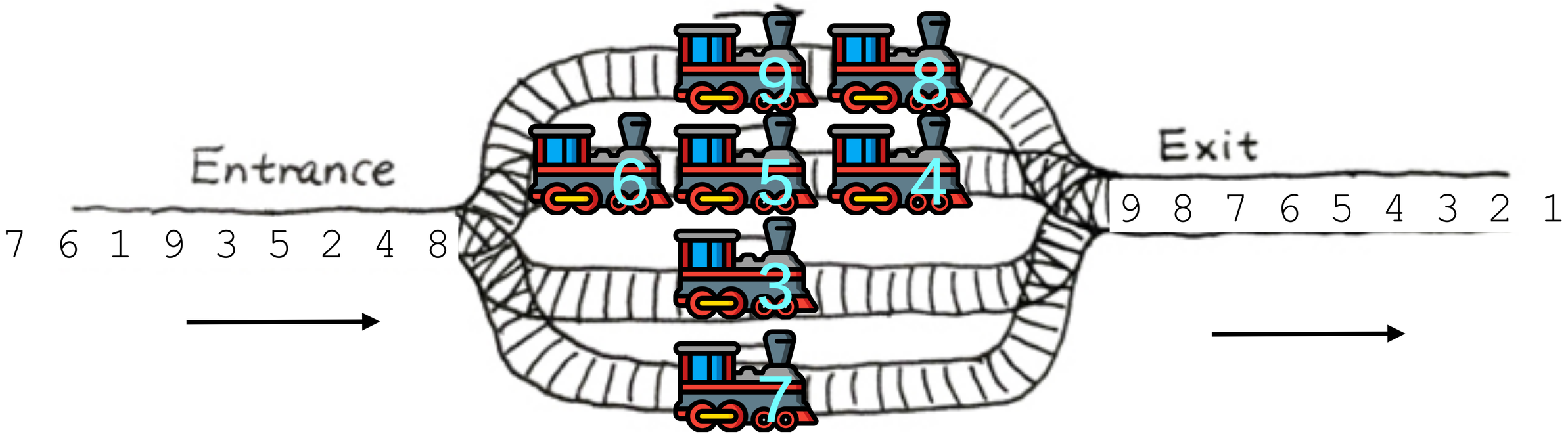

Entrance
7 6 1 9 3 5
6
9
8
5
4
3
2
1
Exit
9 8 7 6 5 4 3 2 1

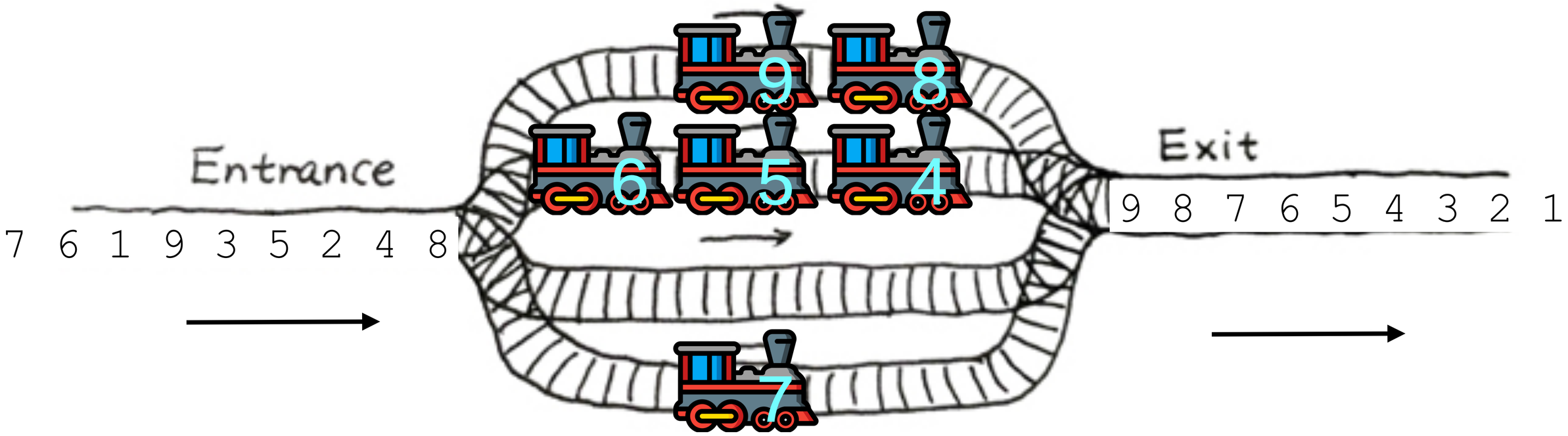

Entrance
7 6 1 9 3 5 2 4 8
9
8
6
5
4
3
2
1
Exit
9 8 7 6 5 4 3 2 1

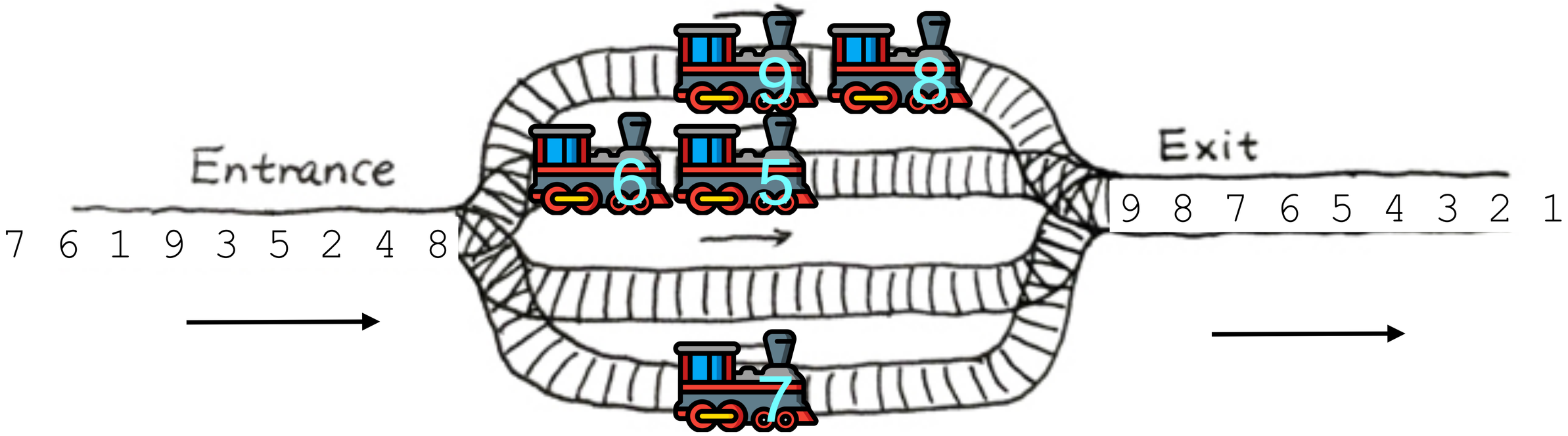

Entrance
7 6 1 9 3 5
Exit
9 8 7 6 5 4 3 2 1

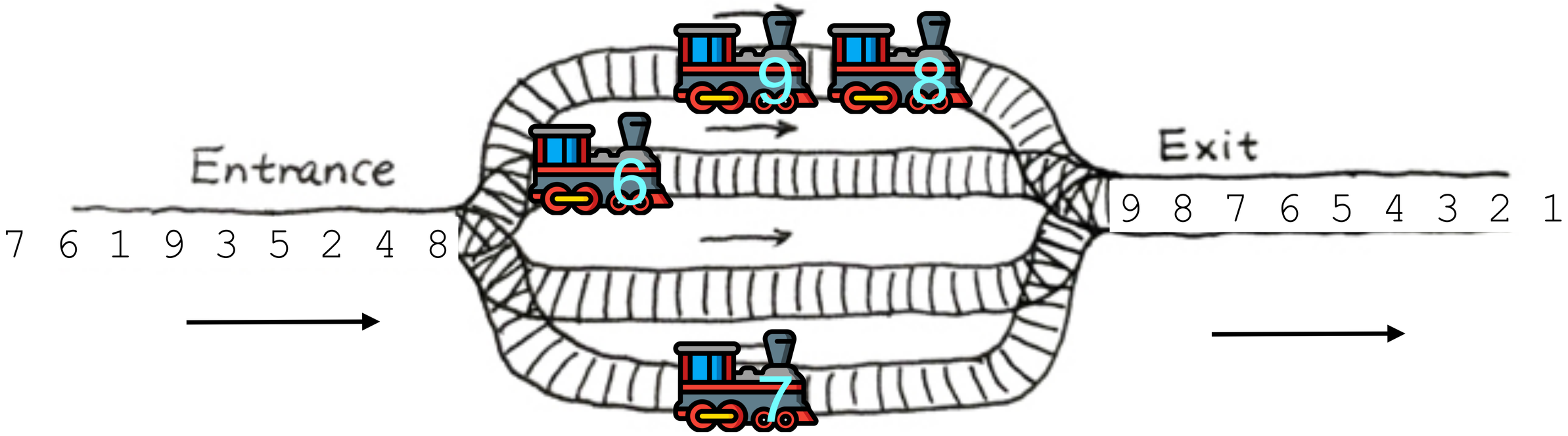

Entrance
7 6 1 9 3 5 2 4 8
9
8
6
5
4
3
2
7
1
Exit
9 8 7 6 5 4 3 2 1

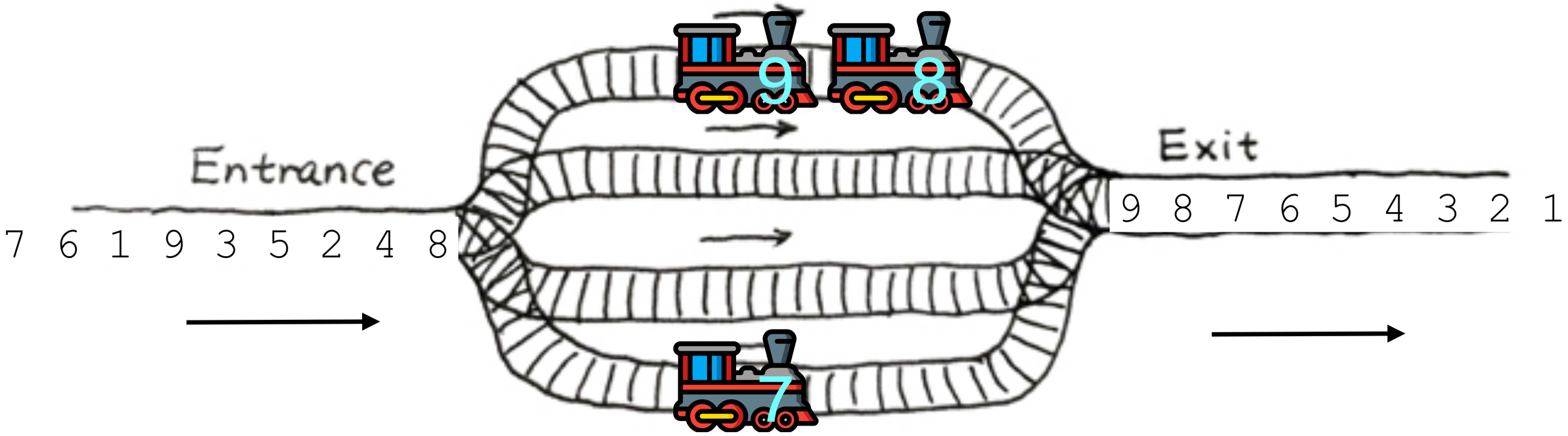

Entrance
7 6 1 9 3 5 2 4 8
9
8
6
5
4
3
2
7
Exit
9 8 7 6 5 4 3 2 1

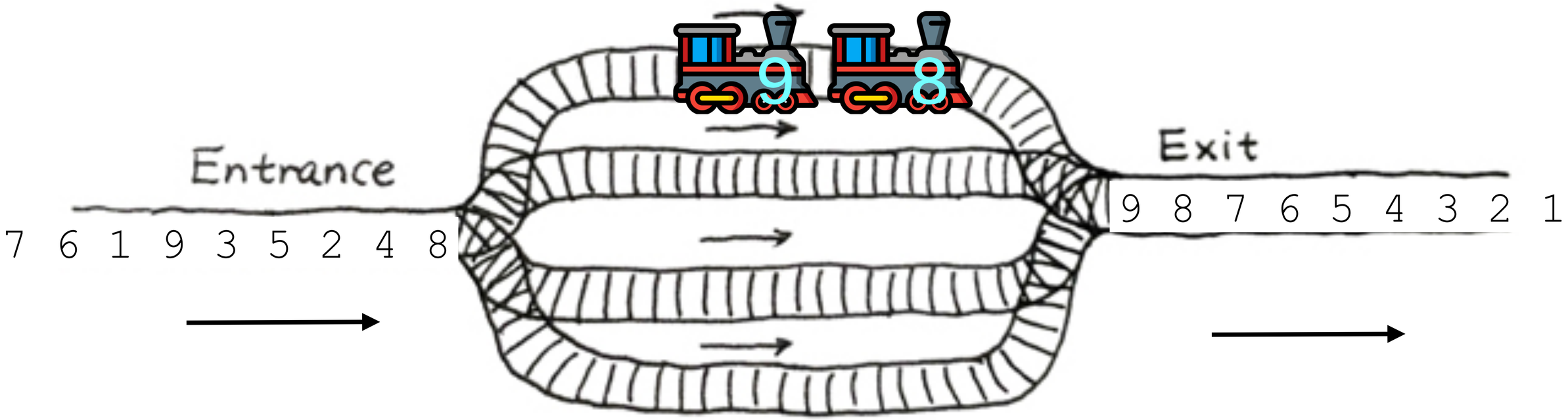

Entrance
7 6 1 9 3 5 2 4 8
9
8
6
5
4
3
7
Exit
9 8 7 6 5 4 3 2 1

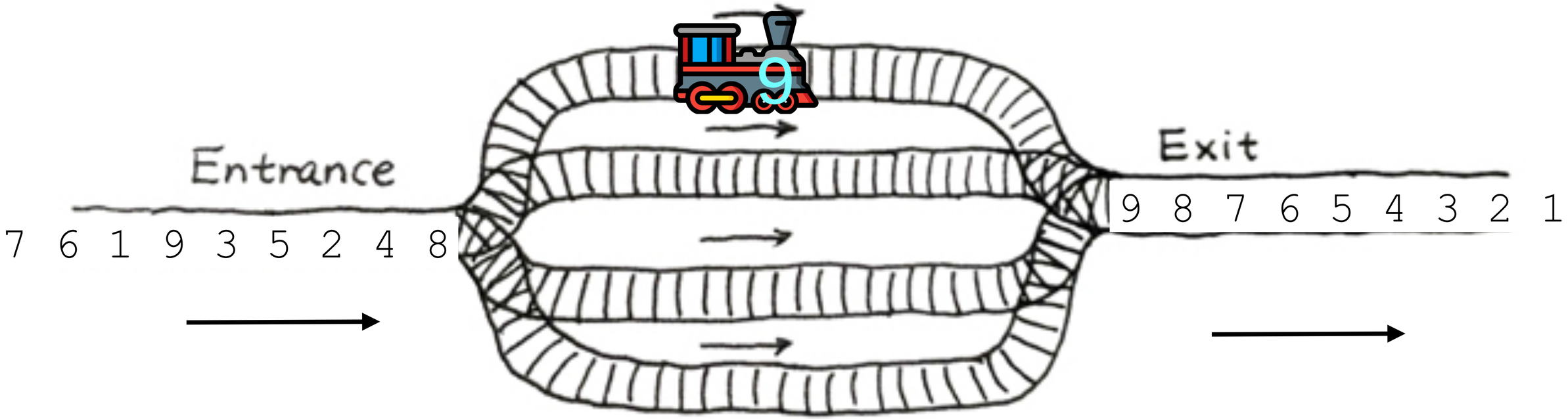

Entrance
7 6 1 9 3 5 2 4 8
9
8
6
5
4
7
Exit
9 8 7 6 5 4 3 2 1

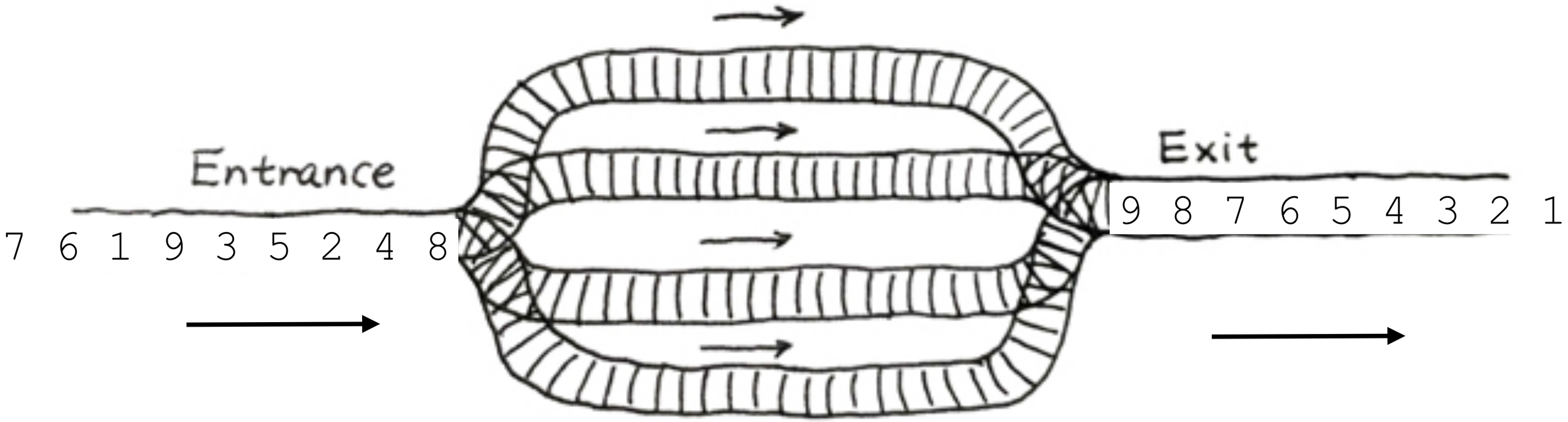

Entrance
7 6 1 9 3 5 2 4 8
9
8
6
5
7
Exit
9 8 7 6 5 4 3 2 1


Entrance
7 6 1 9 3 5 2 4 8
9
8
6
7
Exit
9 8 7 6 5 4 3 2 1


Entrance
7 6 1 9 3 5 2 4 8
9
8
7
Exit
9 8 7 6 5 4 3 2 1


9
8
Entrance
7 6 1 9 3 5 2 4 8
Exit
9 8 7 6 5 4 3 2 1

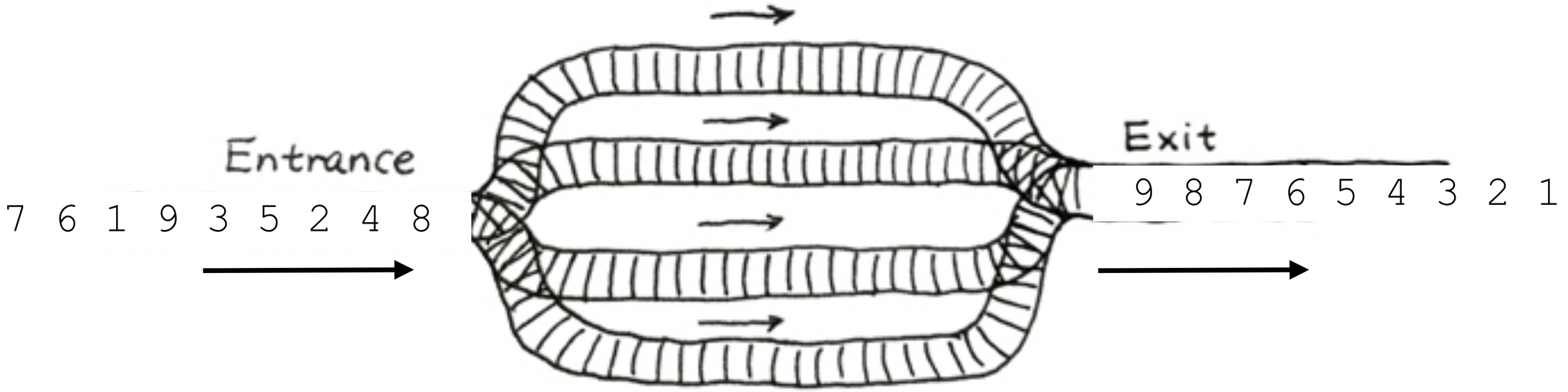

Entrance
7 6 1 9 3 5 2 4 8
9
Exit
9 8 7 6 5 4 3 2 1


Entrance
7 6 1 9 3 5 2 4 8
Exit
9 8 7 6 5 4 3 2 1

栈与队列的应用

[2016] 3. 设有下图所示的火车车轨，入口到出口之间有 n 条轨道，列车的行进方向均为从左至右，列车可驶入任意一条轨道。现有编号为 1～9 的 9 列列车，驶入的次序依次是 8, 4, 2, 5, 3, 9, 1, 6, 7。若期望驶出的次序依次为 1~9，则 n 至少是_____。

A. 2　　B. 3　　C. 4　　D. 5



栈与队列的应用

[2017] 2. 下列关于栈的叙述中，错误的是_____。

I. 采用非递归方式重写递归程序时必须使用栈。

II. 调用函数时，系统要用栈保存必要信息。

III. 只要确定了入栈次序，就可确定出栈次序。

IV. 栈是一种受限的线性表，允许在其两端进行操作。

A. 仅 I　　B. 仅 I、II、III　　C. 仅 I、III、IV　　D. 仅 II、III、IV

栈与队列的应用

[2017] 2. 下列关于栈的叙述中，错误的是_____。

I. 采用非递归方式重写递归程序时必须使用栈。

II. 调用函数时，系统要用栈保存必要信息。

III. 只要确定了入栈次序，就可确定出栈次序。

IV. 栈是一种受限的线性表，允许在其两端进行操作。

A. 仅 I　　B. 仅 I、II、III　　C. 仅 I、III、IV　　D. 仅 II、III、IV

栈与队列的应用

[2018] 1. 若栈 S_1 中保存整数，栈 S_2 中保存运算符，函数 F() 依次执行下述各步操作：

（1）从 S_1 中依次弹出两个操作数 a 和 b；

（2）从 S_2 中弹出一个运算符 op；

（3）执行相应的运算 b op a；

（4）将运算结果压入 S_1 中。

假定 S_1 中的操作数依次是 5, 8, 3, 2（2 在栈顶），S_2 中的运算符依次是 *, −, +（+ 在栈顶）。调用 3 次 F() 后，S_1 栈顶保存的值是_____。

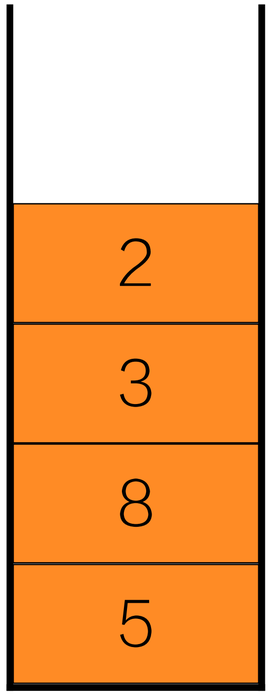
A. −15　　B. 15　　C. −20　　D. 20

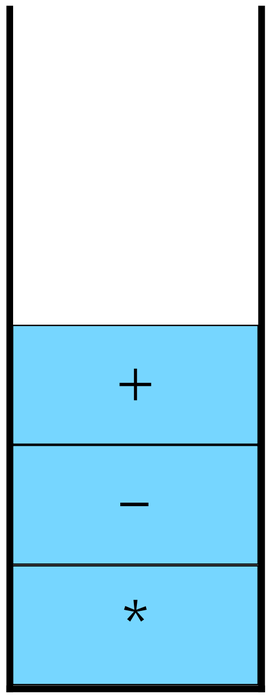
[2018] 1. 若栈 S_1 中保存整数，栈 S_2 中保存运算符，函数 F() 依次执行下述各步操作：

（1）从 S_1 中依次弹出两个操作数a 和 b；　（2）从 S_2 中弹出一个运算符 op；

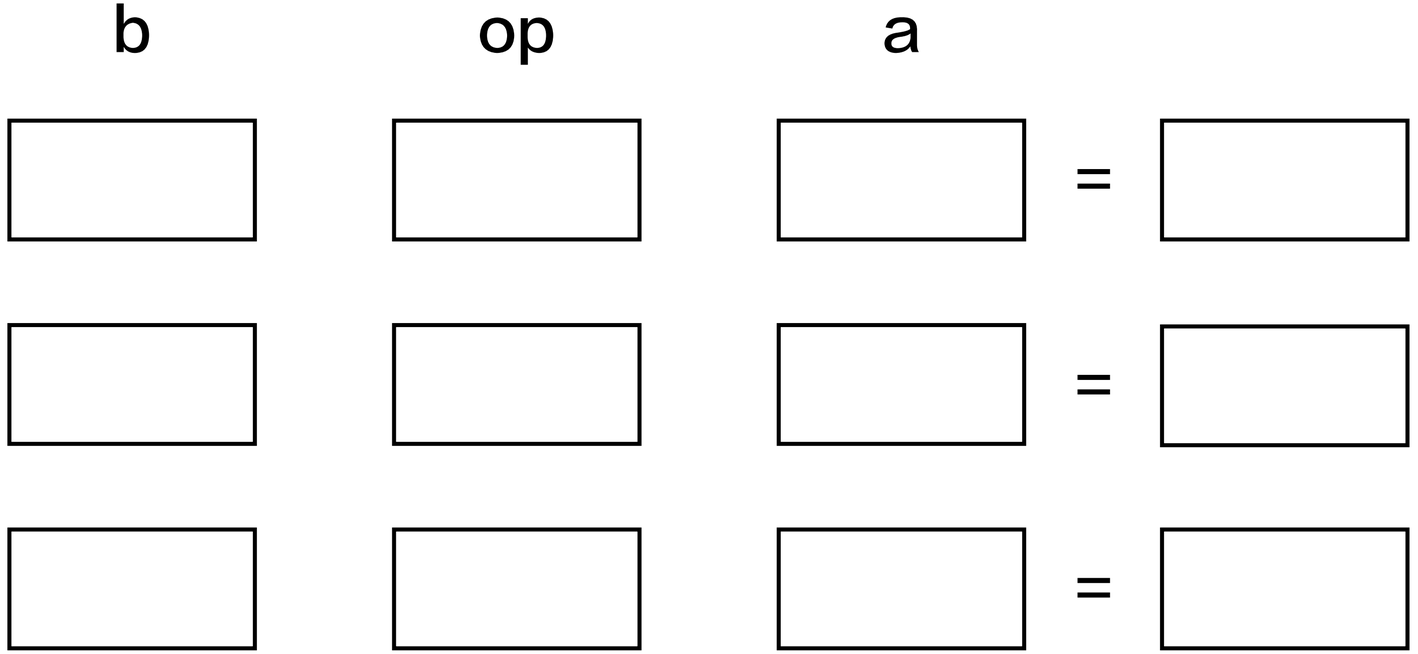
（3）执行相应的运算 b op a；　（4）将运算结果压入 S_1 中。

S_1





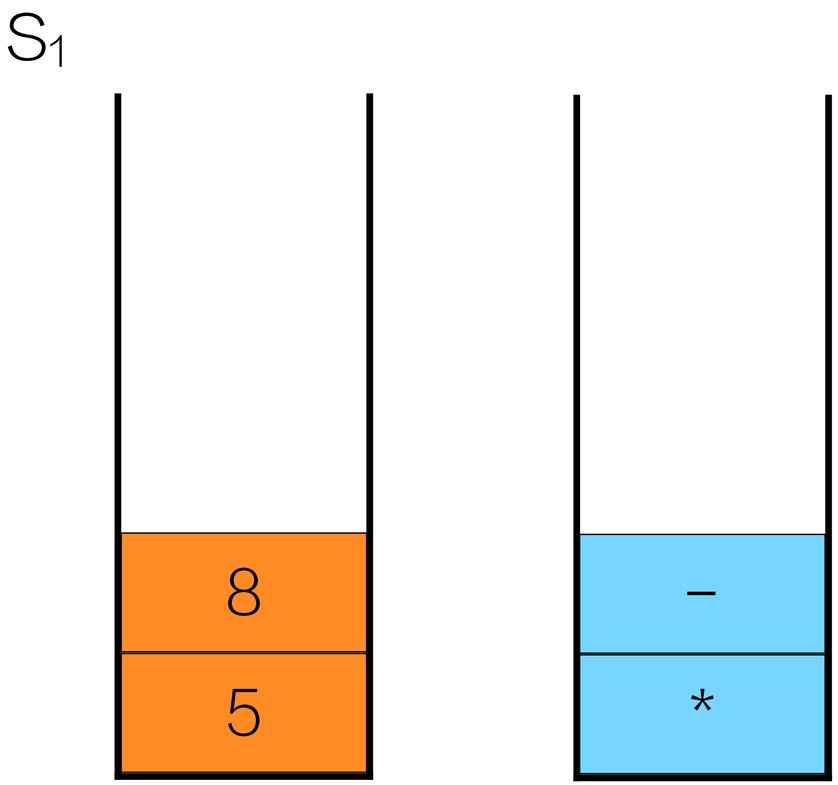
S_2

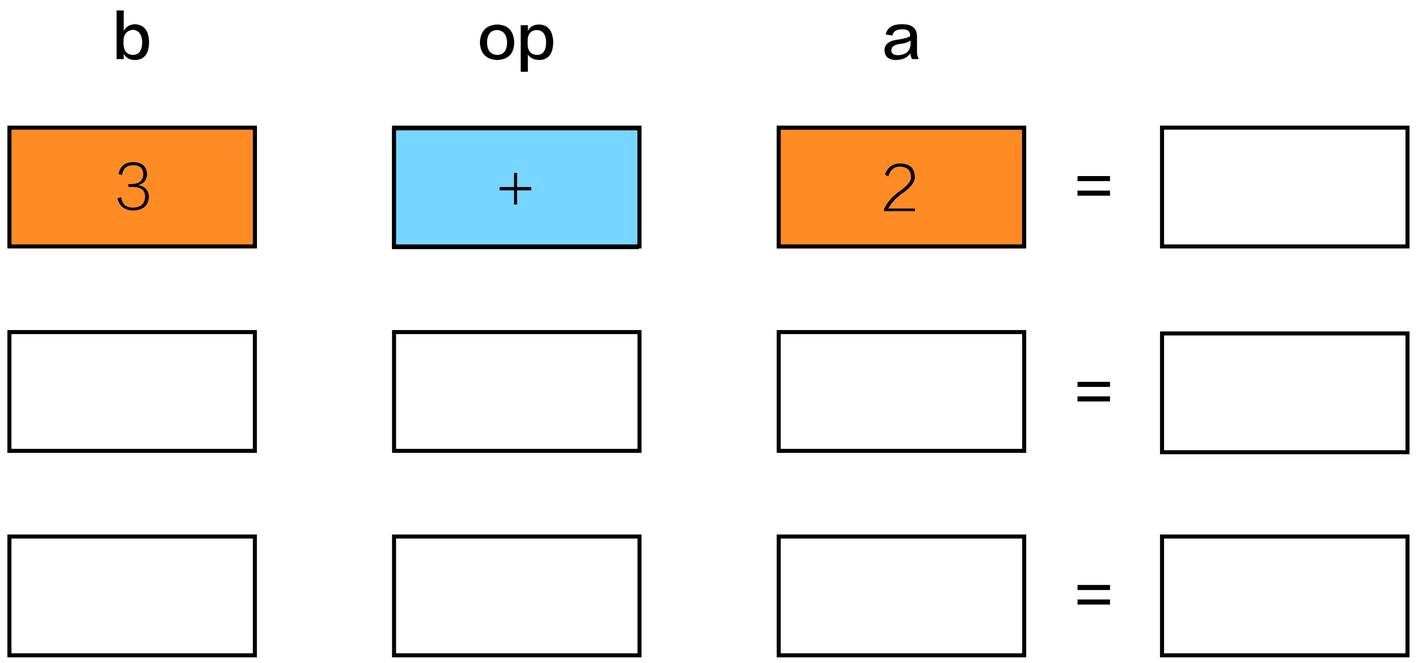


[2018] 1. 若栈 S_1 中保存整数，栈 S_2 中保存运算符，函数 F() 依次执行下述各步操作：

（1）从 S_1 中依次弹出两个操作数a 和 b；　（2）从 S_2 中弹出一个运算符 op；

（3）执行相应的运算 b op a；　（4）将运算结果压入 S_1 中。

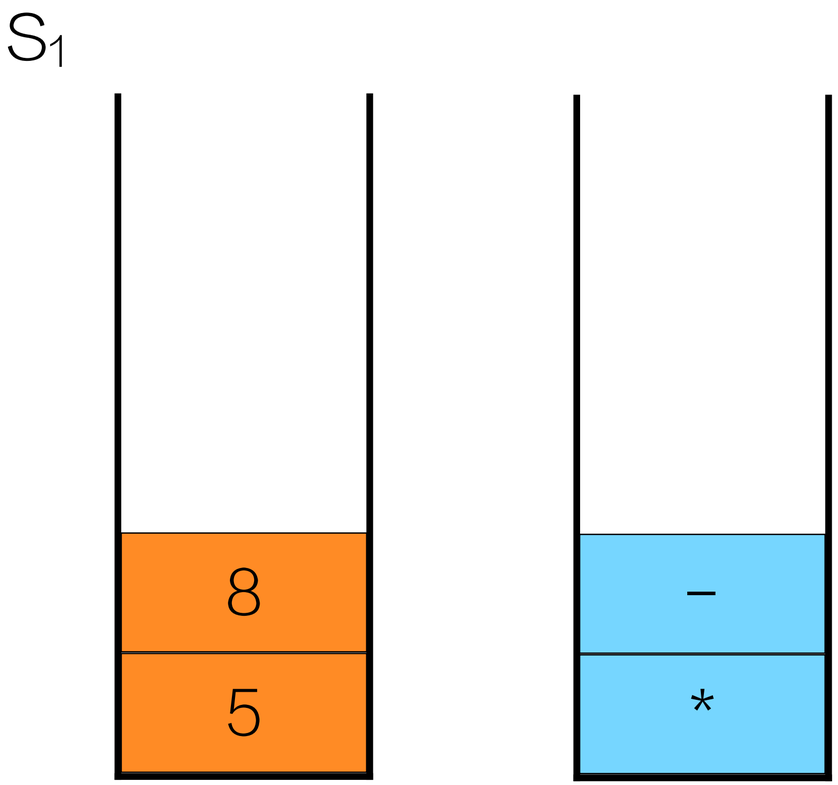


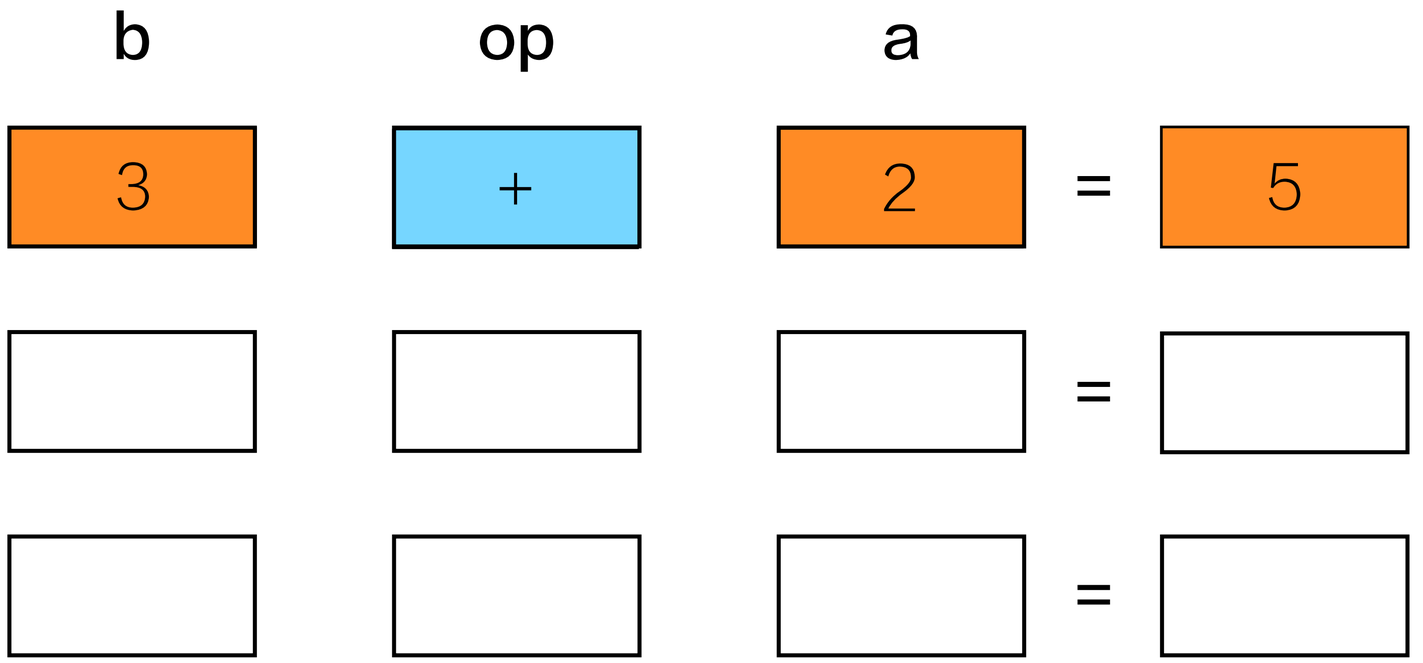


[2018] 1. 若栈 S_1 中保存整数，栈 S_2 中保存运算符，函数 F() 依次执行下述各步操作：

（1）从 S_1 中依次弹出两个操作数a 和 b；　（2）从 S_2 中弹出一个运算符 op；

（3）执行相应的运算 b op a；　（4）将运算结果压入 S_1 中。

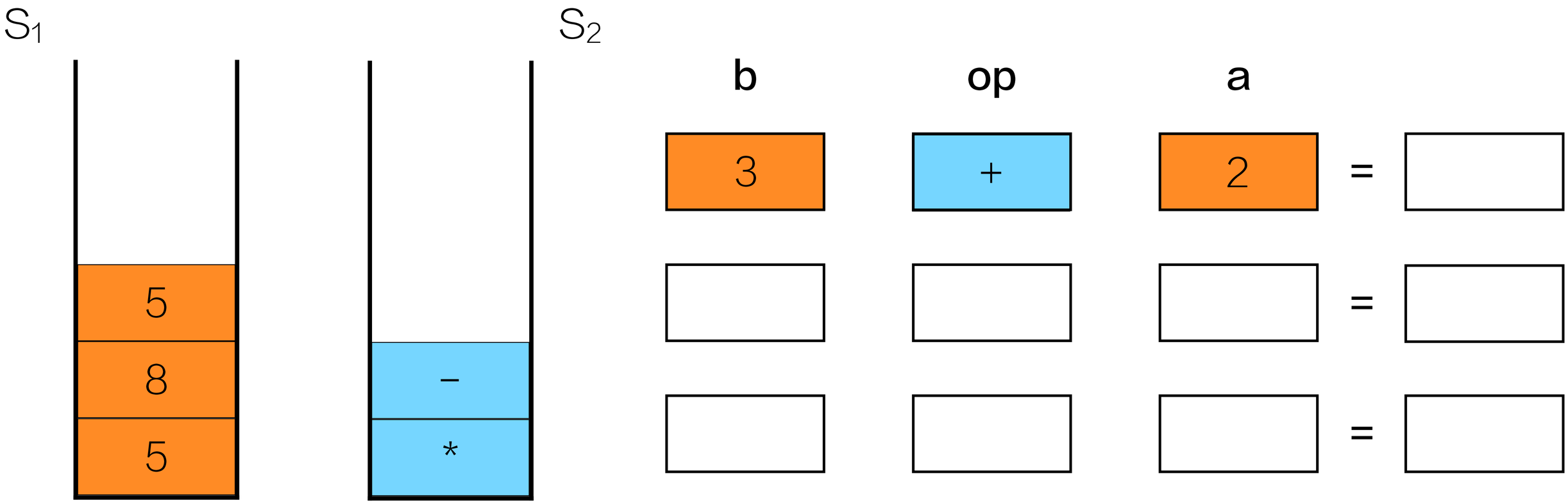




[2018] 1. 若栈 S_1 中保存整数，栈 S_2 中保存运算符，函数 F() 依次执行下述各步操作：

（1）从 S_1 中依次弹出两个操作数a 和 b；　（2）从 S_2 中弹出一个运算符 op；

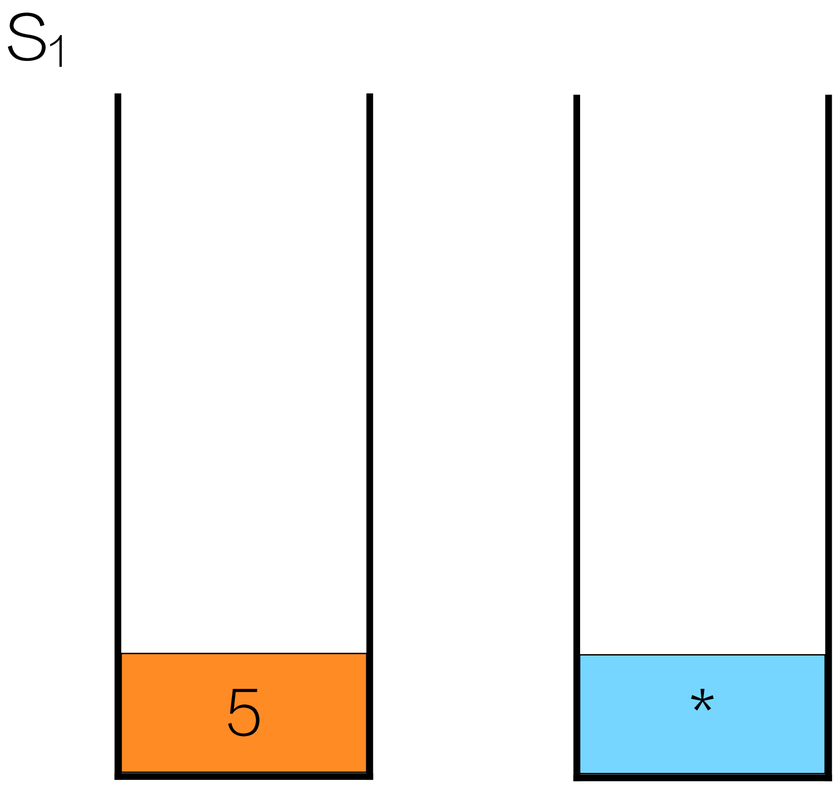
（3）执行相应的运算 b op a；　（4）将运算结果压入 S_1 中。

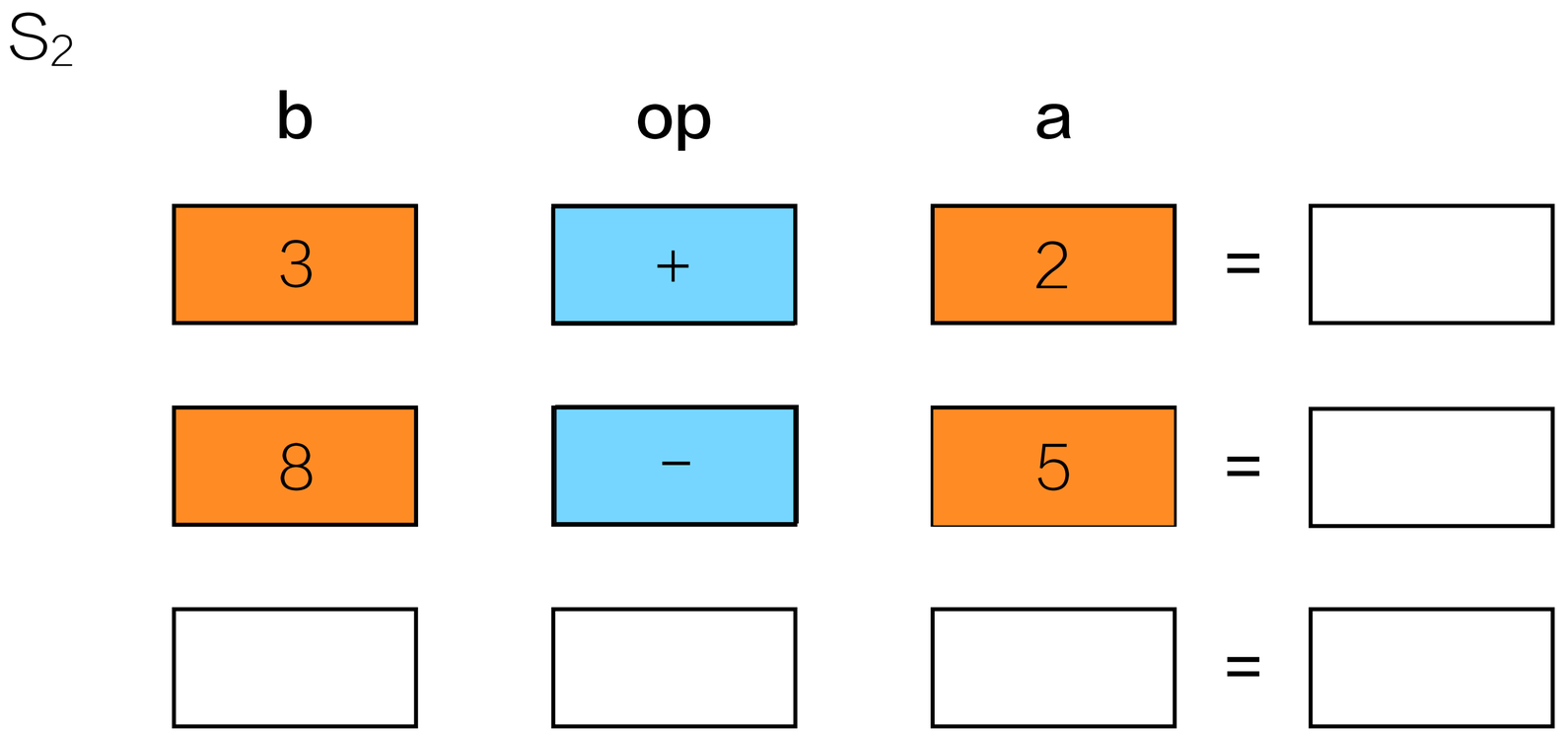


[2018] 1. 若栈 S_1 中保存整数，栈 S_2 中保存运算符，函数 F() 依次执行下述各步操作：

（1）从 S_1 中依次弹出两个操作数a 和 b；　（2）从 S_2 中弹出一个运算符 op；

（3）执行相应的运算 b op a；　（4）将运算结果压入 S_1 中。

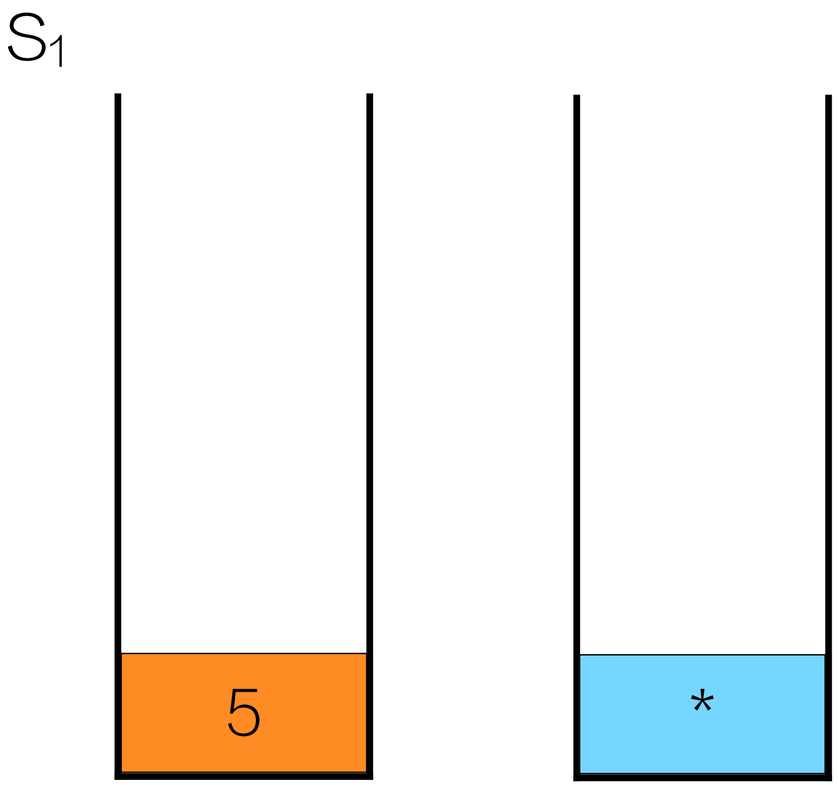


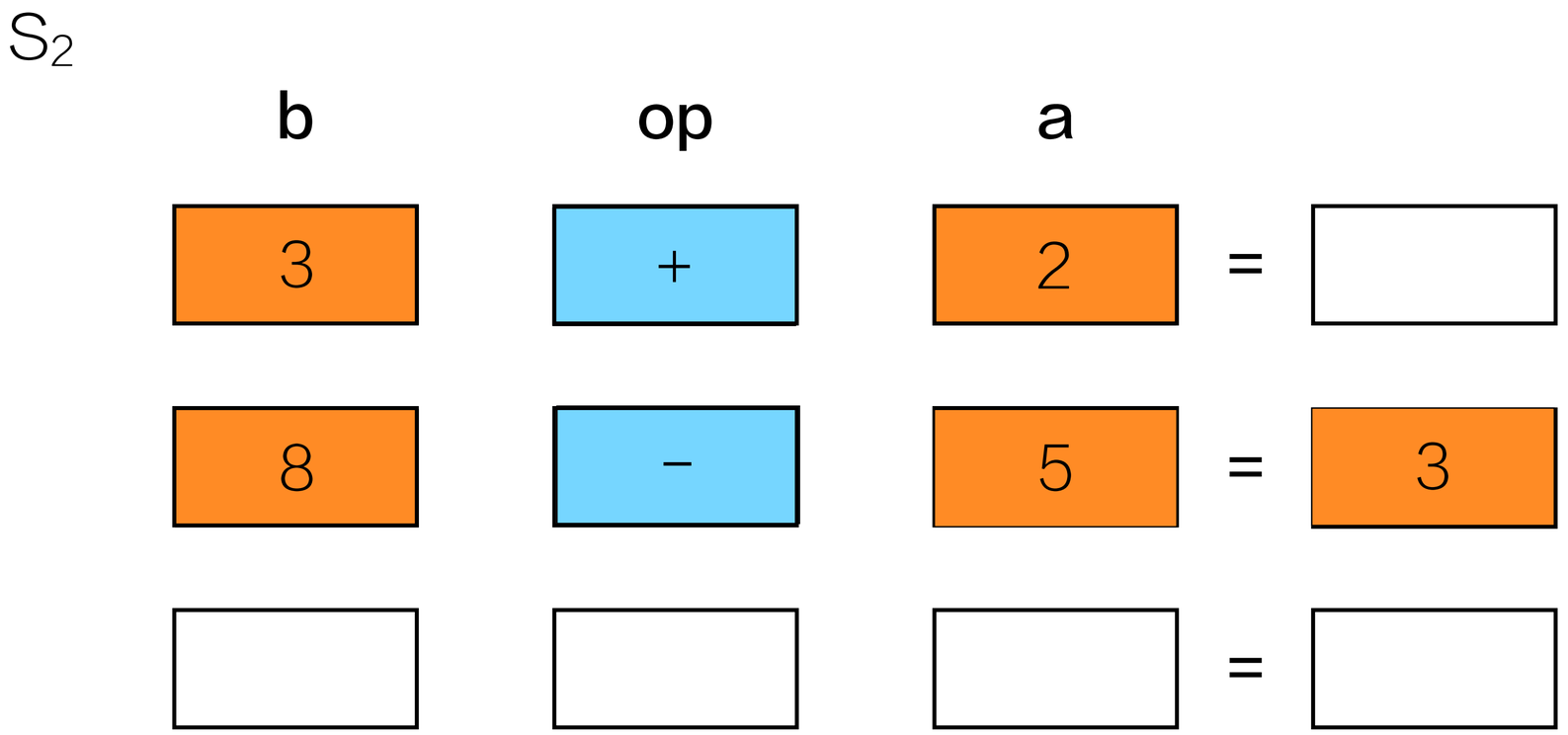


[2018] 1. 若栈 S_1 中保存整数，栈 S_2 中保存运算符，函数 F() 依次执行下述各步操作：

（1）从 S_1 中依次弹出两个操作数a 和 b；　（2）从 S_2 中弹出一个运算符 op；

（3）执行相应的运算 b op a；　（4）将运算结果压入 S_1 中。

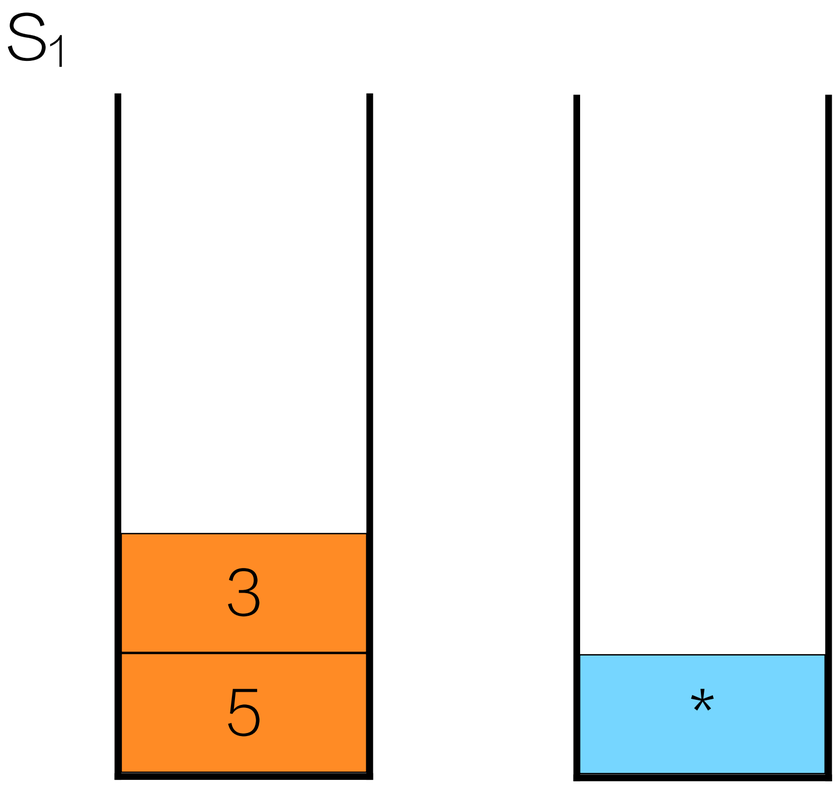


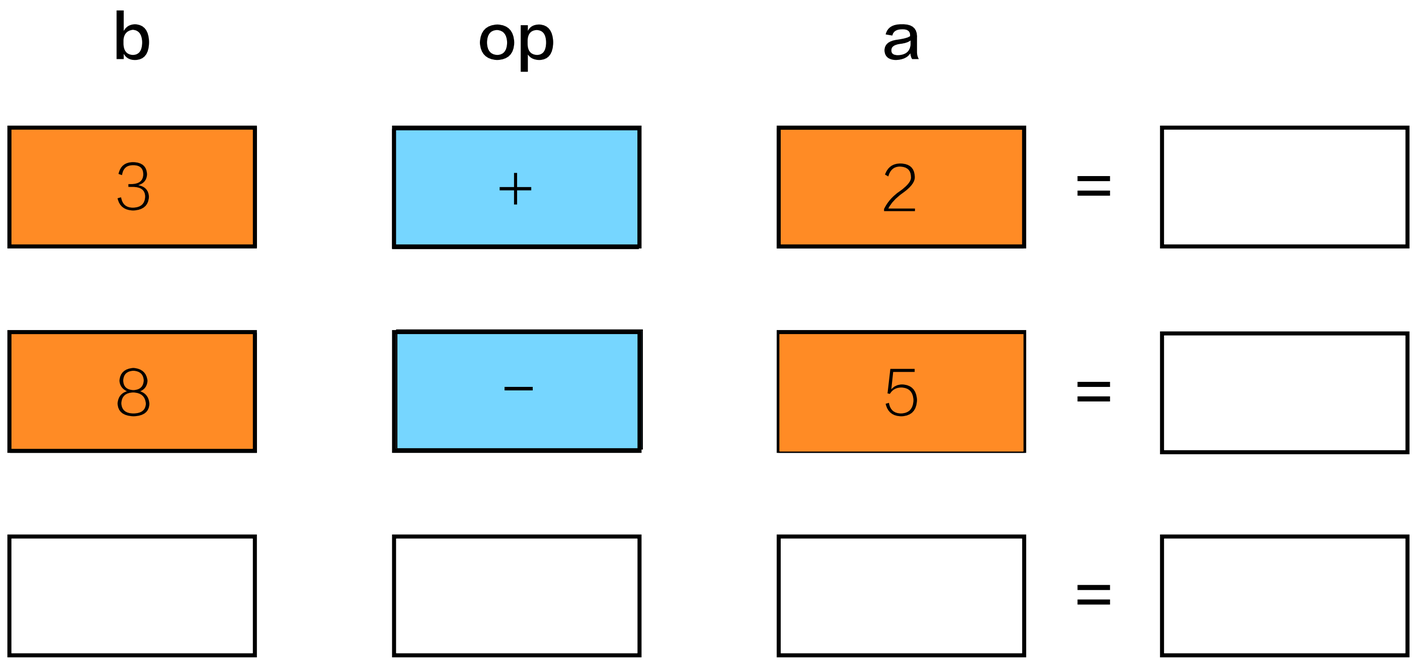


[2018] 1. 若栈 S_1 中保存整数，栈 S_2 中保存运算符，函数 F() 依次执行下述各步操作：

（1）从 S_1 中依次弹出两个操作数a 和 b；　（2）从 S_2 中弹出一个运算符 op；

（3）执行相应的运算 b op a；　（4）将运算结果压入 S_1 中。

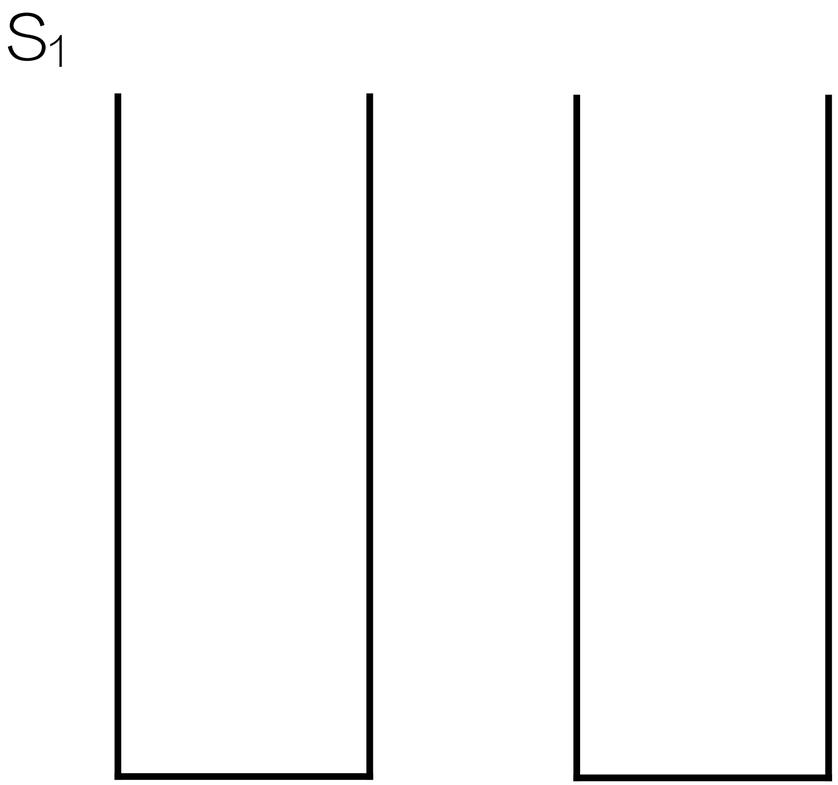




[2018] 1. 若栈 S_1 中保存整数，栈 S_2 中保存运算符，函数 F() 依次执行下述各步操作：

（1）从 S_1 中依次弹出两个操作数a 和 b；　（2）从 S_2 中弹出一个运算符 op；

（3）执行相应的运算 b op a；　（4）将运算结果压入 S_1 中。

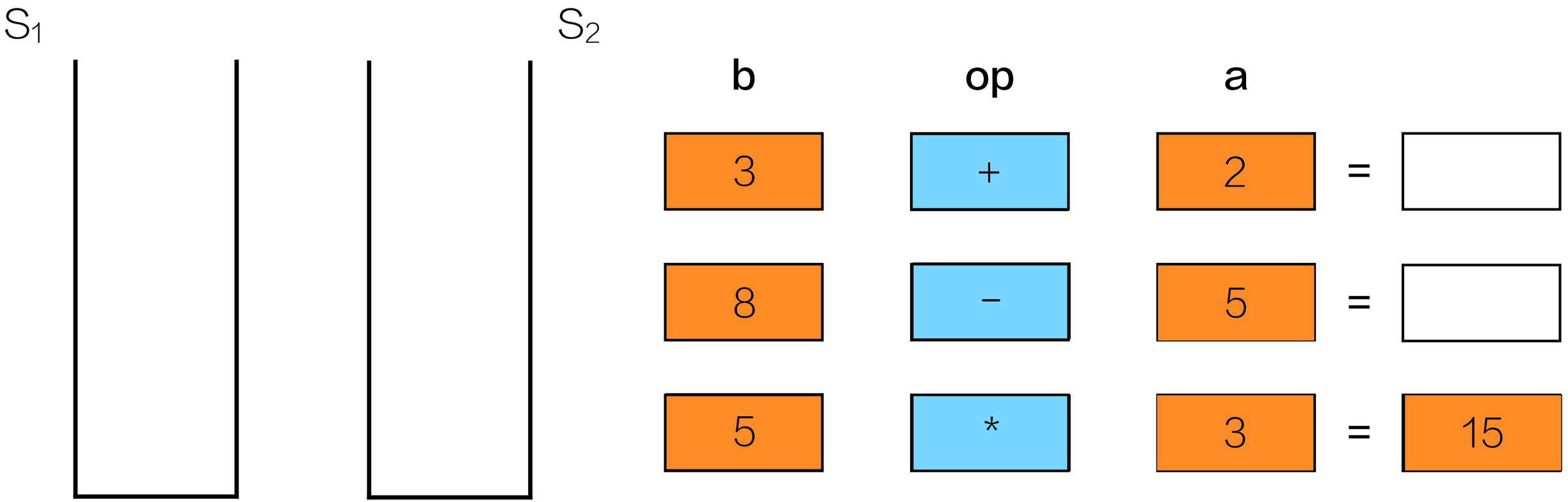


b	op	a		
3	+	2	=	
8	-	5	=	
5	*	3	=	

[2018] 1. 若栈 S_1 中保存整数，栈 S_2 中保存运算符，函数 F() 依次执行下述各步操作：

（1）从 S_1 中依次弹出两个操作数a 和 b；　（2）从 S_2 中弹出一个运算符 op；

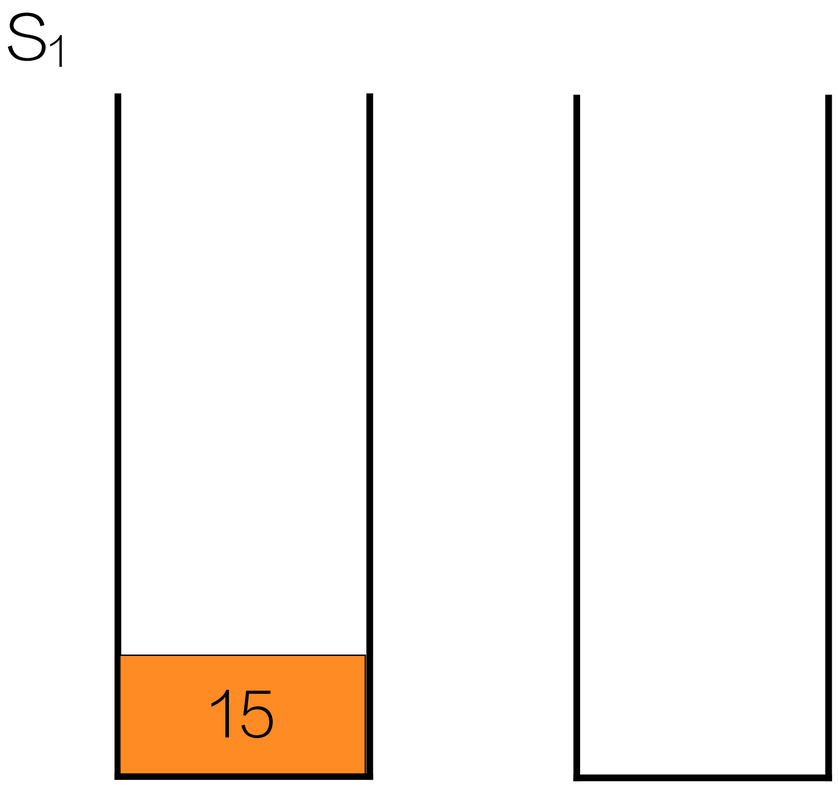
（3）执行相应的运算 b op a；　（4）将运算结果压入 S_1 中。

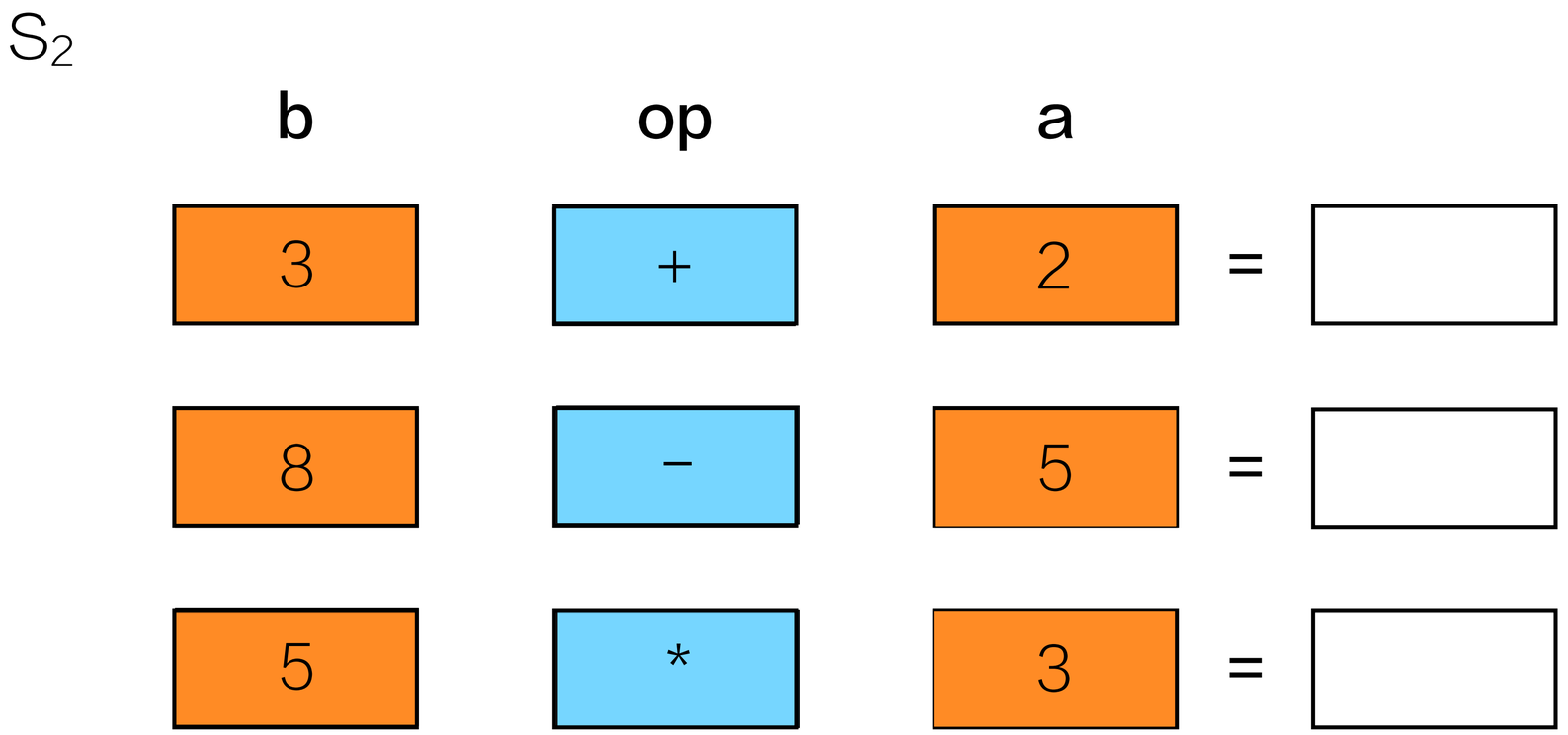


[2018] 1. 若栈 S_1 中保存整数，栈 S_2 中保存运算符，函数 F() 依次执行下述各步操作：

（1）从 S_1 中依次弹出两个操作数a 和 b；　（2）从 S_2 中弹出一个运算符 op；

（3）执行相应的运算 b op a；　（4）将运算结果压入 S_1 中。





栈与队列的应用

[2018] 1. 若栈 S_1 中保存整数，栈 S_2 中保存运算符，函数 F() 依次执行下述各步操作：

（1）从 S_1 中依次弹出两个操作数 a 和 b；

（2）从 S_2 中弹出一个运算符 op；

（3）执行相应的运算 b op a；

（4）将运算结果压入 S_1 中。

假定 S_1 中的操作数依次是 5, 8, 3, 2（2 在栈顶），S_2 中的运算符依次是 *, −, +（+ 在栈顶）。调用 3 次 F() 后，S_1 栈顶保存的值是_____。

A. −15　　B. 15　　C. −20　　D. 20

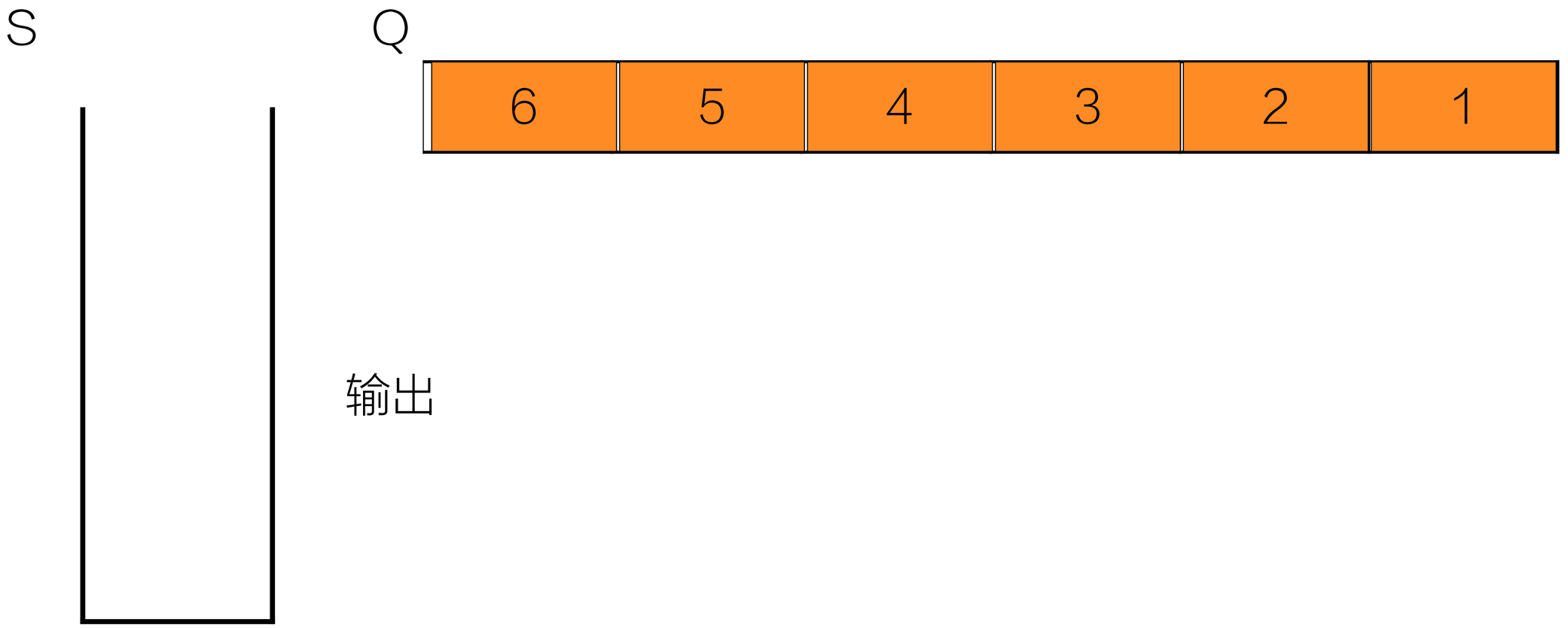
栈与队列的应用

[2018] 2. 现有队列 Q 与栈 S，初始时 Q 中的元素依次是 1, 2, 3, 4, 5, 6（1 在队头），S 为空。若仅允许下列 3 种操作：（1）出队并输出出队元素；（2）出队并将出队元素入栈；（3）出栈并输出出栈元素，则不能得到的输出序列是_____。

A. 1, 2, 5, 6, 4, 3　　B. 2, 3, 4, 5, 6, 1　　C. 3, 4, 5, 6, 1, 2　　D. 6, 5, 4, 3, 2, 1

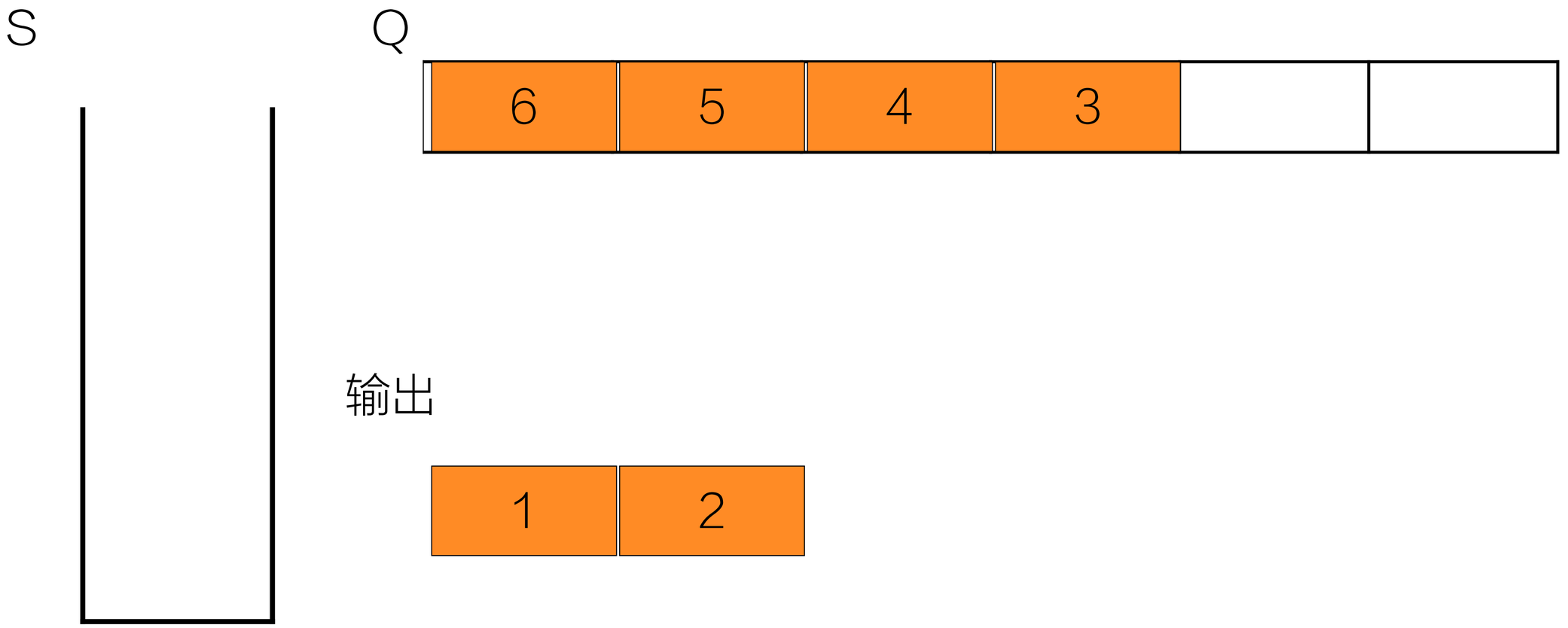
[2018] 2. 现有队列 Q 与栈 S，初始时 Q 中的元素依次是 1, 2, 3, 4, 5, 6（1 在队头），S 为空。若仅允许下列 3 种操作：（1）出队并输出出队元素；（2）出队并将出队元素入栈；（3）出栈并输出出栈元素，则不能得到的输出序列是_____。

A. 1, 2, 5, 6, 4, 3　　B. 2, 3, 4, 5, 6, 1　　C. 3, 4, 5, 6, 1, 2　　D. 6, 5, 4, 3, 2, 1



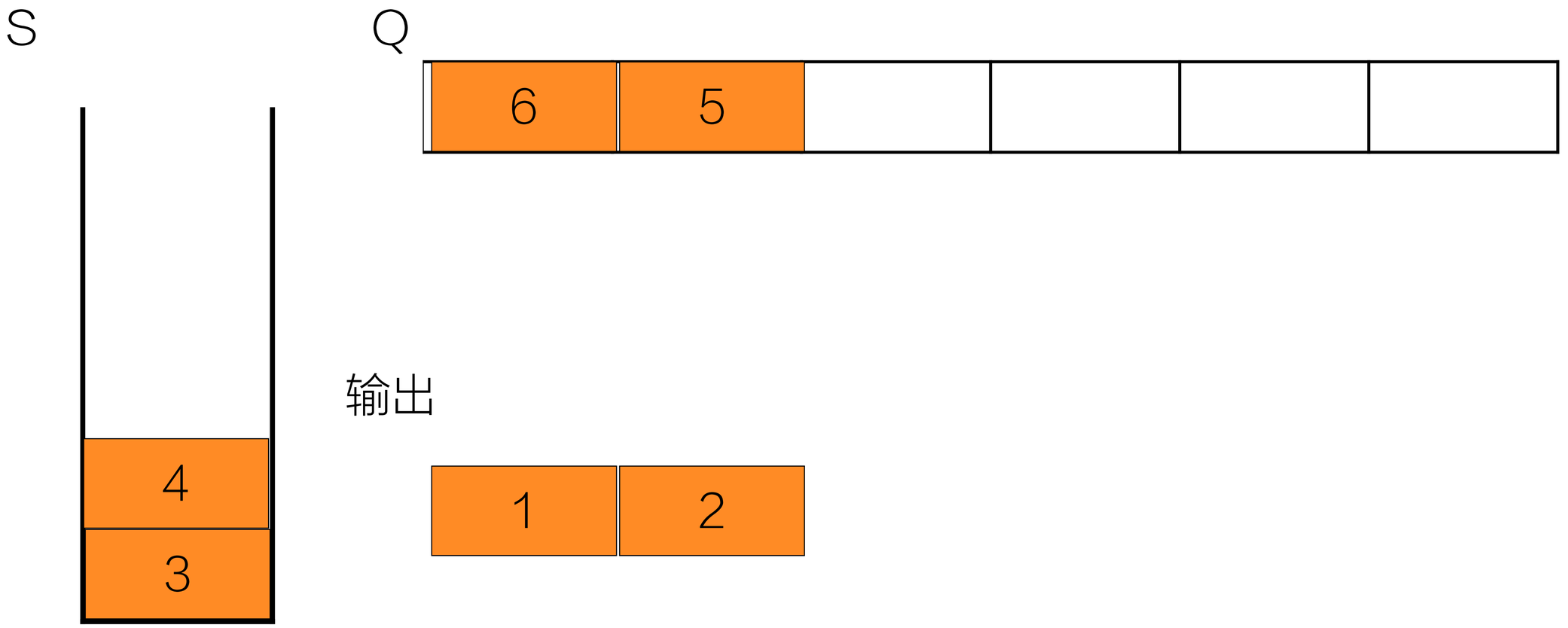
[2018] 2. 现有队列 Q 与栈 S，初始时 Q 中的元素依次是 1, 2, 3, 4, 5, 6（1 在队头），S 为空。若仅允许下列 3 种操作：（1）出队并输出出队元素；（2）出队并将出队元素入栈；（3）出栈并输出出栈元素，则不能得到的输出序列是_____。

A. 1, 2, 5, 6, 4, 3　　B. 2, 3, 4, 5, 6, 1　　C. 3, 4, 5, 6, 1, 2　　D. 6, 5, 4, 3, 2, 1



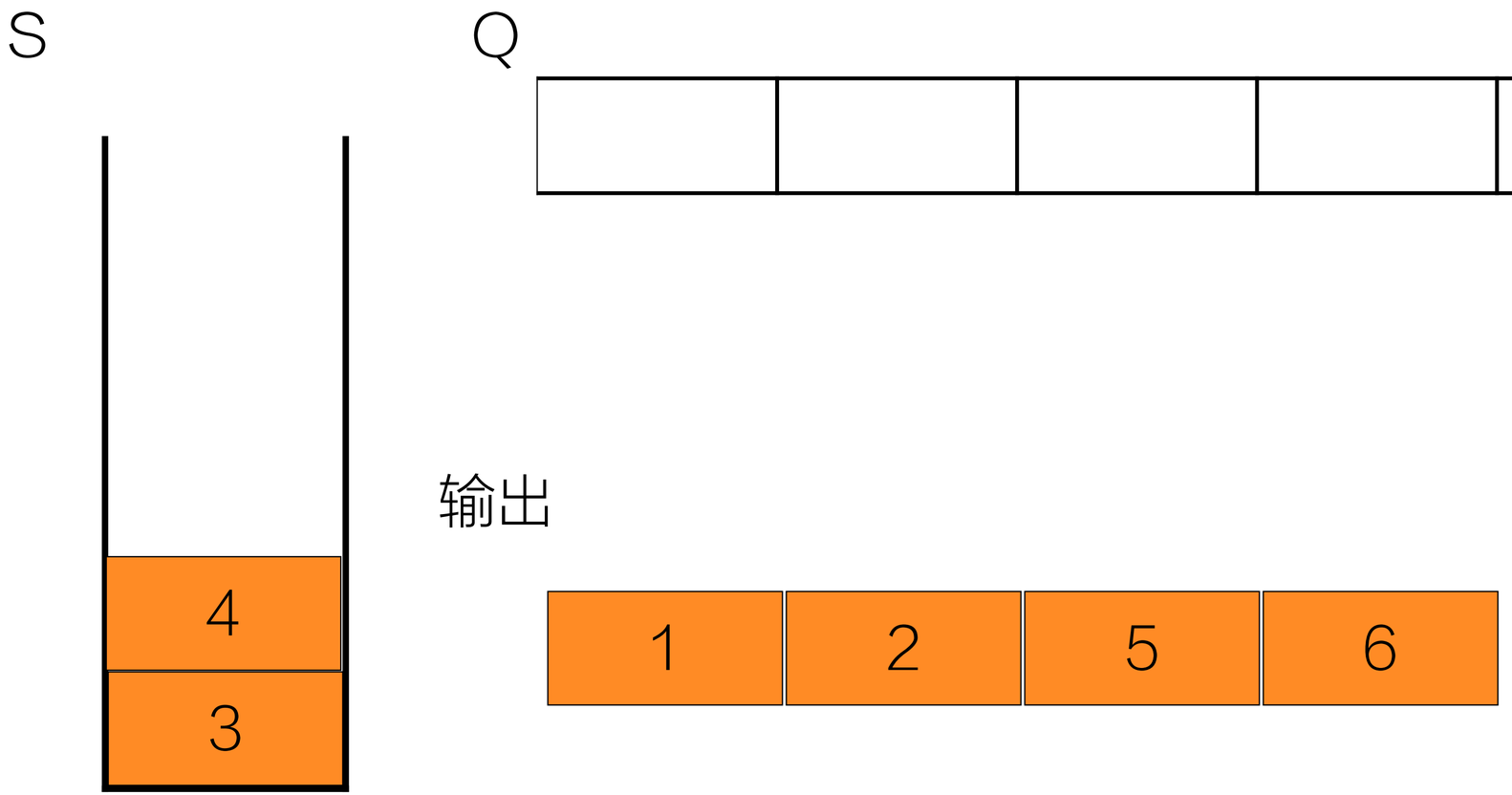
[2018] 2. 现有队列 Q 与栈 S，初始时 Q 中的元素依次是 1, 2, 3, 4, 5, 6（1 在队头），S 为空。若仅允许下列 3 种操作：（1）出队并输出出队元素；（2）出队并将出队元素入栈；（3）出栈并输出出栈元素，则不能得到的输出序列是_____。

A. 1, 2, 5, 6, 4, 3　　B. 2, 3, 4, 5, 6, 1　　C. 3, 4, 5, 6, 1, 2　　D. 6, 5, 4, 3, 2, 1



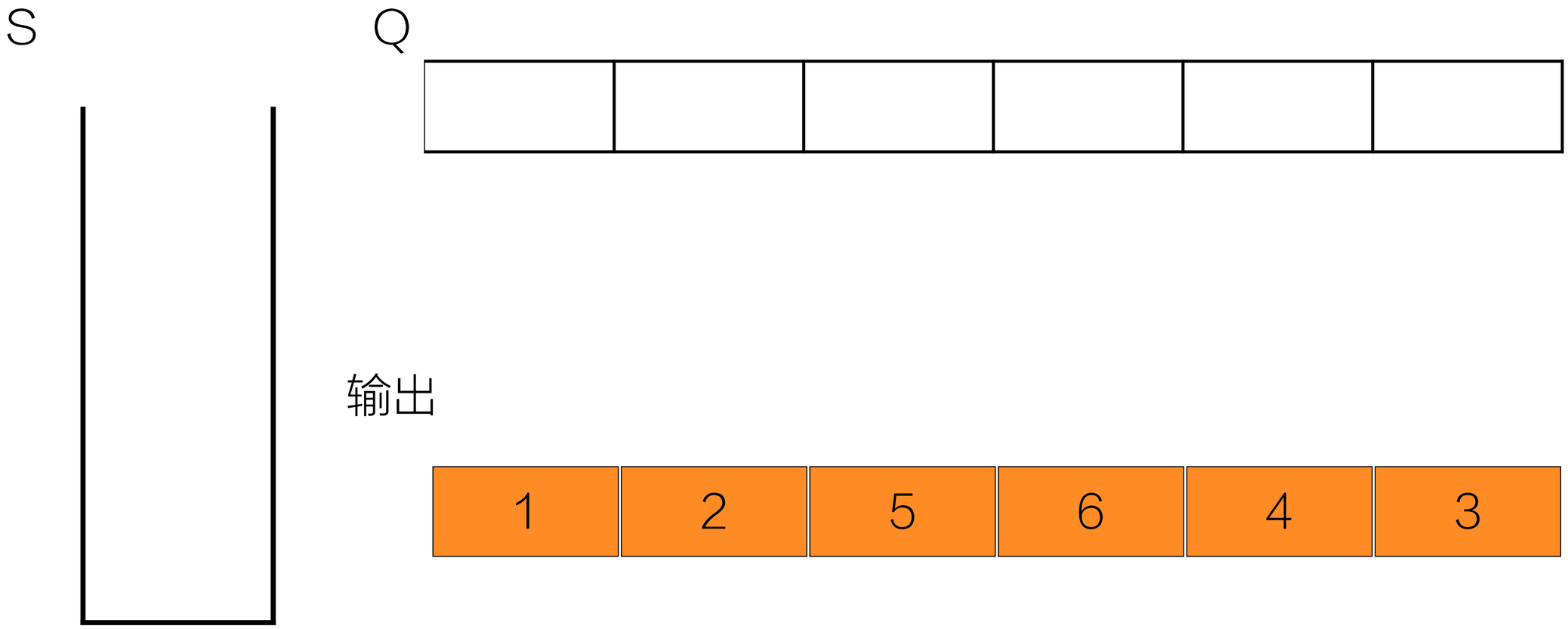
[2018] 2. 现有队列 Q 与栈 S，初始时 Q 中的元素依次是 1, 2, 3, 4, 5, 6（1 在队头），S 为空。若仅允许下列 3 种操作：（1）出队并输出出队元素；（2）出队并将出队元素入栈；（3）出栈并输出出栈元素，则不能得到的输出序列是_____。

A. 1, 2, 5, 6, 4, 3　　B. 2, 3, 4, 5, 6, 1　　C. 3, 4, 5, 6, 1, 2　　D. 6, 5, 4, 3, 2, 1



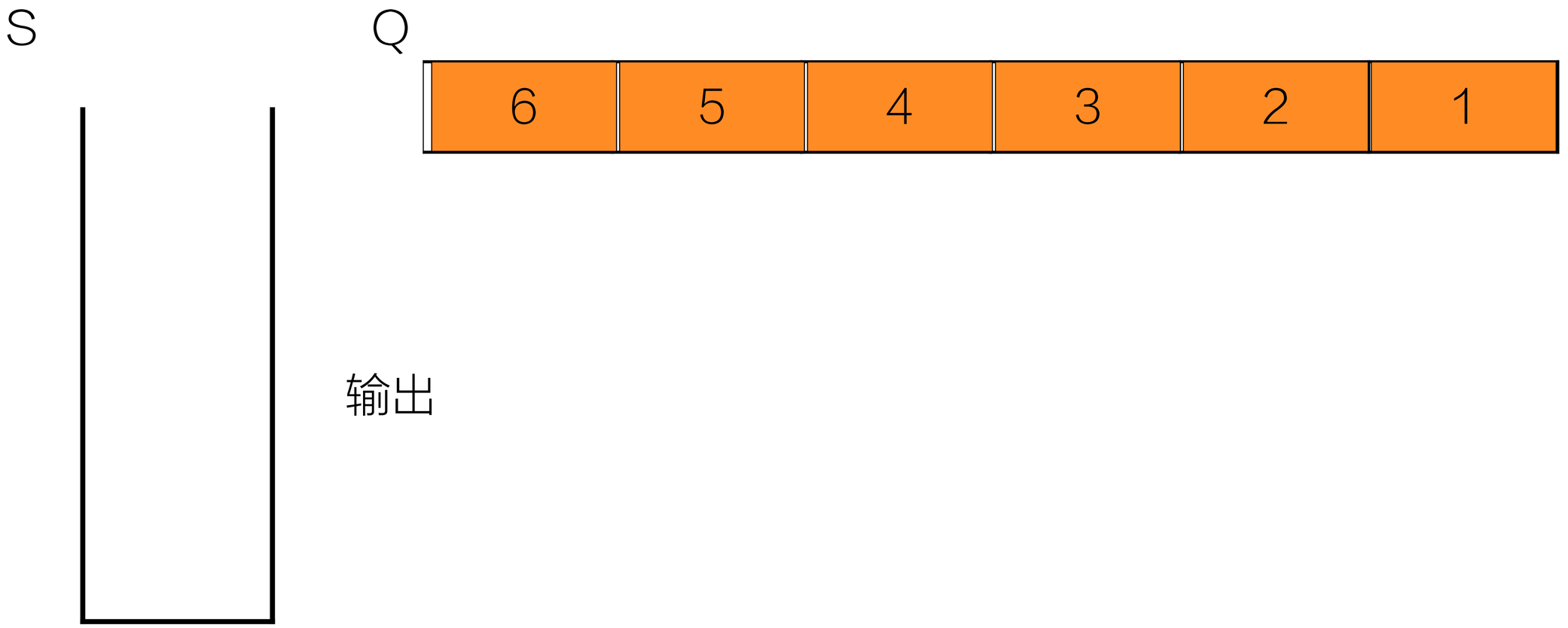
[2018] 2. 现有队列 Q 与栈 S，初始时 Q 中的元素依次是 1, 2, 3, 4, 5, 6（1 在队头），S 为空。若仅允许下列 3 种操作：（1）出队并输出出队元素；（2）出队并将出队元素入栈；（3）出栈并输出出栈元素，则不能得到的输出序列是_____。

A. 1, 2, 5, 6, 4, 3　　B. 2, 3, 4, 5, 6, 1　　C. 3, 4, 5, 6, 1, 2　　D. 6, 5, 4, 3, 2, 1



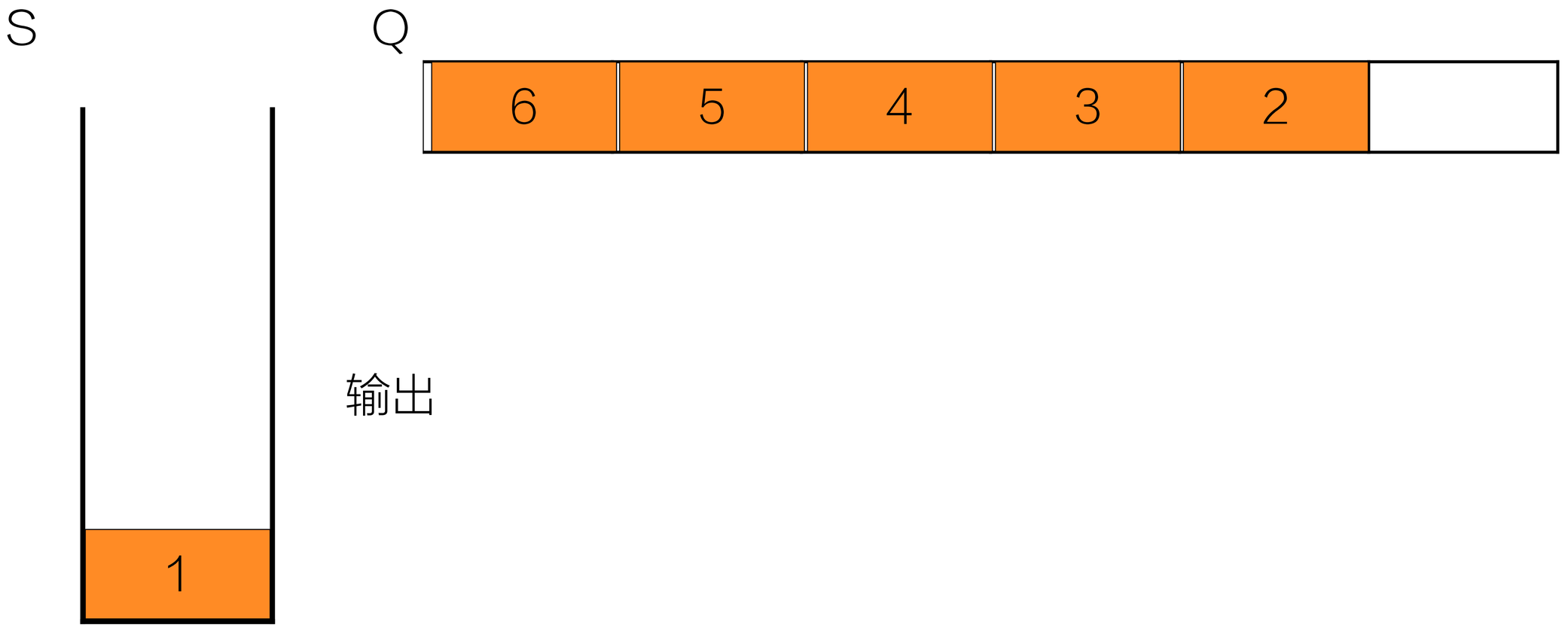
[2018] 2. 现有队列 Q 与栈 S，初始时 Q 中的元素依次是 1, 2, 3, 4, 5, 6（1 在队头），S 为空。若仅允许下列 3 种操作：（1）出队并输出出队元素；（2）出队并将出队元素入栈；（3）出栈并输出出栈元素，则不能得到的输出序列是_____。

A. 1, 2, 5, 6, 4, 3　　B. 2, 3, 4, 5, 6, 1　　C. 3, 4, 5, 6, 1, 2　　D. 6, 5, 4, 3, 2, 1



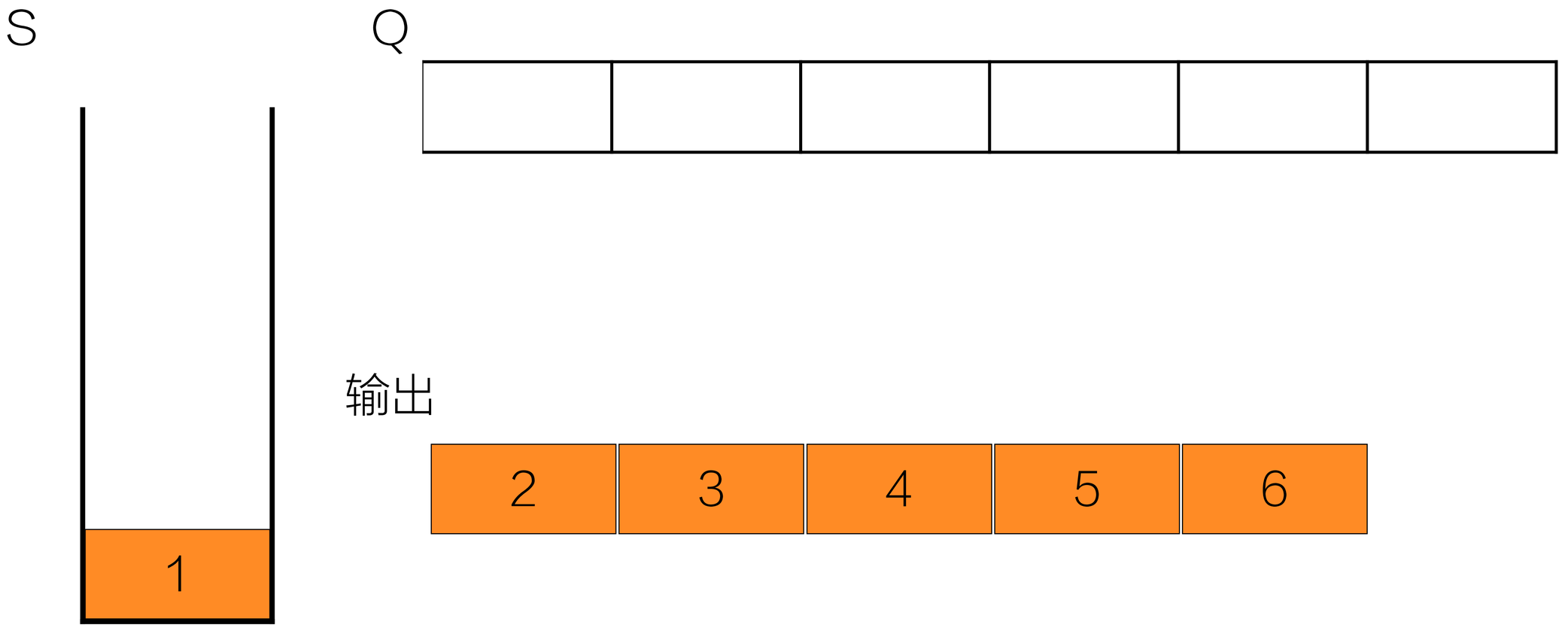
[2018] 2. 现有队列 Q 与栈 S，初始时 Q 中的元素依次是 1, 2, 3, 4, 5, 6（1 在队头），S 为空。若仅允许下列 3 种操作：（1）出队并输出出队元素；（2）出队并将出队元素入栈；（3）出栈并输出出栈元素，则不能得到的输出序列是_____。

A. 1, 2, 5, 6, 4, 3　　B. 2, 3, 4, 5, 6, 1　　C. 3, 4, 5, 6, 1, 2　　D. 6, 5, 4, 3, 2, 1



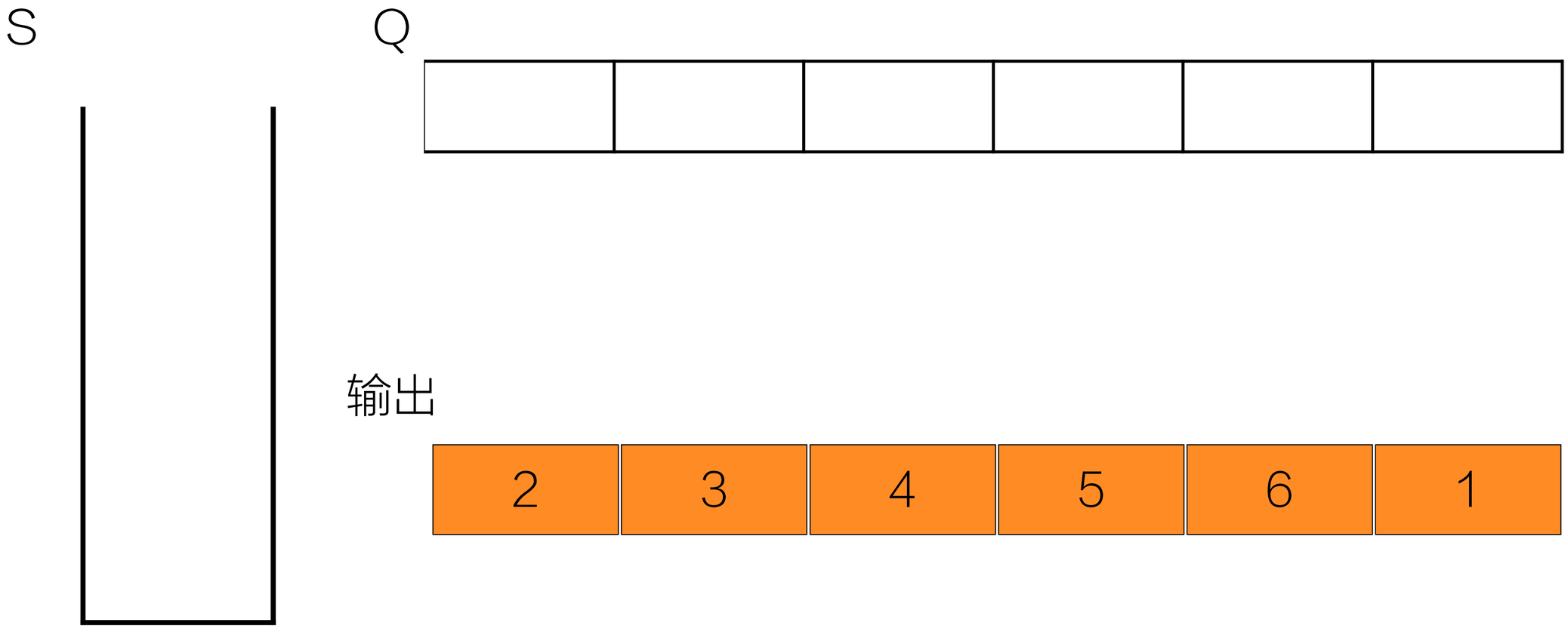
[2018] 2. 现有队列 Q 与栈 S，初始时 Q 中的元素依次是 1, 2, 3, 4, 5, 6（1 在队头），S 为空。若仅允许下列 3 种操作：（1）出队并输出出队元素；（2）出队并将出队元素入栈；（3）出栈并输出出栈元素，则不能得到的输出序列是_____。

A. 1, 2, 5, 6, 4, 3　　B. 2, 3, 4, 5, 6, 1　　C. 3, 4, 5, 6, 1, 2　　D. 6, 5, 4, 3, 2, 1



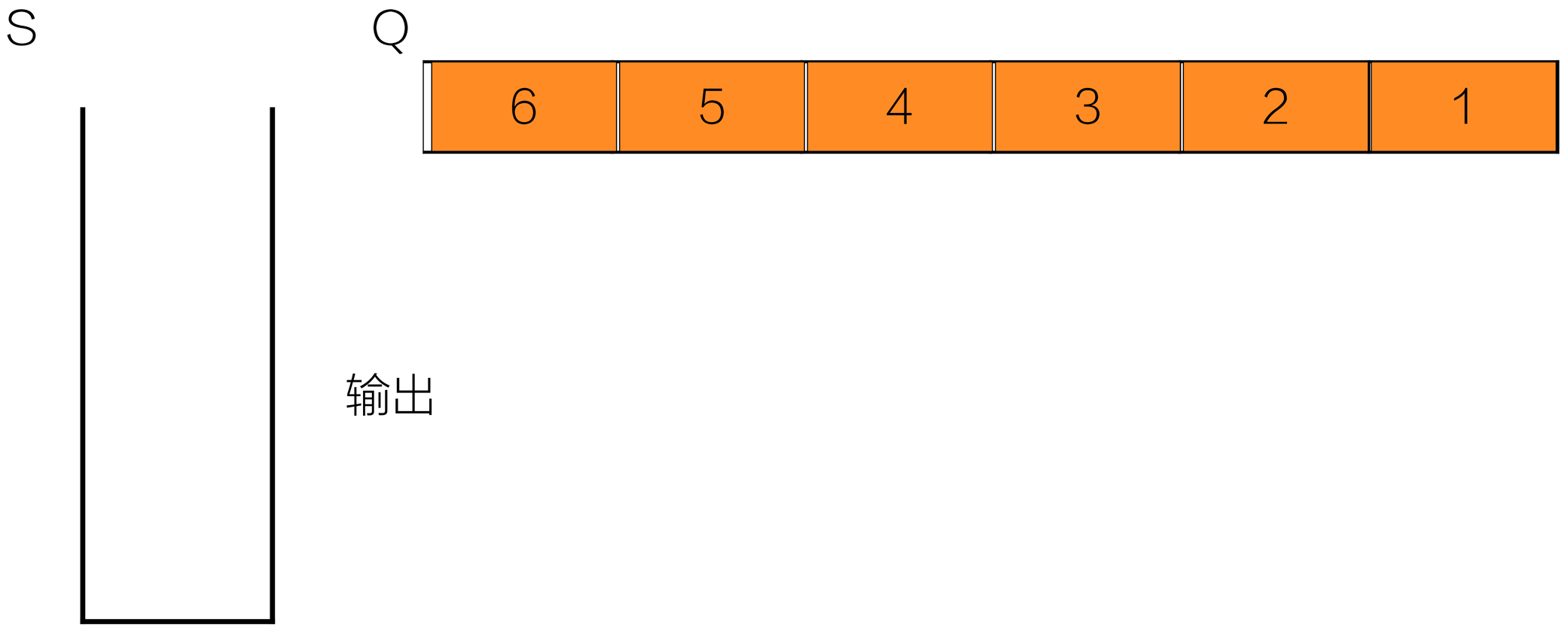
[2018] 2. 现有队列 Q 与栈 S，初始时 Q 中的元素依次是 1, 2, 3, 4, 5, 6（1 在队头），S 为空。若仅允许下列 3 种操作：（1）出队并输出出队元素；（2）出队并将出队元素入栈；（3）出栈并输出出栈元素，则不能得到的输出序列是_____。

A. 1, 2, 5, 6, 4, 3　　B. 2, 3, 4, 5, 6, 1　　C. 3, 4, 5, 6, 1, 2　　D. 6, 5, 4, 3, 2, 1



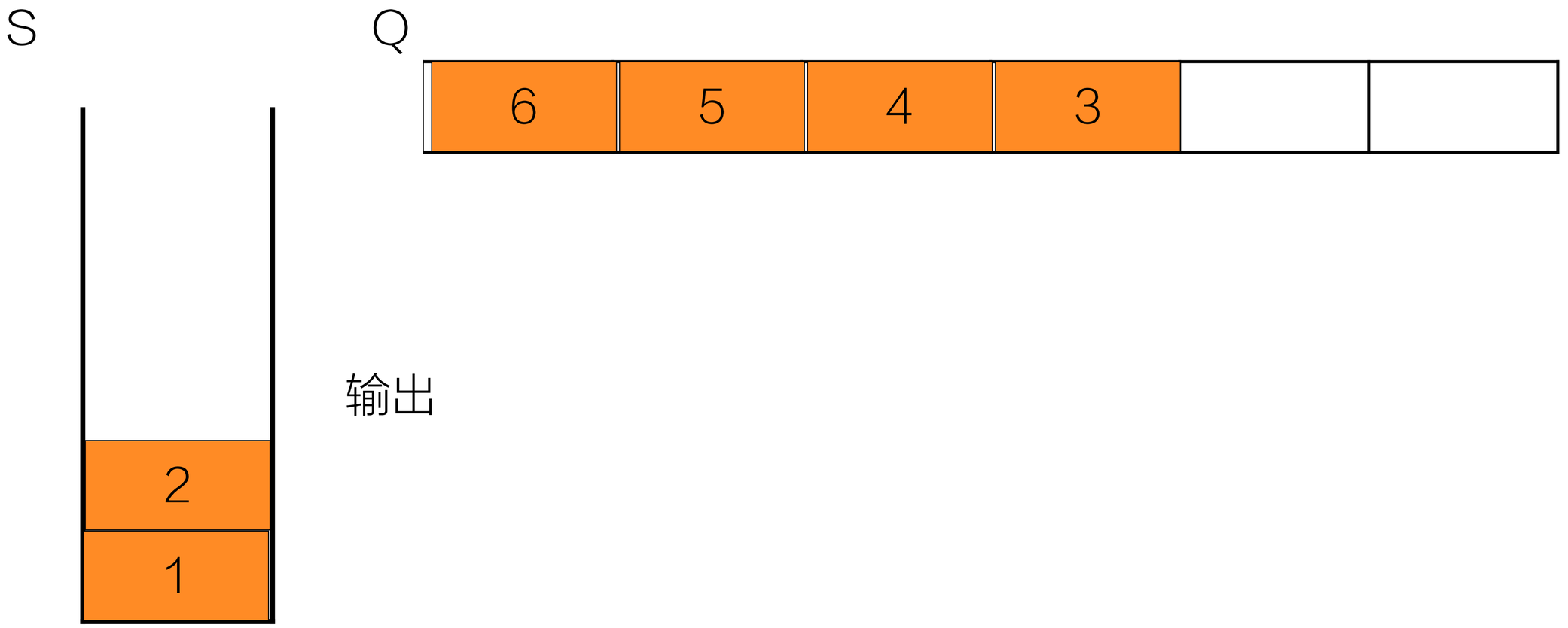
[2018] 2. 现有队列 Q 与栈 S，初始时 Q 中的元素依次是 1, 2, 3, 4, 5, 6（1 在队头），S 为空。若仅允许下列 3 种操作：（1）出队并输出出队元素；（2）出队并将出队元素入栈；（3）出栈并输出出栈元素，则不能得到的输出序列是_____。

A. 1, 2, 5, 6, 4, 3　　B. 2, 3, 4, 5, 6, 1　　C. 3, 4, 5, 6, 1, 2　　D. 6, 5, 4, 3, 2, 1



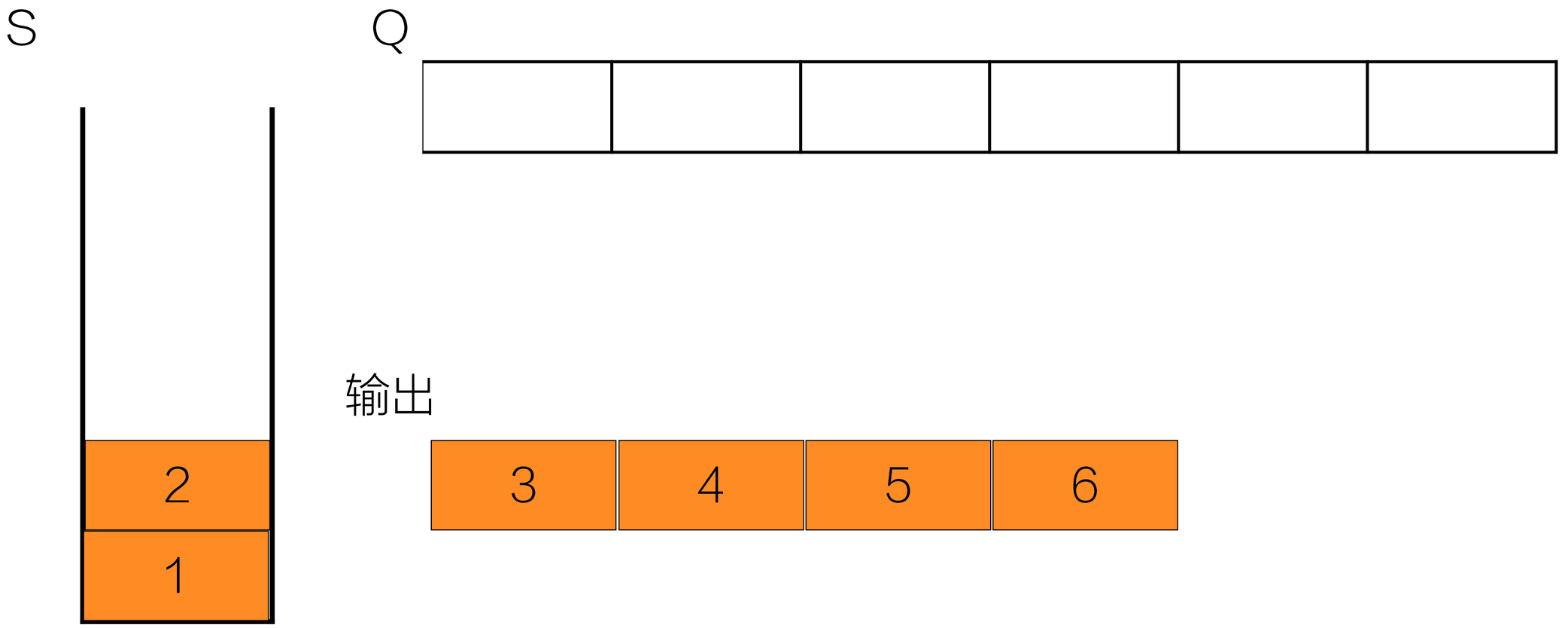
[2018] 2. 现有队列 Q 与栈 S，初始时 Q 中的元素依次是 1, 2, 3, 4, 5, 6（1 在队头），S 为空。若仅允许下列 3 种操作：（1）出队并输出出队元素；（2）出队并将出队元素入栈；（3）出栈并输出出栈元素，则不能得到的输出序列是_____。

A. 1, 2, 5, 6, 4, 3　　B. 2, 3, 4, 5, 6, 1　　C. 3, 4, 5, 6, 1, 2　　D. 6, 5, 4, 3, 2, 1



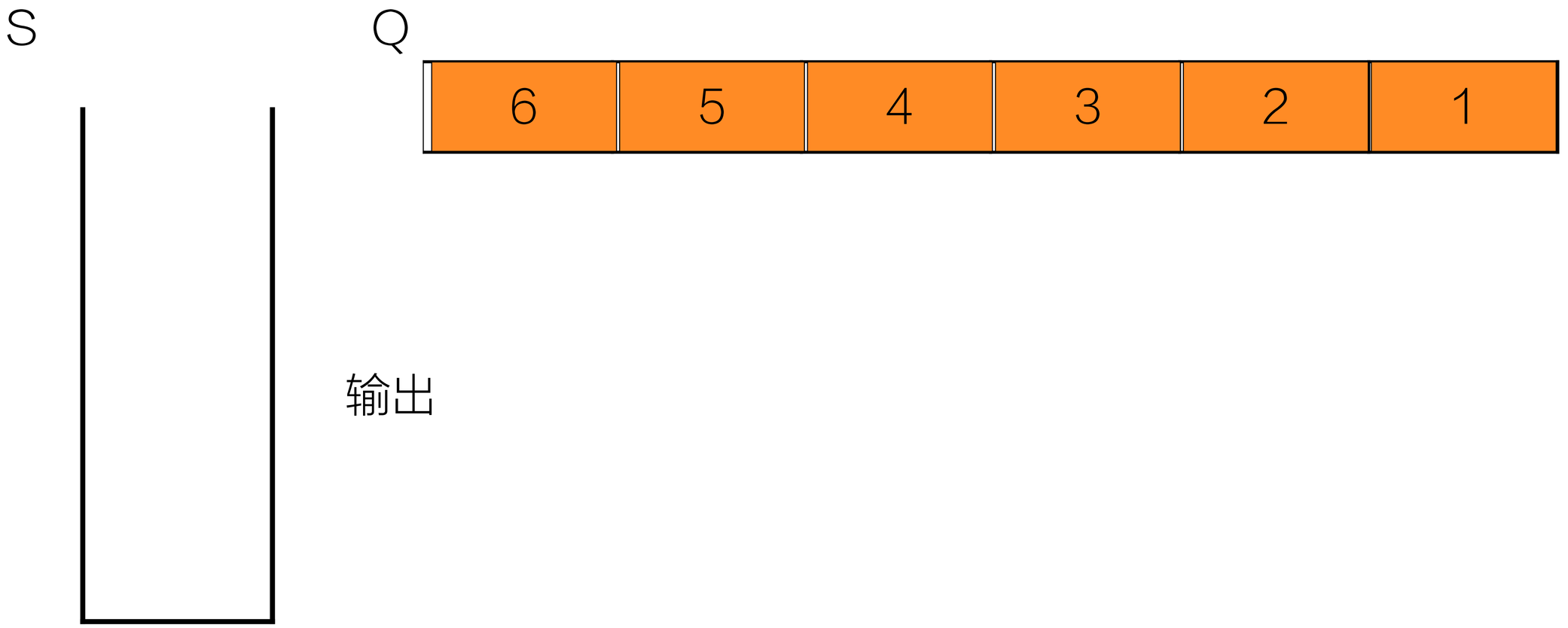
[2018] 2. 现有队列 Q 与栈 S，初始时 Q 中的元素依次是 1, 2, 3, 4, 5, 6（1 在队头），S 为空。若仅允许下列 3 种操作：（1）出队并输出出队元素；（2）出队并将出队元素入栈；（3）出栈并输出出栈元素，则不能得到的输出序列是_____。

A. 1, 2, 5, 6, 4, 3　　B. 2, 3, 4, 5, 6, 1　　C. 3, 4, 5, 6, 1, 2　　D. 6, 5, 4, 3, 2, 1



[2018] 2. 现有队列 Q 与栈 S，初始时 Q 中的元素依次是 1, 2, 3, 4, 5, 6（1 在队头），S 为空。若仅允许下列 3 种操作：（1）出队并输出出队元素；（2）出队并将出队元素入栈；（3）出栈并输出出栈元素，则不能得到的输出序列是_____。

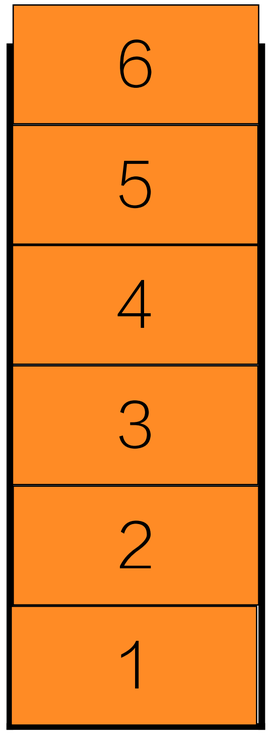
A. 1, 2, 5, 6, 4, 3　　B. 2, 3, 4, 5, 6, 1　　C. 3, 4, 5, 6, 1, 2　　D. 6, 5, 4, 3, 2, 1



[2018] 2. 现有队列 Q 与栈 S，初始时 Q 中的元素依次是 1, 2, 3, 4, 5, 6（1 在队头），S 为空。若仅允许下列 3 种操作：（1）出队并输出出队元素；（2）出队并将出队元素入栈；（3）出栈并输出出栈元素，则不能得到的输出序列是_____。

A. 1, 2, 5, 6, 4, 3　　B. 2, 3, 4, 5, 6, 1　　C. 3, 4, 5, 6, 1, 2　　D. 6, 5, 4, 3, 2, 1

S



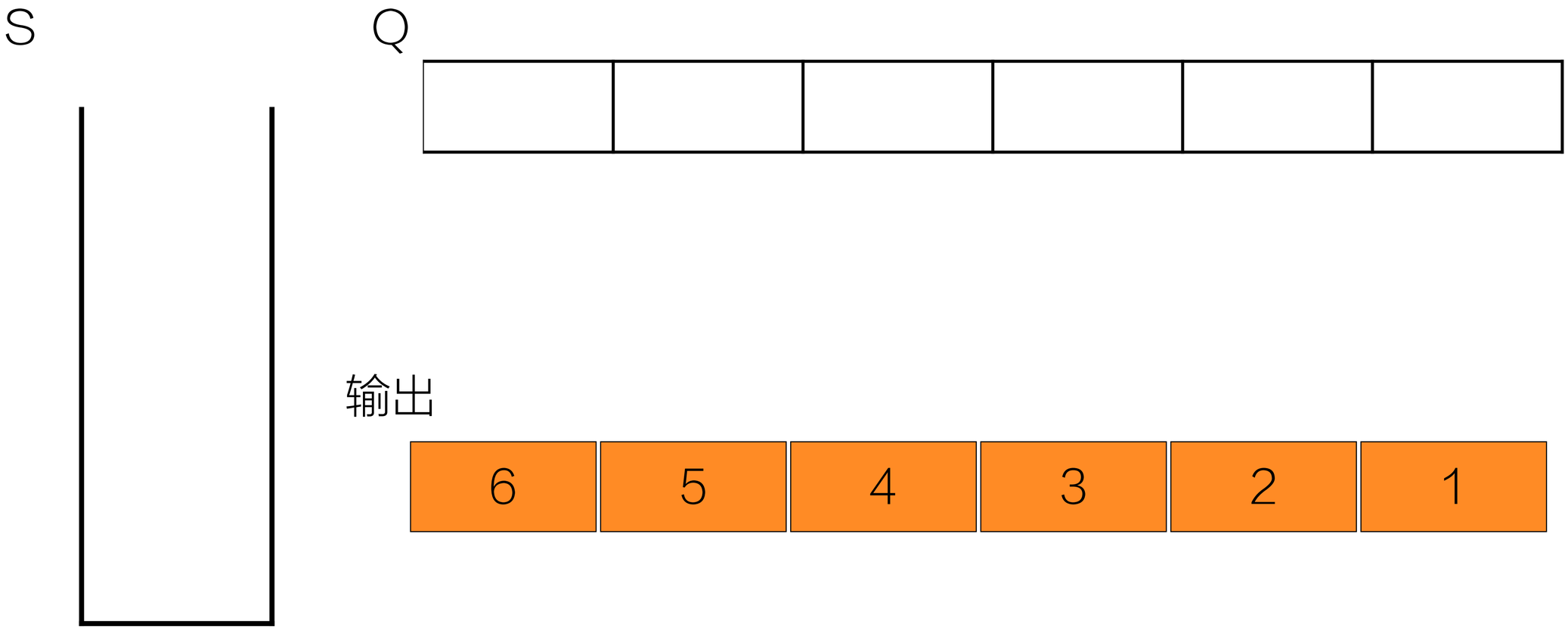
Q



输出

[2018] 2. 现有队列 Q 与栈 S，初始时 Q 中的元素依次是 1, 2, 3, 4, 5, 6（1 在队头），S 为空。若仅允许下列 3 种操作：（1）出队并输出出队元素；（2）出队并将出队元素入栈；（3）出栈并输出出栈元素，则不能得到的输出序列是_____。

A. 1, 2, 5, 6, 4, 3　　B. 2, 3, 4, 5, 6, 1　　C. 3, 4, 5, 6, 1, 2　　D. 6, 5, 4, 3, 2, 1



栈与队列的应用

[2018] 2. 现有队列 Q 与栈 S ， 初始时 Q 中的元素依次是 1, 2, 3, 4, 5, 6（1 在队头），S 为空。若仅允许下列 3 种操作：（1）出队并输出出队元素；（2）出队并将出队元素入栈；（3）出栈并输出出栈元素，则不能得到的输出序列是_____。

A. 1, 2, 5, 6, 4, 3　　B. 2, 3, 4, 5, 6, 1　　C. 3, 4, 5, 6, 1, 2　　D. 6, 5, 4, 3, 2, 1

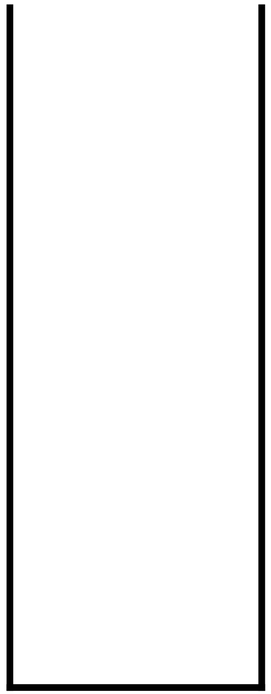
栈与队列的应用

[2020] 2. 现有空栈 S 进行 Push 和 Pop 操作，入栈序列为 a, b, c, d, e，经过 Push, Push, Pop, Push, Pop, Push, Push, Pop 操作后得到的出栈序列是_____。

A. b, a, c　　B. b, a, e　　C. b, c, a　　D. b, c, e

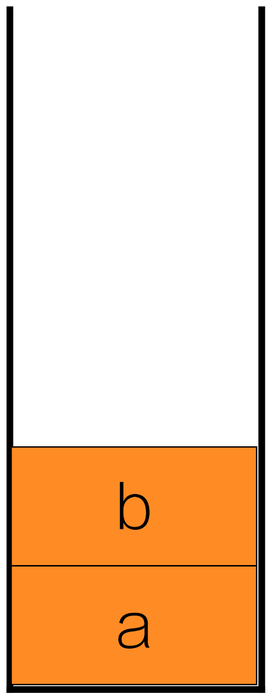
[2020] 2. 现有空栈 S 进行 Push 和 Pop 操作，入栈序列为 a, b, c, d, e，经过 Push, Push, Pop, Push, Pop, Push, Push, Pop 操作后得到的出栈序列是_____。

A. b, a, c　　B. b, a, e　　C. b, c, a　　D. b, c, e



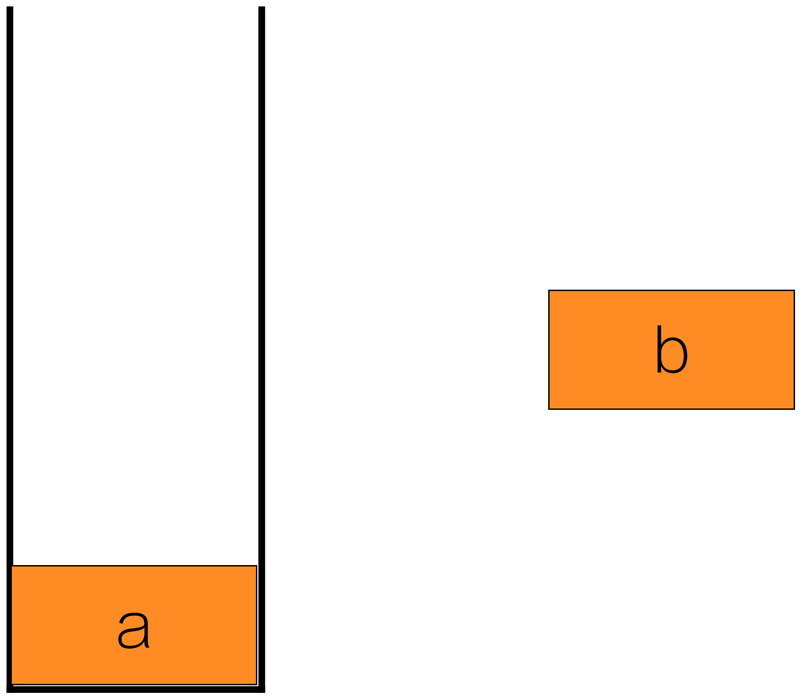
[2020] 2. 现有空栈 S 进行 Push 和 Pop 操作，入栈序列为 a, b, c, d, e，经过 Push, Push, Pop, Push, Pop, Push, Push, Pop 操作后得到的出栈序列是_____。

A. b, a, c　　B. b, a, e　　C. b, c, a　　D. b, c, e



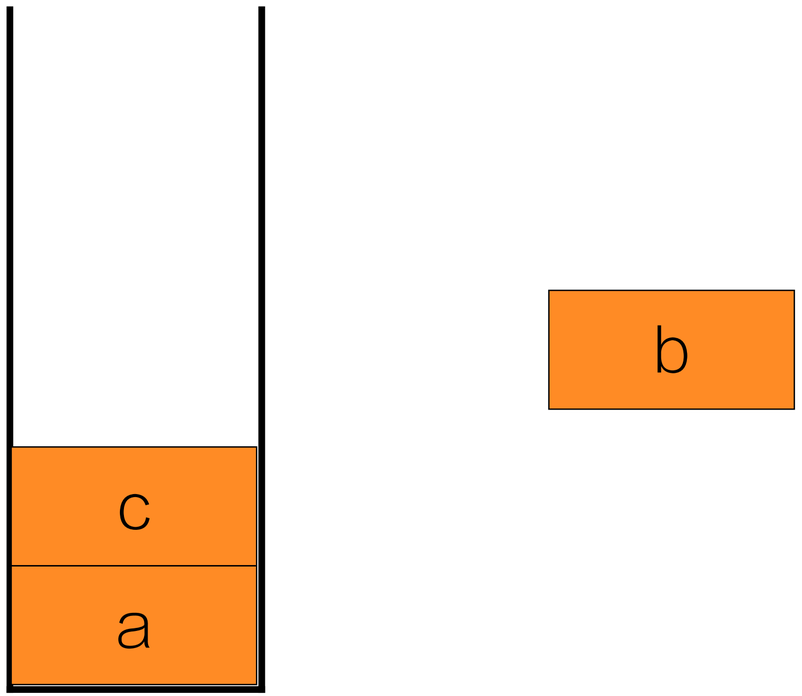
[2020] 2. 现有空栈 S 进行 Push 和 Pop 操作，入栈序列为 a, b, c, d, e，经过 Push, Push, Pop, Push, Pop, Push, Push, Pop 操作后得到的出栈序列是_____。

A. b, a, c　　B. b, a, e　　C. b, c, a　　D. b, c, e



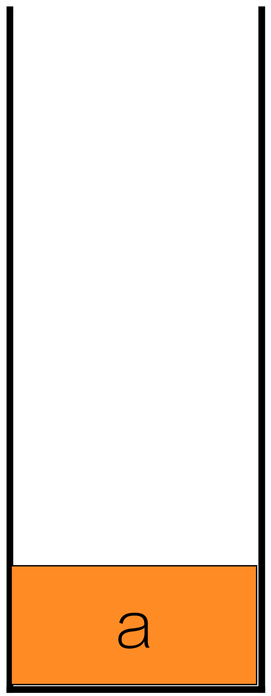
[2020] 2. 现有空栈 S 进行 Push 和 Pop 操作，入栈序列为 a, b, c, d, e，经过 Push, Push, Pop, Push, Pop, Push, Push, Pop 操作后得到的出栈序列是_____。

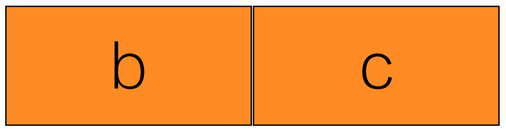
A. b, a, c　　B. b, a, e　　C. b, c, a　　D. b, c, e



[2020] 2. 现有空栈 S 进行 Push 和 Pop 操作，入栈序列为 a, b, c, d, e，经过 Push, Push, Pop, Push, Pop, Push, Push, Pop 操作后得到的出栈序列是_____。

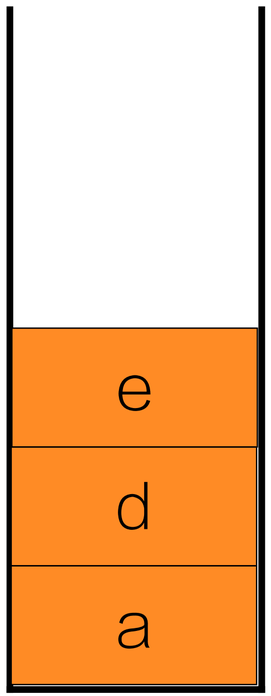
A. b, a, c　　B. b, a, e　　C. b, c, a　　D. b, c, e

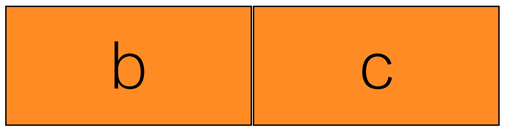




[2020] 2. 现有空栈 S 进行 Push 和 Pop 操作，入栈序列为 a, b, c, d, e，经过 Push, Push, Pop, Push, Pop, Push, Push, Pop 操作后得到的出栈序列是_____。

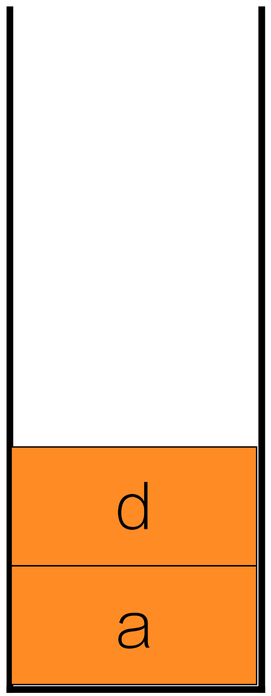
A. b, a, c　　B. b, a, e　　C. b, c, a　　D. b, c, e

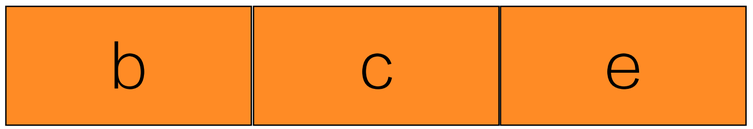




[2020] 2. 现有空栈 S 进行 Push 和 Pop 操作，入栈序列为 a, b, c, d, e，经过 Push, Push, Pop, Push, Pop, Push, Push, Pop 操作后得到的出栈序列是_____。

A. b, a, c　　B. b, a, e　　C. b, c, a　　D. b, c, e





栈与队列的应用

[2020] 2. 现有空栈 S 进行 Push 和 Pop 操作，入栈序列为 a, b, c, d, e，经过 Push, Push, Pop, Push, Pop, Push, Push, Pop 操作后得到的出栈序列是_____。

A. b, a, c　　B. b, a, e　　C. b, c, a　　D. b, c, e

栈与队列的应用

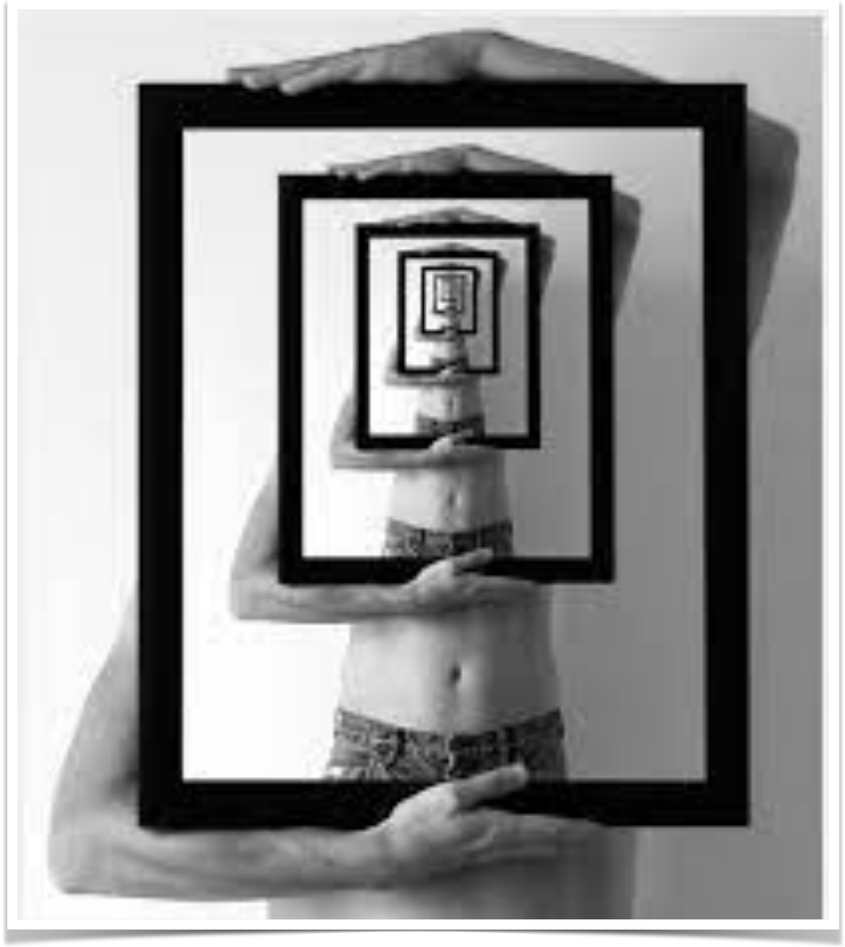
[2021] 2. 已知初始为空的队列 Q 的一端仅能进行入队操作，另外一端既能进行入队操作又能进行出队操作。若 Q 的入队序列是 1, 2, 3, 4, 5，则不能得到的出队序列是_____。

A. 5, 4, 3, 1, 2　　B. 5, 3, 1, 2, 4　　C. 4, 2, 1, 3, 5　　D. 4, 1, 3, 2, 5

递归

在函数调用过程中，调用自己本身





计算机 1～n 的和 – 非递归方式

```
int fun(int n)
{
    int sum = 0;
    for (int i = 1; i <= n; i++)
    {
        sum += i;
    }
    return sum;
}
```

案例：栈与队列 / 递归 / test1.c

计算机 1～n 的和 – 递归方式

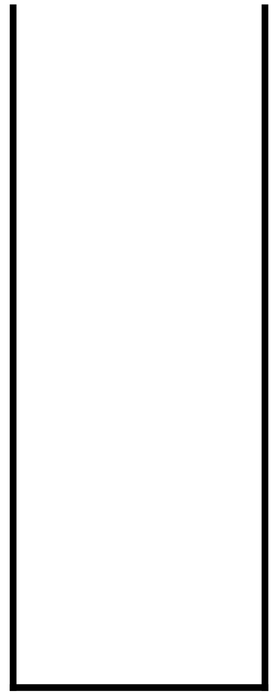
```
int fun(int n)
{
    if (n == 1)
    {
        return 1;
    }
    else
    {
        return fun(n-1) + n;
    }
}
```

案例：栈与队列 / 递归 / test2.c

计算机 1～n 的和 – 递归方式

```
int fun(int n)
{
    if (n == 1)
    {
        return 1;
    }
    else
    {
        return fun(n-1) + n;
    }
}
```

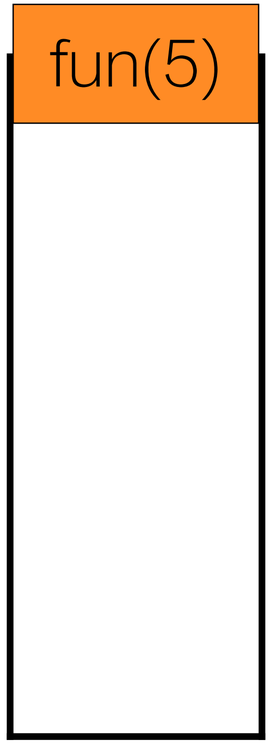
案例：栈与队列 / 递归 / test2.c



计算机 1～n 的和 – 递归方式

```
int fun(int n)
{
    if (n == 1)
    {
        return 1;
    }
    else
    {
        return fun(n-1) + n;
    }
}
```

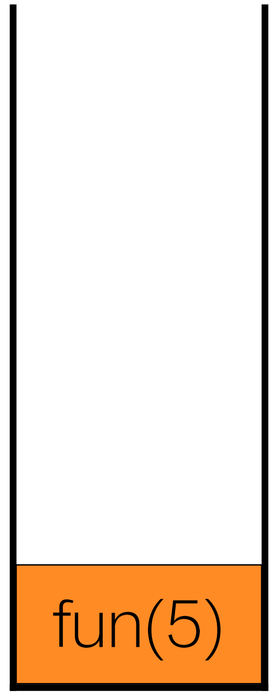
案例：栈与队列 / 递归 / test2.c



计算机 1～n 的和 – 递归方式

```
int fun(int n)
{
    if (n == 1)
    {
        return 1;
    }
    else
    {
        return fun(n-1) + n;
    }
}
```

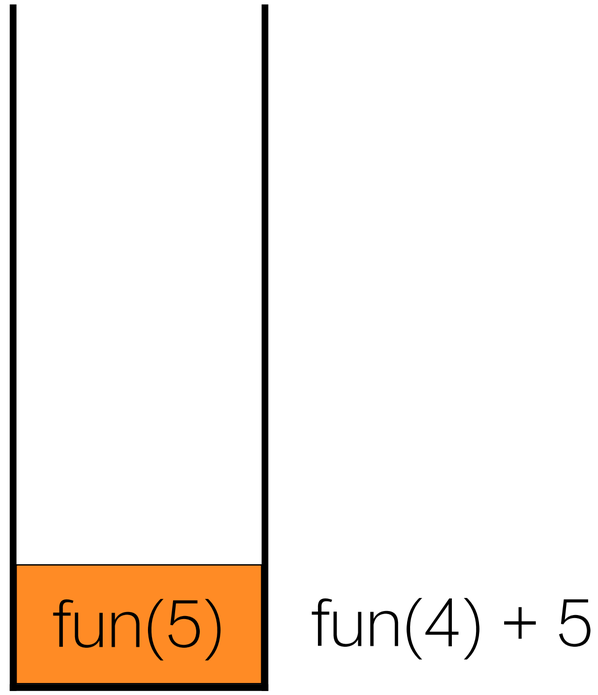
案例：栈与队列 / 递归 / test2.c



计算机 1～n 的和 – 递归方式

```
int fun(int n)
{
    if (n == 1)
    {
        return 1;
    }
    else
    {
        return fun(n-1) + n;
    }
}
```

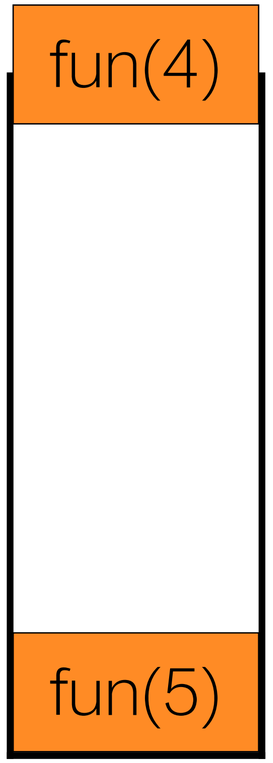
案例：栈与队列 / 递归 / test2.c



计算机 1～n 的和 – 递归方式

```
int fun(int n)
{
    if (n == 1)
    {
        return 1;
    }
    else
    {
        return fun(n-1) + n;
    }
}
```

案例：栈与队列 / 递归 / test2.c

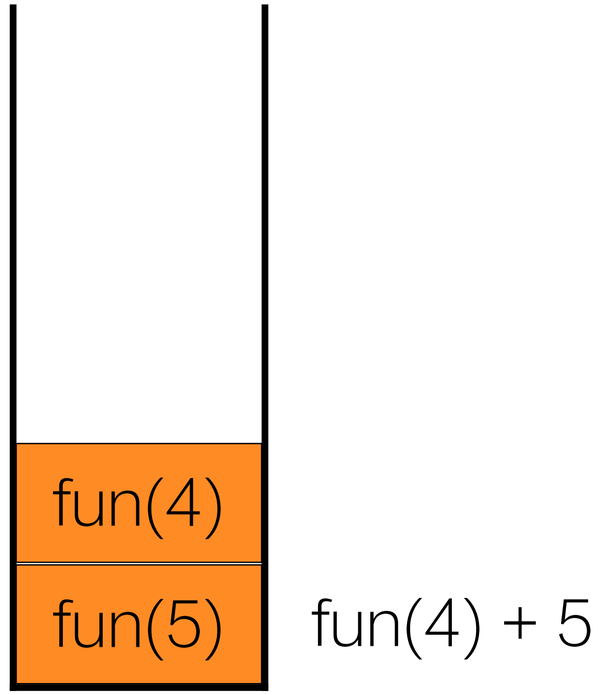


fun(4) + 5

计算机 1～n 的和 – 递归方式

```
int fun(int n)
{
    if (n == 1)
    {
        return 1;
    }
    else
    {
        return fun(n-1) + n;
    }
}
```

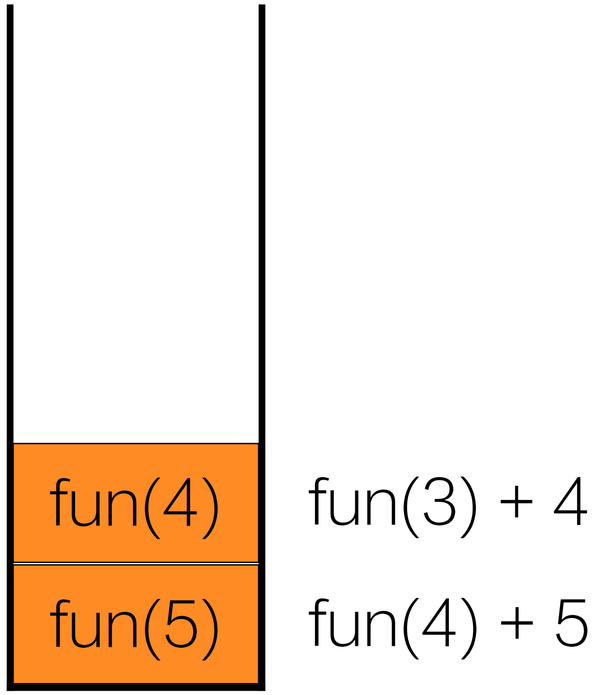
案例：栈与队列 / 递归 / test2.c



计算机 1～n 的和 – 递归方式

```
int fun(int n)
{
    if (n == 1)
    {
        return 1;
    }
    else
    {
        return fun(n-1) + n;
    }
}
```

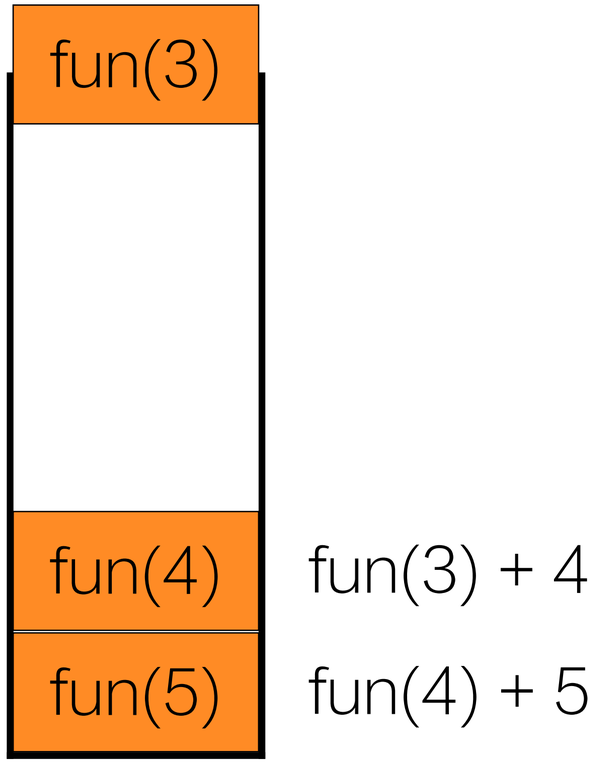
案例：栈与队列 / 递归 / test2.c



计算机 1～n 的和 – 递归方式

```
int fun(int n)
{
    if (n == 1)
    {
        return 1;
    }
    else
    {
        return fun(n-1) + n;
    }
}
```

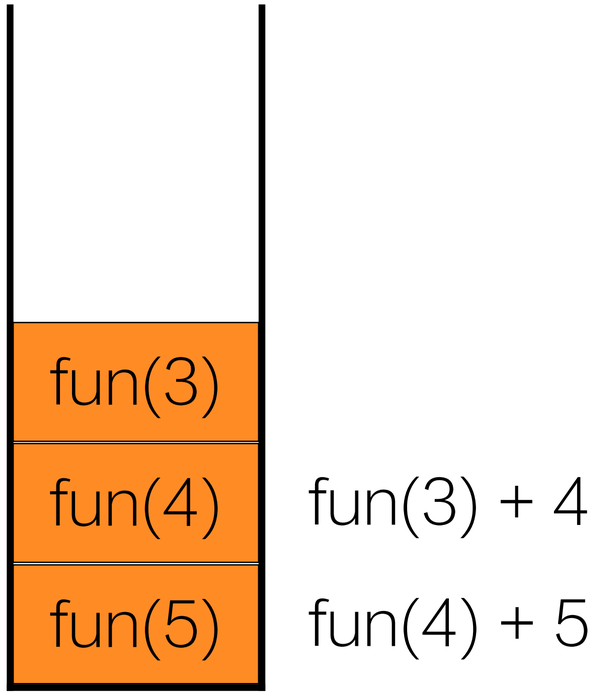
案例：栈与队列 / 递归 / test2.c



计算机 1～n 的和 – 递归方式

```
int fun(int n)
{
    if (n == 1)
    {
        return 1;
    }
    else
    {
        return fun(n-1) + n;
    }
}
```

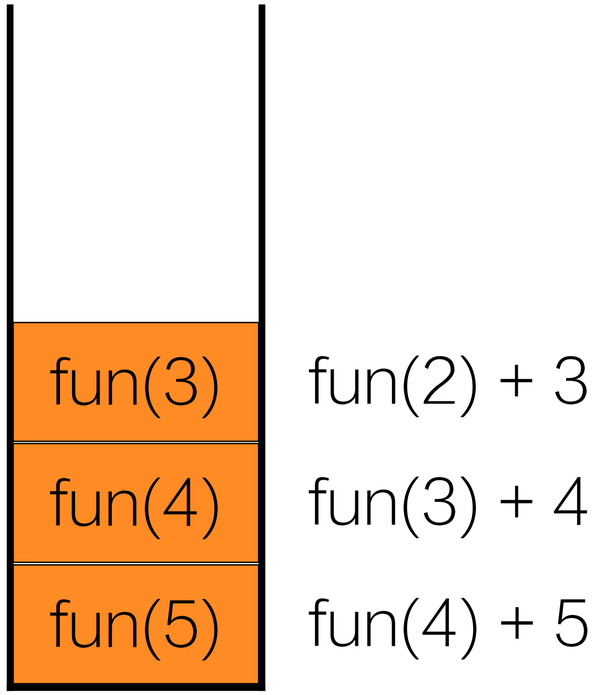
案例：栈与队列 / 递归 / test2.c



计算机 1～n 的和 – 递归方式

```
int fun(int n)
{
    if (n == 1)
    {
        return 1;
    }
    else
    {
        return fun(n-1) + n;
    }
}
```

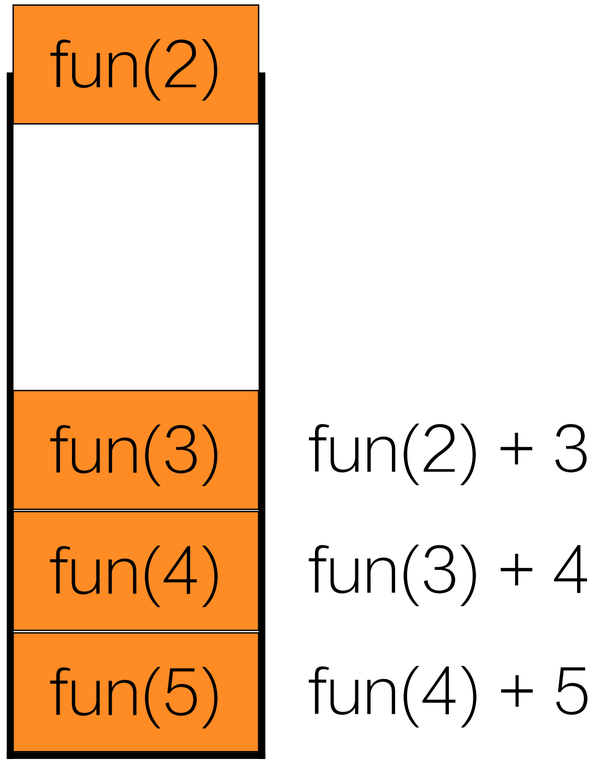
案例：栈与队列 / 递归 / test2.c



计算机 1～n 的和 – 递归方式

```
int fun(int n)
{
    if (n == 1)
    {
        return 1;
    }
    else
    {
        return fun(n-1) + n;
    }
}
```

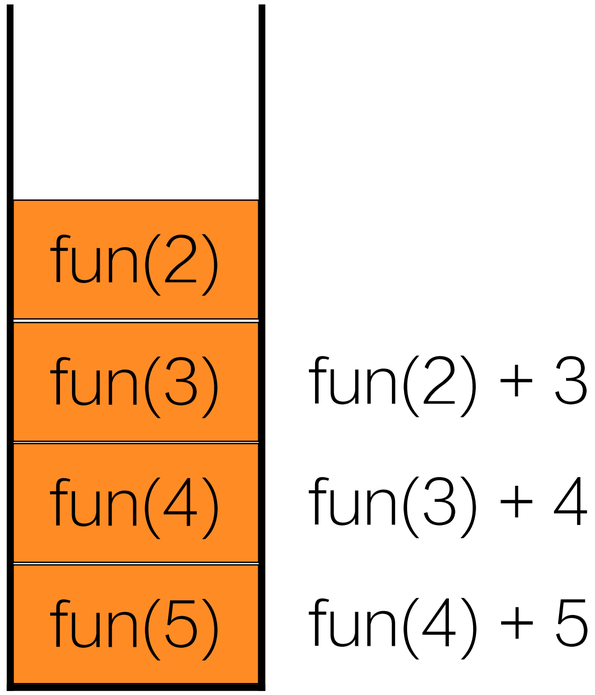
案例：栈与队列 / 递归 / test2.c



计算机 1～n 的和 – 递归方式

```
int fun(int n)
{
    if (n == 1)
    {
        return 1;
    }
    else
    {
        return fun(n-1) + n;
    }
}
```

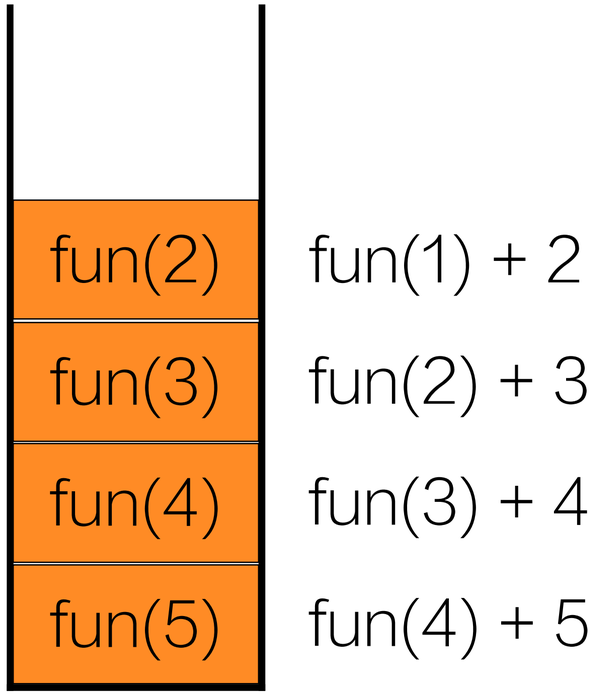
案例：栈与队列 / 递归 / test2.c



计算机 1～n 的和 – 递归方式

```
int fun(int n)
{
    if (n == 1)
    {
        return 1;
    }
    else
    {
        return fun(n-1) + n;
    }
}
```

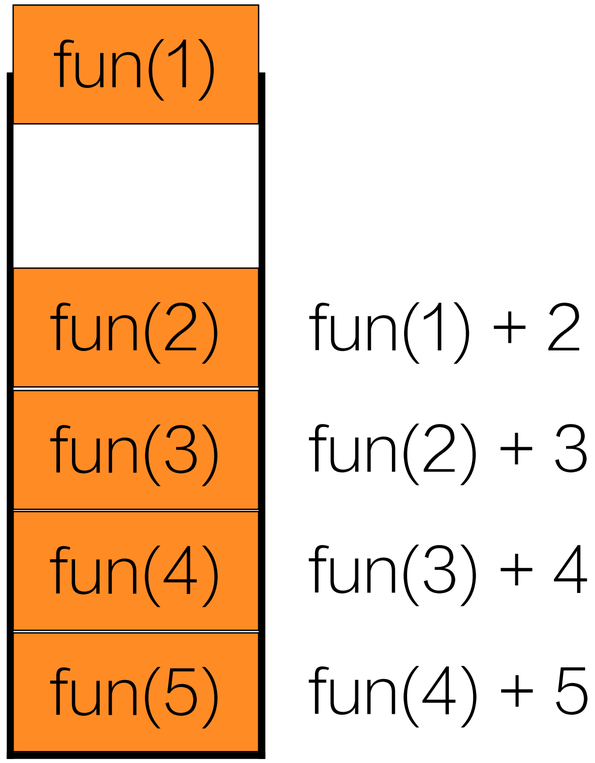
案例：栈与队列 / 递归 / test2.c



计算机 1～n 的和 – 递归方式

```
int fun(int n)
{
    if (n == 1)
    {
        return 1;
    }
    else
    {
        return fun(n-1) + n;
    }
}
```

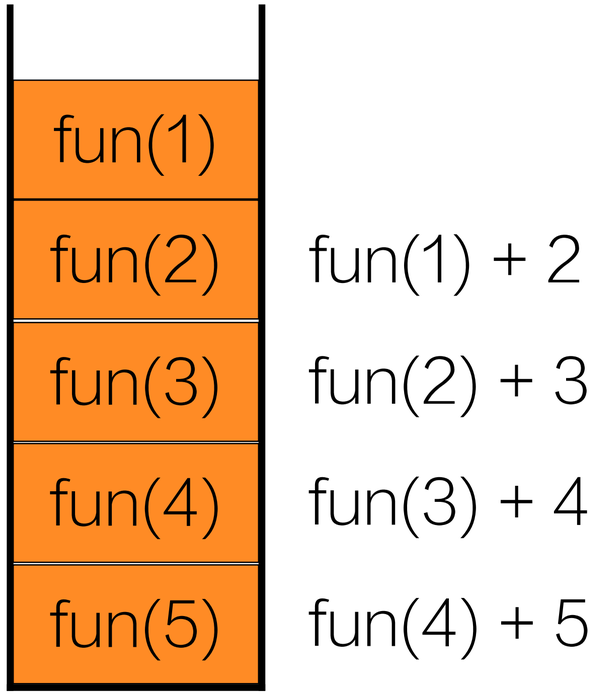
案例：栈与队列 / 递归 / test2.c



计算机 1～n 的和 – 递归方式

```
int fun(int n)
{
    if (n == 1)
    {
        return 1;
    }
    else
    {
        return fun(n-1) + n;
    }
}
```

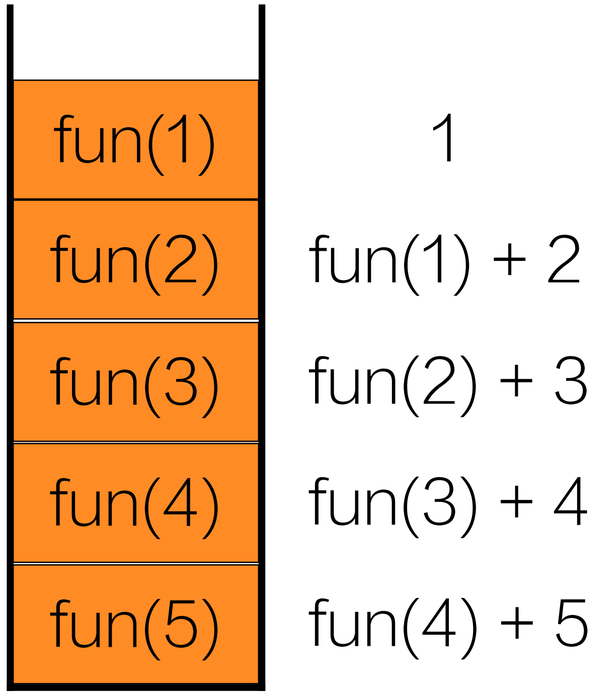
案例：栈与队列 / 递归 / test2.c



计算机 1～n 的和 – 递归方式

```
int fun(int n)
{
    if (n == 1)
    {
        return 1;
    }
    else
    {
        return fun(n-1) + n;
    }
}
```

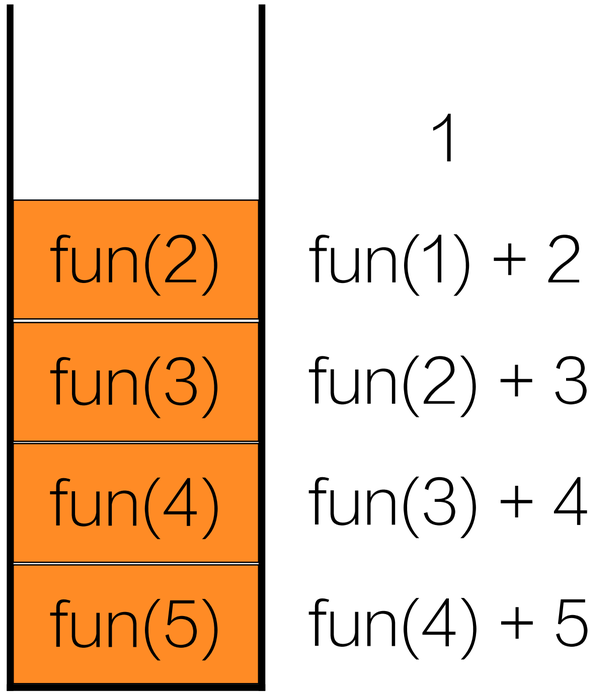
案例：栈与队列 / 递归 / test2.c



计算机 1～n 的和 – 递归方式

```
int fun(int n)
{
    if (n == 1)
    {
        return 1;
    }
    else
    {
        return fun(n-1) + n;
    }
}
```

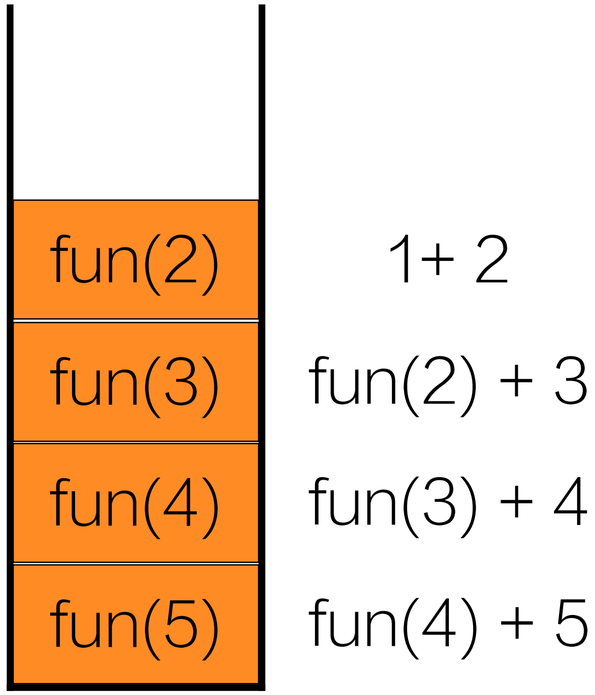
案例：栈与队列 / 递归 / test2.c



计算机 1～n 的和 – 递归方式

```
int fun(int n)
{
    if (n == 1)
    {
        return 1;
    }
    else
    {
        return fun(n-1) + n;
    }
}
```

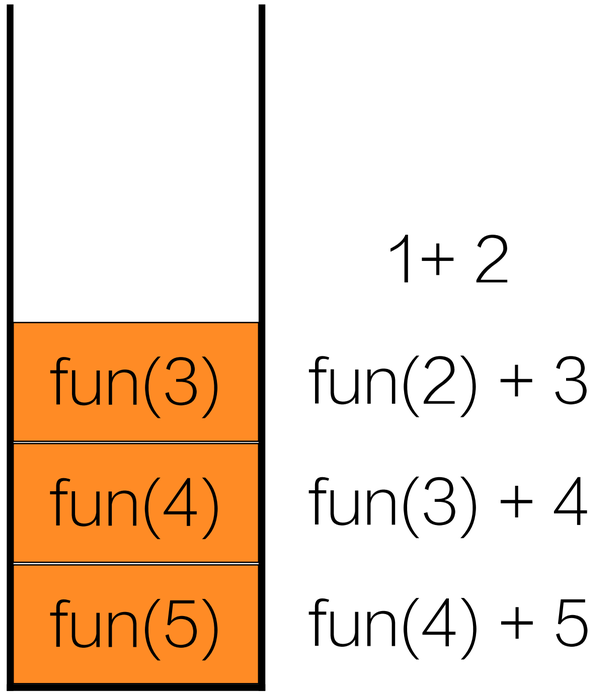
案例：栈与队列 / 递归 / test2.c



计算机 1～n 的和 – 递归方式

```
int fun(int n)
{
    if (n == 1)
    {
        return 1;
    }
    else
    {
        return fun(n-1) + n;
    }
}
```

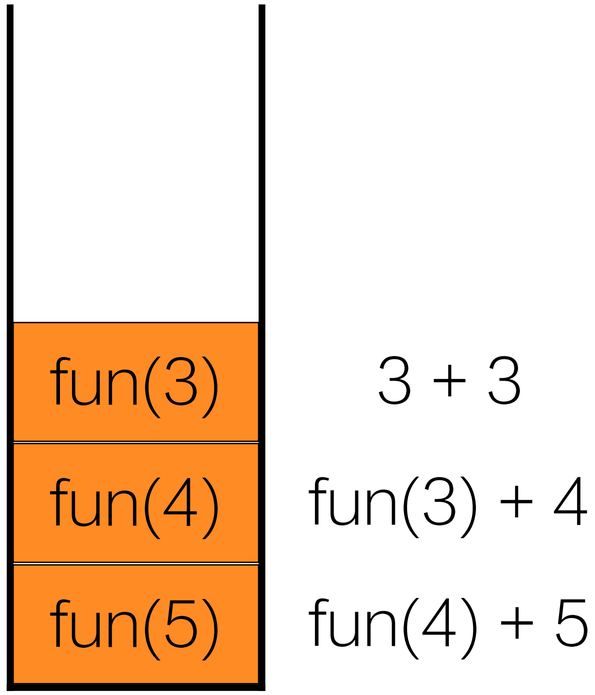
案例：栈与队列 / 递归 / test2.c



计算机 1～n 的和 – 递归方式

```
int fun(int n)
{
    if (n == 1)
    {
        return 1;
    }
    else
    {
        return fun(n-1) + n;
    }
}
```

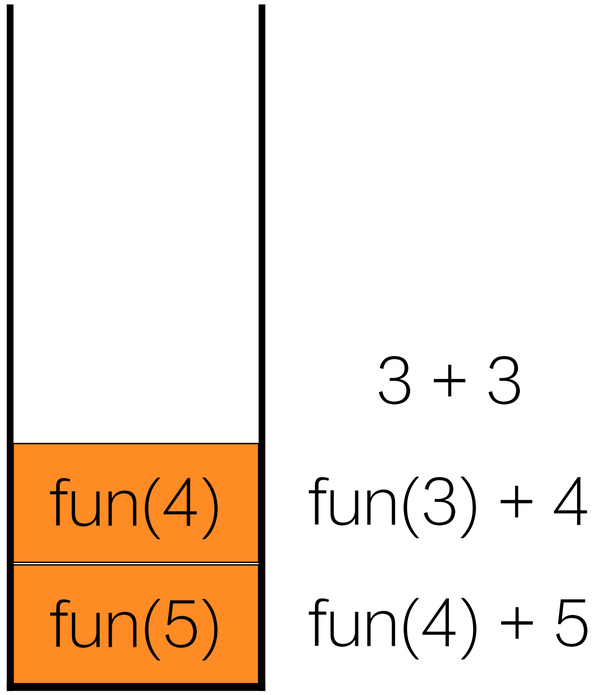
案例：栈与队列 / 递归 / test2.c



计算机 1～n 的和 – 递归方式

```
int fun(int n)
{
    if (n == 1)
    {
        return 1;
    }
    else
    {
        return fun(n-1) + n;
    }
}
```

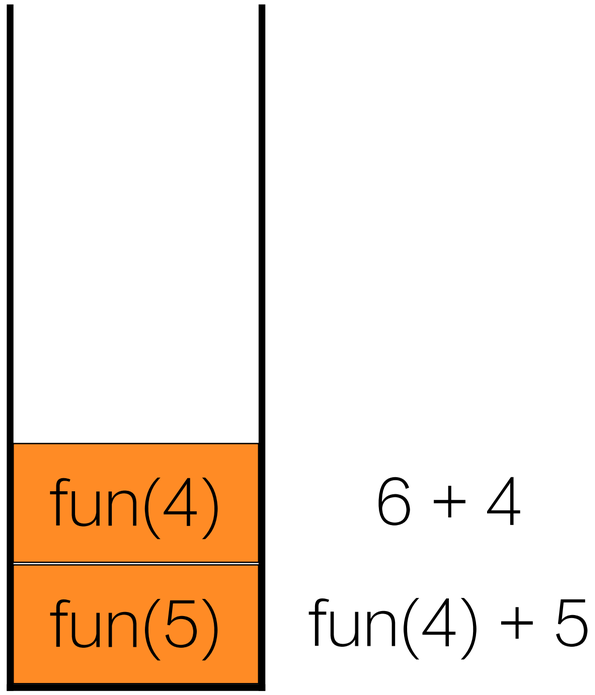
案例：栈与队列 / 递归 / test2.c



计算机 1～n 的和 – 递归方式

```
int fun(int n)
{
    if (n == 1)
    {
        return 1;
    }
    else
    {
        return fun(n-1) + n;
    }
}
```

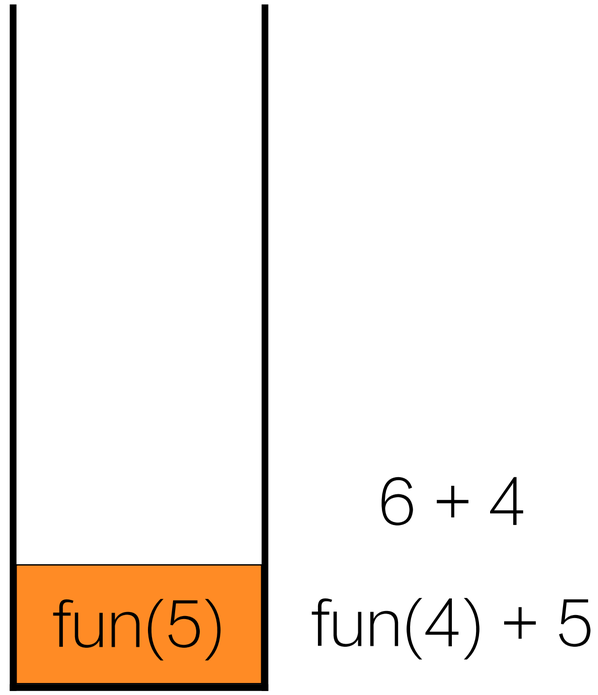
案例：栈与队列 / 递归 / test2.c



计算机 1～n 的和 – 递归方式

```
int fun(int n)
{
    if (n == 1)
    {
        return 1;
    }
    else
    {
        return fun(n-1) + n;
    }
}
```

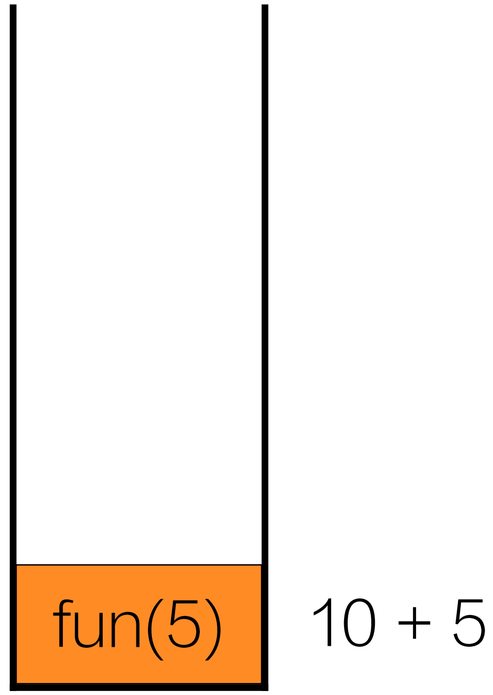
案例：栈与队列 / 递归 / test2.c



计算机 1～n 的和 – 递归方式

```
int fun(int n)
{
    if (n == 1)
    {
        return 1;
    }
    else
    {
        return fun(n-1) + n;
    }
}
```

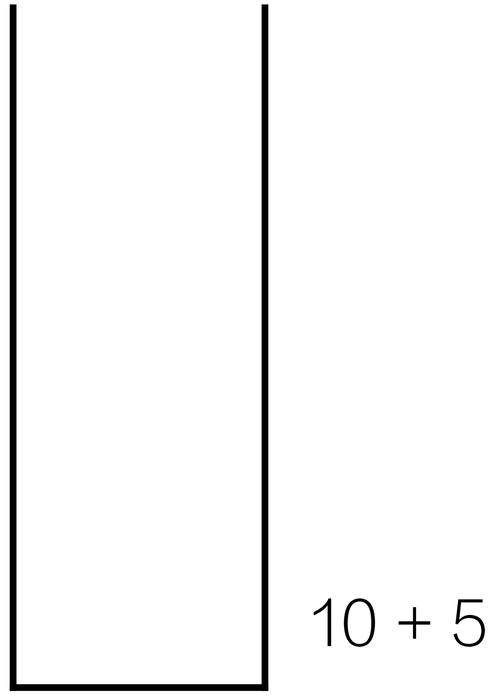
案例：栈与队列 / 递归 / test2.c



计算机 1～n 的和 – 递归方式

```
int fun(int n)
{
    if (n == 1)
    {
        return 1;
    }
    else
    {
        return fun(n-1) + n;
    }
}
```

案例：栈与队列 / 递归 / test2.c



斐波那契数列

斐波那契数列（Fibonacci sequence），又称黄金分割数列，因数学家莱昂纳多·斐波那契（Leonardo Fibonacci）以兔子繁殖为例子而引入，故又称“兔子数列”，其数值为：1、1、2、3、5、8、13、21、34……

第 1、2 项为 1，从第 3 项开始每一项的值为前两项的和

求斐波那契数列第n项值 – 非递归方式

```
int fibonacci(int n)
{
    int last1 = 1;
    int last2 = 1;
    int result = 0;
    for (int i = 3; i <= n; i++)
    {
        result = last1 + last2;
        last2 = last1;
        last1 = result;
    }
    return result;
}
```

案例：栈与队列 / 递归 / test3.c

求斐波那契数列第n项值 – 递归方式

案例：栈与队列 / 递归 / test4.c

```
int fibonacci(int n)
{
    if (n == 1 || n == 2)
    {
        return 1;
    }
    else
    {
        return fibonacci(n - 1) + fibonacci(n - 2);
    }
}
```

表达式求值

中缀表达式	后缀表达式
5+6*3	563*+
b*c/d	bc*d/
(5+6)*7	56+7*
x/y-z+i*j-x*y	xy/z-ij*+xy*-
x*y-6	xy*6-

枚举

将变量的值一一列举出来，变量的值只限于列举出来的值的范围内

在枚举中列出的每一个值，称为枚举元素

每一个枚举元素由系统定义了一个用序号表示的数值，从 0 开始，分别为 0，1，2 …

语法：

```
enum 枚举名
{
    枚举元素……
}
```

案例：栈与队列 / 表达式求值 / test1.c

案例：栈与队列 / 表达式求值 / test2.c

后缀表达式求值

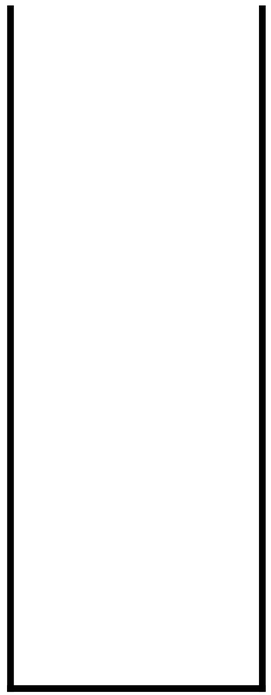
82/2+56*-

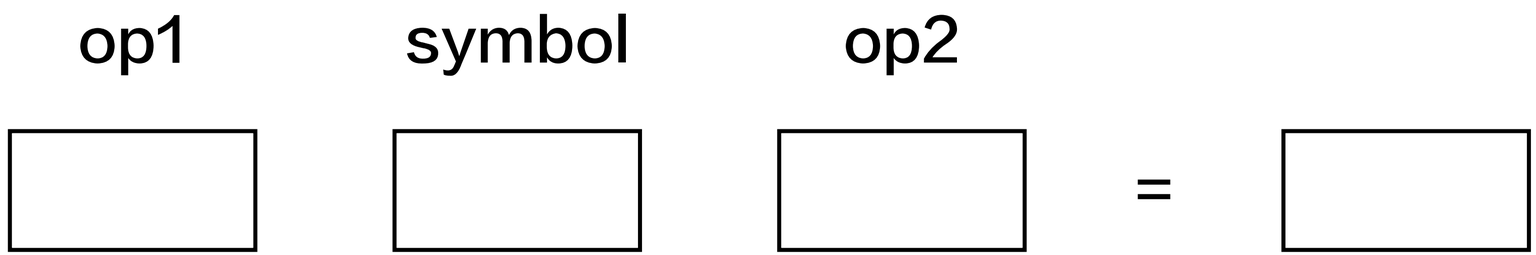
后缀表达式求值

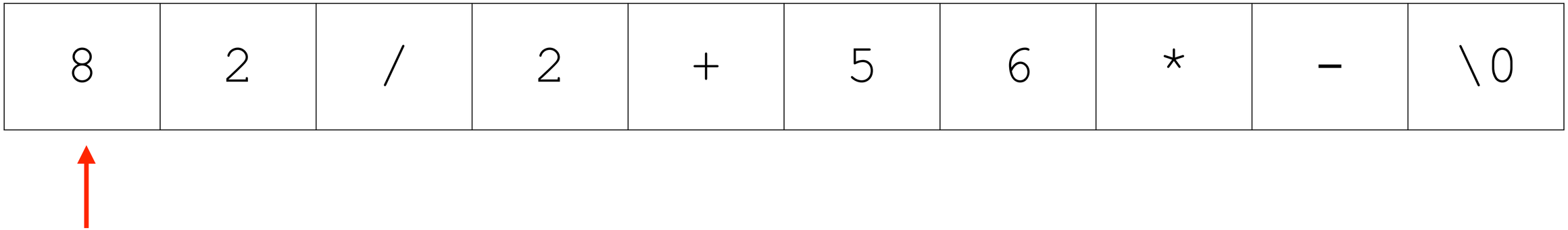
82/2+56*-

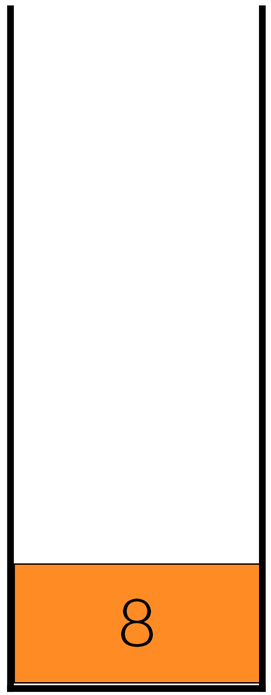
8	2	/	2	+	5	6	*	-	\0

8	2	/	2	+	5	6	*	-	\0

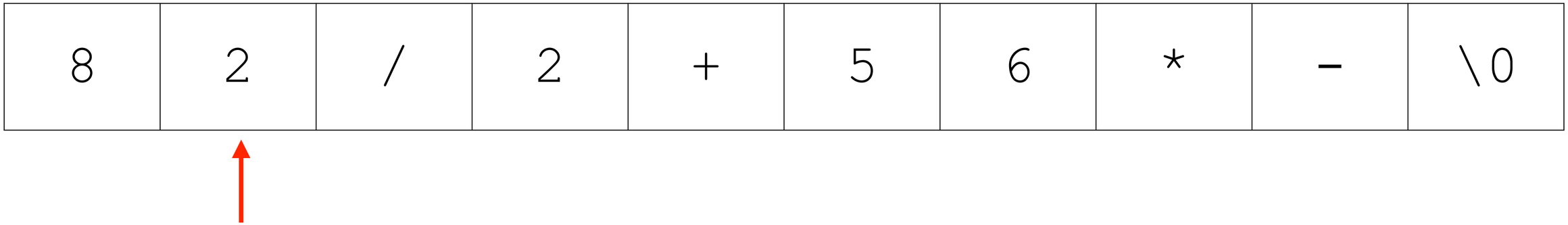


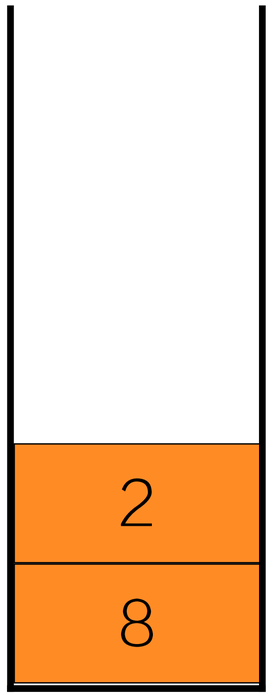






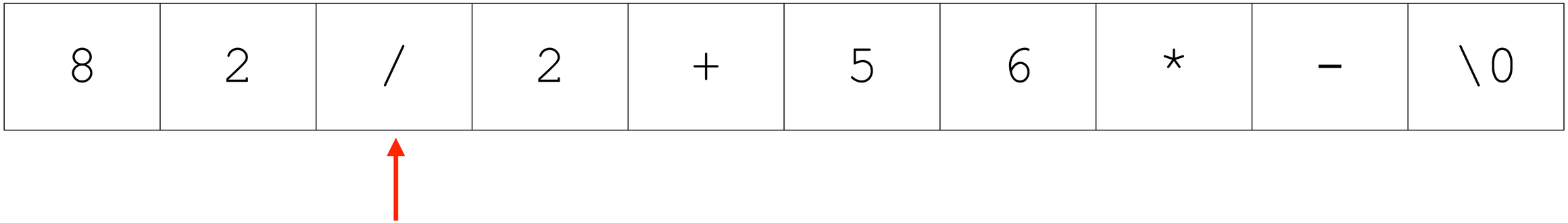
op1 symbol op2 =

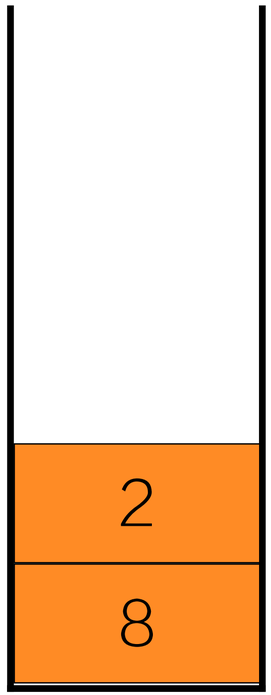




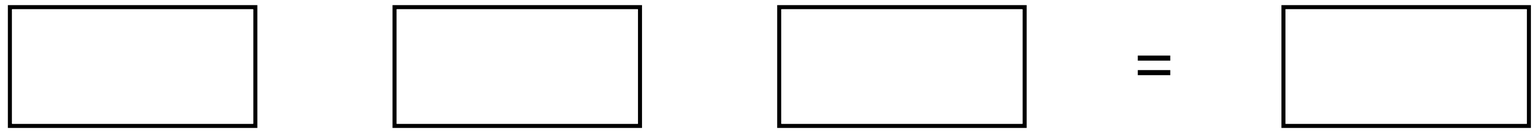
op1 symbol op2

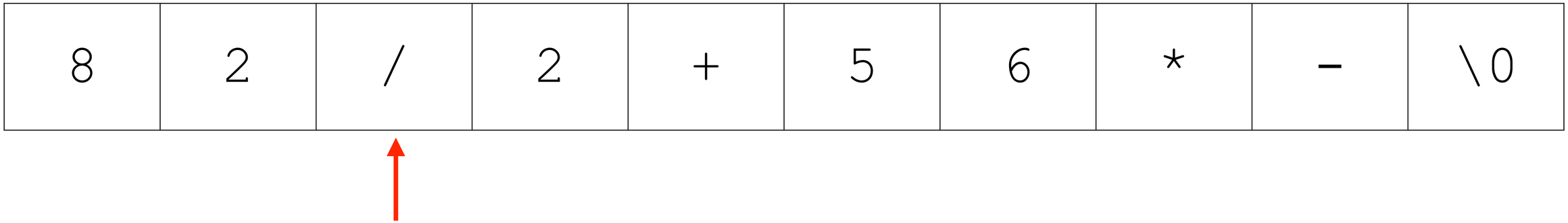
=

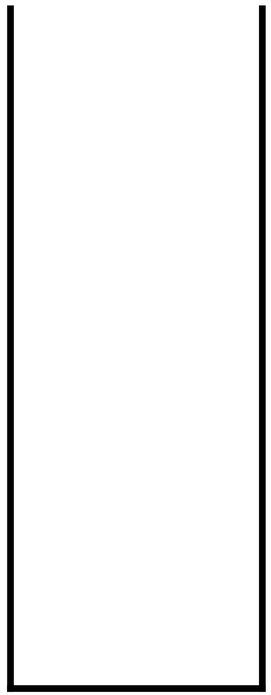


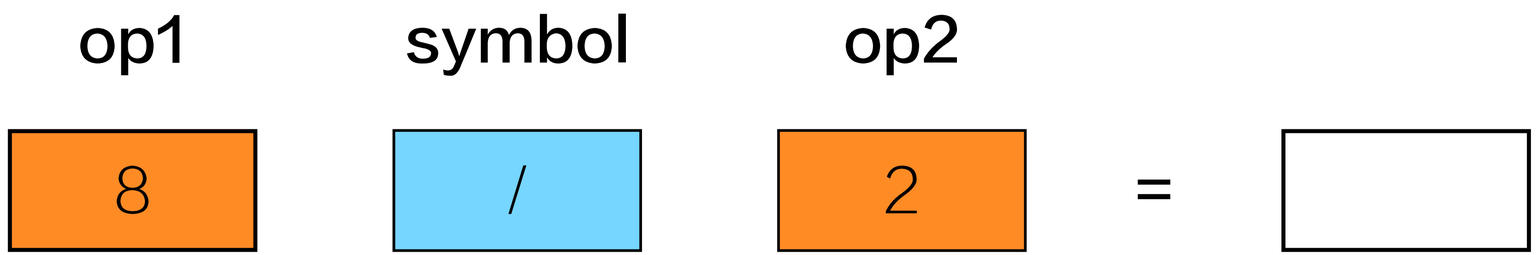


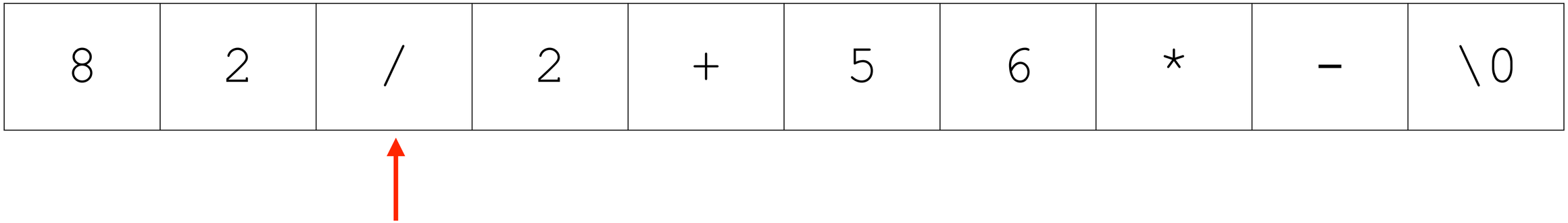
op1 symbol op2

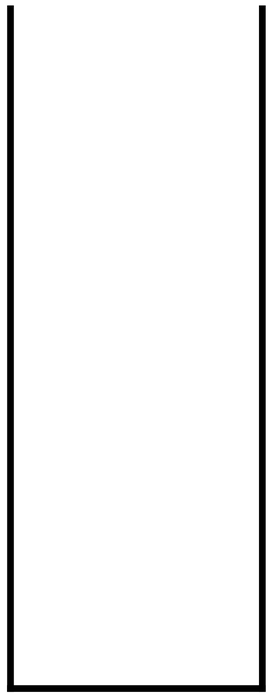


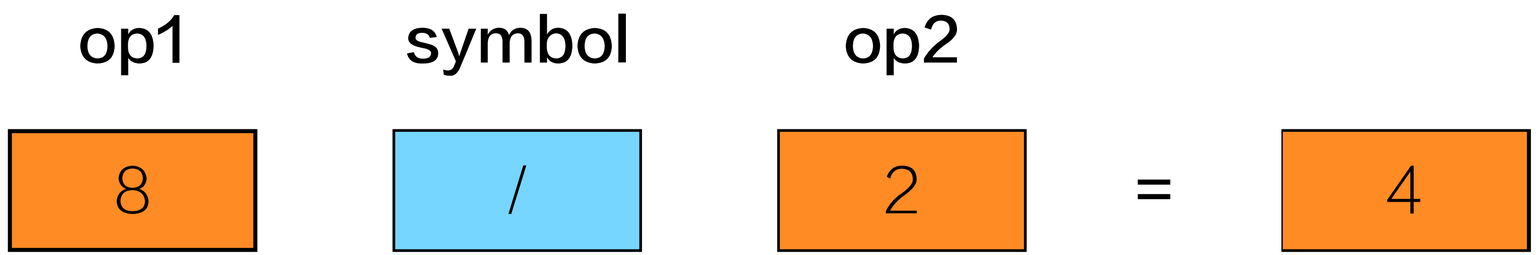

8
2
/
2
+
5
6
*
-
\0

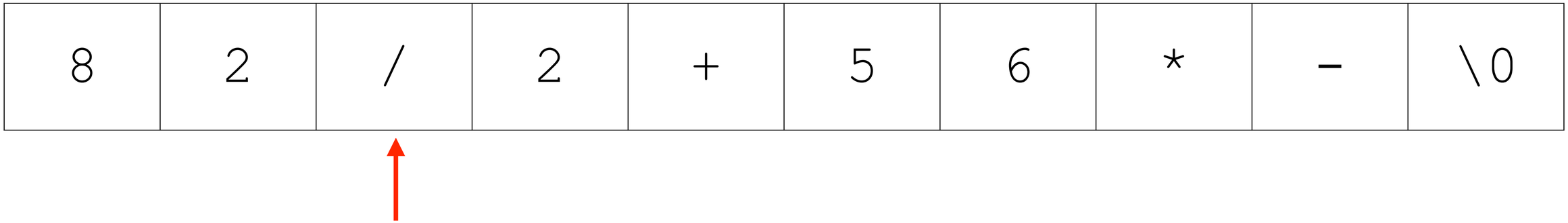


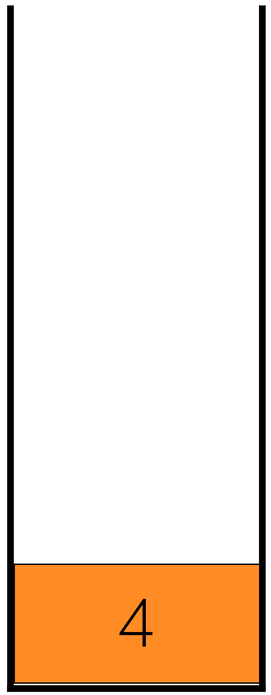

op1
symbol
op2
8
/
2
=

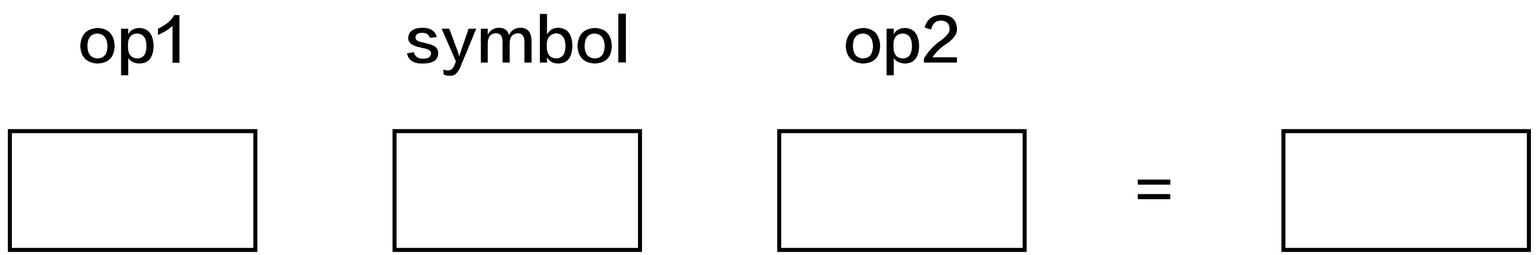

8
2
/
2
+
5
6
*
-
\0

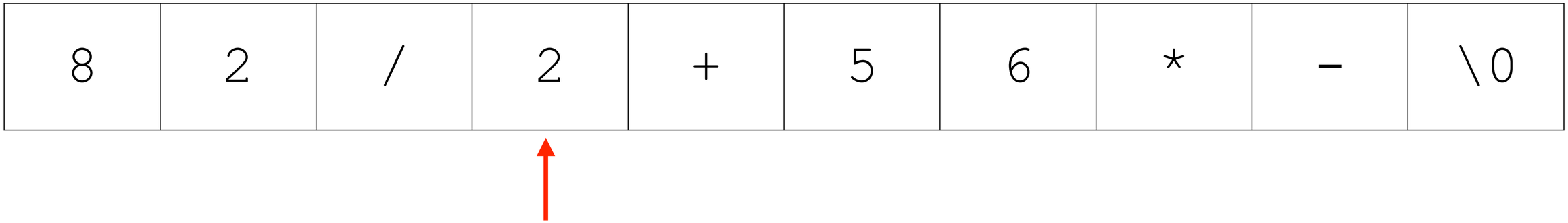


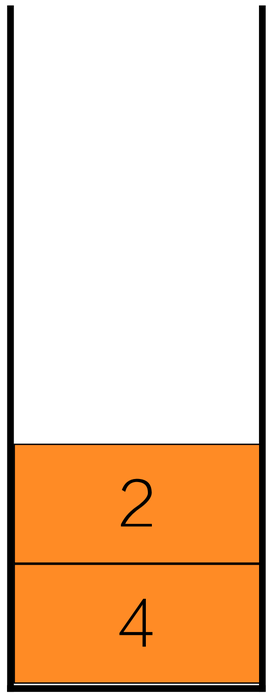

op1
symbol
op2
8
/
2
=
4

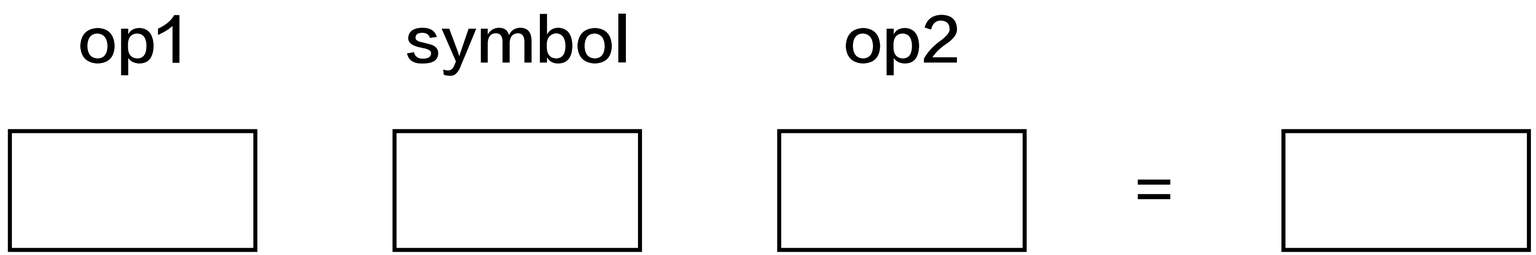

8
2
/
2
+
5
6
*
-
\0

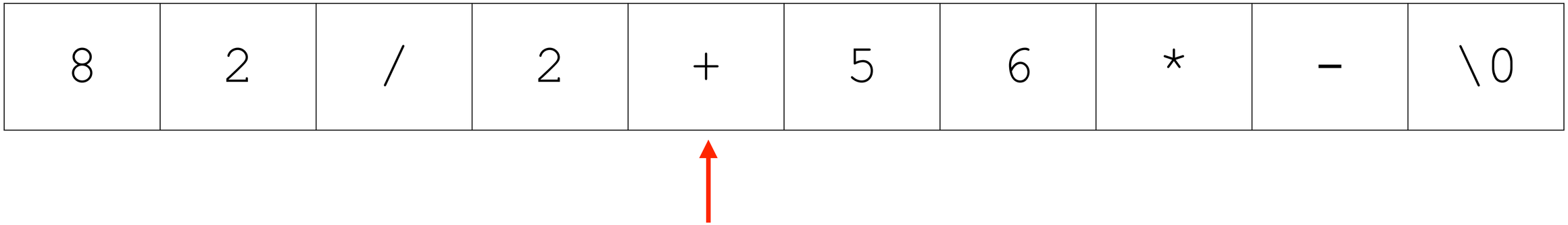

4

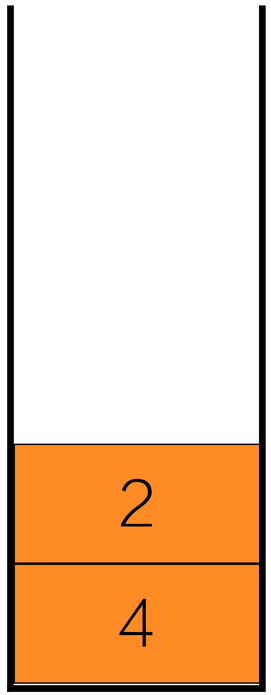

op1
symbol
op2
=

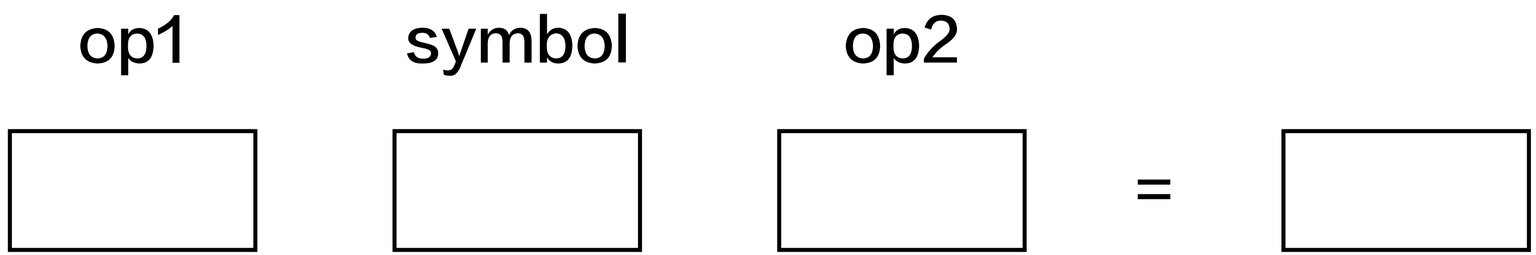

8
2
/
2
+
5
6
*
-
\0

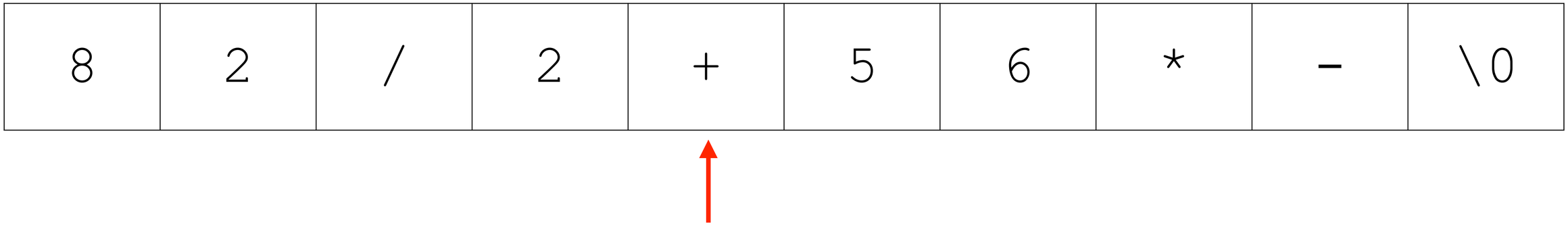

2
4

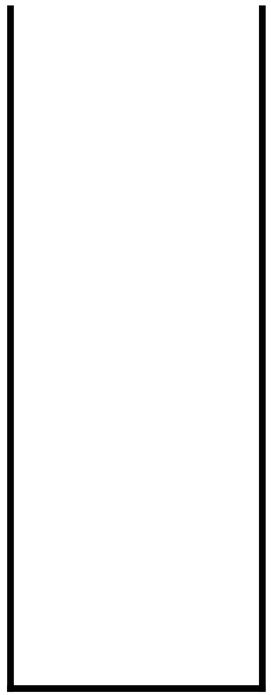

op1
symbol
op2
=

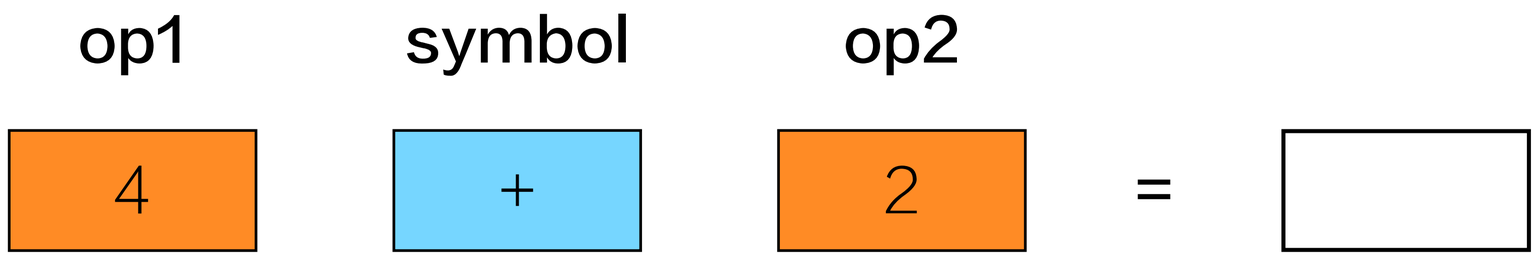

8
2
/
2
+
5
6
*
-
\0

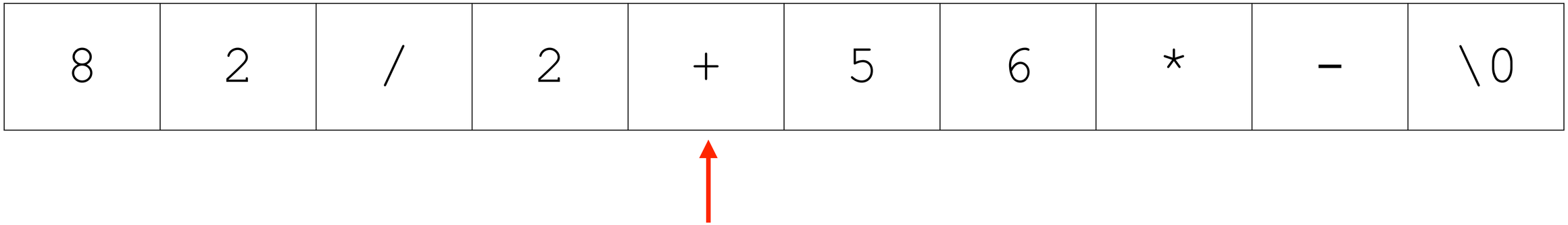

2
4

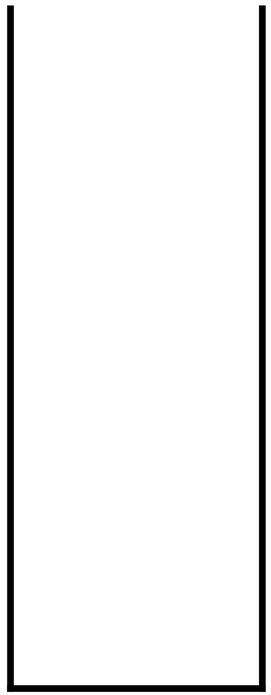

op1
symbol
op2
=

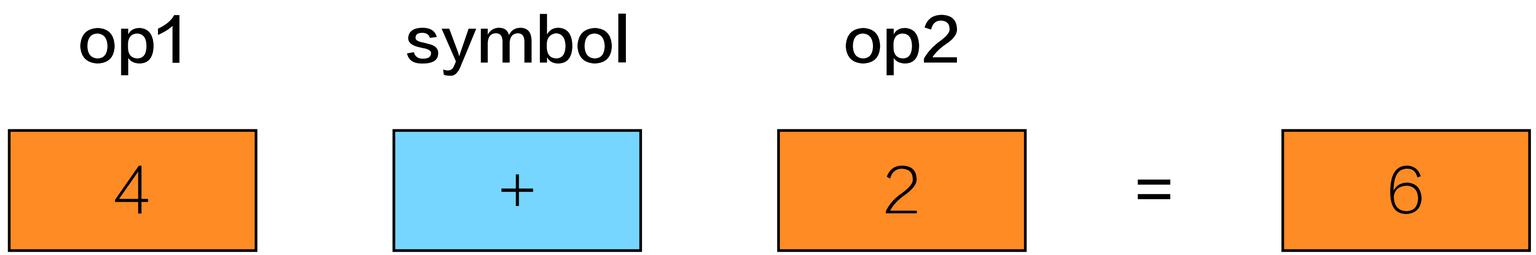

8
2
/
2
+
5
6
*
-
\0

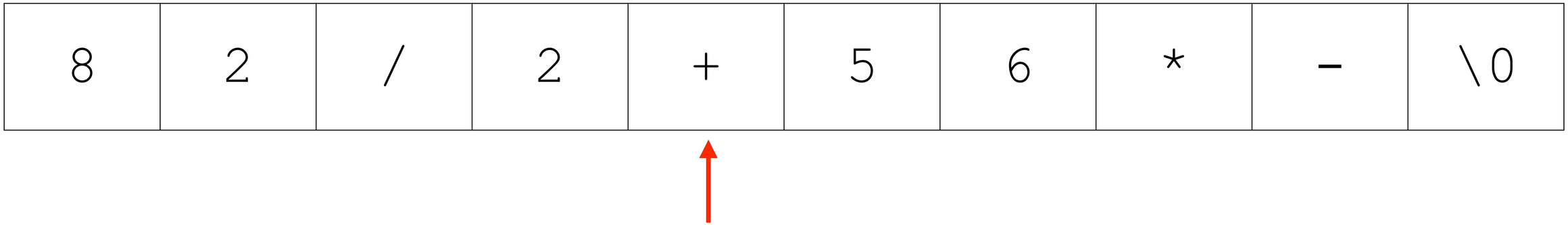


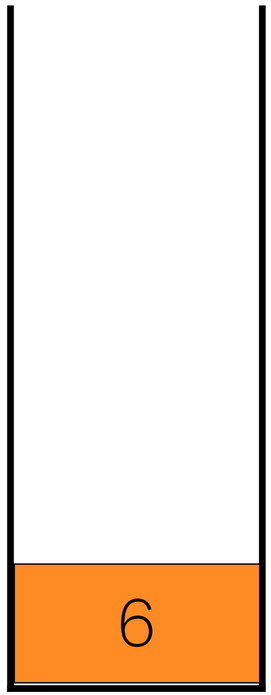

op1
symbol
op2
4
+
2
=

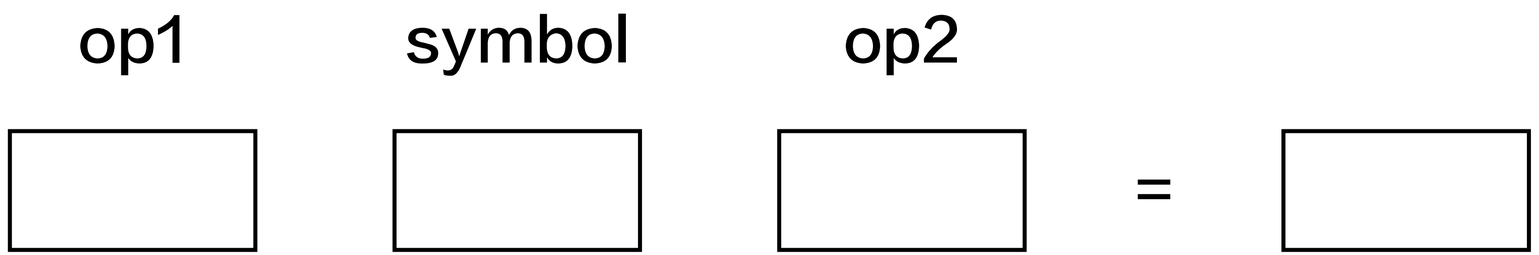

8
2
/
2
+
5
6
*
-
\0

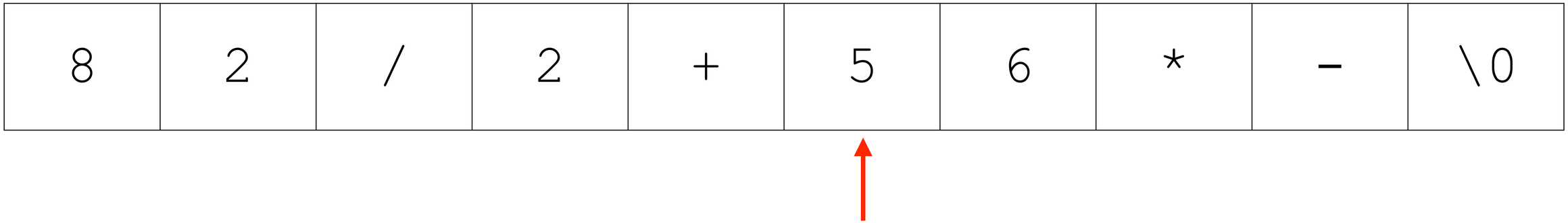


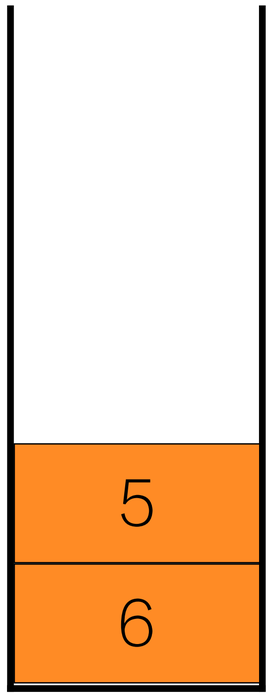

op1
symbol
op2
4
+
2
=
6

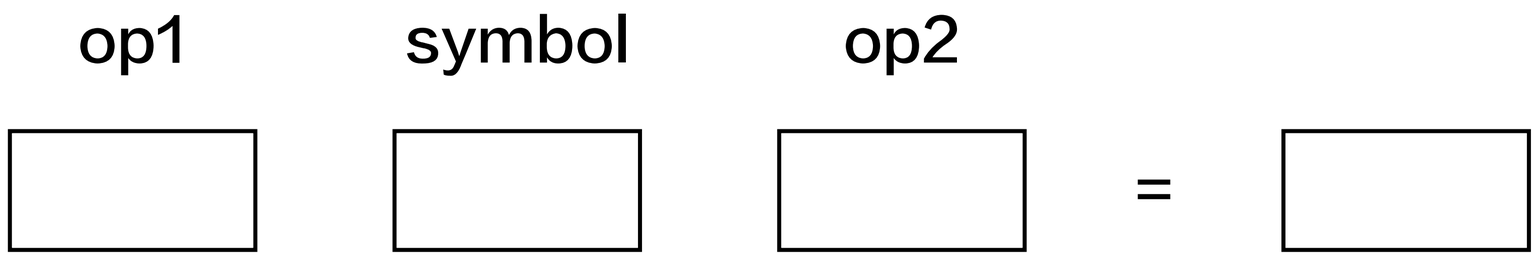

8
2
/
2
+
5
6
*
-
\0

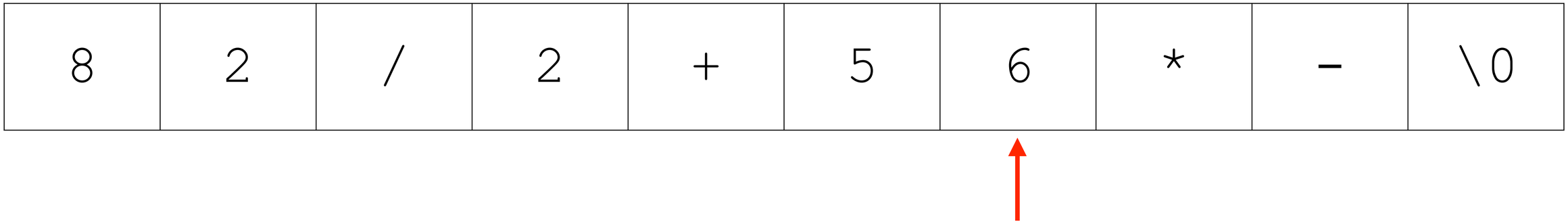

6

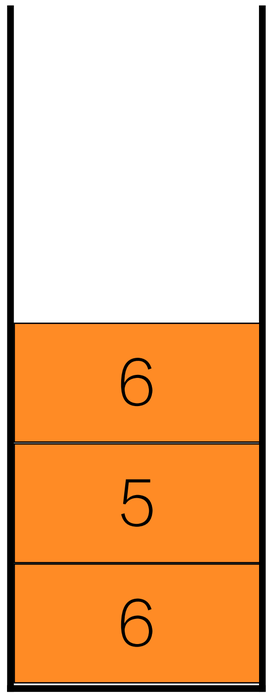

op1
symbol
op2
=

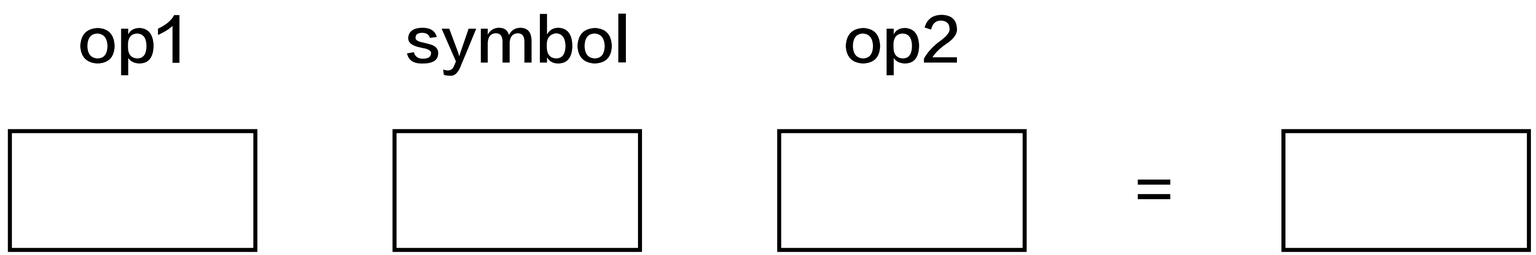

8
2
/
2
+
5
6
*
-
\0

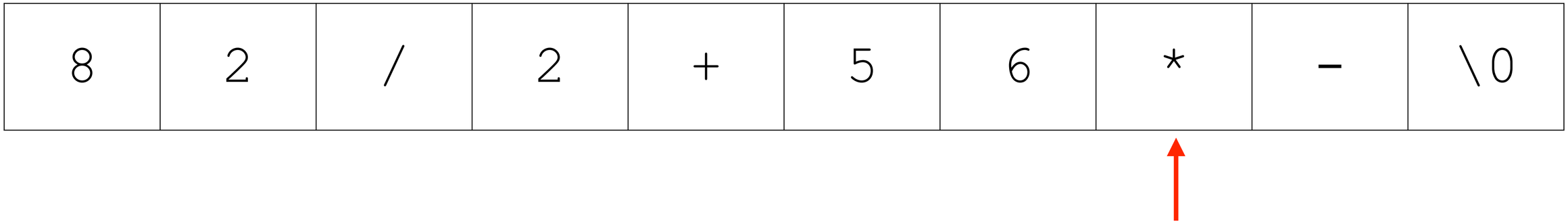

5
6

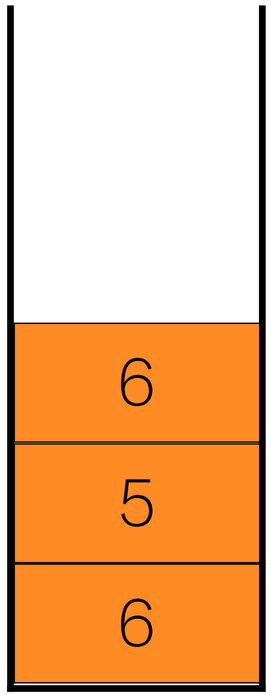

op1
symbol
op2
=

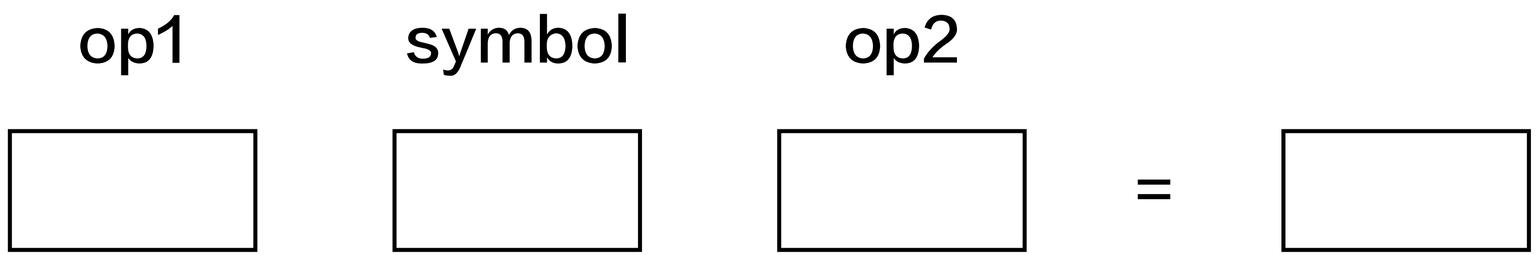

8
2
/
2
+
5
6
*
-
\0

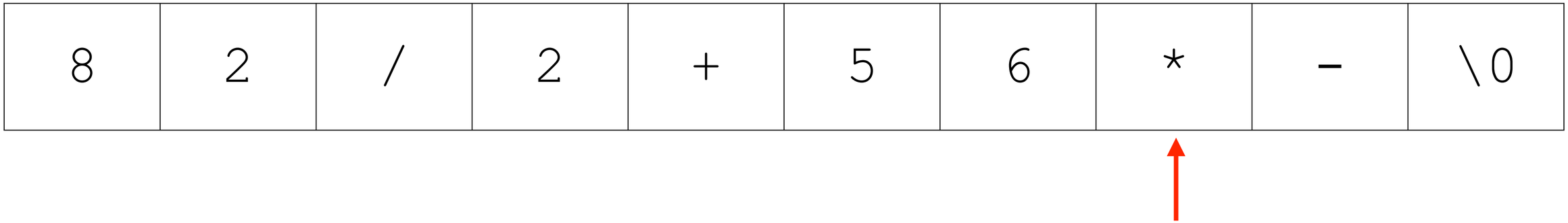

6
5
6

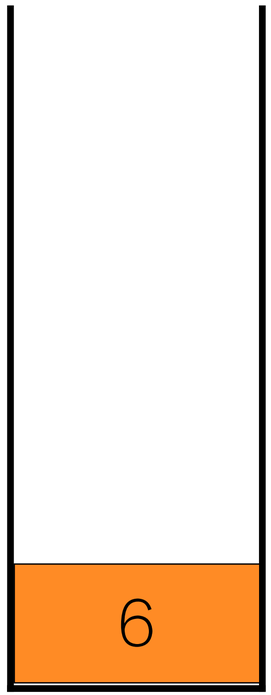

op1
symbol
op2
=

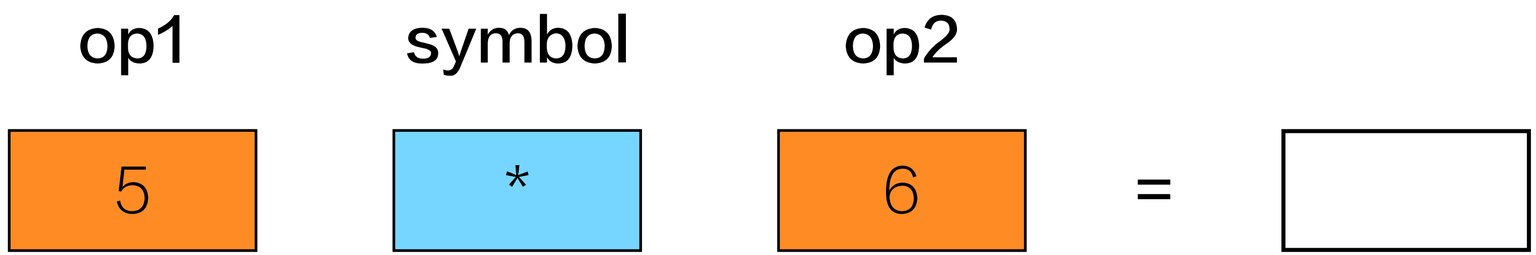

8
2
/
2
+
5
6
*
-
\0

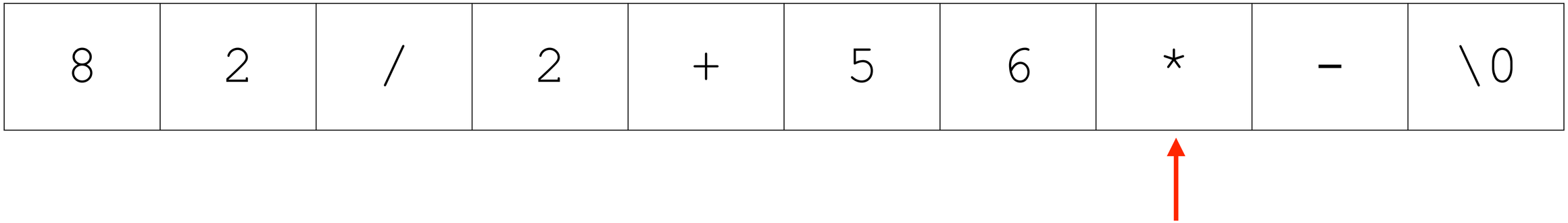

6
5
6

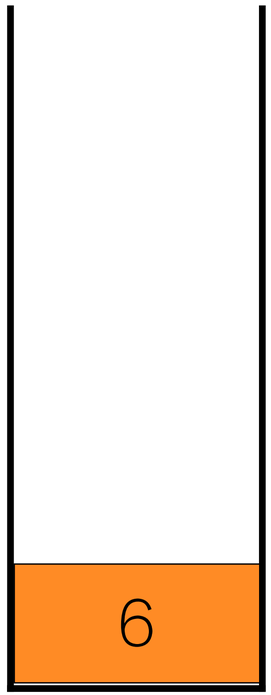

op1
symbol
op2
=

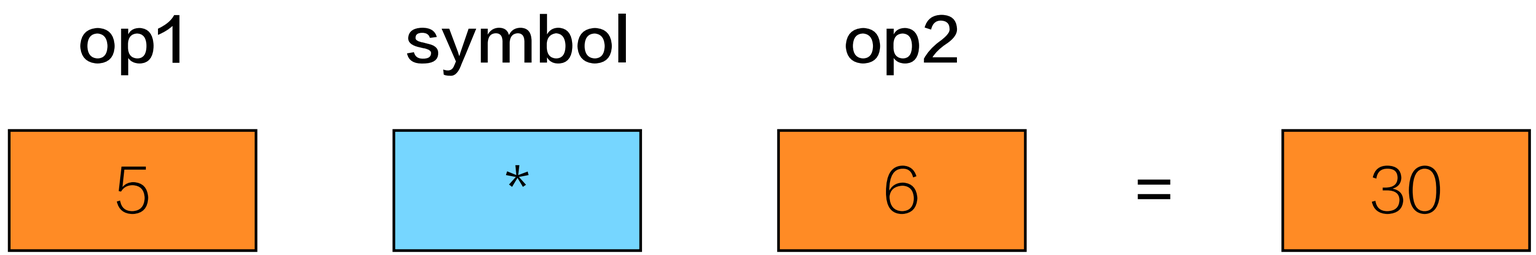

8
2
/
2
+
5
6
*
-
\0

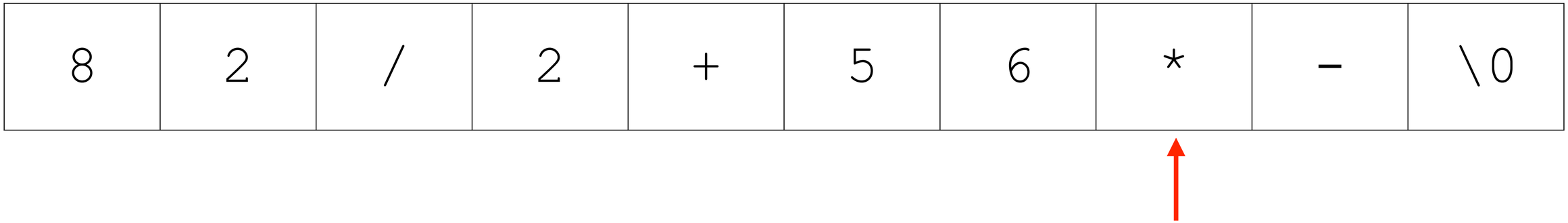

6

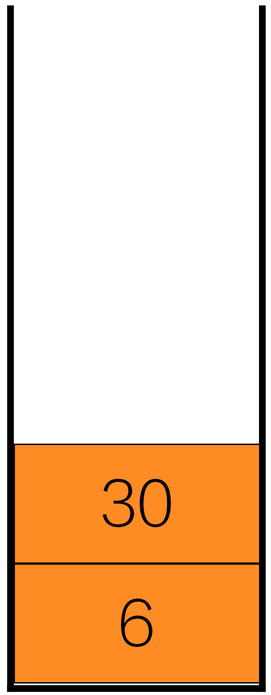

op1
symbol
op2
5
*
6
=

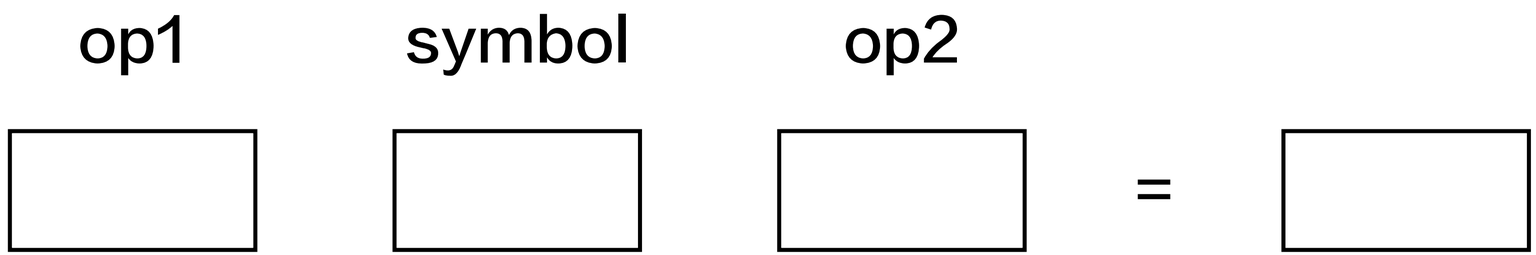

8
2
/
2
+
5
6
*
-
\0

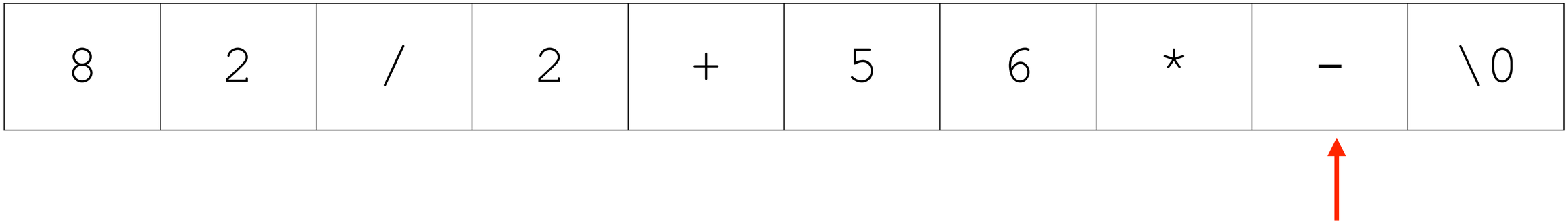

6

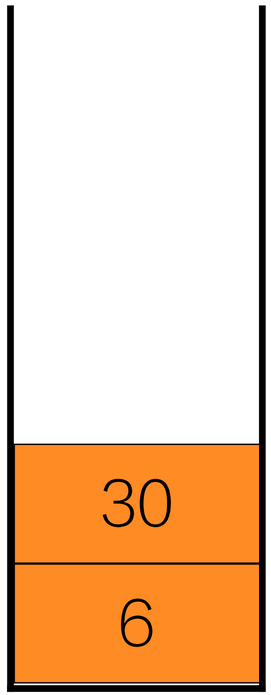

op1
symbol
op2
5
*
6
=
30

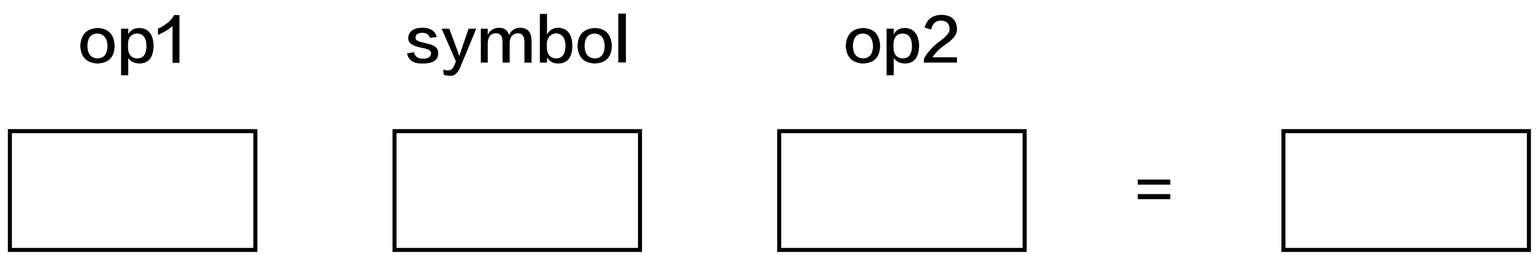

8
2
/
2
+
5
6
*
-
\0

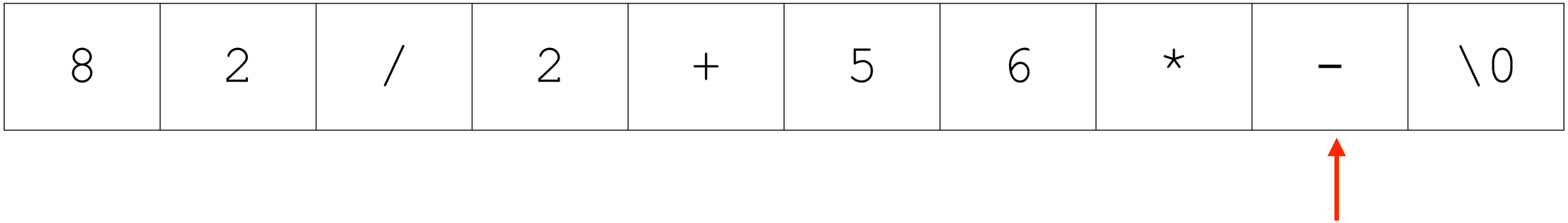

30
6

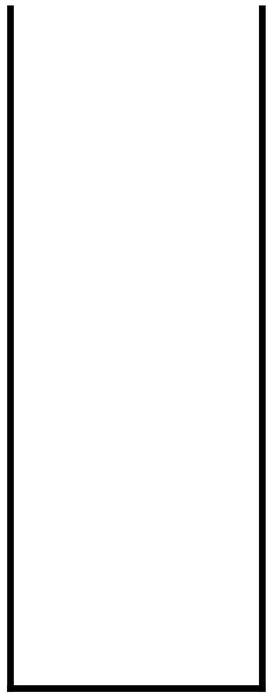

op1
symbol
op2
=

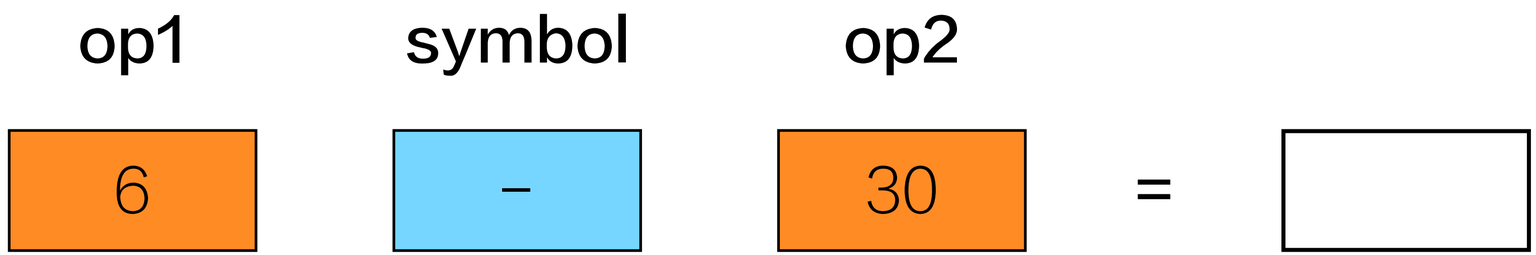

8
2
/
2
+
5
6
*
-
\0


30
6

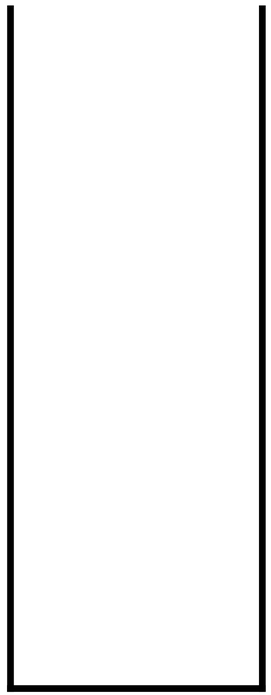

op1
symbol
op2
=


8
2
/
2
+
5
6
*
-
\0

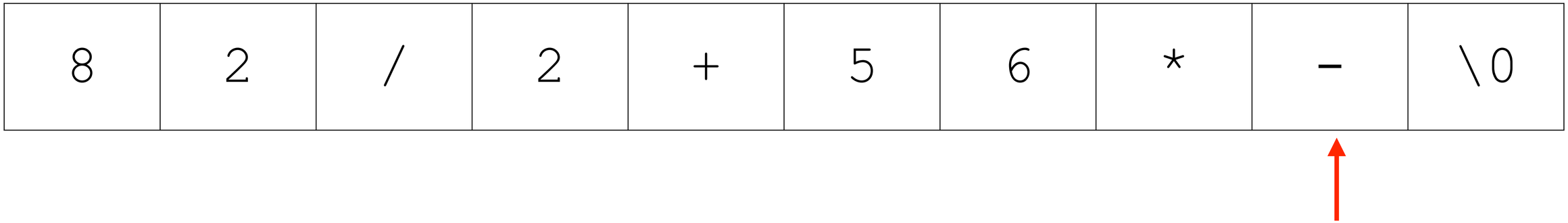


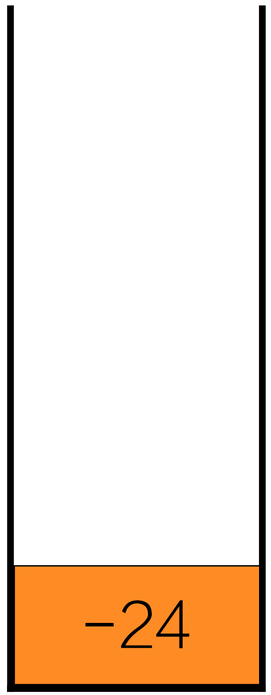

op1
symbol
op2
6
-
30
=

8	2	/	2	+	5	6	*	-	\0



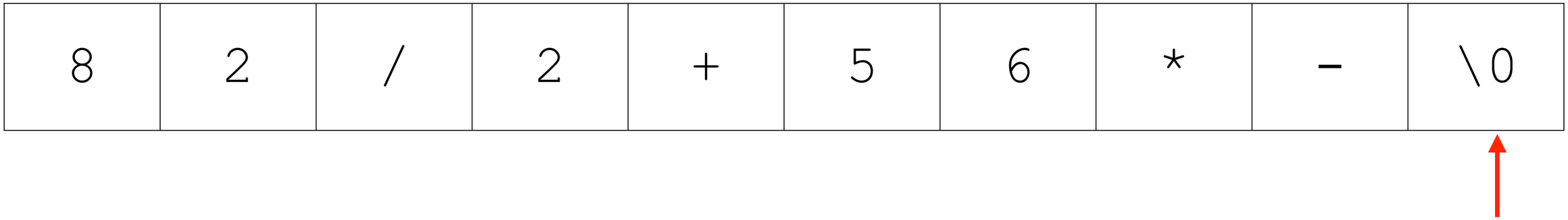
op1	symbol	op2		
6	-	30	=	-24

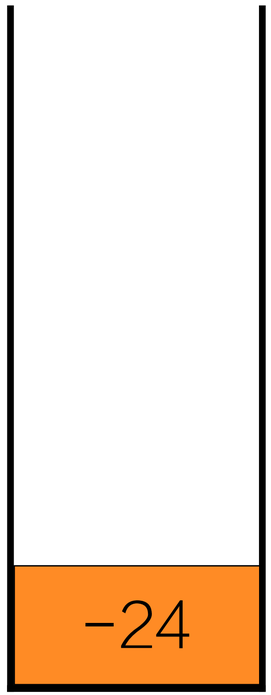


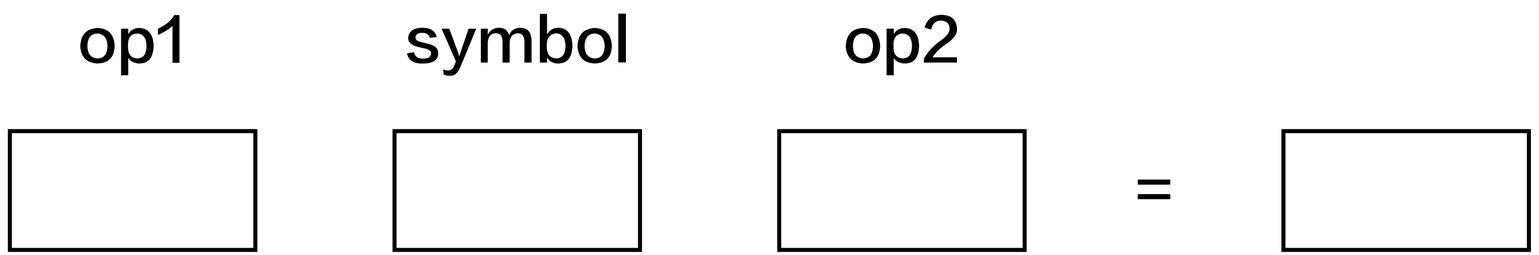


op1 symbol op2

=







案例：栈与队列 / 表达式求值 / test3.c

中缀表达式转后缀表达式

中缀表达式	后缀表达式
5+6*3	563*+
b*c/d	bc*d/
(5+6)*7	56+7*
x/y-z+i*j-x*y	xy/z-ij*+xy*-
x*y-6	xy*6-

中缀表达式转后缀表达式

中缀表达式： x/(i-j)*y  后缀表达式： xij-/y*

中缀表达式转后缀表达式

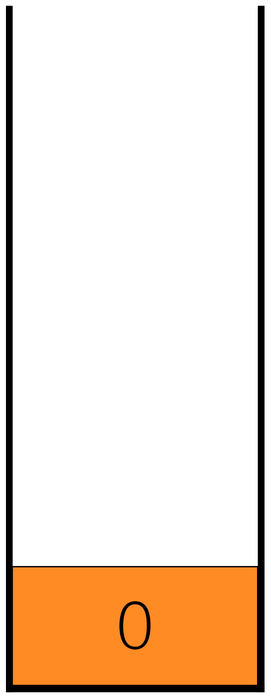
中缀表达式： x/(i-j)*y ⟶ 后缀表达式： xij-/y*

x	/	(	i	-	j	)	*	y	\0

中缀表达式： x/(i-j)*y ⟶ 后缀表达式： xij-/y*

x	/	(	i	-	j	)	*	y	\0

如果是操作数，直接输出

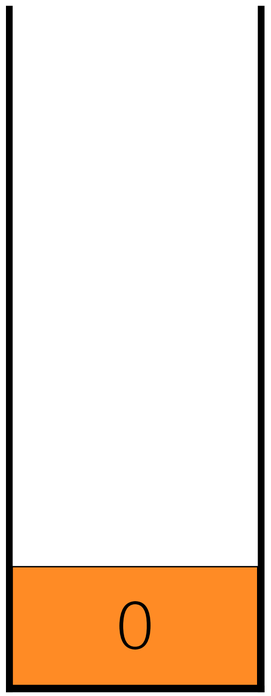


转换后的输出内容：

中缀表达式： x/(i-j)*y ⟶ 后缀表达式： xij-/y*

x	/	(	i	-	j	)	*	y	\0

如果是操作数，直接输出

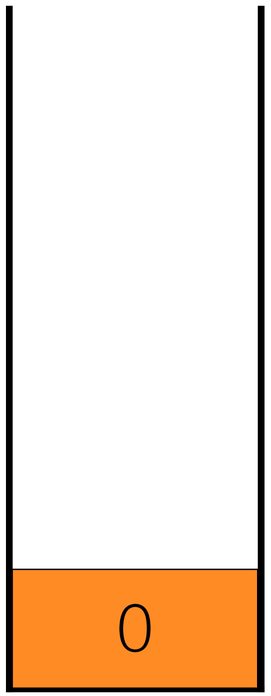


转换后的输出内容：x

中缀表达式： x/(i-j)*y ⟶ 后缀表达式： xij-/y*

x	/	(	i	-	j	)	*	y	\0

如果是运算符，判断优先级，如果优先级大于栈顶元素，压入栈中



转换后的输出内容：x

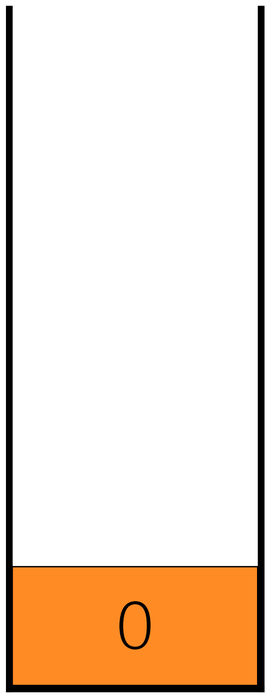
符　号	运 算 符	优先级[1]	结合性
( ) [ ] -> .	函数调用 数组元素 结构或共用体成员	17	自左向右
- - + +	自增，自减[2]	16	自左向右
- - + + ! ~ - + & * sizeof	自减，自增[3] 逻辑非 补 一元减或加 求址或间址 求容量（字节）	15	自右向左
(type)	强制类型转换	14	自右向左
* / %	乘	13	自左向右
+ -	二元加或减	12	自左向右
<< >>	移位	11	自左向右
> >= < <=	关系运算	10	自左向右
== !=	等于	9	自左向右
&	按位与	8	自左向右
^	按位异或	7	自左向右
|	按位或	6	自左向右
&&	逻辑与	5	自左向右
||	逻辑或	4	自左向右
?:	条件	3	自右向左
= += -= /= *= %= <<= >>= &= ^= |=	赋值	2	自右向左
,	逗号	1	自左向右

C: A Reference Manual C:参考手册

中缀表达式： x/(i-j)*y ⟶ 后缀表达式： xij-/y*

x	/	(	i	-	j	)	*	y	\0

如果是运算符，判断优先级，如果优先级大于栈顶元素，压入栈中

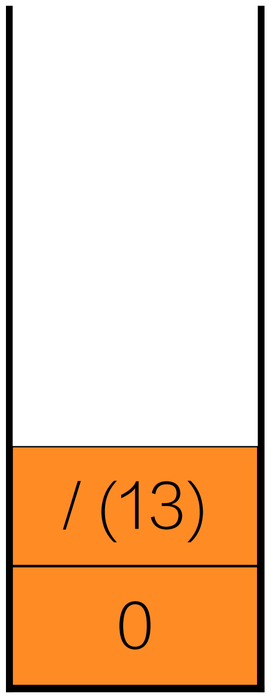


转换后的输出内容：x

中缀表达式： x/(i-j)*y ⟶ 后缀表达式： xij-/y*

x	/	(	i	-	j	)	*	y	\0

如果是运算符，判断优先级，如果优先级大于栈顶元素，压入栈中



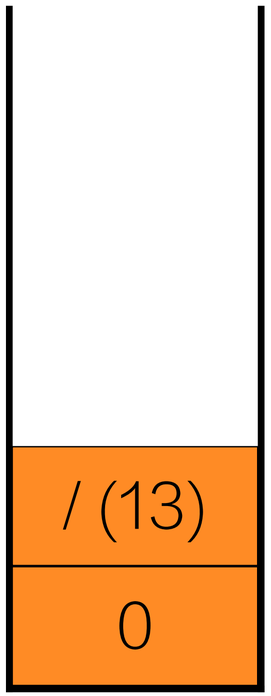
转换后的输出内容：x

中缀表达式： x/(i-j)*y ⟶ 后缀表达式： xij-/y*

x	/	(	i	-	j	)	*	y	\0

如果是运算符，判断优先级，如果优先级大于栈顶元素，压入栈中

当左括号在栈外时，属于最高优先级，当左括号在栈里时，属于最低优先级



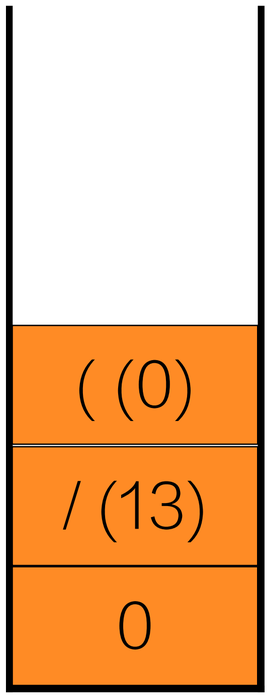
转换后的输出内容：x

中缀表达式： x/(i-j)*y ⟶ 后缀表达式： xij-/y*

x	/	(	i	-	j	)	*	y	\0

如果是运算符，判断优先级，如果优先级大于栈顶元素，压入栈中

当左括号在栈外时，属于最高优先级，当左括号在栈里时，属于最低优先级

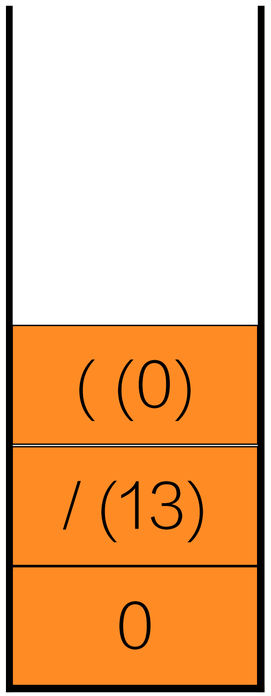


转换后的输出内容：x

中缀表达式： x/(i-j)*y ⟶ 后缀表达式： xij-/y*

x	/	(	i	-	j	)	*	y	\0

如果是操作数，直接输出

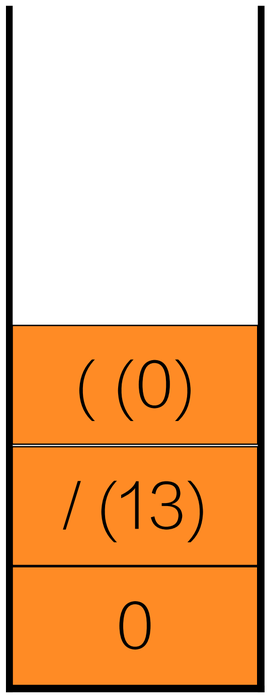


转换后的输出内容：x

中缀表达式： x/(i-j)*y ⟶ 后缀表达式： xij-/y*

x	/	(	i	-	j	)	*	y	\0

如果是操作数，直接输出

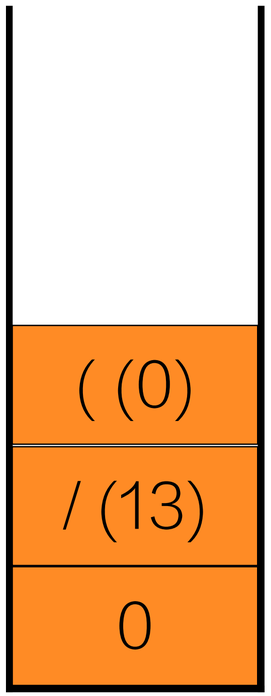


转换后的输出内容：xi

中缀表达式： x/(i-j)*y ⟶ 后缀表达式： xij-/y*

x	/	(	i	-	j	)	*	y	\0

如果是运算符，判断优先级，如果优先级大于栈顶元素，压入栈中

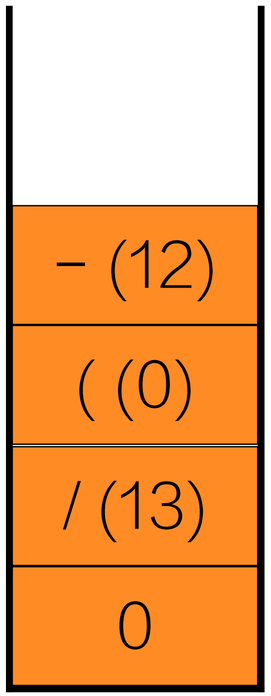


转换后的输出内容：xi

中缀表达式： x/(i-j)*y ⟶ 后缀表达式： xij-/y*

x	/	(	i	-	j	)	*	y	\0

如果是运算符，判断优先级，如果优先级大于栈顶元素，压入栈中

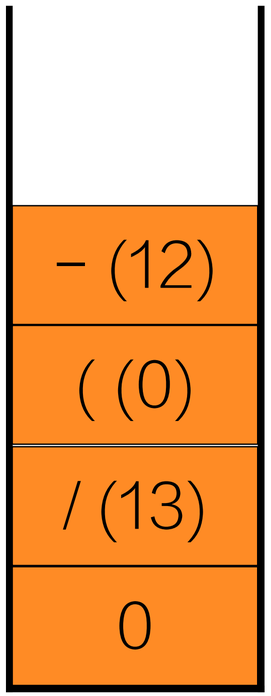


转换后的输出内容：xi

中缀表达式： x/(i-j)*y ⟶ 后缀表达式： xij-/y*

x	/	(	i	-	j	)	*	y	\0

如果是操作数，直接输出

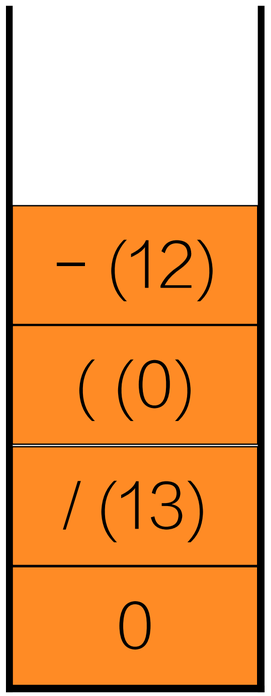


转换后的输出内容：xi

中缀表达式： x/(i-j)*y ⟶ 后缀表达式： xij-/y*

x	/	(	i	-	j	)	*	y	\0

如果是操作数，直接输出

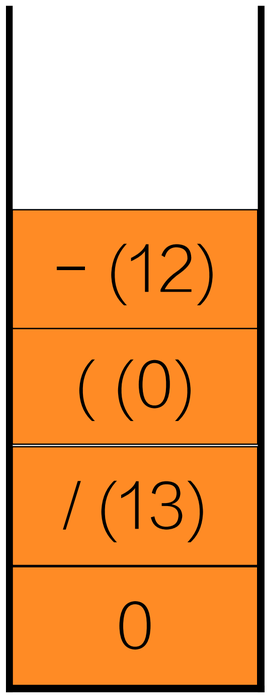


转换后的输出内容：xij

中缀表达式： x/(i-j)*y ⟶ 后缀表达式： xij-/y*

x	/	(	i	-	j	)	*	y	\0

如果是右括号，且栈顶元素不是左括号，持续出栈并输出，直到栈顶为左括号出栈结束
最后将左括号也出栈

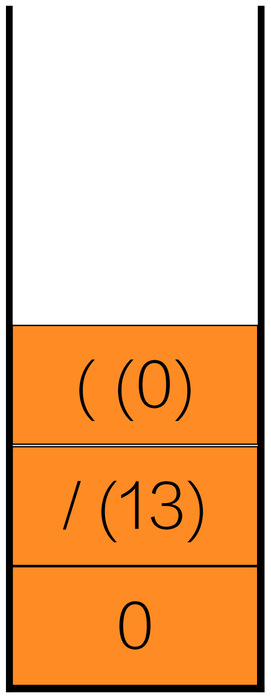


转换后的输出内容：xij

中缀表达式： x/(i-j)*y ⟶ 后缀表达式： xij-/y*

x	/	(	i	-	j	)	*	y	\0

如果是右括号，且栈顶元素不是左括号，持续出栈并输出，直到栈顶为左括号出栈结束
最后将左括号也出栈

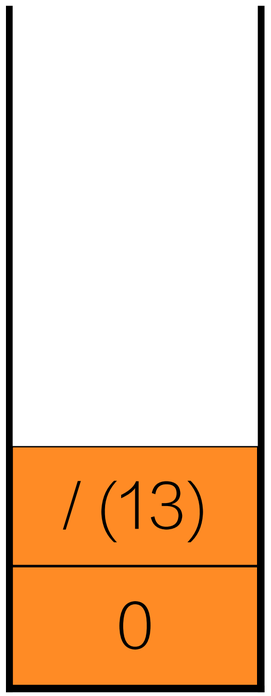


转换后的输出内容：xij-

中缀表达式： x/(i-j)*y ⟶ 后缀表达式： xij-/y*

x	/	(	i	-	j	)	*	y	\0

如果是右括号，且栈顶元素不是左括号，持续出栈并输出，直到栈顶为左括号出栈结束
最后将左括号也出栈

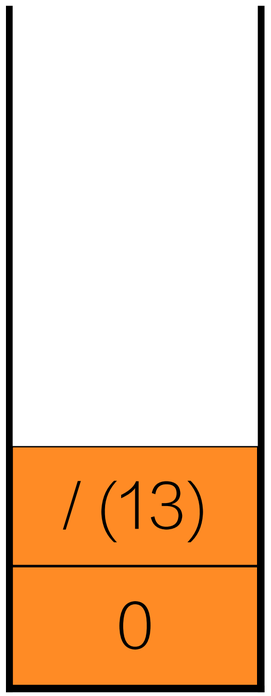


转换后的输出内容：xij-

中缀表达式： x/(i-j)*y ⟶ 后缀表达式： xij-/y*

x	/	(	i	-	j	)	*	y	\0

如果是运算符，判断优先级，如果优先级大于栈顶元素，压入栈中

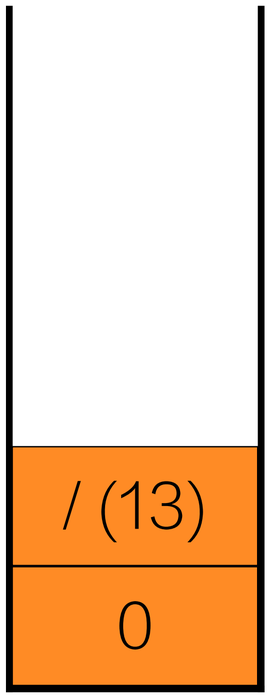


转换后的输出内容：xij-

中缀表达式： x/(i-j)*y ⟶ 后缀表达式： xij-/y*

x	/	(	i	-	j	)	*	y	\0

如果是运算符，判断优先级，如果优先级大于栈顶元素，压入栈中
否则将栈顶元素出栈输出，然后将当前运算符压入栈中

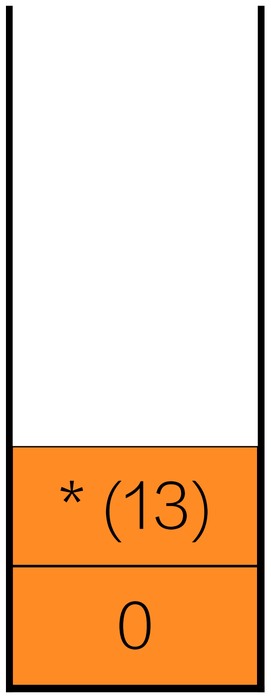


转换后的输出内容：xij-

中缀表达式： x/(i-j)*y ⟶ 后缀表达式： xij-/y*

x	/	(	i	-	j	)	*	y	\0

如果是运算符，判断优先级，如果优先级大于栈顶元素，压入栈中
否则将栈顶元素出栈输出，然后将当前运算符压入栈中

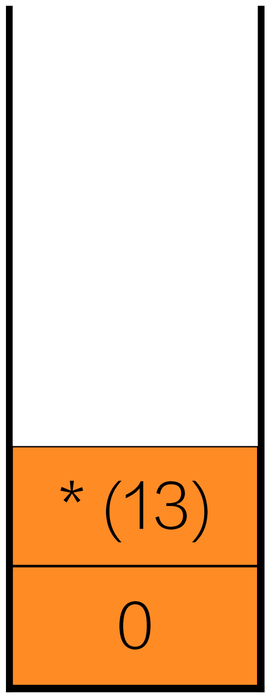


转换后的输出内容：xij-/

中缀表达式： x/(i-j)*y ⟶ 后缀表达式： xij-/y*

x	/	(	i	-	j	)	*	y	\0

如果是操作数，直接输出

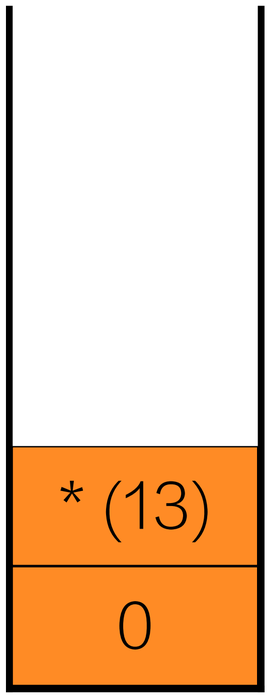


转换后的输出内容：xij-/

中缀表达式： x/(i-j)*y ⟶ 后缀表达式： xij-/y*

x	/	(	i	-	j	)	*	y	\0

如果是操作数，直接输出

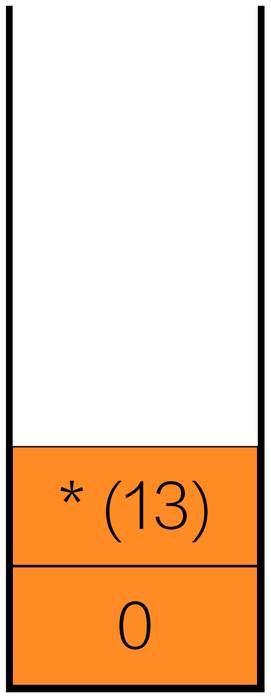


转换后的输出内容：xij-/y

中缀表达式： x/(i-j)*y ⟶ 后缀表达式： xij-/y*

x	/	(	i	-	j	)	*	y	\0

最后将栈中元素依次输出

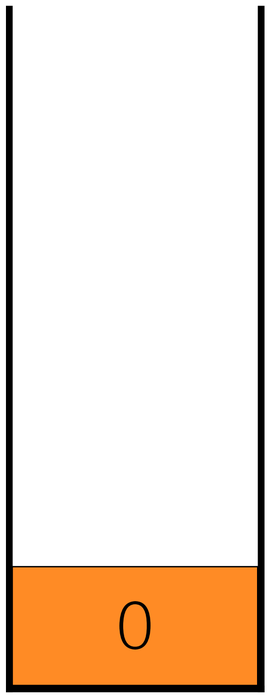


转换后的输出内容：xij-/y

中缀表达式： x/(i-j)*y ⟶ 后缀表达式： xij-/y*

x	/	(	i	-	j	)	*	y	\0

最后将栈中元素依次输出



转换后的输出内容：xij-/y*

栈与队列

维度	栈	队列
逻辑结构	和线性表一样，数据元素之间存在一对一的关系	和线性表一样，数据元素之间存在一对一的关系
存储结构	顺序存储： 存储空间预先分配，可能会出现空间闲置或栈满溢出现象；数据元素个数不能自由扩充	顺序存储（常设计成循环队列形式）： 存储空间预先分配，可能会出现空间闲置或队满溢出现象；数据元素个数不　能自由扩充
	链式存储： 动态分配，不会出现闲置或栈满溢出现象；数据元素个数可以自由扩充	链式存储： 动态分配，不会出现闲置或队满溢出现象；数据元素个数可以自由扩充
运算规则	插入和删除在表的一端（栈顶）完成，后进先出	插入运算在表的一端（队尾）进行，删除运算在表的另一端（队头）进行，先进先出